

ANÁLISE E MODELAGEM DE SISTEMAS E GERÊNCIA DE CONFIGURAÇÃO

Leandro Agostini do Amaral;
Ronnie E. S. Santos.

LIVRO 1

ANÁLISE E MODELAGEM DE
SISTEMAS

LIVRO 2

GERÊNCIA DE CONFIGURAÇÃO



ANÁLISE E MODELAGEM DE SISTEMAS (LIVRO 1)



Presidente do Conselho de Administração	Janguiê Diniz
Diretor-presidente	Jânio Diniz
Diretoria Executiva de Ensino	Adriano Azevedo
Diretoria Executiva de Serviços Corporativos	Joaldo Diniz
Diretoria de Ensino a Distância	Enzo Moreira
Autoria	Leandro Agostini do Amaral
Projeto Gráfico e Capa	DP Content

DADOS DO FORNECEDOR

Análise de Qualidade, Edição de Texto, Design Instrucional,
Edição de Arte, Diagramação, Design Gráfico e Revisão.

© Ser Educacional 2019

Rua Treze de Maio, nº 254, Santo Amaro

Recife-PE – CEP 50100-160

*Todos os gráficos, tabelas e esquemas são creditados à autoria, salvo quando indicada a referência.

Informamos que é de inteira responsabilidade da autoria a emissão de conceitos.

Nenhuma parte desta publicação poderá ser reproduzida por qualquer meio
ou forma sem autorização.

A violação dos direitos autorais é crime estabelecido pela Lei n.º 9.610/98 e punido pelo
artigo 184 do Código Penal.

Imagens de ícones/capa: © Shutterstock

Boxes



ASSISTA

Indicação de filmes, vídeos ou similares que trazem informações complementares ou aprofundadas sobre o conteúdo estudado.



CITANDO

Dados essenciais e pertinentes sobre a vida de uma determinada pessoa relevante para o estudo do conteúdo abordado.



CONTEXTUALIZANDO

Dados que retratam onde e quando aconteceu determinado fato; demonstra-se a situação histórica do assunto.



CURIOSIDADE

Informação que revela algo desconhecido e interessante sobre o assunto tratado.



DICA

Um detalhe específico da informação, um breve conselho, um alerta, uma informação privilegiada sobre o conteúdo trabalhado.



EXEMPLIFICANDO

Informação que retrata de forma objetiva determinado assunto.



EXPLICANDO

Explicação, elucidação sobre uma palavra ou expressão específica da área de conhecimento trabalhada.

Sumário

Unidade 1 - Introdução à análise e desenvolvimento de sistemas

Objetivos da unidade	12
A importância da modelagem	13
Modelagem gráfica X textual	14
Finalidades e benefícios da modelagem	15
Princípios de modelagem	17
Atividades de análise e projeto	20
Levantamento de requisitos	21
Análise, validação e verificação de modelos	23
Projeto (desenho)	23
Implementação, testes e implantação	24
Manutenção e evolução	24
Participantes do processo	27
Análise e projeto orientado a objeto	28
Orientação a objetos	28
Princípios da orientação a objetos	30
Benefícios da orientação a objetos	33
Análise orientada a objetos	34
Projeto orientado a objetos	35
Descrição de caso de uso	37
Sintetizando	38
Referências bibliográficas	39

Sumário

Unidade 2 - Introdução à UML e Ferramentas CASE

Objetivos da unidade	42
Introdução à UML	43
Objetivos da UML	44
Componentes da especificação da UML	45
Mecanismos de uso geral.....	46
Evolução da UML.....	46
Visão dos modelos	49
Ferramentas CASE (<i>Computer-Aided Software Engineering</i>).....	52
Classificação das ferramentas CASE	54
Verificação de ferramentas CASE existentes.....	56
Enterprise Architect	56
ArgoUML.....	59
Visual Paradigm	62
BOUML	65
Umbrello UML Modeller	66
Outras	67
Sintetizando.....	69
Referências bibliográficas	70

Sumário

Unidade 3 - Diagramas UML e suas aplicações

Objetivos da unidade	73
Activity diagram.....	74
Class diagram.....	80
Communication diagram	82
Component diagram	85
Composite structure diagram.....	88
Deployment diagram.....	93
Interaction overview diagram	98
Object diagram.....	101
Sintetizando.....	105
Referências bibliográficas	106

Sumário

Unidade 4 - O uso dos diagramas UML e a engenharia reversa

Objetivos da unidade	108
Package diagram.....	109
Profile diagram	113
State machine diagram	115
Sequence diagram	119
Timing diagram	124
Use case diagram.....	127
Engenharia reversa com UML.....	132
Sintetizando.....	134
Referências bibliográficas	135

Apresentação

O conteúdo sobre análise e modelagem de sistemas integra de modo fundamental o currículo do curso para formar uma base de conceitos e fazer com que o aluno, enquanto futuro desenvolvedor ou gestor de projetos, adquira habilidades de compor e interpretar diferentes modelos para produção de software com qualidade. Isso norteará, de modo adequado, todo processo de desenvolvimento de software, minimizando os riscos com a adoção de modelagens, evitando retrabalho, prejuízos e estresse no desenvolvimento de componentes errados ou incoerentes com a necessidade real dos clientes.

Como linha tecnológica, será utilizada a *Unified Modeling Language* (UML) em sua segunda versão para modelar os softwares no paradigma de orientação a objetos. De modo prático, serão vistos, com exemplos, imagens, explicações e os principais tipos de diagramas da UML.

Por fim, será abordado como realizar a engenharia reversa de um software utilizando a UML, orientando o(a) aluno(a) sobre como iniciar a análise e o projeto de evolução de um software já existente a partir da verificação e entendimento dos requisitos nele implementados.

Desejamos a todos um excelente estudo!

O autor



O **professor Leandro Agostini do Amaral** é doutor (2019) e mestre (2014) em Ciências de Computação e Matemática Computacional, na linha de pesquisa de Engenharia de Software e Sistemas de Informação/Sistemas Web e Multimídia Interativos, pela Universidade de São Paulo (USP) e graduado em Engenharia de Computação (2010) pela mesma universidade. Tem publicações em anais de congressos e periódicos importantes, além de possuir experiência na área da indústria com empreendedorismo em startups e pesquisa em análise e modelagem de sistemas.

Currículo Lattes:

<http://lattes.cnpq.br/6798864284254487>

Dedico este material, primordialmente, a Deus e aos meus pais, Wilson e Marlene, que sempre me auxiliaram ao longo da minha formação, acreditando em mim e ressaltando a importância do esforço e do respeito ao próximo. Agradeço, ainda, aos meus sábios mestres, que me guiaram nessa trajetória de aprendizado contínuo.

UNIDADE

1

INTRODUÇÃO
À ANÁLISE E
DESENVOLVIMENTO
DE SISTEMAS



ser
educacional

Objetivos da unidade

- Apresentar elementos que evidenciem, de modo teórico e prático, a importância da modelagem no processo do desenvolvimento de software;
- Fornecer ao aluno conceitos básicos de princípios que regem a técnica de modelagem de software;
- Ampliar conhecimentos básicos sobre as diferentes atividades da fase de análise para investigação e verificação do que será feito, com execução de desenvolvimentos para a adequada construção de software;
- Apresentar o projeto orientado a objetos, com suas características e vantagens de utilização, formando a base do aluno para realizar a análise e projeto de software;
- Descrever os conceitos envolvidos em casos de uso, enfatizando sua importância, o que são e quando devem ser utilizados.

Tópicos de estudo

● A importância da modelagem

● Modelagem gráfica X textual

● Finalidades e benefícios da modelagem

● Notação

● Princípios de modelagem

● Atividades de análise e projeto

- Levantamento de requisitos
- Análise, validação e verificação de modelos

● Implementação

● Testes

● Implementação, testes e implantação

● Manutenção e evolução

● Participantes do processo

● Atividades de análise e projeto

- Orientação a objetos
- Princípios da orientação a objetos
- Benefícios da orientação a objetos
- Análise orientada a objetos
- Projeto orientado a objetos

● Descrição de caso de uso



A importância da modelagem

A cada dia, os softwares têm ganhado maior importância no cotidiano das pessoas, já que praticamente todo equipamento elétrico/eletrônico contém algum software que rege seu funcionamento. Até mesmo os softwares mais simples têm uma significativa complexidade inerente, necessitando, portanto, de técnicas sistêmicas da área de Engenharia de Software (ES) para sua criação.

Nesse contexto, a modelagem se enquadra em uma das mais importantes técnicas para planejamento, produção, evolução e manutenção de software, já que permite registrar e manipular informações do produto que está sendo trabalhado.

A modelagem pode ser entendida como um meio de planejamento que utiliza modelos, normalmente gráficos, para a construção eficiente dos componentes de software.

A importância desse planejamento é comparada à construção civil, em que são realizadas esquematizações visuais de todos os elementos que constituem uma obra, desde paredes, janelas e telhado até aspectos elétricos, hidráulicos e estruturais.

Na Engenharia Civil, as plantas de uma obra constituem os modelos que permitem a instalação da construção, contendo todos os dados que parametrizaram o trabalho dos operários.

Isso permite, além do alinhamento de ações dos operários, a compra dos suprimentos em quantidade adequada e a existência de instrumentos para o acompanhamento da obra pelos engenheiros. Assim, eles podem verificar a adequação do que está sendo construído ao que foi modelado e em relação ao cronograma de execução.

De acordo com Deboni, no livro *Modelagem orientada a objetos com a UML*, publicado em 2003, a planta é um modelo que representa graficamente o produto final. Por exemplo: a planta de um edifício permite que o cliente avalie o andamento de sua construção de acordo com as datas de entrega de cada fase da obra. Além disso, a planta apresenta notações padronizadas, como a escala de suas medidas, para facilitar o entendimento dos leitores e dirimir possíveis dúvidas.

Assim, no âmbito da **construção civil**, fica fácil identificar o caos que se tornaria uma construção sem a sua modelagem e projeto. Seria difícil explicar aos operários o que fazer e os detalhes das tarefas em cada etapa de construção.



CURIOSIDADE

É importante ressaltar que, nos municípios, são exigidos o registro e a aprovação dos projetos de construção civil, pois as obras devem seguir regulamentos especiais, visando o bem comum e segurança da população. Quanto aos softwares, em poucos casos é exigida a aprovação de projetos, ocorrendo apenas para alguns softwares de geração de documentos financeiros e para sistemas de alto risco. O que vem acontecendo na atualidade em relação aos softwares é a necessidade de aprovação de aplicativos para dispositivos móveis em lojas virtuais.

Ainda conforme Deboni (2003), deve ser valorizada a atividade de criação de modelos no desenvolvimento de software de modo a i) reduzir as incertezas do produto; ii) aproximar a expectativa da realização; iii) facilitar a padronização e a automação dos projetos; iv) melhorar a reutilização de componentes e, por fim, v) aumentar a maturidade da engenharia de software nas equipes de desenvolvimento.

Nesse contexto, o autor Bezerra, no livro *Princípios de análise e projeto de sistemas com UML*, publicado em 2007, relata que os projetos de desenvolvimento de software têm uma característica intrínseca: sua complexidade gradativa, que aumenta conforme cresce o tamanho do sistema.

Desse modo, são necessárias ferramentas que auxiliem na gestão dessa complexidade, visando o desenvolvimento de softwares com sucesso e que possam evoluir com o tempo. E uma característica frequente das empresas que criam softwares com sucesso, de médio e grande porte, é a utilização, com maturidade, da modelagem em seus processos.



Modelagem gráfica X textual

Existem diversos tipos de modelos no desenvolvimento de software para representar diferentes estruturas de seu funcionamento. No entanto, existe uma categoria que se destaca nesse contexto: a dos **modelos gráficos**.

Os modelos gráficos são desenhos visuais que seguem determinado padrão lógico, normalmente denominados **diagramas**. Um diagrama, por sua vez, pode ser entendido como uma coleção de elementos gráficos com semântica bem definida.



A modelagem gráfica traz como vantagem a facilidade de leitura e interpretação. No entanto, no desenvolvimento de software, normalmente os desenhos são acompanhados de informações textuais para explicar e conferir informações complementares.

Como exemplo do uso de sucesso da modelagem gráfica, podemos citar a alocação de máquinas, servidores e clientes em um projeto, em que são utilizados **ícones de computadores e de nuvens** para definir a arquitetura desejada. Nesse cenário, um integrante da equipe consegue visualizar mais facilmente a ideia representada no modelo do que em um texto com a listagem das máquinas.

Existe um conceito que prevê o uso dos modelos não apenas como referência documental, mas também como artefatos vivos, servindo de entrada para ferramentas de geração de código. Essa evolução no uso de modelos se chama **desenvolvimento orientado a modelos**, ou *model-driven development* (MDD), já contando com ferramentas para seu suporte e pertencendo a uma arquitetura padronizada pelo OMG – *Object Management Group*: a chamada arquitetura dirigida a modelos, ou *model-driven architecture* (MDA).

Ou seja, a modelagem vem ganhando cada vez mais importância no mercado de desenvolvimento de software e, conseqüentemente, mais recursos técnicos para sua utilização.

Assim, não se assuste se partes significativas de desenvolvimento de software hoje feitas por programadores via digitação de código forem substituídas pela criação de modelos gráficos de “arrastar e soltar” no mouse.



Finalidades e benefícios da modelagem



De acordo com Blaha e Rumbaugh, no livro *Modelagem e projetos baseados em objetos*, publicado em 2006, a modelagem pode ser entendida com uma técnica de composição de modelos com abstrações de diferentes visões de sistema. Esses modelos têm várias finalidades, agregando benefícios ao projeto, tais como:

- **Teste de uma entidade antes da construção:** é possível realizar verificações e validações nos modelos antes da construção do software. Esse procedimento é análogo aos testes de modelos de aviões, carros, barcos, túneis de vento e tanques de água;

- **Comunicação com o cliente:** podem ser feitos modelos de tela que funcionam como protótipos, permitindo que os clientes entendam melhor o projeto, validando decisões e indicando sugestões aos projetistas;

- **Visualização:** modelos podem ser vistos como storyboards de filmes, que contemplam fluxo de ideias e transições, como um livro de história em quadrinhos, sem a necessidade da escrita detalhada do filme. Do mesmo modo, na Engenharia de Software, podem ser desenhadas transições das interações de usuários e fluxos de gravação de dados em servidores;

- **Redução de complexidade:** finalidade que incorpora as anteriores pela necessidade humana de se lidar com uma quantidade limitada de informações de cada vez. Com as separações de aspectos em diferentes modelos e abstração de informações, o software fica mais fácil de ser entendido e projetado.

Bezerra, por sua vez, em seu livro *Princípios de análise e projeto de sistemas com UML*, publicado em 2007, adiciona a esses pontos as seguintes finalidades da modelagem:

- **Comunicação entre as pessoas envolvidas:** os modelos permitem uma boa difusão de informações sobre os desenvolvimentos não somente para entendimento dos clientes, mas também entre toda a equipe de trabalho envolvida, desde programadores até gerentes e outros interessados (chamados de stakeholders);

- **Redução de custos de desenvolvimento:** qualquer desenvolvimento está propenso a erros e, se eles forem encontrados em fases de modelagem, a correção se torna significativamente mais barata. O alinhamento com os desejos dos clientes e com a validação do modelo também promove a diminuição de riscos;

- **Previsão de comportamento do sistema:** o registro de ações que o software deve realizar no modelo faz com que não haja surpresas no comportamento do produto final.



Notação



De acordo com Quatrani, no livro *Modelagem visual com Rational Rose 2000 e UML*, publicado em 2001, o modo sistêmico de escrita de modelos, com a definição de elementos próprios, é chamado de **notação**, sendo considerada a “cola” que mantém o processo unido. A notação tem três papéis fundamentais:

- **Servir como a linguagem** para comunicar as decisões que não são óbvias ou que não podem ser entendidas a partir do código-fonte;
- **Oferecer elementos com semânticas ricas** para captar importantes decisões estratégicas;
- **Ser um modo de trabalho concreto.**

É importante ressaltar que, apesar da clara utilidade das notações, em um primeiro momento elas podem ser consideradas dificultadoras, já que o leitor precisa entendê-las. Um exemplo de seu uso pode ser visto no desenho de placas de trânsito: o condutor deve conhecer as simbologias para compreender o seu significado. Na área de Tecnologia da Informação (TI), um exemplo que pode ser citado é a forma de setas e o tracejado em linhas, que significam diferentes propriedades em alguns diagramas de projeto de software.

Um ponto interessante no uso de notações é a possibilidade de apoio em seu uso com ferramentas de software para a criação dos modelos. As ferramentas, dessa forma, podem criar ícones entendíveis pelos seus usuários (conhecedores do padrão de notação) e realizar checagens da utilização correta do conceito embutido na notação.



Princípios de modelagem

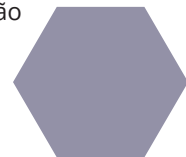


Segundo os renomados autores Booch, Rumbaugh e Jacobson, no livro *UML: guia do usuário*, publicado em 2005, existem quatro princípios básicos que norteiam essas atividades de planejamento em questão:

1º princípio: a escolha dos modelos influencia a maneira como determinado problema é atacado e como uma solução é definida.

Esse princípio se refere a escolher bem os modelos com os quais você deseja trabalhar. Em relação aos softwares, a escolha de modelos varia de acordo com a visão de mundo do projetista. Projetistas distintos podem criar modelos variados a partir dos mesmos requisitos. Nesse caso, a experiência e o conhecimento do projetista são essenciais para produzir modelos que representem menor custo e maior eficiência do software.

Outra questão importante para esse princípio é que a criação de projetos que requeiram tecnologias desconheci-



das pode dificultar o andamento do trabalho, pois existe uma curva de aprendizado influenciando o tempo de desenvolvimento do software.

2º princípio: cada modelo poderá ser expresso em diferentes níveis de precisão.

Ao desenvolver um software complexo, pode ser necessário o uso de uma visão panorâmica. Em outros casos, pode ser requisitado o retorno ao nível micro da base de componentes. Desse modo, os melhores tipos de modelos são os que permitem a escolha do grau de detalhamento das informações (como um zoom em fotografias), dependendo de quem esteja fazendo a visualização e para qual necessidade deseja fazê-la.

Por exemplo, o usuário final irá focar em questões relacionadas à visualização da interface gráfica do software e às funcionalidades que pode acessar. O desenvolvedor, por sua vez, focará na maneira como os objetos funcionam e se relacionam.

3º princípio: os melhores modelos estão relacionados à realidade.

Devem ser priorizados modelos que tenham forte conexão com a realidade e, nos casos em que essa conexão seja fraca, saber, de modo exato, como esses modelos são diferentes do mundo real.

Todos os modelos objetivam simplificar a realidade. Dessa forma, você deve ter sempre a preocupação de que a simplificação não oculte detalhes importantes do software, ignorando possíveis funcionalidades futuras.

4º princípio: nenhum modelo único é suficiente. Qualquer sistema não trivial será mais bem investigado por meio de um pequeno conjunto de modelos quase independentes e com vários pontos de vista.

Nesse contexto, a expressão “quase independentes” significa modelos que possam ser criados e estudados separadamente, mas que continuam inter-relacionados.

Para compreender esse princípio com base no paradigma de softwares orientados a objetos, é necessário recorrer a cinco visões complementares e inter-relacionadas:

- **Visão de casos de uso:** expor os requisitos do sistema na forma de atores e suas ações;
- **Visão de projeto:** capturar o vocabulário do problema a ser resolvido;
- **Visão de processo:** modelar a distribuição dos processos e das atividades concorrentes do software;

- **Visão de implementação:** expor questões técnicas de engenharia dos componentes do software;

- **Visão de implantação:** detalhar características da distribuição física de um software e seus componentes e conexões.

Essas visões podem ser vistas de modo mais didático a partir do esquema apresentado no Diagrama 1.

DIAGRAMA 1. VISÕES COMPLEMENTARES E INTER-RELACIONADAS DE UM SOFTWARE



Fonte: BEZERRA, 2007, p. 17. (Adaptado).

Como pode ser visto no Diagrama 1, a visão de casos de uso está no centro do esquema, demonstrando sua importância para a descrição de possíveis usos do software em um determinado domínio.

Cada uma dessas visões pode conter diferentes aspectos estruturais e aspectos comportamentais, sendo que, em conjunto, elas formam a base do projeto de software.



Atividades de análise e projeto

Antes de descrever as atividades de análise e projeto, é preciso apresentar bem as diferenças de cada uma dessas macrofases.

Na fase de análise, é verificado o **domínio do problema**. É preciso, então, ser visto o que está ocorrendo, com seus detalhes e características de demandas que devem ser resolvidas em um determinado software.

Na fase de projeto, por sua vez, são identificados meios e elementos para solucionar o problema identificado. Nesta fase, a chave da questão está na palavra **como**, uma vez que nela é projetada e modelada a maior parte do software, incluindo seus componentes e sua arquitetura, que contempla a divisão de estruturas.

Essa diferenciação básica de fases pode ser mais bem compreendida pela esquematização da Figura 1.



Figura 1. Diferenciação básica das fases de análise e projeto de software.

Como pode ser visto na Figura 1, existem dois domínios bem separados: um já existente, que é o **domínio do problema**, em que é necessário investigar o que está ocorrendo em detalhes, e outro **domínio da solução**, que deve ser criado na fase de projeto.

Em suma, é necessário verificar o contexto das necessidades reais de um software em determinado ambiente, sendo essa a fase de análise que ocorre no domínio do problema. A partir de então, na **fase de projeto**, são tomadas decisões técnicas para criar a solução de software que, ao ser implementada, irá atender às demandas e aos usuários previstos, tudo correlacionado com a fase de análise.



CURIOSIDADE

Na prática, o analista de sistemas atua na fase de projeto e na codificação dos sistemas.

Para tratar essa questão, incluindo a fase de projeto, o Ministério da Educação (MEC), em seu Catálogo Nacional de Cursos Superiores de Tecnologia, publicado em 2016, define o nome do curso que forma esse tipo de profissionais como Tecnologia em Análise e Desenvolvimento de Sistemas.

Tanto na análise quanto no projeto tem-se a definição de várias atividades, que podem ser dispostas em diferentes encadeamentos de acordo com o modelo de ciclo de vida adotado. Desse modo, com as fases de análise e projeto bem definidas, vamos nos concentrar em apresentar as atividades comuns no processo de desenvolvimento de software, que são tarefas de construção, entrega e evolução do software.



Levantamento de requisitos

Também conhecida como elicitação ou identificação de requisitos, essa etapa de análise consiste na compreensão e registro de detalhes do problema que o software visa solucionar.

Tais detalhes de problemas correspondem às necessidades de desenvolvimento do software, sendo chamados de **requisitos**. Cada requisito está relacionado a uma condição ou capacidade implementada no software para resolução de problemas de um domínio.

Um **domínio**, no contexto de levantamento de requisitos, é uma área de conhecimento ou uma atividade específica com terminologia própria. Ele está ligado ao que é relevante para inclusão no software.

Existem várias técnicas para auxiliar os profissionais a realizarem o estudo exploratório de domínio do ambiente para o levantamento de requisitos. Dentre essas técnicas, podemos citar a realização de entrevistas com participantes e contratantes envolvidos no software, entrevistas com especialistas no negócio, a observação do ambiente de usuário e a inspeção em sistemas já existentes.

Os requisitos de software podem ser divididos em três tipos:

- **Funcionais:** declaram as regras específicas do negócio. Por exemplo: clientes efetuando uma compra e um lojista cadastrando um produto no software;
- **Não funcionais:** compreendem características sistêmicas de qualidade para suporte aos requisitos funcionais. Essas características, que não estão diretamente relacionadas ao foco do software, envolvem segurança, confiabilidade, usabilidade e desempenho;
- **Normativos:** definem restrições e condições do ambiente relacionadas ao campo de atuação do software. Como exemplo podemos relatar as questões de plataformas tecnológicas, aspectos legais, políticas adotadas pelo contratante e compatibilidade do software com sistemas.

O resultado do levantamento de requisitos é o **documento de requisitos**, que contém a lista detalhada das necessidades do software a ser desenvolvido.

É interessante ressaltar que, além de guiar os desenvolvimentos, o documento de requisitos é um importante instrumento de registro e consenso entre os desenvolvedores e o cliente, podendo integrar o contrato de criação do software. Assim, é estabelecido o escopo do sistema, que consiste na identificação dos **requisitos** implementados no produto final.



DICA

O analista de requisitos deve buscar as necessidades e recursos do software que realmente vão agregar soluções para os usuários. Nesse contexto, é importante incluir na identificação dos requisitos quais características o sistema não deve exibir. Por exemplo, informações em excesso em uma tela podem confundir o usuário, tirando sua atenção dos dados que realmente são importantes naquele momento.

Mesmo sendo um guia e um termo de consenso entre desenvolvedores e o cliente, não é correto entender o documento de requisitos como algo estático. Além disso, a importância dessa fase é alta, uma vez que um erro identificado tardiamente implica na construção de um software que não corresponde às expectativas do usuário.



Análise, validação e verificação de modelos

A **análise** corresponde à etapa na qual os profissionais analistas fazem um estudo detalhado dos requisitos identificados, de modo a construir modelos para representar o problema e as soluções do software a ser construído.

O foco da análise é construir estratégias de solução sem se preocupar com o ambiente tecnológico utilizado. Em outras palavras, há uma abstração em relação aos detalhes tecnológicos, devendo interessar o que está ocorrendo no momento e o que o software deve fazer.

A **validação** se preocupa em assegurar que as necessidades do cliente estejam sendo atendidas pelo software. Nesse caso, temos uma frase que resume bem essa preocupação: o software correto está sendo construído? Desse modo, é importante a proximidade com os usuários, que devem ter entendimento do que está sendo feito, sem ambiguidades em relação à compreensão do que foi incluído no software.

Já as atividades de **verificação** analisam se os modelos estão em conformidade com os requisitos identificados. Para tanto, podemos usar a seguinte frase como resumo dessa análise: o software está sendo construído corretamente? Em outras palavras, dados os requisitos corretos, devem ser produzidos os modelos coerentes em relação a esses requisitos.



Projeto (desenho)

No projeto, deve ser determinado como o software irá atender aos requisitos com base nos recursos tecnológicos existentes. Nesta etapa, já temos a dependência com aspectos técnicos e restrições de plataforma e implementação. Desse modo, características tecnológicas são adicionadas nos modelos construídos em fases anteriores.

Nesta fase, tem-se duas atividades principais: o **projeto da arquitetura**, ou de alto nível, contendo o modo de distribuição de componente, e o **projeto detalhado**, ou de baixo nível, sendo modeladas as colaborações entre os objetos de cada módulo.

No projeto detalhado, são construídos o projeto de interação e interface com o usuário e o projeto de banco de dados, com a definição das tabelas caso se trate de estruturas relacionais de persistência.

É perceptível que o processo de desenvolvimento de software é incremental, sempre direcionando as ideias de um plano mais geral para desenvolvimentos em arquivos de código-fonte, que são próximos da plataforma de utilização e geram um produto tangível ao usuário.

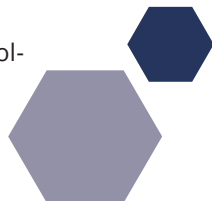


Implementação, testes e implantação

Na **implementação**, o software é escrito em alguma linguagem de programação por meio da **codificação**. Ocorre, então, uma tradução dos modelos anteriormente elaborados em código-fonte, que será traduzido ou executado para interação com o usuário do software.

Para essa tradução, podem ser utilizadas algumas transformações automáticas via software para geração de código-fonte, mas a maior parte do trabalho ainda é feita pelos **programadores**.

Nesse ponto, com a evolução das tecnologias de desenvolvimento, existem partes de código que podem ser reutilizadas, oferecendo maior agilidade na codificação. Tais partes podem ser componentes, bibliotecas de classes e frameworks, que são estruturas maiores que podem conter auxílios de programação e, até mesmo, sugestões de arquitetura.



CURIOSIDADE

Existe uma falsa impressão, especialmente para leigos, de que o desenvolvimento de software é extremamente dependente de excelentes programadores. Esses profissionais são importantes e fundamentais no processo, no entanto, para se ter um software de qualidade, é necessário uma equipe com diferentes profissionais capacitados (gestores de projeto, analistas e testadores).

Os **testes** envolvem tarefas para verificar o software que vão desde sua implementação, com observação de código-fonte e de suas demais estruturas internas, até o seu funcionamento, a partir da execução do software com dados selecionados pelo profissional de modo a abranger o maior número de funções possível.

Tais dados utilizados são chamados de **massa de teste** ou **casos de teste**. O produto resultante dessa fase é o relatório de testes, que deve incluir o comportamento do sistema que recebeu a massa de testes e os retornos. Assim, será verificado se a saída dada pelo software correspondeu à esperada.

Na **implantação**, ou *deployment*, é feito o empacotamento, a distribuição e a instalação do software para utilização do usuário. Em alguns casos, a implantação envolve a migração de dados ou de softwares completos para o novo sistema adotado.

Esse processo tem evoluído com o tempo em benefício do usuário. Em caso de sistemas web, nos quais há um servidor que disponibiliza o software, a implantação é feita pelos desenvolvedores. No caso de aplicativos para dispositivos móveis, ou mobile, as lojas virtuais fazem sua atualização dos smartphones e demais dispositivos do usuário remota e automaticamente.



Manutenção e evolução



A **manutenção** envolve as atividades de modificação de software depois que ele foi implantado. Mesmo em um software amplamente testado, é necessário realizar sua manutenção, pois é impossível descobrir todos os erros na etapa de teste e os requisitos de ambiente mudam com certa frequência.

As modificações para manutenção, sejam preventivas ou corretivas, podem ser simples, afetando pequenas porções de código-fonte, ou mais extensas e complexas, cuja finalidade é corrigir erros maiores como falhas de especificação e projeto.

Em cada modificação de manutenção do software, deve ser feita a atualização da modelagem, do documento de requisitos e de qualquer outro componente da especificação que seja afetado. Esse processo pode ser visto no Diagrama 2.

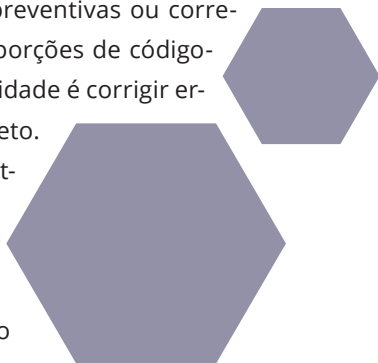
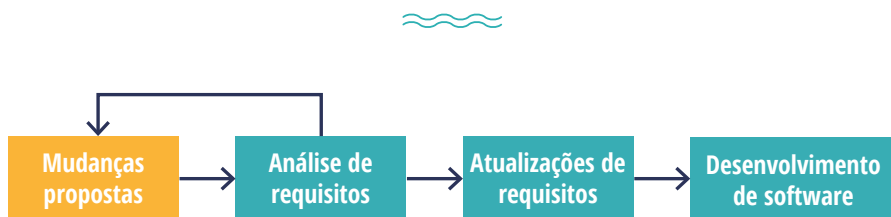


DIAGRAMA 2. IMPLEMENTAÇÃO DE MUDANÇAS EM UM SOFTWARE



Fonte: MMERVILLE, 2011, p. 167. (Adaptado).

Conforme pode ser visto no Diagrama 2, o processo de alteração começa com a descrição de uma proposta de modificação e continua com a análise dos requisitos envolvidos nessa alteração. A partir daí, no cenário ideal (que nem sempre é visto nas organizações), são feitas atualizações no documento de requisitos e, por fim, o software é desenvolvido.

Uma questão que não foi mostrada no Diagrama 2, devido ao fato dele estar focado apenas na parte da implementação de uma mudança, é a necessidade de novos testes (locais e globais) e a liberação da versão corrigida aos usuários. No caso de ser liberado apenas um programa ou script de modificação de um software maior, essa distribuição é chamada de patch.

Quando temos um software em utilização, as falhas são percebidas logo e podem causar efeitos ruins ao negócio. Nesse caso, ocorrem as chamadas **manutenções emergenciais** devido à urgência na resolução dos problemas.

No entanto, como dica, aconselha-se garantir que o processo de qualidade seja seguido nessas modificações. Isso envolve, além de competência técnica e tecnológica, o bom uso da inteligência emocional da equipe para que possíveis danos não sejam agravados.

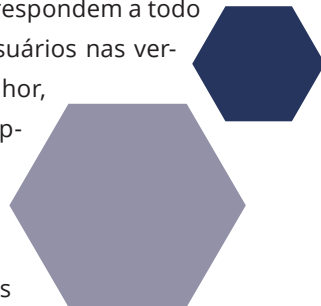
De acordo com Pressman, que escreveu o livro *Engenharia de Software: uma abordagem profissional*, publicado em 2011, o perigo de uma abordagem de emergência mal realizada é tornar os modelos incoerentes devido às alterações de correções serem feitas diretamente e apenas no código-fonte (sem a documentação necessária).

Outra questão a ser vista é que a correção pontual poder ter implicações na estrutura global

do software e em futuras manutenções. Uma opção que pode ser feita para amenizar o problema é refazer ou revisar totalmente os reparos de emergência e suas implicações.

As atividades de **evolução** de um software correspondem a todo o processo de verificação de necessidades dos usuários nas versões em utilização e à criação de um produto melhor, com incorporação de novas funcionalidades e adaptação a mudanças organizacionais ou de negócios.

Tais atividades também objetivam atualizar o software em relação a novos ambientes de utilização (dispositivos mais avançados, conexões melhores etc.), uma vez que essas mudanças tecnológicas ambientais mudam constantemente.



Participantes do processo



Dada a significativa complexidade natural do desenvolvimento de software, as atividades são realizadas por um grupo de pessoas, denominado **equipe** ou **time de trabalho**.

Tendo isso em mente, iremos definir algumas funções comuns nos processos de desenvolvimento de software:

- **Gerente de projeto:** coordena as atividades de construção do software, incluindo a parte de orçamentos e de acompanhamento do cronograma de trabalho estabelecido;

- **Analista:** define os requisitos do software a partir do conhecimento sobre o negócio e da comunicação com especialistas. Ele faz a ponte de comunicação entre os profissionais da computação e os profissionais do negócio;

- **Projetista:** integra a equipe de desenvolvimento, avaliando alternativas de solução e gerando a especificação de uma solução computacional detalhada;

- **Programador:** realiza a codificação das estruturas definidas pelo projetista e implementa o software. Em alguns vocabulários, esse cargo também é conhecido como desenvolvedor;

- **Administrador de banco de dados:** esse profissional realiza os procedimentos de interação com as bases de dados e sistemas gerenciadores de bancos

de dados, incluindo a definição de tabelas, otimização dos bancos e bases de dados e integridade das informações;

- **Especialista de domínio:** detém o conhecimento sobre a área ou o negócio em que o software realiza soluções. Geralmente, o cliente-usuário é um especialista do domínio, diferentemente do cliente-contratante, que tem por função encomendar o desenvolvimento do software (podendo ou não ter conhecimentos do domínio);

- **Avaliadores de qualidade:** analisam a adequação do processo de desenvolvimento e do produto de software de acordo com os padrões de qualidade estabelecidos no projeto.

Na prática, na maioria das equipes de desenvolvimento, especialmente em micro e pequenas empresas, existem analistas que programam e programadores realizando análises. Além disso, frequentemente os gerentes de projeto se comportam como avaliadores de qualidade no processo de desenvolvimento de software.



Análise e projeto orientado a objeto

No início do uso dos computadores, a comunidade de TI se preocupava com a definição de softwares enquanto uma lógica sequencial de comandos que se adequava bem às necessidades da época.

Com o uso maior dos computadores e sua proliferação no cotidiano das pessoas, os softwares foram também crescendo em termos de tamanho de código e complexidade, demandando a criação de processos melhores de desenvolvimento.

Tendo isso em mente, veremos, neste tópico, o histórico de evolução das linguagens de programação, que foi o pilar para a criação do paradigma de orientação a objetos. Além disso, são apresentadas as vantagens dessa evolução e como pode ser feito o processo de desenvolvimento.



Orientação a objetos

O paradigma da orientação a objetos surgiu inicialmente no final dos anos 60, com a linguagem **SIMULA**. Já nos anos 70, ela foi parte importante da linguagem Smalltalk, desenvolvida pela empresa Xerox.

Naquela época, esse paradigma era tratado como inovação, sendo mais utilizadas, na prática, as linguagens estruturadas com blocos de código sequenciais combinados com acesso a partes de dados. No entanto, a semente para um novo modo de pensar o software estava plantada.

Os pilares da orientação a objetos são:

- qualquer coisa é um objeto;
- objetos realizam tarefas por meio da chamada de serviços a outros objetos;
- cada objeto pertence a uma determinada classe;
- a classe é um repositório para comportamentos (localizados nos métodos);
- as classes são organizadas em hierarquias.

Ainda, a grande ideia da orientação a objetos é usar elementos de processamento encapsulados em softwares que se comunicam por meio de passagem de mensagens, em vez do compartilhamento direto de dados.

Para facilitar o entendimento, os conceitos básicos de classes e objetos podem ser vistos de modo gráfico conforme apresentado na Figura 2.

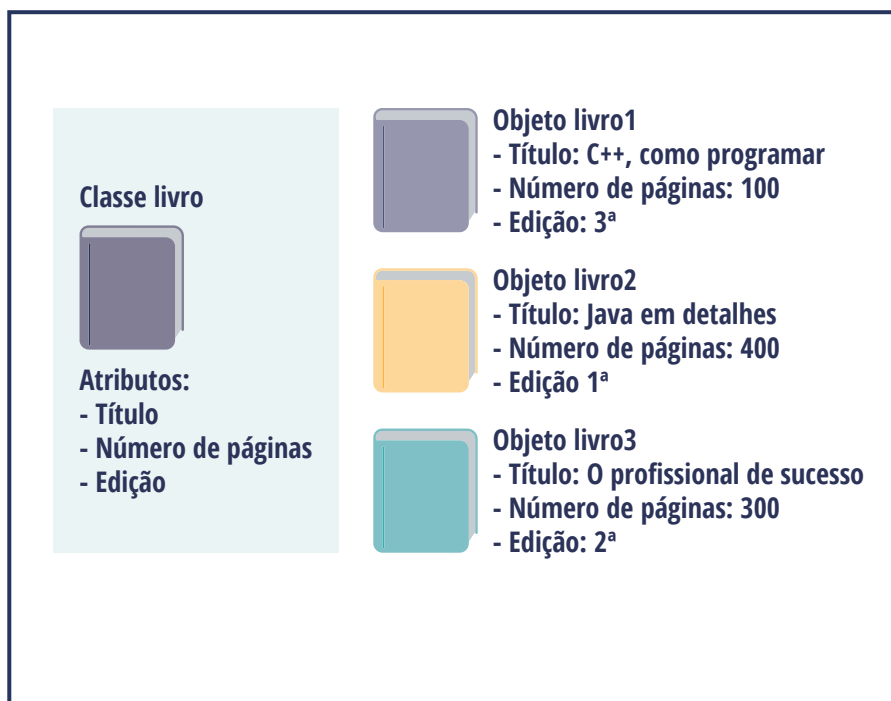


Figura 2. Esquema de uma classe.

Conforme pode ser visto na Figura 2, a classe organiza um formato de informações de livros com os seguintes atributos: título, número de páginas e edição. Tais atributos são preenchidos na instanciação dos objetos, que ocorre em tempo de execução do software. Nesse sentido, três objetos da classe livro foram instanciados no exemplo apresentado.

Em linguagens orientadas a objetos, os objetos são criados ou instanciados de modo dinâmico (em tempo de execução do software), sendo alocados na memória do dispositivo. Essa criação de um objeto realiza seu processo de construção, chamando o método construtor, que é o primeiro a ser chamado, e iniciando seu ciclo de vida.

Em Java e em diversas outras linguagens, o método construtor não tem valor de retorno e utiliza o nome da própria classe para ser chamado, podendo ter diferentes assinaturas para ser chamado de diferentes modos.

É importante mencionar que os conceitos de orientação a objetos são aplicáveis e devem integrar todo o desenvolvimento do sistema, não sendo apenas um modo de escrita de código-fonte. Em outras palavras, envolve o modo de pensar no software, com o comportamento das estruturas de dados representadas principalmente nos objetos.



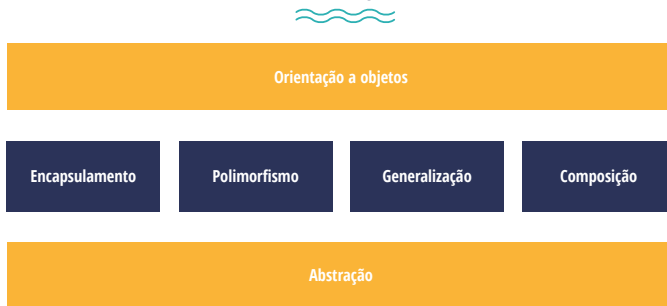
Princípios da orientação a objetos



O primeiro princípio e o mais básico da orientação a objetos é o da **abstração**, que é análogo ao que ocorre no processo mental de gerenciamento da complexidade de um objeto.

No Diagrama 3, é possível ver os princípios da orientação a objetos, com destaque para a abstração, que é a base para os demais.

DIAGRAMA 3. PRINCÍPIOS DA ORIENTAÇÃO A OBJETOS COMO APLICAÇÕES BÁSICAS DA ABSTRAÇÃO



Fonte: MMERVILLE, 2011, p. 167. (Adaptado).

O conceito de abstração permite criar uma visualização específica e simplificada, apresentando, de modo separado, aspectos importantes de finalidades específicas. Com isso, limita-se o universo real, de modo que possamos entendê-lo melhor.

O princípio do **encapsulamento** ou o ocultamento de informações, por sua vez, pode ser visto como uma analogia a uma cápsula que agrupa e protege algo. Nesse sentido, encapsular código representa agrupar e proteger dados que podem ser acessados externamente. Esse princípio evita que pedaços de um código-fonte de um software se tornem tão independentes a ponto de fazerem com que uma mudança tenha efeitos em cascata.

O encapsulamento é usado, então, para ocultar e proteger a programação de métodos e valores de atributos. Métodos publicamente acessíveis são, geralmente, fornecidos na classe (chamados *getters* para obtenção de dados de atributos e *setters* para armazenamento de dados de atributos) para acessar os valores das instâncias de objetos.

No entanto, é importante ressaltar que não é recomendado a tradução de termos como *get* e *set* para a língua portuguesa, pois a utilização dos nomes em inglês permite que recursos de linguagens como Java possam realizar operações automáticas, sendo esse conceito chamado de **introspecção**.

Existe um apoio ferramental para geração das classes, de seus atributos e até mesmo de métodos do tipo *getters* e *setters* nos ambientes integrados de desenvolvimento, mais conhecidos pela sigla em inglês IDEs, de *Integrated Development Environment*. São exemplos bem conhecidos de softwares de apoio do

tipo IDEs, que fazem geração de código no paradigma de orientação a objetos em Java, o Eclipse e o Netbeans.

Na Figura 3, é possível ver a tela que oferece o recurso da geração de código padrão para o acesso de atributos via métodos *getters* e *setters* no Eclipse.

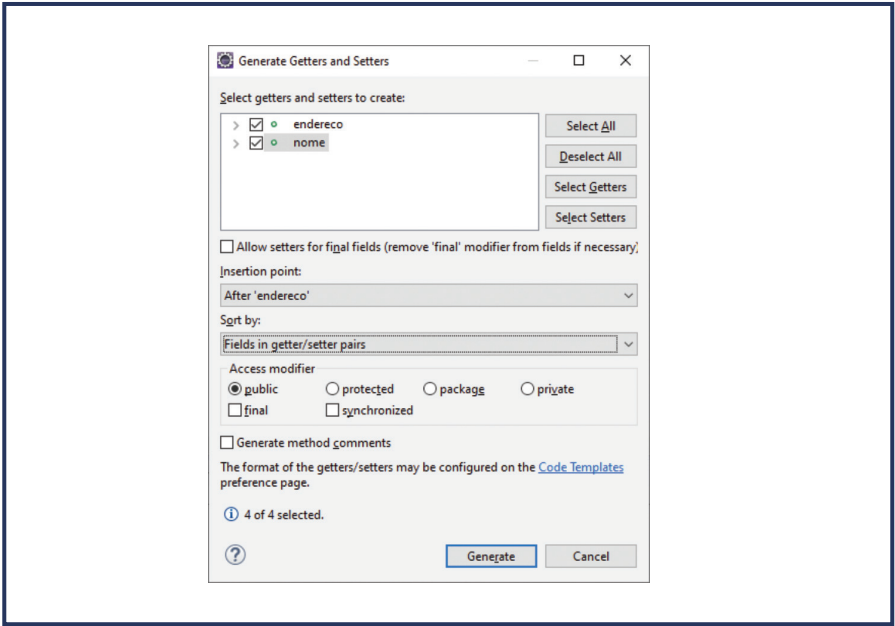


Figura 3. Captura de tela com geração automática de métodos *getters* e *setters* no Eclipse.

Como pode ser visto na 3, o usuário necessita marcar os atributos nos quais deseja gerar os códigos automáticos. As demais opções são referentes ao estilo do código a ser gerado, podendo ser alterado o lugar de inserção do texto e os modificadores (o padrão é *public*, ou público em português, para que eles possam ser acessados por métodos de outras classes).

O princípio da **generalização**, ou de herança, por sua vez, rege o relacionamento entre elementos gerais (chamados de superclasses ou classes-mães) e elementos mais específicos desses itens (chamados subclasses ou classes-filhas). Com a generalização, objetos da classe-filha podem ser utilizados em qualquer local no qual a classe-mãe ocorra, mas não vice-versa. Desse modo, a filha herda as propriedades da mãe, principalmente seus atributos e operações.

Em um uso comum da generalização, as classes-filhas têm atributos e operações além daqueles encontrados nas respectivas mães. Ou seja, são desenvolvidos novos métodos, adicionados atributos ou atualizados os métodos existentes.

O método de uma filha que tenha a mesma assinatura de um método da mãe prevalece em relação à operação da mãe, princípio esse conhecido como **polimorfismo**.

Com o princípio do polimorfismo, é possível utilizar a mesma invocação de método que irá originar diferentes resultados, de acordo com o objeto que responder por aquele método. Com isso, a adição de novas classes a um software pode ser realizada com o mínimo de modificações de código.

Por fim, o princípio da **composição** se refere a quando um objeto contém outros. Por exemplo, uma classe de um carro herda propriedades da classe veículo e contém quatro instâncias de objetos da classe roda. Diferentemente da herança, que significa, de modo geral, uma relação “é um”, a composição representa a relação “tem um”.



Benefícios da orientação a objetos



Os softwares desenvolvidos no paradigma da orientação a objetos têm uma melhor relação com o mundo real, visto que o raciocínio das pessoas está diretamente ligado ao conceito de objetos.

Para os autores Deitel e Deitel, que escreveram o livro *Java: como programar*, publicado em 2005, um benefício que motiva a adoção desse paradigma é a possibilidade concreta de desenvolvimento mais rápido de aplicações, principalmente pela capacidade de reuso e melhor organização de software.

Com a proximidade aos conceitos do mundo real, é possível realizar uma modelagem mais natural, facilitando a criação e o entendimento dos diagramas. Além disso, esse paradigma oferece outros benefícios:

- Favorece a continuidade de projeto, pois a mesma notação é utilizada desde a análise até o projeto e a implementação;
- Incentiva os desenvolvedores a trabalharem com o domínio da aplicação durante a maior parte do ciclo de vida;
- Valida, via modelos, etapas iniciais de modo facilitado, podendo reduzir,

assim, a quantidade de erros com a consequente diminuição do tempo nas etapas de codificação e teste, visto que os problemas são detectados mais cedo e corrigidos antes da implementação;

- Melhora a comunicação entre desenvolvedores, usuários e stakeholders, que utilizam os modelos como facilitadores de entendimento do que está sendo feito;

- Reduz o tempo de manutenção, pois as revisões são mais fáceis e mais rápidas de serem realizadas em um software mais bem organizado e com documentação arquitetural concisa;

- Favorece a reutilização, pois prevê a construção de estruturas utilizadas em vários locais pelo acionamento dos métodos em objetos;

- Facilita a divisão do trabalho e a interação da equipe, visto que as classes têm interfaces definidas nos modelos. Assim, os desenvolvedores podem criar novas classes, que fazem interação com outras sem conhecer a implementação dessas últimas;

- Previne o espalhamento indevido de código-fonte pela separação de escopos e pelo ocultamento e encapsulamento de informação dentro dos objetos. Assim, alterações realizadas em uma classe não irão afetar outras de modo imprevisível.



Análise orientada a objetos



Na década de 1980, os computadores começaram a ficar mais poderosos e acessíveis às organizações, e, consequentemente, os softwares se tornaram maiores. As técnicas tradicionais de desenvolvimento de software utilizadas, conhecidas como análise e projeto estruturado, já não se adequavam tão bem aos sistemas de informação médios, uma vez que possuíam cerca de 50.000 a 500.000 linhas de código.

Além disso, nesse tipo de desenvolvimento estruturado, ficava a critério dos desenvolvedores a divisão do código em partes menores, não havendo sistematização para uma organização eficiente de código.

Nesse contexto, como solução para a inadequação das técnicas tradicionais, surgiu, no início da década de 1990, um novo paradigma de modelagem: a análise orientada a objetos, aproveitando dos conceitos básicos de orientação a objetos e requerendo um novo modo de trabalhar na modelagem, com foco

em abstrair e entender os problemas. Colaboradores de expressão na criação desse novo paradigma são Sally Shlaer, Stephen Mellor, Rebecca Wirfs-Brock, James Rumbaugh, Grady Booch e Ivar Jacobson.

O objetivo básico da análise orientada a objetos é identificar classes, a partir das quais objetos serão representados como instâncias. Ela envolve as seguintes tarefas:

- **Identificação de atores:** definição de quem atua nos sistemas, disparando eventos para cenários ou casos de utilização comuns;

- **Identificação de classes de objetos:** busca pelas entidades que serão as classes. São candidatos a classes:

- Entidades externas: outros sistemas e dispositivos;
- Estruturas do domínio do problema: veículo, relatório e mostradores;
- Papéis ou funções: cliente, gerente e vendedor;
- Unidades organizacionais: grupo e equipe;
- Lugares: recepção, garagem e sala.

- **Especificação de atributos:** lista de todos os detalhes da classe que especificarão os objetos pelo preenchimento dos atributos definidos. Exemplo: a classe pessoa tem como atributos nome, e-mail e telefone;

- **Definição de métodos:** detalhe dos comportamentos dos objetos e como será o acesso dos seus atributos;

- **Comunicações entre objetos:** registro do modo como os objetos trocam informações via comunicação de métodos e utilização de conteúdo dos atributos definidos.

No momento da definição de requisitos, **nomes** (substantivos) são potenciais candidatos a classes e **verbos** são potenciais candidatos a métodos. No entanto, a experiência do analista, a modelagem e o entendimento do contexto, incluindo a verificação das necessidades do usuário, são fundamentais para classificar possíveis objetos e métodos.



Projeto orientado a objetos

No projeto orientado a objetos, a partir da análise feita, com a adequada identificação dos requisitos, são feitos detalhamentos técnicos das classes identificadas. Esse projeto também inclui a definição de uma arquitetura de

alto nível, chamada de **arquitetura do sistema**, sendo especificadas políticas padronizadas para funcionamento de todo o software.

Em relação às classes, nesse momento, são adicionados nos modelos de análise vários detalhes técnicos necessários para implementação das classes em código-fonte.

O foco trabalhado, então, é a definição das estruturas de dados e algoritmos necessários para implementação de cada classe. Por exemplo, usaremos uma estrutura de dados do tipo fila para implementação de uma classe que conterá os produtos de um carrinho de compras em uma loja virtual.

Note que, nessa etapa de projeto, são incluídas as classes que não fazem parte da análise, mas que são necessárias na implementação. Esse é o caso da classe fila, que citamos no exemplo anterior. Ou seja, o projeto passa a ter mais detalhes técnicos das soluções a serem incorporados no software.

Nessa etapa, os projetistas fazem também o desenvolvimento de estruturas para armazenamento dos dados da aplicação, de acordo com a arquitetura idealizada. Vale ressaltar que a definição das classes e seus atributos auxilia bastante na criação das tabelas, no caso de ser feita a opção pelo uso de sistema gerenciadores de bancos de dados relacionais.

No caso de ser feita a opção pelo uso de bancos de dados orientados a objetos, a persistência dos objetos é feita de modo facilitado, já que existe uma maior aderência conceitual e tecnológica desses bancos com o paradigma orientado a objetos.

De um modo geral, esse é o projeto para desenvolvimento de software considerando a orientação a objetos. No entanto, diversas especializações foram criadas para nortear os desenvolvedores. Essas especializações são extensas, devendo, assim, ser alvo de outras disciplinas.

Como exemplo, podemos citar o Processo Unificado da Rational, ou RUP, sigla em inglês para *Rational Unified Process*, que é bem rigoroso e burocrático. Além disso, há também o desenvolvimento ágil, que é dividido em curtas iterações, ou sprints, sendo menos burocrático do que o RUP.



Descrição de caso de uso

Os casos de uso, também conhecidos pelo termo em inglês *use-cases*, correspondem a cenários de utilização de um software e são fruto do trabalho de obtenção de requisitos. Eles foram criados em 1993 e, desde então, são utilizados amplamente pela comunidade de desenvolvimento de software, com boa aderência ao projeto orientado a objetos.

Em seu modo mais simples, os **casos de uso** descrevem uma interação e identificam os atores ou agentes envolvidos nela. Além disso, eles identificam como o software se comporta em diferentes situações que podem ocorrer durante sua operação. Descreve, assim, comportamentos do sistema, seu ambiente, os atores envolvidos e a relação entre os dois.

Um **caso de uso** é uma sequência de ações executadas de modo ordenado no software, revelando um padrão de utilização para realização de alguma tarefa específica. O caso de uso é acionado por um ator e produz um resultado que contribui para os objetivos do sistema. Suas características básicas são:

- Um caso de uso modela o diálogo entre atores e o sistema, ou mesmo entre casos de uso;
- Um caso de uso é iniciado por um ator para invocar uma funcionalidade do sistema, ou pode ser acionado por um outro caso de uso;
- Um caso de uso é um fluxo de eventos completo e consistente;
- O conjunto de todos os casos de uso representa todas as situações possíveis de utilização do sistema.



ASSISTA

No vídeo *Entenda o diagrama de casos de uso | #7*, é possível entender como funciona e como se utiliza o caso de uso.





Essa Unidade se iniciou com a apresentação da importância da modelagem, com sua história e motivações para utilização em benefício do desenvolvimento de software de qualidade, com maior previsibilidade e comunicação efetiva com o cliente e usuários na fase de construção das soluções.

A partir da base de conceitos de modelagem, foi feita, então, a apresentação teórica e a demonstração prática das diferentes atividades da fase de análise, para investigação e verificação do que será feito, e da fase de projeto, com detalhamentos das soluções em termos de componentes de software e suas arquiteturas.

Vista a importância da modelagem e da análise, foi apresentado o projeto orientado a objetos, com suas características e vantagens de utilização, formando a base do aluno para realizar a análise e projeto de software nesse tipo de projeto, que aproxima os conceitos da computação à realidade das pessoas.

Como parte final dessa Unidade, considerando o foco em modelagem da unidade, foram abordados também os detalhes dos conceitos envolvidos em casos de uso, enfatizando sua importância, o que são e quando utilizá-los

Referências bibliográficas



BEZERRA, E. **Princípios de análise e projeto de sistemas com UML**. 2. ed. Rio de Janeiro: Elsevier, 2007.

BLAHA, M.; RUMBAUGH, J. **Modelagem e projetos baseados em objetos**. 2. ed. Rio de Janeiro: Elsevier, 2006.

BOOCH, G; RUMBAUGH, J; JACOBSON, I. **UML: guia do usuário**. 2. ed. Rio de Janeiro: Editora Campus, 2005.

DEBONI, J. E. Z. **Modelagem orientada a objetos com a UML**. São Paulo: Futura, 2003.

DEITEL, H.; DEITEL P. **Java: como programar**. 6. ed. São Paulo: Pearson Prentice Hall, 2005.

ELLIOTT, E. **Composing Software**. 1. ed. British Columbia: Leanpub, 2018.

ENTENDA O DIAGRAMA DE CASOS DE USO #7. Postado por Estudo na web. (13 min. 58 s.). color. port. son. Disponível em: <<https://www.youtube.com/watch?v=xrcgbMQdM8Y>>. Acesso em: 13 nov. 2019.

GUEDES, G. T. A. **UML 2: uma abordagem prática**. 2. ed. São Paulo: Novatec Editora, 2011.

JACOBSON, I.; CHRISTERSON, M.; JONSSON P. e ÖVERGAARD, G. **Object-oriented software engineering**. 4. ed. Wokingham: Addison Wesley, 1993.

KLEPPE, A; WARMER, J.; BAST, W. **MDA Explained - The Model Driven Architecture: Practice and Promise**. 1. ed. Hoboken: Addison-Wesley, 2003.

LUCRÉDIO, D. **Uma abordagem orientada a modelos para reutilização de software**. 2009. 277 f. Tese de Doutorado – Instituto de Ciências Matemáticas e Computação, Universidade de São Paulo, São Carlos, 2009. Disponível em: <<https://teses.usp.br/teses/disponiveis/55/55134/tde-02092009-140533/pt-br.php>>. Acesso em: 13 nov. 2019.

MEC. **Catálogo Nacional de Cursos Superiores de Tecnologia**. 3. ed. Brasília: Ministério da Educação, 2016. Disponível em: <http://portal.mec.gov.br/index.php?option=com_docman&view=download&alias=98211-cncst-2016-a&category_slug=outubro-2018-pdf-1&Itemid=30192>. Acesso em: 13 nov. 2019.

PRESSMAN, R. S. **Engenharia de software: uma abordagem profissional**. 7. ed. Porto Alegre: AMGH, 2011.

QUATRANI, T. **Modelagem visual com Rational Rose 2000 e UML**. Rio de Janeiro: Editora Ciência Moderna Ltda., 2001.

SCHACH, S. R. **An introduction to object-oriented systems analysis and design with UML and the unified process**. Boston: McGraw-Hill Irwin, 2004.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

UNIDADE

2

INTRODUÇÃO À UML E
FERRAMENTAS CASE



ser
educacional

Objetivos da unidade

- Apresentar a Introdução à Unified Modelling Language (UML), com sua definição, importância da análise, modelagem e projeto na Engenharia de Software;
- Explicar como funciona o padrão de notação dentro de um processo de desenvolvimento de software, no qual é fundamental ter boa comunicação entre os membros da equipe;
- Evidenciar fatos que motivaram a criação da UML, incluindo todo o panorama de desenvolvimento de software das décadas de 1980 e 1990;
- Detalhar a evolução da UML após sua criação, com descrição de eventos importantes e detalhes de características técnicas que foram incorporadas em suas versões;
- Ampliar a visão dos diferentes modelos presentes na UML. Para tanto, é apresentada a taxonomia dos diagramas contida na especificação oficial da UML, verificando questões estruturais e comportamentais envolvidas;
- Apresentar os conceitos envolvidos nas Ferramentas CASE, incluindo sua definição, importância no desenvolvimento de software, detalhes de como fazer uma boa escolha de ferramenta e análise de algumas importantes ferramentas disponíveis no mercado, sejam essas pagas ou gratuitas.

Tópicos de estudo

- **Introdução à UML**
 - Objetivos da UML
 - Componentes da especificação da UML
 - Mecanismos de uso geral
- **Evolução da UML**
- **Visão dos modelos**
- **Ferramentas CASE (*Computer-Aided Software Engineering*)**
 - Classificação das ferramentas CASE
 - Verificação de ferramentas CASE existentes
- Enterprise Architect
- ArgoUML
- Visual Paradigms
- BOUML
- Umbrello UML Modeller
- Outras



Introdução à UML

Unified Modelling Language (UML) é uma linguagem para especificar, visualizar e documentar modelos de software no paradigma orientado a objetos, utilizando uma notação padrão.

Vale ressaltar que a UML não é um método de desenvolvimento, o que significa que ela não lhe diz o que fazer primeiro, o que fazer depois ou como projetar o seu software, mas lhe ajuda a criar e a visualizar suas estruturas e a tecer uma comunicação efetiva entre os membros da equipe. Desse modo, podemos dizer que a UML se associa a um processo, constituindo-se de um instrumento de trabalho para formar um método.

O padrão da UML é gerenciado pelo **Object Management Group** (OMG), consórcio internacional de empresas que define os padrões da orientação a objetos. Isso torna a linguagem segura, com especificação de qualidade validada e internacionalmente compreensível, sendo esses os principais fatores que a fazem ser amplamente utilizada na indústria para descrever graficamente os detalhes do software.

A significativa aceitação da UML pela comunidade de desenvolvedores e pela indústria de software é uma prova da sua força, validando sua importância.

A UML é composta por elementos de modelos que representam as diferentes partes de um software. Esses elementos são utilizados para criar diagramas que representam uma determinada parte ou um ponto de vista do software.

São definidos vários modelos de diagrama na UML. Não se utiliza, obrigatoriamente, todos os modelos em todos os projetos. Deve-se utilizar o que melhor representar o contexto do negócio. Ou seja, minimizar o número de modelos produzidos reduz os custos e o tempo gasto no projeto.

Nessa mesma linha de pensamento, o autor Roger Pressman relata, no livro *Engenharia de software: uma abordagem profissional*, de 2001, que é possível suprimir partes não relevantes ao aspecto que está sendo modelado para evitar congestionar o modelo com dados sem relevância. Portanto, a omissão de uma característica específica não significa necessariamente que ela esteja ausente.

Faz parte da notação da UML uma vasta gama de símbolos gráficos bem definidos para a representação de artefatos de software. Para cada símbolo utilizado, há uma sintaxe e uma semântica bem definidas.

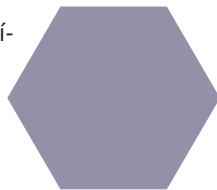
Com a utilização da UML, é possível o intercâmbio de dados de modelos entre uma diversidade de ferramentas, além da criação de diferentes repositórios para armazenamento de modelos que se tornam soluções reutilizáveis com boa interoperabilidade em uma ou mais organizações.



Objetivos da UML

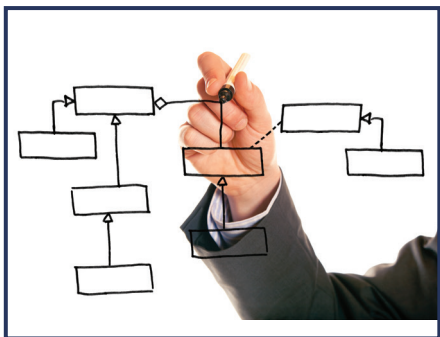
Os objetivos principais da UML, de forma geral, são:

- Prover aos membros da equipe de desenvolvimento de software uma linguagem de modelagem visual pronta para a utilização no desenvolvimento e comunicação de modelos ricos em significados;
- Oferecer mecanismos de extensibilidade e especialização para ampliar os conceitos principais;
- Suportar especificações de projeto independentes das linguagens de programação e do processo de desenvolvimento;
- Fornecer uma base formal de entendimento de uma linguagem de modelagem;
- Encorajar o crescimento do mercado de ferramentas de software orientadas a objeto;
- Avançar nos processos de desenvolvimento de software pela possibilidade de uso de ferramentas de modelagem visual de objetos com interoperabilidade;
- Especificar elementos de notação legíveis por humanos para representar a modelagem de conceitos, bem como regras para combiná-los em uma variedade de tipos de diagrama diferentes, correspondentes a distintos aspectos dos sistemas modelados;
- Suportar conceitos de desenvolvimento de alto nível, como componentes, colaboração, frameworks e padrões;
- Integrar boas práticas de projeto em diferentes visões complementares.





Componentes da especificação UML



A UML é formada por especificações que definem padrões, ou seja, para que ela seja utilizada, na prática, deve haver a junção de várias documentações incluídas em seu projeto pelo OMG. Cada uma dessas documentações provê facilidades e estruturações da linguagem para suportar seus recursos e ser inteligível para os desenvolvedores de software,

de ferramentas CASE e demais usuários do padrão.

Esses componentes da especificação da UML, a partir de sua versão 2.0, correspondem a quatro documentos:

- **Superestrutura:** define as construções da UML a nível de usuário, utilizadas para modelar a estrutura e o comportamento de um sistema com os seguintes elementos:

- **Itens:** componentes dos diagramas;

- **Relacionamentos:** ligam itens para formar relações de associação e herança;

- **Diagramas:** desenhos gráficos que formam modelos.

- **Infraestrutura:** define o metamodelo da UML com um núcleo de metalinguagem, podendo ser reutilizado para definir outras arquiteturas de metamodelos, além de definir mecanismos de personalização e adaptação da UML. Assim, podem ser criados dialetos para plataformas e domínios específicos;

- **OCL (Object Constraint Language) ou linguagem de restrição de objeto:** linguagem formada com o objetivo de escrever regras e fórmulas para definir comportamentos e restrições em elementos dos modelos, incluindo semânticas próprias. É possível, ainda, serem definidas pré e pós-condições de operações;

- **UML (Diagram Interchange) ou intercâmbio de diagramas UML:** é a soma das informações gráficas com os arquivos XMI. XMI (*XML Metadata Interchange*) é um padrão do OMG baseada em XML, com o objetivo de trocar informações. Seu uso mais comum é na persistência (gravação) e troca de metadados entre ferramentas de modelagem.



Mecanismos de uso geral

Existem alguns mecanismos de operação de elementos que valem em toda a UML, sendo a base para o entendimento das diferentes visões e características técnicas da linguagem. Eles podem ser sintetizados da seguinte forma:

- **Estereótipos:** ampliam o vocabulário da UML, permitindo a criação de novos tipos de blocos de construção derivados dos já existentes, mas específicos a determinados problemas. Eles personalizam itens por meio de construções específicas para um domínio, plataforma ou método de desenvolvimento. Os estereótipos podem ser predefinidos, como `<<interface>>`, `<<document>>`, `<<control>>`, `<<entity>>` ou personalizados, como `<<router>>`, `<<database>>` etc;

- **Notas:** símbolo gráfico considerado adorno que contém comentários textuais anexados a um elemento ou a uma coleção de elementos. As notas são utilizadas para anexar informações a um modelo, como requisitos, revisões e explicações;

- **Pacotes:** recurso de separação que organiza elementos de modelagem em conjuntos maiores que possam ser manipulados como grupos. Realiza, portanto, o agrupamento de itens semanticamente relacionados;

- **Tagged values:** conjunto de valores pré-definidos para um elemento. Um tagged value é um par de valores que pode ser usado para adicionar propriedades a elementos de um modelo. Na UML 2, os tagged values podem ser aplicados apenas a elementos que usam um estereótipo com uma definição da marcação ou tag;

- **Restrições:** especificação de regras que delimitam um conjunto de valores ou situações possíveis para um determinado elemento. É um recurso utilizado para definir condições que devem ser mantidas como verdadeiras para que o modelo seja bem formado.



Evolução da UML

Na década de 1980, mesmo com a orientação a objetos já conhecida, existiam várias metodologias, técnicas e notações distintas para representar os softwares orientados a objetos. Naquela época, no início de um projeto de desenvolvimento de software, era preciso selecionar um método e, geralmente, treinar a equipe na notação escolhida. A ausência de um padrão dificultava a difusão e adoção da tecnologia de orientação a objetos.

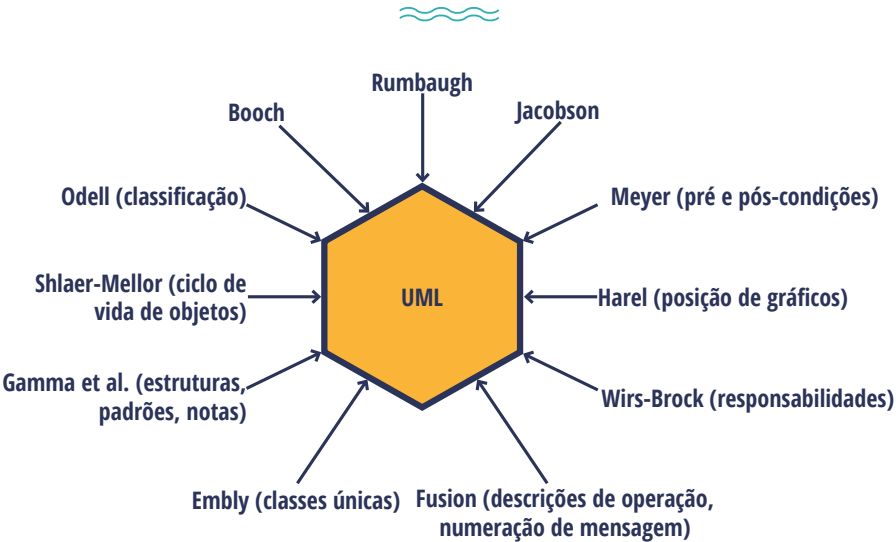
O autor Bezerra, no livro *Princípios de análise e projeto de sistemas com UML*, publicado em 2007, complementa que várias propostas para modelagem orientada a objetos se proliferaram do início dos anos 80 até a primeira metade da década de 90, havendo a problemática comum de duas técnicas possuírem notações diferentes para modelar as mesmas perspectivas de um software. Ademais, cada técnica tinha seus pontos fortes e fracos em relação à notação proposta.

A partir dessa problemática, ficava evidente a necessidade de definição de uma padronização, o que se realizou a com a criação, em 1996, da Unified Modelling Language (UML) a partir da junção de esforços de vários pesquisadores contribuintes.

Dentre esses contribuintes se destacam três autores (conhecidos como três amigos), com união das melhores características do trabalho que estavam realizando: Grady Booch, com seu Booch Method; James Rumbaugh, com seu método OOSE (*Object-Oriented Software Engineering*) e Ivar Jacobson, com sua técnica OMT (*Object Modeling Technique*) e seu método Objectory.

No Diagrama 1, é possível visualizar todos os autores da área de metodologia orientada a objetos que contribuíram para o início da UML.

DIAGRAMA 1. AUTORES QUE CONTRIBUÍRAM PARA A CRIAÇÃO DA UML



Fonte: QUATRANI, 2000, p. 29. (Adaptado).

A partir de seu lançamento, empresas atuantes na área de desenvolvimento de software (como Microsoft, Oracle, HP, Rational e IntelliCorp) reconheceram a qualidade do trabalho inicial e passaram a contribuir para o projeto da UML. Como resultado, a UML foi adotada, em 1997, pelo OMG como uma linguagem padrão de modelagem.

No Quadro 1, é apresentado o histórico de versões formais da UML em ordem decrescente da especificação oficial presente no site oficial da linguagem.

TABELA 1. HISTÓRICO DE VERSÕES FORMAIS DA UML



Versão	Data de adoção
2.5.1	Dezembro de 2017
2.4.1	Julho de 2011
2.3	Maio de 2010
2.2	Janeiro de 2009
2.1.2	Outubro de 2007
2.0	Julho de 2005
1.5	Março de 2003
1.4	Setembro de 2001
1.3	Fevereiro de 2000
1.2	Julho de 1999
1.1	Dezembro de 1997

Fonte: OMG, 2019.

É possível notar, a partir dos dados presentes no Quadro 1, que a UML teve uma evolução contínua, com periodicidade de lançamentos de novas versões formais a cada um ou dois anos até sua versão 2.4.1, datada de julho de 2011. Depois dessa data, houve um período maior sem lançamentos de quase seis anos até o anúncio da nova versão, a 2.5.1.

Um fato marcante da evolução da UML ocorreu em 2004, ano em que a versão 1.4.2 foi transformada no padrão ISO/IEC 19501, oferecendo, com isso, maior reconhecimento e confiabilidade para sua utilização em indústrias e outras grandes corporações.

O lançamento da UML 2.0, feito em julho de 2005, merece um grande destaque na evolução da linguagem, sendo a maior revisão da história da UML, que se consagrou como uma das mais expressivas linguagens para modelagem de sistemas orientados a objetos. Nessa revisão, houve mudanças significativas na própria arquitetura da UML, refinamentos, aumento de qualidade dos elementos visuais e introdução de quatro novos diagramas (passando de nove para 13): de comunicação, de estrutura composta, de visão geral de interação e de tempo.

A motivação dos desenvolvedores da linguagem UML a lançar a versão 2.0 foi reestruturá-la e refiná-la de maneira a torná-la mais fácil de se aplicar, implementar e adaptar, melhorando sua precisão e capacidade de reutilização.

Em relação à **UML 1.0**, a especificação na versão 2.5.1 foi aprimorada com definições significativamente mais precisas de suas regras e semânticas, abstratas e de sintaxe, contando com uma estrutura de linguagem mais modular e com uma capacidade bastante aprimorada para modelagem de sistemas de larga escala.



ASSISTA

Para saber mais sobre a utilização da UML, assista ao vídeo do canal Estude na Web, que explana sucintamente a história e a importância da UML.



Alguns livros e artigos descrevem a UML como uma linguagem que contém 13 diagramas. Isso ocorre porque esses materiais estão desatualizados, não contando com o 14º diagrama, que foi incorporado em 2009 na versão 2.2 da UML: o diagrama de perfil, que é utilizado para especificar plataformas e domínios.



Visão dos modelos

Uma das motivações para a criação da UML foi a necessidade de uma linguagem completa o bastante para incluir as várias e complexas necessidades da modelagem de software orientado a objetos.

A solução para essa questão envolveu a adoção de uma arquitetura de múltiplas visões na UML, separando os modelos do software em diferentes visões que tratam de aspectos complementares.

Assim, cada visão dos modelos na UML utiliza notações e diagramas próprios. É como se o software fosse modelado em camadas, sendo que certos diagramas mostram o software de modo mais geral, com uma visão externa, e outros oferecem a visão de uma camada mais específica, contendo detalhes de processos, por exemplo.

Para melhor compreensão, podemos fazer uma analogia das diferentes visões dos modelos na UML com os diversos tipos de projetos de uma construção civil. Estes são enviados para uma empresa construtora, envolvendo plantas de instalações elétricas, com foco na fiação e tomadas, e plantas baixas, com foco na distribuição dos cômodos.

De modo resumido, as visões da UML, em sua versão 2.5.1, abordam diferentes visões em 14 tipos de diagramas, divididos em duas grandes categorias: de estrutura (sete diagramas) e de comportamento (sete diagramas).

Em um diagrama de estrutura, é mostrada a composição estática dos objetos e seus relacionamentos em um sistema. Exemplos de relacionamentos importantes que podem ser documentados nessa estrutura são os de generalização (herança) e composição. Em outras palavras, esses elementos são descritos em uma especificação independentemente do tempo, tendo uma descrição fixa e contínua em todo o projeto.

Ademais, os elementos em um diagrama de estrutura representam os conceitos significativos de um aplicativo e podem incluir conceitos abstratos, do mundo real e de implementação. Por exemplo, um diagrama de estrutura para um software de compra de passagens aéreas pode incluir elementos que representam algoritmos de atribuição de assentos e aquisição de passagens.

Os diagramas de estrutura não mostram detalhes do comportamento dinâmico, ilustrados por diagramas comportamentais. No entanto, eles podem mostrar relacionamentos, com os comportamentos dos elementos exibidos nos diagramas de estrutura.

Nas visões de estrutura, normalmente é especificada a arquitetura de implementação, que descreve os componentes que formam o software e a arquitetura de hardware, podendo conter, por exemplo: Diagramas de Componentes, Diagramas de Implantação e Diagramas de Perfil.

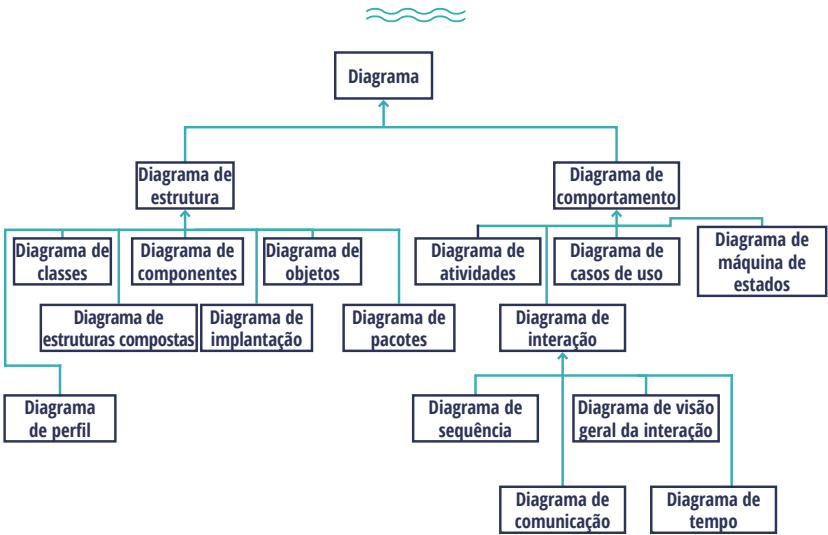
Também é possível especificar a estrutura de suporte, que detalha as partes que formam o sistema e suas relações internas. São exemplo desse tipo de estrutura: Diagramas de classes e Diagramas de pacotes.

Já os diagramas de comportamento, por sua vez, mostram modelos que contém o comportamento dinâmico dos objetos em um software, incluindo métodos, colaborações, atividades em fluxo de dados e histórico de estados. O comportamento dinâmico de um software pode ser descrito como uma série de alterações no seu ambiente ao longo do tempo.

Desse modo, modelos de comportamento dinâmicos descrevem a estrutura dinâmica do sistema com detalhes das interações entre os objetos. Interações, por sua vez, incluem a sequência de solicitações de serviço feitas pelos objetos e as mudanças de estado que são disparadas pelas interações de objetos.

Os diagramas de comportamento podem ser classificados em vários outros tipos. A taxonomia completa dos diagramas da UML pode ser vista no Diagrama 2.

DIAGRAMA 2. TAXONOMIA DE DIAGRAMAS DA UML



Fonte: OMG, 2019, p. 685. (Adaptado).

Como pode ser visto no Diagrama 2, os 14 diagramas da UML são divididos em categorias hierárquicas. Além disso, é necessário ressaltar que a notação utilizada nessa figura segue a UML, com uso de triângulos apontados para cima e com o elemento no topo estendido (havendo uma herança de conceitos). Por exemplo, o diagrama de interação estende suas propriedades para quatro tipos de diagramas, a saber: de sequência, de comunicação, de visão geral de interação e de tempo.



Ferramentas CASE (*Computer-Aided Software Engineering*)



As ferramentas CASE, da sigla em inglês Computer-Aided Software Engineering, são softwares que auxiliam a equipe de Engenharia de Software na execução de uma ou mais atividades existentes no processo de criação e evolução de software.

É possível criar diagramas e outros modelos da UML usando softwares convencionais de edição gráfica, como o Microsoft Paint ou Powerpoint. No entanto, eles são genéricos, não auxiliando na combinação correta de elementos próprios da notação UML e, tampouco, emitindo relatórios, gerando códigos e fazendo controle de atividades de desenvolvimento.

Assim, tem-se uma vantagem de uso das ferramentas CASE, pois são específicas para desenvolvimento de software e a maioria delas provê suporte para a UML, facilitando a utilização dessa linguagem de modo correto e com maior agilidade.

O autor Pichiliani, que escreveu o artigo “Mapeamento de Software para permitir a colaboração síncrona”, publicado em 2006, afirma que as principais vantagens do uso de ferramentas CASE envolvem o aumento da qualidade e produtividade dos produtos, a redução de trabalho manual e a melhoria na eficiência de elaboração dos modelos. Como consequência dessas vantagens, o time do projeto pode dedicar mais tempo para a tomada de decisões e para a gestão das mudanças, além de outras tarefas importantes do projeto, como a documentação e o controle de defeitos, erros e falhas.

Mas qual ferramenta CASE devo utilizar? Essa é uma pergunta comum feita por integrantes de equipes de desenvolvimento de software, já que existem várias delas à disposição no mercado, sejam gratuitas ou pagas.

Nesse sentido, a escolha correta da ferramenta CASE é essencial para o sucesso de um projeto de desenvolvimento de software, uma vez que o risco de um baixo desempenho é comum nos projetos e interfere negativamente no trabalho da equipe.

Apesar da importância dessa escolha, infelizmente há escassez de publicações na literatura especializada que fazem uma análise das ferramentas CASE existentes. Também não há, para esse domínio, uma ferramenta “bala de prata”, que seja aplicável em todos os projetos.

Em um primeiro momento, podemos pensar em escolher uma ferramenta CASE apenas dentre as opções de software gratuito. No entanto, esse pensamento pode prejudicar uma boa escolha, especialmente para projetos grandes e complexos, em que as ferramentas CASE pagas podem compensar por terem mais funções e suporte com respaldo contratual, minimizando tempo de desenvolvimento e riscos de problemas que podem surgir no andamento do processo.

Imagine um projeto no qual apenas uma ferramenta paga tem um recurso necessário para uma determinada aplicação, economizando horas de analistas. A organização desenvolvedora deve, então, realizar o cálculo da economia de horas humanas, que pode ser proporcionada por essa ferramenta frente aos custos de licenças.

Além disso, no caso da utilização das ferramentas CASE pagas, existe uma relação de compromisso na compra da aplicação, em que a fornecedora tem o dever de manter um produto e o respectivo suporte de qualidade. E isso não existe, dessa maneira, no caso de utilização de um software gratuito, ficando a pergunta: “como cobrar suporte ou resolução de um possível bug de uma entidade fornecedora ou comunidade criadora de um software gratuito?”

Por outro lado, para desenvolvimento de softwares de baixa ou média complexidade com capital financeiro limitado para trabalho, pode compensar ou ser mais adequado o uso de ferramentas CASE gratuitas com recursos suficientes para atender às necessidades do projeto.

Nessa linha, diversas ferramentas pagas possuem versões menores ou mais simples, com custos menores e até mesmo versões de assinatura mensal de baixo custo para aderência a projetos pequenos.

Nesse contexto, o desenvolvimento de aplicações pequenas tem menor demanda de recursos e não requerem uma ferramenta de modelagem pesada. Assim, é um erro pensar em não utilizar tais ferramentas nesses casos, pois é ressaltado que sempre é uma boa ideia ter o apoio ferramental, já que mesmo uma ferramenta mais simples irá facilitar a construção de modelos que irão auxiliar na construção das aplicações.

Desse modo, para boa escolha da ferramenta CASE, além do custo, devem ser vistos os seguintes aspectos principais:

- **Aderência dos processos adotados:** a ferramenta deve estar alinhada com os processos adotados pela empresa, evitando que ela altere significativamente as atividades organizacionais, de forma que se adapte às rotinas necessárias para o funcionamento da ferramenta CASE adotada;

- **Necessidades do projeto frente aos recursos oferecidos:** não adianta usar uma ferramenta que não cobre as necessidades reais do projeto. Por exemplo, não ter um ambiente de especificação de projeto de interfaces de usuário em um projeto de sistema web em que a usabilidade é requisito essencial;

- **Documentação existente:** é necessário verificar o tamanho da base de conhecimento fornecida pelo fabricante e por terceiros, incluindo fóruns de usuários. Repositórios de documentação facilitam a resolução de problemas comuns que surgem no momento de utilização da ferramenta;

- **Comunidade que a utiliza:** quanto maior a comunidade que a utiliza, melhor. Dessa forma, será mais fácil encontrar profissionais habituados com sua utilização. Uma ferramenta de sucesso em outras organizações e com uma comunidade grande de uso são fatores para motivar a equipe que a adotará;

- **Risco de ser descontinuada:** conheça o histórico de atualizações da ferramenta e quem está por trás dela, dando preferência a que tem mais tempo de mercado e com uma grande organização que a suporte. Isso minimizará o risco de escolha de uma ferramenta que futuramente pode ser descontinuada, acarretando em custos para migração de uma nova ferramenta.

Uma vez selecionada a ferramenta CASE, é preciso, ainda, treinar os membros da equipe para sua utilização e tratar a possível relutância desses em usá-la.



Classificação das ferramentas CASE

Uma primeira classificação das ferramentas CASE pode ser efetuada pelos seguintes critérios relacionados às fases do processo às quais as ferramentas se aplicam:

- **Upper-case:** especializadas na fase de concepção do software (ferramentas de análise e especificação e modelagem de requisitos);

- **Lower-case:** utilizadas na fase de implementação (incluindo desenho técnico, de edição e compilação de código e de testes);

• **Integrated- case:** cobrem todo o ciclo de vida do software, desde os requisitos do sistema até o controle final da qualidade.

Para uma classificação mais detalhada em relação aos recursos oferecidos e quanto às partes e fases do projeto em que as ferramentas CASE podem atuar, utilizamos as seguintes categorias:

• **Documentação:** são geradas descrições textuais e relatórios a partir de elementos modelados graficamente. Desse modo, a documentação textual fica coerente com os diagramas;

• **Planejamento e gerenciamento de projetos:** podem ser cadastradas e gerenciadas tarefas do projeto, reunindo detalhes da problemática envolvida e duração estimada da atividade e desenvolvedor atribuído. Podem, ainda, ser gerados relatórios úteis ao gestor de projeto;

• **Especificações formais:** é feito um auxílio para criação de modelos formais, que são ligados a uma sintaxe e notações bem definidas. Além disso, elas são difíceis de serem feitas manualmente;

• **Comunicação:** são providos meios tecnológicos colaborativos para troca organizada e profissional de mensagens entre membros do projeto;

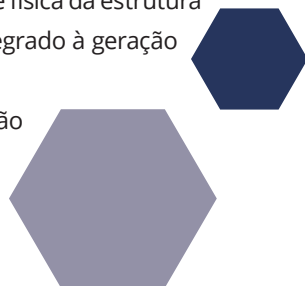
• **Análise e projeto de software:** os editores de modelos atuam como facilitadores para análise, registrando os requisitos obtidos e, posteriormente, provendo o desenho da solução tecnológica com diagramação das classes;

• **Projeto e desenvolvimento de interfaces:** as ferramentas CASE podem produzir diagramas que representam a interação dos sistemas com os usuários, incluindo o desenho das telas e seus componentes;

• **Programação:** existe significativo trabalho de produção de código que pode ser automatizado a partir de detalhes dos modelos. Então, as ferramentas CASE geram código de modo padrão e com alta fidelidade aos modelos de origem;

• **Desenho de bases de dados:** definição lógica e física da estrutura das bases de dados, podendo esse recurso ser integrado à geração de código;

• **Gerenciamento de configuração:** organização dos diversos arquivos de programas, documentação e dados. Na prática, é difícil coordenar todos esses arquivos, suas versões, autoria e possíveis conflitos pela edição ao mesmo tempo;



- **Controle de qualidade:** as ferramentas contêm mecanismos que auxiliam na qualidade do software, desde análises automáticas dos modelos, com questionamento de certas decisões dos desenvolvedores, até o registro dos casos de testes e os resultados da execução dos mesmos;

- **Engenharia reversa:** é comum encontrarmos softwares legados com códigos extensos, sem documentação e consequente dificuldade de seu entendimento. Para esses casos, as ferramentas CASE podem ler esse código e produzir modelos para facilitar o entendimento humano.

Existem ferramentas que realizam apoio em um desses pontos mencionados, que são chamadas de **horizontais**, atuando em funções específicas, e outras que atuam de modo mais amplo, de modo integrado em várias frentes, sendo chamadas de **verticais**.



Verificação de ferramentas CASE existentes

Agora, não há nada melhor para aprender sobre essas ferramentas do que vendo exemplos, não é mesmo? Tendo isso em mente, entre as diversas ferramentas existentes atualmente no mercado, iremos detalhar algumas nos subtópicos seguintes.

O critério para inclusão de uma ferramenta neste material é o de ser um projeto ativo, com pelo menos uma atualização em seis meses. Uma exceção a esse critério foi a inclusão da ArgoUML: apesar de sua última versão ser de 2011, ela ainda apresenta boa utilização no mercado e na área acadêmica, principalmente por ser o maior projeto de ferramenta CASE gratuita e de código livre.



Enterprise Architect

A Enterprise Architect é uma ferramenta CASE colaborativa criada em 2000 para modelagem, design e gerenciamento de etapas do processo de desenvolvimento de software baseado em UML e padrões similares.

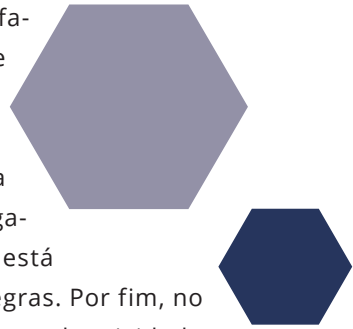
Em termos de distribuição, ela é uma ferramenta paga que fornece uma versão de avaliação, modalidade conhecida como trial, em que a aplicação funciona de modo completo por 30 dias.

Esta ferramenta CASE tem uma longa e comprovada reputação no mercado, sendo utilizada em mais de 160 países. Originalmente, foi projetada como uma ferramenta de modelagem, suportando a UML 1.1. Com o tempo, o produto evoluiu para incluir outras especificações UML: 1.3, 2.0, 2.1, 2.3, 2.4.1 e 2.5.

Na Figura 1, é possível ver a interface de usuário da ferramenta Enterprise Architect.



Como pode ser visto na Figura 1, a interface de usuário da ferramenta, principalmente pela parte superior organizada em abas, se assemelha a um utilitário da família Microsoft Office, apesar de não haver vínculo com a Microsoft. No painel à esquerda, há um navegador de itens de projeto e, no painel do meio, está sendo editado um diagrama de modelo de regras. Por fim, no painel à direita, tem-se a edição de um diagrama de atividades, que contempla a criação de um pedido em uma loja.



Os principais recursos oferecidos pela ferramenta Enterprise Architect podem ser descritos da seguinte maneira:

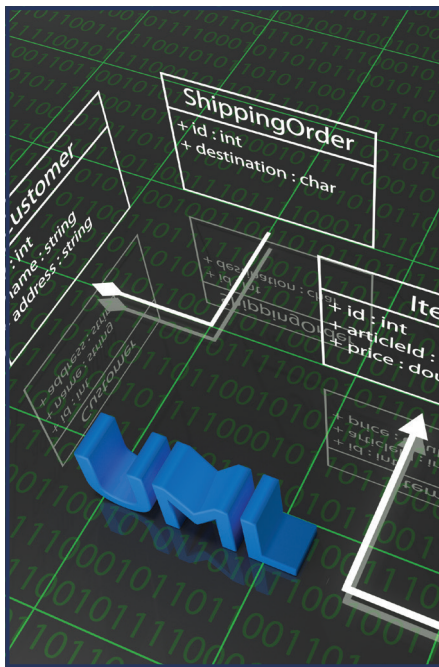
- Criação de modelos de informações, softwares e hardwares, usando UML;
- Produção de documentação detalhada nos formatos RTF, PDF e HTML;
- Geração e reversão de códigos de mais de dez linguagens de programação;
- Modelagem de bases de dados e reversão de esquemas de banco de dados;
- Modelagem das dependências entre os elementos, a dinâmica do sistema e do estado;
- Modelagem das hierarquias de classe, a implantação, os componentes e os detalhes de implementação;
- Registro de problemas do projeto, as tarefas e o glossário do sistema;
- Compartilhamento, importação de modelos e controle de versões usando o formato XMI;
- Uso de perfis UML para criar extensões personalizadas para modelagem específica de domínio;
- Gravação e leitura de diagramas completos como padrões;
- Conexão a repositórios de bancos de dados compartilhados;
- Execução de transformações de modelo para modelo;
- Criação e compartilhamento de visualizações dinâmicas dos elementos de modelo e conjuntos de diagramas;
- Criação de mapas mentais, modelos de processos de negócios e diagramas de fluxo de dados usando UML.



ArgoUML

A ArgoUML é a principal ferramenta CASE de modelagem UML gratuita e de código aberto, incluindo suporte para todos os diagramas da UML 1.4 padrão. Ela é executada em qualquer sistema operacional que suporte a plataforma Java e está disponível em dez idiomas, incluindo o português brasileiro.

A ArgoUML foi criada no ano de 1998, na Universidade da Califórnia, em Berkeley, com o objetivo de ser um ferramental de software poderoso e acessível para todos. Para esse objetivo ser conquistado, ela conta com o **apoio financeiro de doações** e com as contribuições técnicas colaborativas da comunidade de desenvolvedores de código livre, chamada Tigris.



CURIOSIDADE

A ArgoUML solicita apoio financeiro de doações opcionais de recursos no site oficial de seu projeto. Isso é importante para cobrir os custos de manutenção, já que a distribuição da ferramenta é feita de modo totalmente gratuito. Nem mesmo propagandas ou outros artifícios de geração de renda são utilizados no projeto. Desse modo, para recebimento das doações, é informado o endereço físico para envio de uma ordem de pagamento e um link para envio de recursos de modo eletrônico via PayPal.

A ArgoUML foi distribuída inicialmente na Licença BSD Open Source e, atualmente, está sob a Eclipse Public License (EPL) 1.0, sempre com aquisição livre e com código-fonte aberto, porém com mudanças em detalhes na forma de colaboração.

Na Figura 2, é possível visualizar a interface de usuário da ferramenta ArgoUML.

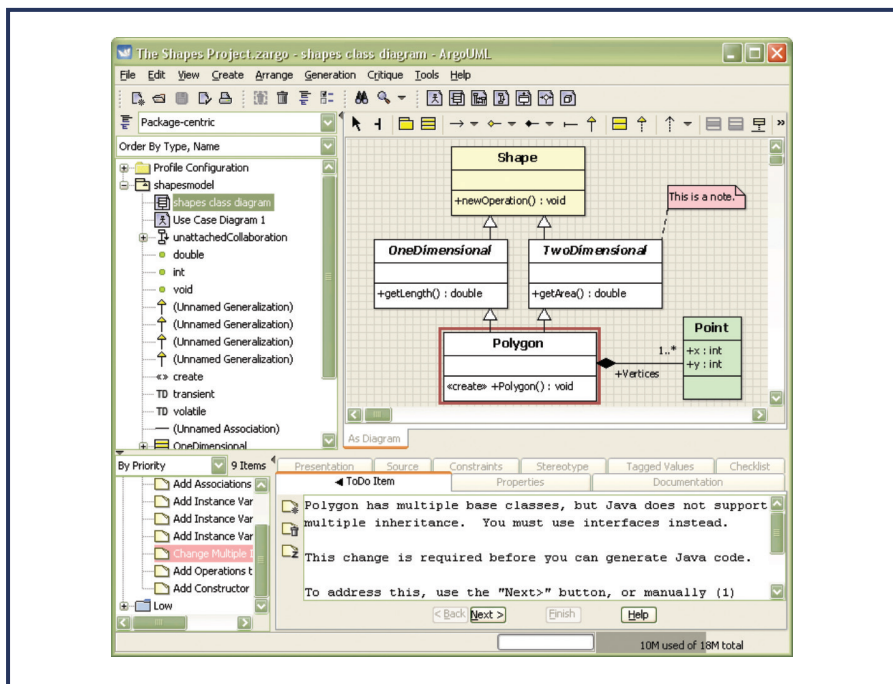


Figura 2. Captura de tela da ferramenta ArgoUML.

Conforme pode ser visto na Figura 2, a tela da ArgoUML é dividida em quatro áreas:

- **Esquerda superior:** visualização hierárquica de itens do projeto atual;
- **Direita superior:** editor para a parte selecionada do projeto. Neste caso, um diagrama de classe;
- **Esquerda inferior:** visualização da lista to do, que apresenta os itens a serem feitos agrupados por prioridade;
- **Direita inferior:** detalhes do objeto selecionado no diagrama to do e abas para outras opções de um item, como propriedades e documentação relacionada.

O grande diferencial da ArgoUML em relação a outras ferramentas CASE são os recursos cognitivos embutidos no produto. Em vez de ser apenas um diagramador, documentador e gerador de código, a ferramenta em questão procura orientar e auxiliar o desenvolvedor na construção dos modelos. Essa ajuda provém de várias regras que são aplicadas continuamente, verificando inconsistências, erros comuns e sugerindo próximos passos.

Outro ponto importante que a difere das demais é o fato de seu código ser aberto, possibilitando que você faça mudanças no comportamento da ferramenta, caso necessário. Tais adaptações podem incluir a adição de um mecanismo de bate-papo (chat), um editor de diagramas da UML colaborativo e travas para controle de concorrência, além de elementos de percepção remota, como lista de elementos travados etc.

Os tipos de diagramas que podem ser criados utilizando a ArgoUML incluem: diagrama de classe, diagrama de estado, diagrama de atividade, diagrama de caso de uso, diagrama de interação, diagrama de distribuição e diagrama de sequência. Além disso, os formatos de arquivo utilizados pela ArgoUML para exportar os diagramas são os seguintes: GIF, PNG, PostScript (simples e encapsulado), PGML e SVG.

Outras funcionalidades interessantes encontradas no ArgoUML são:

- compatibilidade com o padrão XML: permite utilização de modelos criados com o UML em outros programas;
- suporte ao OCL;
- produção de códigos para Java, Python, C, C++, C# e PHP;
- estrutura modular para a engenharia reversa;
- críticos automáticos de design para melhoria dos modelos.

Ela é uma ferramenta extensível, podendo incorporar novas funções de acordo com a disponibilidade de recursos, que são disponibilizados em subprojetos. A lista desses subprojetos e suas descrições pode ser vista no Quadro 2.

QUADRO 2. NOME E DESCRIÇÃO DOS SUBPROJETOS DA ARGOUML



Nome	Descrição
argoprint	Geração de uma visão imprimível dos modelos.
argouml-andromda	Plug-in para executar a AndromDA.
argouml-cpp	Suporte à linguagem C++.
argouml-csharp	Suporte à linguagem C#.
argouml-db	Ferramenta para modelagem de bancos de dados.
argouml-de	Adiciona idioma alemão.
argouml-documentation	Documentação de auxílio ao usuário.

argouml-downloads	Downloads da ferramenta.
argouml-en-gb	Adiciona idioma inglês britânico.
argouml-es	Adiciona idioma espanhol.
argouml-fr	Adiciona idioma francês.
argouml-gen	Scripts para checagens estáticas, compilações noturnas e tarefas administrativas.
argouml-groovy	Suporte à linguagem Groovy.
argouml-i18n	Coleção de projetos de tradução de idiomas.
argouml-i18n-zh	Adiciona idioma chinês.
argouml-idl	Suporte à linguagem IDL.
argouml-it	Adiciona idioma italiano.
argouml-java	Suporte à linguagem Java.
argouml-nb	Adiciona idioma norueguês.
argouml-php	Suporte à linguagem PHP.
argouml-pt	Adiciona idioma português de Portugal.
argouml-pt-br	Adiciona idioma português brasileiro.
argouml-ru	Adiciona idioma russo.
argouml-ruby	Suporte à linguagem Ruby.
argouml-sql	Suporte à linguagem SQL.
argouml-stats	Publicação de resultados de verificações estáticas e compilações noturnas.
argoumlinstaller	Empacotamento da ArgoUML na forma de um instalador.
seeds	Subprojetos ainda não maduros.



Visual Paradigm

A Visual Paradigm é uma ferramenta CASE UML criada em 2002 que suporta UML 2, SysML e a notação de modelagem de processos de negócios (BPMN) do OMG.

Além do suporte à modelagem, essa ferramenta fornece recursos de geração de relatórios e engenharia de código, incluindo geração de código. Ela

realiza engenharia reversa de diagramas a partir do código e fornece engenharia de ida e volta, conhecida como round-trip engineering, para várias linguagens de programação.

Ela é uma ferramenta paga que apresenta dois modelos comerciais: a aquisição de sua licença para uso perpétuo, com preços que variam entre US\$ 99,00 a US\$ 1.999,00, ou assinatura mensal, com preços que variam de US\$ 6,00 a US\$ 89,00. Essa variação de preços é proporcional à quantidade de recursos oferecidos pela ferramenta.

A Visual Paradigm também possui uma versão web que oferece suporte a uma ampla variedade de necessidades de visualização, desde design de software, modelagem de dados, mapeamento de processos de negócios, análise estratégica, mapeamento mental e agendamento de projetos, além de ser amplamente adotada em diferentes setores como negócios, educação e unidades sociais. Como existe um custo para sua aquisição, o fornecedor criou um **programa de apoio ao ensino** com o objetivo de incentivar a adoção da ferramenta por profissionais em formação.



DICA

Existe um programa de apoio ao ensino que oferece, de modo gratuito, as licenças educacionais para que os estudantes possam usar o ambiente proporcionado pela ferramenta Visual Paradigma. Esse programa oferece software de diagramação on-line gratuito para escolas, que podem ser usados em todos os níveis acadêmicos - escolas de Ensino Médio, colégios, faculdades, universidades etc.

Além disso, é possível listar algumas de suas funcionalidades:

- **Ferramentas para UML & SysML:** capture e registre, de modo sistemático, os requisitos funcionais com o diagrama de caso de uso UML;
- **Ferramentas e diagrama BPMN:** visualize os fluxos de trabalho empresariais. Comunique ideias de processos de negócios com os diagramas BPMN;
- **UML para código e código para UML:** gere código-fonte do modelo de classe UML ou vice-versa;
- **Gerencie projetos:** registre atividades com instruções, exemplos e ferramentas para o desenvolvimento incremental de entregas;

- **Formate sua saída de modo versátil:** exporte seu design em uma gama de formatos de arquivos;
- **Gere documentos do projeto:** gere documentos completos e profissionais com diversos detalhes técnicos do projeto;
- **Possua outras diversidades de recursos:** suporte a várias línguas e desenvolvimento de plug-ins.

Um de seus recursos principais é a captura de requisitos funcionais com a ferramenta gráfica de geração de diagrama de casos de uso da UML.

Cada caso de uso em um diagrama representa um objetivo de negócios de alto nível, que gera um resultado mensurável em relação aos valores de negócios. Os atores são conectados aos casos de uso para representar papéis que interagem com as funções. A diagramação de casos de uso feita na ferramenta Visual Paradigm pode ser vista na Figura 3.

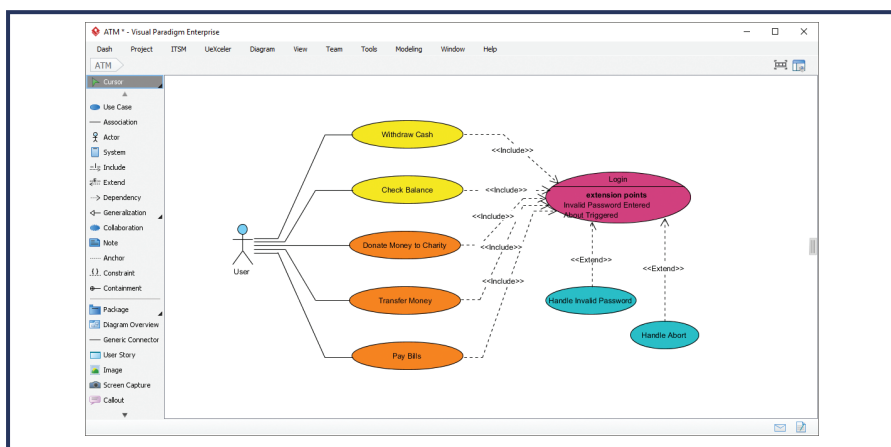


Figura 3. Captura de tela de diagrama de casos de uso na ferramenta Visual Paradigm.

Como pode ser visto na Figura 3, a interface de usuário da ferramenta Visual Paradigm é bem organizada e intuitiva, apresentando, no painel esquerdo, os elementos que podem ser utilizados em um diagrama, que é montado no painel à direita.

Além disso, é possível ver o ator, chamado usuário (representado pelo boneco azul à esquerda do diagrama), executando cinco atividades, que incluem a atividade de log-in, ou seja, que necessitam que o usuário esteja em ambiente restrito após acessar com suas credenciais, geralmente formadas por nome de usuário (log-in) e senha.



Os principais pontos da BOUML são:

- As razões iniciais que motivaram o autor para o desenvolvimento da BOUML foram, principalmente, o prazer de desenvolver livremente uma ferramenta, a esperança de vê-la sendo usada por muitas pessoas e o alto preço das ferramentas de ML comerciais.

The screenshot displays the Eclipse IDE interface for a project named 'Cloud - deploy (deploy.prj)'. The 'Project Explorer' on the left shows the 'API BASE' package structure. The main editor window shows the 'main()' method in 'Cloud.java', which interacts with 'UmlClass' and 'UmlAttribute' objects. The bottom window shows the 'UML Inheritance Diagram' for the 'API BASE' package, illustrating the relationships between 'UmlClass', 'UmlAttribute', 'UmlOperation', and 'UmlRelation' classes.

API BASE

- UmlClass
- UmlAttribute
- UmlOperation
- UmlRelation

UML Inheritance Diagram

```

classDiagram
    class UmlClass {
        +String name
        +String package
        +String type
        +String class
        +String operation
        +String relation
    }
    class UmlAttribute {
        +String name
        +String package
        +String type
        +String class
        +String operation
        +String relation
    }
    class UmlOperation {
        +String name
        +String package
        +String type
        +String class
        +String relation
    }
    class UmlRelation {
        +String name
        +String package
        +String type
        +String class
        +String operation
        +String relation
    }
    UmlClass <|-- UmlAttribute
    UmlClass <|-- UmlOperation
    UmlClass <|-- UmlRelation
    
```

returns a new attribute named 's' created under 'parent'

In case it cannot be created the name is already used or invalid, 'parent' cannot contain it etc... in Java and produce a RuntimeException in Java

[operation] created (out parent: UmlClass, in s: s): UmlAttribute

ANÁLISE E MODELAGEM DE SISTEMAS

Como pode ser visto na Figura 4, a tela da ferramenta BOUML possui dois painéis principais: um à esquerda, contendo uma árvore para acesso hierárquico, e um à direita, para abrir os diferentes editores de diagramas separados em janelas. No exemplo em questão, tem-se duas janelas: a primeira contendo um diagrama de sequência e a segunda contendo um diagrama de classes.

Devido a violações de licença, o autor decidiu interromper a versão gratuita da BOUML e passou a desenvolver versões não gratuitas, iniciando pela 5.0, até a 6.12. No entanto, ao final de maio de 2017, a BOUML foi distribuída novamente como um software livre.



Umbrello UML Modeller



A Umbrello UML Modeller é uma ferramenta CASE para modelagem UML gratuita e integrada ao projeto KDE, que é um importante conjunto de aplicativos para utilização no sistema operacional Linux. Essa ferramenta é utilizada para modelar o próprio projeto KDE por grande parte de seus desenvolvedores.

Este projeto foi iniciado por Paul Hensgen como um de seus projetos universitários. O nome original do aplicativo era Modelador UML e Paul fez todo o desenvolvimento até o final de 2001. Quando o programa atingiu sua versão 1.0, em 2002, Paul se retirou da equipe de desenvolvimento. No entanto, como um software livre e de código aberto, o programa continuou a evoluir e está sendo mantido por um grupo de desenvolvedores de diferentes partes do mundo. Em setembro de 2002, o projeto mudou seu nome de Modelador UML para Umbrello UML Modeller.

Apesar de ter origem e estar integrada no projeto KDE, a ferramenta funciona em outros sistemas operacionais além do Linux, tendo suporte para o Windows e o MacOS.

Especialmente durante as fases de análise e desenho, o Umbrello UML Modeller auxilia na obtenção de um produto de qualidade. Além disso, a Umbrello UML Modeller 2.11 suporta os seguintes tipos de diagrama da UML:

- Diagrama de classe;
- Diagrama de sequência;
- Diagrama de colaboração;
- Diagrama de caso de uso;

- Diagrama de estado;
- Diagrama de atividade;
- Diagrama de distribuição.

Na Figura 5, é possível ver a interface de usuário da ferramenta CASE Umbrello UML Modeller.

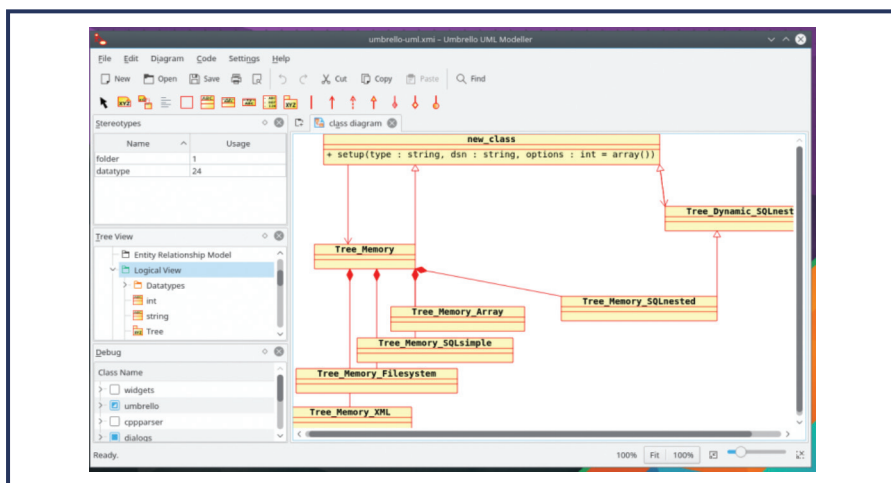


Figura 5. Tela da ferramenta Umbrello.

Como pode ser visualizado na Figura 5, tem-se a edição de um diagrama de classes na ferramenta, com visualização e alteração de propriedades dos elementos na parte esquerda superior.

A Umbrello UML Modeller pode gerar código-fonte a partir de várias linguagens de programação baseadas no seu Modelo UML, para auxiliá-lo no início com a implementação do seu projeto. O código gerado consiste em declarações de classe, com seus métodos e atributos de modo, para que você possa preencher as lacunas e fornecer a funcionalidade das suas operações.



Outras

A StarUML é uma ferramenta de modelagem UML licenciada sob uma versão modificada da GNU GPL até 2014, quando uma versão reescrita 2.0 foi lançada sob uma licença proprietária. Ou seja, ela evoluiu tecnicamente e se tornou uma ferramenta paga.

Outra ferramenta CASE interessante é a MagicDraw, que é paga, sendo bem completa e podendo ser obtida por meio do acesso ao seu site oficial. A MagicDraw permite o desenho de processos de negócios e modelagem de sistemas com suporte ao trabalho em equipe. Trata-se de uma ferramenta projetada para analistas de negócios, analistas de software, programadores, engenheiros e analistas de documentação, uma vez que essa ferramenta facilita a análise e projeto no paradigma orientado a objetos com uso de sistemas de banco de dados.

Ela fornece mecanismos para engenharia de código (com o apoio da engenharia reversa de Java, C, C#, CL (MSIL) e linguagens de programação CORBA IDL, bem como a modelagem do esquema do banco de dados e geração de DDL.

Por fim, é possível mencionar a Astah, que é paga e pode ser obtida por meio do acesso ao seu site oficial. A Astah apresenta diferentes versões, que vão desde as mais simples até as mais avançadas, sempre focando em permitir o trabalho ágil na engenharia de software. A ideia presente nela é oferecer um suporte próximo ao cliente, com conteúdo amplo e atendimento especializado para auxílio aos seus usuários, além da de utilização dos recursos oferecidos.



Esta unidade se iniciou com a apresentação da introdução à UML, com sua definição e apontamentos teóricos e práticos segundo renomados autores. Foi ainda ressaltada a importância da padronização de notação feita pela UML para os processos de trabalho na complexa indústria de desenvolvimento de software, que engloba análise, modelagem e projeto na engenharia de software. Vários outros benefícios da UML também foram apontados e comentados.

A partir da base de conceitos introduzida, foi vista a evolução da UML, com descrição de eventos importantes ocorridos no contexto estudado e detalhes de características técnicas que foram incorporadas em suas versões.

Nessa plataforma, considerando o foco em UML da unidade, foram abordados também os detalhes da visão dos modelos presentes na linguagem em questão. Pela taxonomia dos diagramas apresentada, foi possível ter uma boa noção do que a UML contempla.

Como parte final desta unidade, foi visto o conceito de ferramentas CASE e foram apresentadas algumas das mais importantes ferramentas disponíveis no mercado. Com isso, você teve a oportunidade de verificar a importância de usar uma boa ferramenta CASE no processo de desenvolvimento de software, orientado a objetos usando UML.

Referências bibliográficas



- ARGOUML. **Site oficial da ArgoUML**. Disponível em: <<http://argouml.tigris.org/>>. Acesso em: 05 nov. 2019.
- BEZERRA, E. **Princípios de análise e projeto de sistemas com UML**. 2. ed. Rio de Janeiro: Elsevier Editora, 2007.
- BLAHA, M.; RUMBAUGH, J. **Modelagem e projetos baseados em objetos**. 2. ed. Rio de Janeiro: Elsevier Editora, 2006.
- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. **UML - guia do usuário**. 2. ed. Rio de Janeiro: Editora Campus, 2005.
- BOUML. **BOUML user manual**. Junho, 2018. Disponível em: <<https://www.bouml.fr/doc/index.html>>. Acesso em: 05 nov. 2019.
- CUNHA, W. S.; COSTA, H.; PARREIRA JR. P. A. Análise de Ferramentas CASE quanto às Boas Práticas de Modelagem de Software com UML. In: **XV Simpósio Brasileiro de Qualidade de Software**, Florianópolis, Santa Catarina, 2016. Disponível em: <<http://www.ic.uff.br/~esteban/files/sbes-prova.pdf>>. Acesso em: 03 dez. 2019.
- DA SILVA, A. M. R.; VIDEIRA, C. A. E. **UML – metodologias e ferramentas CASE**. Lisboa: Editora Centro Atlântico, 2001.
- DEBONI, J. E. Z. **Modelagem orientada a objetos com a UML**. São Paulo: Futura, 2003.
- ENTERPRISE ARCHITECT. **Full Lifecycle Modeling for Business, Software and Systems | Sparx Systems**. Disponível em: <<https://sparxsystems.com/products/ea/>>. Acesso em: 05 nov. 2019.
- GUEDES, G. T. A. **UML 2: uma abordagem prática**. 2. ed. São Paulo: Novatec Editora, 2011.
- INTRODUÇÃO A UML**, CONHEÇA A HISTÓRIA E SAIBA O QUE É. Postado por Estudo na web. (7 min. 52 s.). color. son. port. Disponível em: <<https://www.youtube.com/watch?v=llpdtMcLZ7A&feature=youtu.be>>. Acesso em: 03 dez. 2019.
- JACOBSON, I.; CHRISTERSON, M.; JONSSON P.; ÖVERGAARD, G. **Object-oriented software engineering**. 4. ed. Wokingham: Addison Wesley, 1993.
- OMG. **About the Unified Modeling Language Specification Version 2.5.1**. Disponível em <<https://www.omg.org/spec/UML/About-UML/>>. Acesso em: 05 nov. 2019.

PICHILIANI, M. C. **Mapeamento de software para permitir a colaboração síncrona**. 2006. 170 f. Dissertação de mestrado – Instituto Tecnológico de Aeronáutica, São José dos Campos, 2006. Disponível em: <<http://www.comp.ita.br/~pichilia/argo/TeseVersaoFinal.pdf>>. Acesso em: 08 nov. de 2019.

PRESSMAN, R. S. **Engenharia de software**: uma abordagem profissional. 7. ed. Porto Alegre: AMGH, 2011.

QUATRANI, T. **Modelagem visual com Rational Rose 2000 e UML**. Rio de Janeiro: Editora Ciência Moderna Ltda., 2001.

SCHACH, S. R. **An introduction to object-oriented systems analysis and design with UML and the unified process**. Boston: McGraw-Hill Irwin, 2004.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

UMBRELLO. **Manual da Umbrello - The UML Modeller**. 2014. Disponível em: <https://docs.kde.org/trunk5/pt_BR/kdesdk/umbrello/umbrello.pdf>. Acesso em: 05 nov. 2019.

VISUAL PARADIGM. **Site oficial da Visual Paradigm**. Disponível em: <<https://www.visual-paradigm.com/>>. Acesso em: 05 nov. 2019.

UNIDADE

3

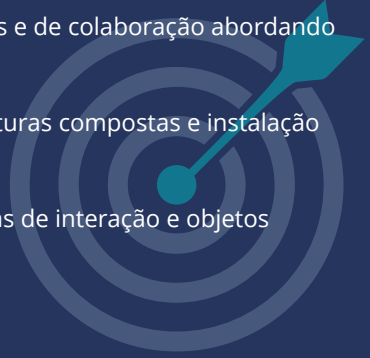
DIAGRAMAS UML E
SUAS APLICAÇÕES



ser
educacional

Objetivos da unidade

- Apresentar os diagramas de atividades, classes e de colaboração abordando as suas principais características e aplicações;
- Abordar os diagramas de componentes, estruturas compostas e instalação expondo as suas atribuições e funcionalidades;
- Observar os aspectos referentes aos diagramas de interação e objetos verificando os seus principais modelos.



Tópicos de estudo

- Activity diagram
- Class diagram
- Communication diagram
- Component diagram
- Composite structure diagram
- Deployment diagram
- Interaction overview diagram
- Object diagram





Activity diagram



No que se refere à modelagem e análise de sistemas, é preciso entender como são organizados os seus diagramas. De imediato, podemos tratar do activity diagram, ou diagrama de atividades. De maneira básica, podemos entender que os diagramas de atividades estão disponíveis dentro da UML direcionada à modelagem de aspectos dinâmicos de sistemas. Você pode estar se questionando: o que de fato representa esse diagrama? Bem, segundo Booch et al (2012), um diagrama de atividade é constituído por um gráfico de fluxo de controle de uma determinada atividade para outra. É um tipo de diagrama que apresenta dois aspectos relevantes: a concorrência e as ramificações de controle.

Segundo Sommerville (2011), os **diagramas de atividades** têm, como função, apresentar as atividades que formam um processo de sistema e o fluxo de controle de uma atividade para a outra. Esses diagramas, quando definidos, são utilizados para a realização da modelagem relacionada aos aspectos dinâmicos inseridos no sistema. É possível, por meio do diagrama de atividades, realizar uma modelagem de fluxo quando este, em um fluxo de controle, muda de um estado para outro em locais distintos.

Vale salientar que os diagramas de atividade se caracterizam por se manterem isoladamente, a fim de realizar algumas ações que vão desde a visualização, passando pela especificação e construção, até chegar à documentação referente à dinâmica estabelecida por um conjunto de objetos ou até mesmo para realizar a modelagem do fluxo de controle dentro de um processo operacional. Como você pode notar, os diagramas de atividades têm a finalidade de evidenciar o fluxo de controle de uma atividade para outra. As atividades resultam em ações desenvolvidas pelas computações atômicas executáveis que indicam basicamente dois resultados: uma alteração do estado do sistema, ou o retorno ao valor anterior (BOOCH et al, 2000).



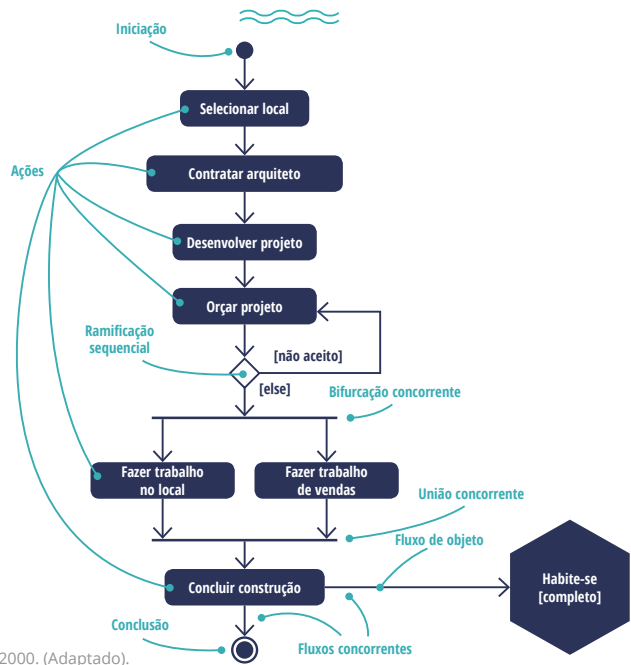
CONTEXTUALIZANDO

A importância dos diagramas de atividades não se limita à modelagem de requisitos dinâmicos dentro de um sistema, uma vez que eles também estão ligados ao desenvolvimento de sistemas executáveis, por sua vez utilizados na engenharia de produção, por exemplo.

E como realizar os passos iniciais para estabelecer um fluxo de trabalho através dos diagramas de atividades? Imagine um fluxo de trabalho de forma análoga ao do desenvolvimento de uma casa. Uma construção habitacional requer uma série de atividades complexas que vão desde a escolha do local até a certificação que habilita o usuário a tomar posse da casa. Na modelagem que envolve sistemas mais complexos de software você irá se deparar com situações similares, e se perguntar, por exemplo, qual a forma mais viável de realizar a modelagem que determina o fluxo de trabalho.

Vamos analisar juntos: você pode elaborar roteiros de cenários estabelecendo a interligação de determinados objetos e mensagens trocadas entre ambos. Para isso, é preciso utilizar diagramas de sequências e colaboração. Por sua vez, a modelagem desses aspectos dinâmicos pode ser realizada por meio de diagramas de atividades. Você pode observar, no diagrama a seguir, que um diagrama de atividades é um fluxograma que evidencia a tarefa realizada ao longo de um determinado período. Vamos então observar as operações executadas entre os objetos (BOOCH et al, 2000).

DIAGRAMA 1. DIAGRAMA DE ATIVIDADES



Vale frisar que os diagramas de atividades se caracterizam por apresentar basicamente três aspectos: estados de atividade e ação, transições e objetos.

Estados de atividade e ação

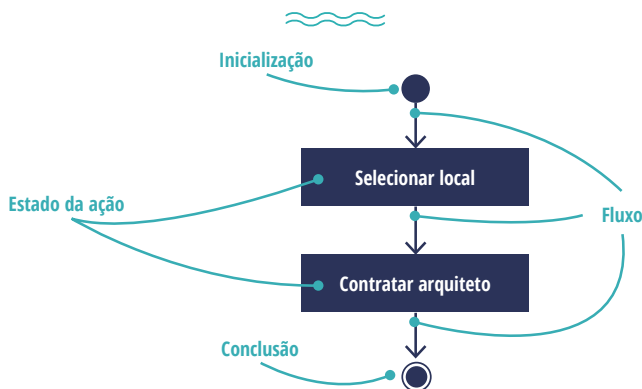
Em um fluxo de controle desenvolvido por um diagrama de atividade é viável mensurar uma expressão ou modelo que estabeleça o conjunto de valor de um requisito ou que volte para um determinado valor. De forma alternada, é possível realizar uma série de ações ligadas ao objeto, por exemplo, a manipulação e destruição de objetos.

Todas essas computações atômicas executáveis são denominadas estados de ação, pois são estados pertencentes a um sistema que simboliza a realização de uma determinada ação. Em contrapartida, os estados de atividades se caracterizam por sua decomposição, onde as suas atividades podem ser simbolizadas pelos diversos diagramas de atividade. São considerados como não atômicos, ou seja, podem ser abortados durante o processo e levam um determinado período para serem completados.

Transições

No momento em que a ação ou atividade pertencente a um estado se encontra suprida, o fluxo do controle salta para o estado posterior. É necessário determiná-lo por meio de transições que apresentem a rota traçada pelo estado de ação ou de atividade. Dentro da UML é preciso compreender que uma transição é simbolizada por meio de uma linha simplificada e uma direção, conforme o diagrama a seguir.

DIAGRAMA 2. TRANSIÇÕES NÃO ATIVADAS

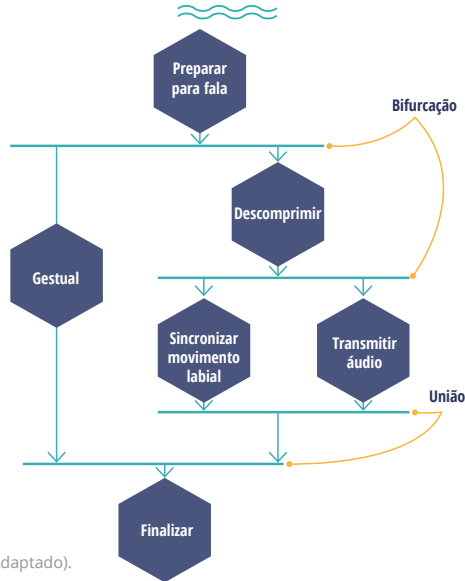


Fonte: BOOCH et al, 2000. (Adaptado).

Nesse contexto, é preciso se atentar a três caminhos utilizados para realizar um fluxo de controle. Primeiramente, é possível tratar da ramificação, que se configura como caminhos alternativos baseados em expressões booleanas, e que poderá apresentar uma transição de entrada e diversas saídas (BOOCH et al, 2000). Vale frisar que durante essas transições de saída não pode haver sobrecarga das proteções para evitar a ambiguidade, porém elas deverão suprir todas as possibilidades existentes para evitar que o fluxo de controle se estanque.

Uma segunda alternativa, principalmente no que se refere à modelagem de fluxos de trabalho de processos de negócios, é a bifurcação e a união utilizada pela barra de sincronização de diversos fluxos de controles concorrentes.

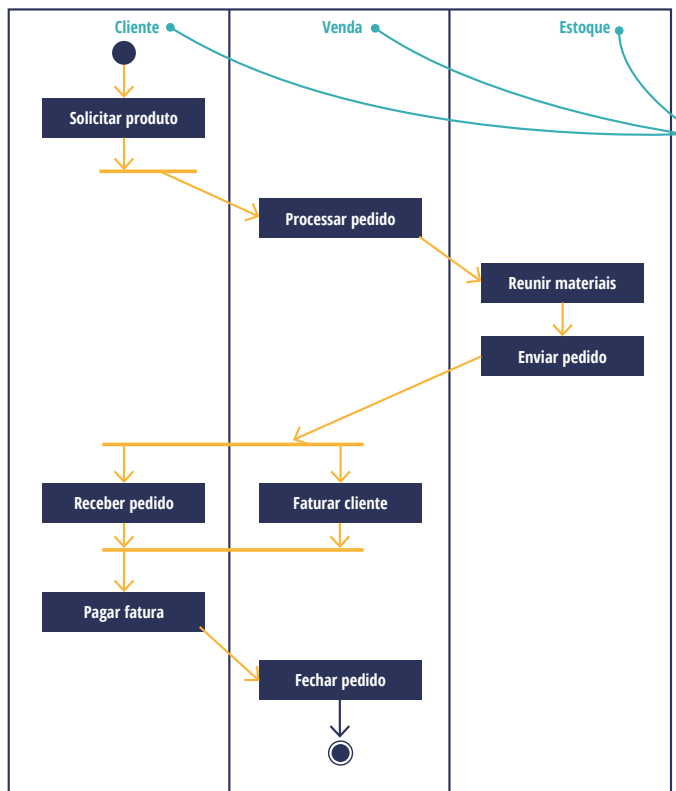
DIAGRAMA 3. BIFURCAÇÃO E UNIÃO



Fonte: BOOCH et al, 2000 . (Adaptado).

A terceira possibilidade se refere ao uso das “raias de natação”, que é extremamente útil em relação a fluxos de trabalho relacionados aos processos de negócios. Sua importância está relacionada ao fato dessas raia dividirem os estados de atividades de um diagrama de atividades em grupos. É importante notar que cada raia simboliza uma responsabilidade elevada ligada a uma etapa da atividade geral pertencente a um diagrama de atividade, sendo implantada por uma ou mais classes. Cada atividade é relacionada a uma raia específica, porém as transições poderão cruzar as outras raia (BOOCH et al, 2000).

DIAGRAMA 4. RAIAS DE NATAÇÃO

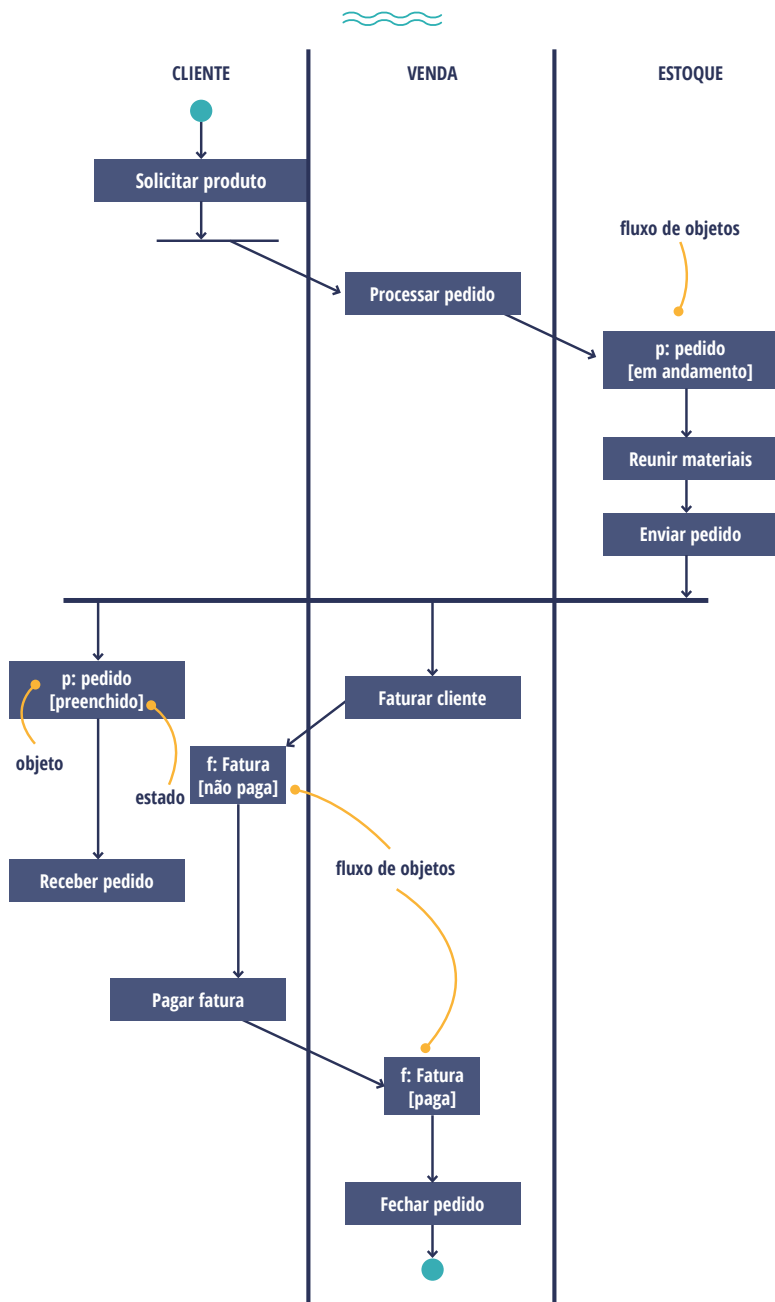


Fonte: BOOCH et al, 2000. (Adaptado).

Objetos

É necessário destacar que os objetos estarão interligados ao fluxo de controle ligado a um diagrama de atividades. Imagine, por exemplo, o processamento de um pedido onde as instâncias pertencentes a ele serão elaboradas por certas atividades, assim como outras atividades que conseguem modificar status de objetos. É viável determinar o que se encontra inserido dentro de um diagrama de atividades. Esse modelo de relacionamento é conhecido como “fluxo de objetos”, pois simboliza a inserção de um objeto dentro de um fluxo de controle. É importante evidenciar, também, que o fluxo de um objeto possibilita que os atributos sejam modificados por meio de um diagrama de atividade.

DIAGRAMA 5. FLUXO DE OBJETOS



Fonte: BOOCH et al, 2000. (Adaptado).



Class diagram



O class diagram, ou diagrama de classes, é observado frequentemente na modelagem de sistemas direcionados a objetos. Se caracteriza por apresentar uma série de classes, colaborações, relacionamentos e interfaces (BOOCH et al, 2000).

Você pode utilizar os diagramas de classe para a execução de uma modelagem que utiliza uma visão estática direcionada ao planejamento de um sistema. Isso implica dizer que, em boa parte dos casos, existe uma interação entre algumas modelagens relacionadas à aspectos como vocabulário, colaborações ou esquemas. Os diagramas de classes servem, ainda, de referência para os diagramas de componentes e de implantação. Esses diagramas são essenciais para a observação e documentação dos modelos denominados “estruturais”. Porém, eles também servem para o desenvolvimento de sistemas executáveis através de áreas relacionadas, por exemplo, à engenharia de produção.



Ao desenvolver uma obra, por exemplo, é necessário levar em consideração que os aspectos estruturais e comportamentais não se dissociam, ou seja, portas e janelas são itens estruturais extremamente importantes, porém as cargas suportadas e o manuseio desses itens são aspectos comportamentais que precisam ser levados em consideração. Ao desenvolver um software, o usuário apresenta uma habilidade de estabelecer o seu planejamento desde o início. Por meio da UML é possível usar os diagramas de classe para observar as características estáticas ligadas ao desenvolvimento e suas relações.



ANÁLISE E MODELAGEM DE SISTEMAS



EXPLICANDO

Os diagramas de classes, ao serem apresentados, podem conter restrições. Eles podem conter pacotes ou subsistemas usados para reunir elementos do seu modelo em um grupo mais extenso.

Como já mencionamos anteriormente, os diagramas de classes são usados para realizar uma modelagem da visão estática. Você pode perceber que essa visão disponibiliza um auxílio aos atributos funcionais de um sistema que estabelece serviços destinados aos usuários finais. Segundo Booch et al. (2012), esta visão estática atribuída à modelagem de um sistema indica que os diagramas de classes podem ser utilizados para algumas atividades, tais como:

- A modelagem do vocabulário de um sistema: define o número de abstrações que pertencem ao sistema analisado e as que se encontram fora dele. Daí a importância em utilizar os diagramas de classes para determinar as abstrações e as suas respectivas obrigações;
- A modelagem de colaborações simples: é necessário compreender que uma colaboração consiste em um agrupamento composto por classes, interfaces e componentes que atuam conjuntamente para possibilitar algum comportamento cooperativo maior do que a soma de todos os elementos;
- Modelagem do esquema lógico de um banco de dados: basta pensar de modo análogo a um projeto elaborado dentro de uma base de dados. Em situações como esta, é natural arquivar informações persistentes em um banco de dados orientado a objetos, por exemplo. Nesse contexto, é possível desenvolver a modelagem de esquemas por meio dos diagramas de classes.



Communication diagram

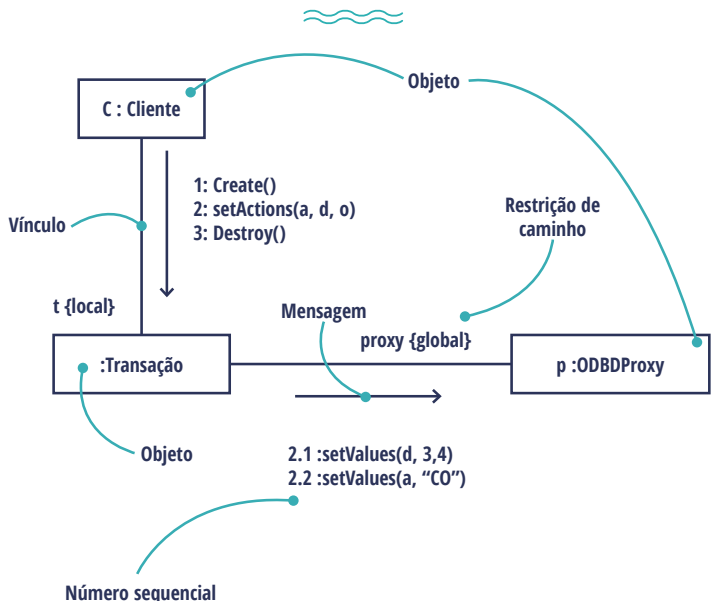


Quando nos referimos aos diagramas de comunicação, é preciso entender que esses modelos de diagramas pertencem aos denominados diagramas de interação, juntamente com o diagrama de sequência. É necessário observar que esses diagramas são usados em uma UML com o objetivo de auxiliar na modelagem direcionada aos aspectos dinâmicos presentes em um sistema.

Devemos entender que um diagrama de comunicação tem como foco a organização dos objetos que estão inseridos em uma interação. Podemos observar o diagrama de comunicação a seguir que, por sua vez, é desenvolvido ao se inse-

rir, primeiramente, os objetos que se apresentam como vértices de um gráfico. Posteriormente, é preciso simbolizar os vínculos que interligam esses objetos na condição de arcos do gráfico. Importante frisar, também, que estes vínculos podem apresentar nomes de papéis como uma maneira de identificação. Ao final, é preciso concretizar esses vínculos através de mensagens enviadas e recebidas pelos objetos. Você irá notar que essa sequência apresenta uma indicação visual referente ao fluxo de controle presente dentro do contexto da organização estrutural dos objetos colaboradores.

DIAGRAMA 7. UM DIAGRAMA DE COMUNICAÇÃO



Fonte: BOOCH et al, 2012. (Adaptado).

Os diagramas de comunicação apresentam características que os diferenciam dos diagramas de sequências, que são utilizados para criar uma modelagem de interações entre os elementos do sistema, mesmo que os agentes externos passem a ser incluídos (SOMMERVILLE, 2011). A primeira distinção se refere ao caminho, pois, primeiramente, existe o caminho que corresponde a uma associação estabelecida. Outros caminhos correspondentes podem ser evidenciados, como, por exemplo, as variáveis locais e globais. Vale frisar que um caminho simboliza a maneira de obter o conhecimento utilizado em um objeto.

A segunda distinção está relacionada ao número de sequência. Para que isso fique mais claro, é preciso notar que, para indicar a ordem temporal presente em uma mensagem, será preciso que o usuário utilize um número na condição de um prefixo da mensagem, por exemplo, “mensagem 1”. Posteriormente, se eleva de forma unitária para cada mensagem nova dentro de um fluxo de controle, por exemplo, “mensagem 2”, “mensagem 3” e assim sucessivamente. Ao demonstrar um aninhamento, uma opção viável é a utilização de uma numeração decimal de Dewey, ou seja, aquele modelo de mensagem onde a primeira mensagem é classificada como 1, a segunda 1.1, que estará aninhada à mensagem 1, e assim sucessivamente.

O usuário pode apresentar o aninhamento até um nível arbitrário de profundidade. Você, caro leitor, irá observar que, dentro de um mesmo vínculo, diversas mensagens poderão ser apresentadas e remetidas de direções distintas, com a possibilidade de cada uma apresentar um número de sequência unificado.

Durante uma boa parte do tempo é possível realizar uma modelagem de fluxos onde os controles são diretos e sequenciais. Porém, também existe a possibilidade também de se desenvolver uma modelagem de fluxos com um nível maior de complexidade, o que certamente irá envolver aspectos ligados à iteração e ramificação.

Uma iteração consiste em uma sequência de mensagens repetidas. Sendo assim, para a realização de uma modelagem de iteração, é necessário utilizar uma expressão de iteração, para que o mesmo exerça a função de prefixo do número de sequência de uma mensagem. Assim, podemos concluir que uma iteração aponta que a mensagem comum, ou aninhada, será repetida em paralelo com a expressão disponibilizada.

Similarmente, uma condição simboliza uma mensagem onde a sua execução é aleatória à análise de uma condição booleana. Para realizar a modelagem de uma condição, é preciso utilizar uma expressão condicional como prefixo do número de sequência de uma mensagem. É preciso compreender, também, que os caminhos alternativos de uma ramificação apresentam a mesma quantidade de sequência, porém cada caminho deve expor uma única diferenciação, onde não exista uma sobreposição entre as condições.





Component diagram



O component diagram, ou diagrama de componentes, é considerado um dos tipos de diagramas direcionados para formar a modelagem que envolve aspectos físicos de um sistema orientado à objetos. É preciso compreender que ele demonstra basicamente dois aspectos importantes dentro de um conjunto de componentes: 1) a organização; e 2) as dependências presentes.

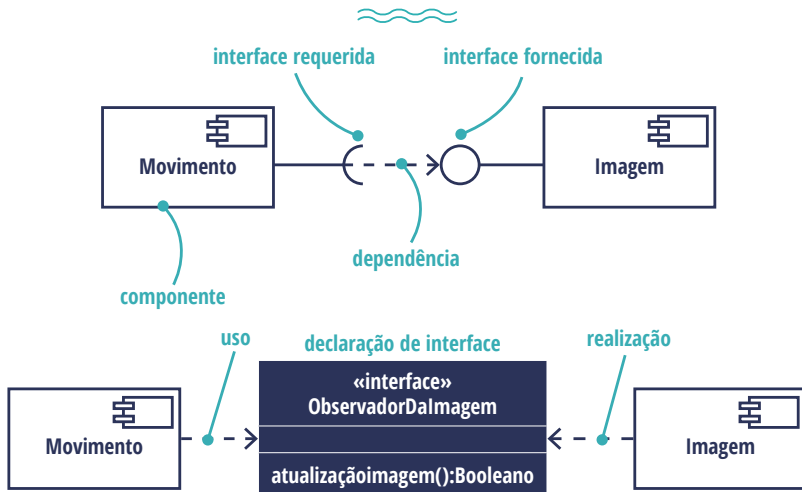
Vale frisar que os diagramas de componentes são utilizados para auxiliar na modelagem que utiliza uma visão estática adequada para a implantação de um determinado sistema. E como isso ocorre? Bem, é necessário utilizar uma modelagem de componentes físicos que se localizam em um nó, como tabelas ou documentos, por exemplo. Eles são considerados, basicamente, como diagramas de classes que têm como foco os componentes pertencentes a um sistema. Assim como ocorre em outros diagramas, o diagrama de componentes também é utilizado para criar sistemas executáveis através da engenharia de produção e reversa.

Vamos compreender melhor esse diagrama voltando ao exemplo da casa. A construção de uma casa vai além do desenvolvimento de projetos. Esses projetos são essenciais para auxiliar na visualização, determinação e documentação do modelo de casa que se objetiva construir. Posteriormente, as plantas arquitetadas darão espaço aos componentes que darão forma e estrutura a esta casa. Mas, e o software?

Bem, quando nos referimos ao software, é preciso desenvolver diagramas referentes aos casos de uso para idealizar sobre a performance almejada pelo sistema. Posteriormente, é necessário determinar o vocabulário do domínio por meio de diagramas de classes. Em seguida, desenvolve-se alguns diagramas, como o diagrama de sequência, por exemplo, para determinar a maneira como os elementos presentes em seu vocabulário irão atuar conjuntamente para realizar a execução dessa performance. De maneira eventual, é preciso mudar os projetos lógicos desenvolvendo componentes do ponto inicial, reutilizando os componentes mais antigos de formas mais renovadas.

A partir de uma UML, este diagrama, assim como os anteriores, é utilizado para observar o aspecto estático pertencente aos componentes físicos e às suas relações, a fim de determinar aspectos referentes à construção.

DIAGRAMA 8. DIAGRAMA DE COMPONENTES E INTERFACES



Fonte: BOOCH et al, 2012. (Adaptado).

Os diagramas de componentes geralmente apresentam aspectos como: componentes interfaces e relacionamentos de dependência, associação, generalização e realização. Eles também apresentam um conjunto de notas e restrições. É importante frisar que os diagramas de componentes apresentam uma série de pacotes, ou até mesmo subsistemas, usados para unir itens pertencentes ao modelo dentro de agrupamentos mais extensos. Esporadicamente, o usuário provavelmente vai incluir instâncias dentro dos diagramas de componentes, em especial quando houver o desejo de observar a instância de uma família de sistemas cuja referência se encontra nos componentes.

Esses diagramas são utilizados para estabelecer a modelagem da visão estática em relação à implementação de um sistema, como já observado em outros tipos de diagramas. O que podemos notar é que essa perspectiva consegue oferecer auxílio ao gerenciamento realizado na configuração das etapas pertencentes a um sistema, constituídas pelos componentes agrupados de diversas maneiras para elaborar um sistema em execução. A utilização dos diagramas de componentes pode ocorrer basicamente de quatro formas. São elas:

1. Na modelagem do código fonte: assim como ocorre em algumas linguagens de programação orientadas a objeto, é possível destacar o código usando

áreas de desenvolvimento que se caracterizam pela integração que arquiva o código-fonte. Sendo assim, os diagramas de componentes conseguem ser empregados para a modelagem de gerenciamento de configuração referente a esses arquivos, que simbolizam elementos do produto do trabalho;

2. Na modelagem de versões do tipo “executáveis”: em relação aos componentes, uma versão tem, como foco, atingir as áreas necessárias para liberar o sistema com execução. Ao realizar a modelagem de uma versão utilizando diagramas de componentes, é possível observar e até documentar decisões referentes às partes físicas que formam um software;

3. Na modelagem de bancos de dados físicos: os esquemas presentes em um banco de dados físico disponibilizam uma API com o objetivo de armazenar informações persistentes. É importante salientar que o modelo de banco de dados físico simboliza os arquivamentos dessas informações, por exemplo, em páginas de um banco orientado a objetos. Cabe aos diagramas de componentes simbolizar diversos modelos de banco de dados físicos;

4. Na modelagem de sistemas adaptáveis: determinados sistemas se caracterizam pelo perfil quase estático, pois seus componentes são inseridos, executados e, posteriormente, desaparecem. Em contrapartida outros sistemas se caracterizam por conseguirem ser mais dinâmicos e

integrar agentes móveis, ou até mesmo componentes que se transferem, com o objetivo de estabelecer uma carga de trabalho equilibrada e impedir o surgimento de falhas. Dentro desse contexto, os diagramas de componentes são usados conjuntamente com determinados digramas UML direcionados para auxiliar na modelagem de performance e simbolizar esses tipos de sistemas.





Composite structure diagram

Podemos observar que, nos modelos UML, os diagramas de estrutura composta apresentam a estrutura interna que contém os denominados classificadores estruturados. Esses classificadores essencialmente usam peças, portas e conectores. Vale ressaltar que um classificador estruturado é responsável por estabelecer a implantação de um classificador e pela inclusão, dentre outros elementos, de uma classe, um componente ou até mesmo um nó utilizado na implementação. Você pode adotar o diagrama de estrutura composta com o intuito de apresentar os detalhes internos pertencentes a um classificador e expor os objetos e funções que atuam conjuntamente para realizar o desempenho do classificador contido.

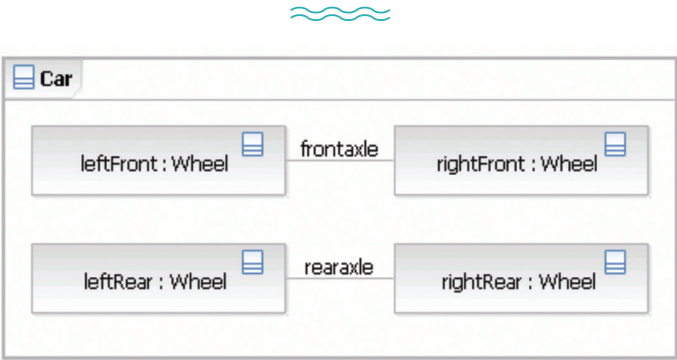
Um aspecto relevante a ser observado se dá pelo fato de o diagrama de estrutura composta apresentar características semelhantes a um diagrama de classe. Porém, esse modelo de diagrama simboliza peças individualizadas no lugar de classes inteiras. No momento que antecede a definição da estrutura interna de um classificador, é preciso realizar basicamente duas ações: 1) demonstrar seu compartimento de estrutura, ou 2) realizar a abertura de um diagrama de estrutura composta. E o que isso pode indicar? Graças a essas ações, é possível definir um modelo para as peças que simbolizam as instâncias contidas em um classificador. Além disso, é possível acrescentar conectores para estabelecer um vínculo entre diversas peças dentro de uma relação que envolve associação ou dependência.

Outro aspecto que devemos tratar se refere às portas que estabelecem o ponto em que interage um classificador com o seu ambiente de desenvolvimento ou com suas peças internas. É possível, também, usar uma porta para determinar as ações que um classificador disponibiliza e deseja do seu ambiente.

É possível estabelecer um modelo para as colaborações e o seu uso dentro dos diagramas de estrutura composta. E o que isso pode indicar? Bem, isso implica afirmar em que uma colaboração expõe os papéis e os requisitos que estabelecem uma performance mais específica de um classificador. O uso de colaboração trata da utilização especial da colaboração para esclarecer a relação existente entre as propriedades de um classificador. Com o objetivo de reconhecer as peças no uso de colaboração, é preciso unir o seu uso com a colaboração em si para, posteriormente, acrescentá-la em um diagrama de estrutura composta.

A seguir, o diagrama de estrutura composta apresenta, por meio de um editor de diagrama, o nome do classificador contido:

DIAGRAMA 9. DIAGRAMA DE ESTRUTURA COMPOSTA



Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).

Por meio desse diagrama é possível identificar a maneira como o diagrama de estrutura composta **reconhece o classificador contido**, Car. Nesse exemplo, a estrutura de diagrama apresenta peças de material composto internas do classificador contido, simbolizando as rodas de um automóvel. Um link de comunicação estabelece uma conexão entre as rodas por meio dos conectores frontaxle e rearaxle.



EXPLICANDO

Ao desenvolver um diagrama de estrutura formado partindo de um classificador Carro, quatro instâncias da classe Roda serão desenvolvidas, onde essas peças são estabelecidas pela composição interna existente na instância Carro e nas rodas vinculadas por meio dos conectores.

O modelo em diagramas da estrutura composta é formado por alguns componentes. Veremos detalhadamente os principais deles.

Peças

Ao longo do texto você verificou o uso intensivo do termo “peças” e, possivelmente, se questionou sobre qual o significado desse termo. Pois bem, em diagramas de estrutura composta, uma peça é um componente de diagrama que simboliza um grupo de diversas instâncias apresentado em um classifica-

dor estruturado contido. Uma peça apresenta a funcionalidade de uma instância dentro de um classificador. O usuário pode desenvolver peças dentro da estrutura de um classificador e em diversos diagramas UML.

Essas peças pertencem à composição. Além disso, o diagrama de estrutura composta define como ocorre a conexão das peças em um classificador contido. Importante frisar que cada peça representa a utilização específica de um tipo que define objetos que podem estar relacionados a uma função. Um usuário pode apresentar peças com o mesmo tipo e cada peça pode conter um grupo distinto de relacionamentos para outras peças.

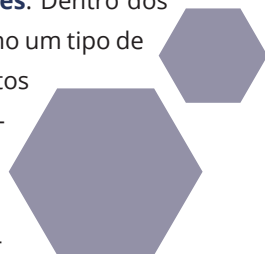
Portas

Dentro desse contexto, uma porta pode ser entendida como forma de interação estabelecida entre uma instância do classificador com o seu ambiente, ou até mesmo o comportamento do classificador e as peças internas. Vale mencionar que o ambiente externo mantém uma relação com as peças internas observadas por meio de uma porta, ou seja, por meio desse mecanismo, essas peças podem ser isoladas de um objeto dentro do seu ambiente. Nesse contexto, os conectores interligam as portas com as propriedades do classificador, o que estabelece uma comunicação entre diversas instâncias.

Você, na condição de usuário desse diagrama, pode estabelecer diversas portas para um classificador com o objetivo de apresentar interações distintas a depender da porta, observando de onde se origina a sua interação. Você irá notar, portanto, que essas portas estarão nomeadas e serão exibidas dentro de um quadro de diagramas em forma de um pequeno quadrado. Além disso, você pode incluir as portas nas peças dispostas internamente nos diagramas de estrutura composta.

Colaborações

Outro aspecto importante se refere às **colaborações**. Dentro dos diagramas UML, é possível definir uma colaboração como um tipo de classificador estruturado. Sua funcionalidade e atributos atuam no sentido de auxiliar na determinação da estrutura interna pertencente a um classificador. É possível empregar uma colaboração para estabelecer as funcionalidades e conexões que são solicitadas na realização de um objetivo específico da colaboração.





DICA

Um objetivo de uma colaboração pode ser o estabelecimento das funções ou os elementos formadores de um classificador. Deixando as funções essenciais isoladas, uma colaboração pode simplificar a estrutura e definir o comportamento dentro de um modelo.

É necessário notar que as classes ou identificadores específicos das instâncias participantes não estão expostas. Isso ocorre somente em aspectos relacionados às funções e aos conectores. Devido a isso, existe a possibilidade de reutilização de uma colaboração, no sentido de estabelecer uma diagramação nos padrões estruturais de objetos, além de estabelecer uma modelagem do seu comportamento comum.

Iremos notar, também, que uma colaboração pode acrescentar classificadores de peças distintos do sistema modelado. Além disso, um único classificador pode executar diferentes funções e participar em diversas colaborações. E o que isso significa? Que uma determinada função dentro de uma colaboração se baseia ou utiliza tipos de um classificador. Entretanto, vale frisar que a colaboração não apresenta fisicamente ou contém o classificador referido.

Usos de colaboração

Mais um componente importante nesse tipo de diagramação se refere ao uso de colaboração. Ele consiste em um elemento de modelo que simboliza um uso de uma colaboração para esclarecer as relações existentes entre as partes de um classificador estruturado. Você, caro leitor, pode empregar o uso de colaboração, por exemplo, para inserir um padrão. Lembrando que um padrão é apresentado por uma colaboração para definir uma situação especial que envolve classes ou instâncias que executam as funções de colaboração determinadas. É possível ter diversos usos de colaboração, abrangendo um agrupamento distinto de funções e conectores direcionados a uma colaboração específica.

Podemos compreender que cada função de colaboração está relacionada a um elemento conectável com um classificador, em um uso de colaboração. Ou seja, é necessário compreender que ao digitar um “uso de colaboração” juntamente com uma “colaboração”, é possível abrir o uso de colaboração dentro de um diagrama de estrutura composta. Posteriormente, iremos observar as funções das partes inseridas na ocorrência.

Qualquer usuário pode acrescentar um conector de ligação de função. Mas o que seria de fato esse conector? Ele consiste em um relacionamento de dependência simples utilizado para estabelecer a ligação e o mapeamento das funções e dos conectores que auxiliam dentro de um classificador, levando em consideração a colaboração específica. É possível acrescentar uma ligação de função levando em consideração os seguintes componentes:

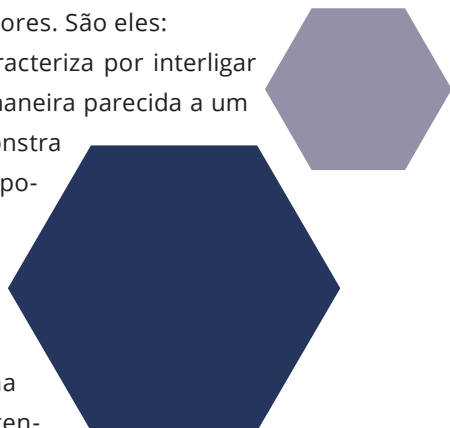
- Duas funções existentes;
- Um uso de colaboração e uma função existente;
- Uma função existente e um novo uso de colaboração;
- Um uso de colaboração existente e uma nova função.

Conectores em classificadores estruturados

Por fim, temos a definição de conectores em classificadores estruturados. Eles podem ser definidos como uma linha que simboliza um relacionamento dentro de um modelo. Vamos entender melhor: quando modelamos a estrutura interna de um classificador, é preciso usar um conector para apontar uma ligação existente e duas ou diversas instâncias pertencentes a uma peça ou porta. O conector estabelece a relação existente entre os objetos ou instâncias que são vinculadas às funções dentro de um mesmo classificador estruturado. Além disso, ele reconhece a comunicação existente entre essas funções, onde o produto gerado acaba especificando de maneira automática o tipo de conector a ser desenvolvido. Os classificadores estruturados apresentam basicamente dois tipos de conectores. São eles:

- **Um conector de montagem:** se caracteriza por interligar duas peças ou portas internas. Atua de maneira parecida a um relacionamento de associações e demonstra que uma peça da composição vincula e disponibiliza serviços que a outra peça solicita;

- **Um conector delegado:** se caracteriza por ligar uma porta dentro da estrutura externa a uma peça ou porta interna. Ele liga todas as partes com uma de suas peças, além de ser exposto contendo uma ponta de seta aberta ao fim da linha de conexão.





Deployment diagram

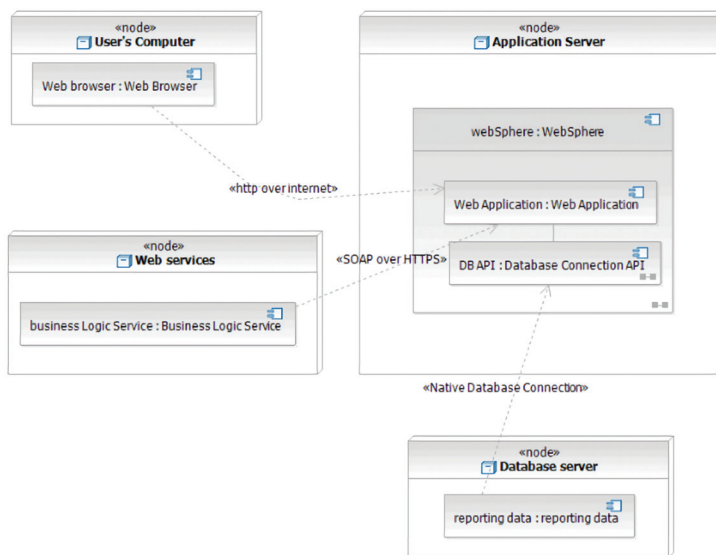


Quando nos referimos ao deployment diagram, ou diagramas de implementação, é preciso observar alguns aspectos. Dentro de uma UML, esses diagramas se caracterizam por modelar a arquitetura física de um sistema. Eles apresentam as relações existentes entre os componentes pertencentes a um software e a um hardware de um sistema com a distribuição física do processamento.

Importante mencionar um aspecto essencial: esses diagramas são conhecidos por disponibilizar certo nível de organização física dos nós dentro de um sistema distribuído. Além disso, os diagramas de implementação apresentam artefatos arquivados em cada nó, por exemplo.

Você pode estar se questionando: qual a finalidade de um nó? Bem, um nó tem, por finalidade principal, simbolizar os dispositivos de hardware e outros dispositivos responsáveis pelo suporte ao ambiente de tempo de execução dentro de um sistema. Vale observar que os caminhos de comunicação e relacionamentos de implementação têm a função de estabelecer modelos nas conexões do sistema.

DIAGRAMA 10. DIAGRAMAS DE IMPLEMENTAÇÃO



Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).

Você irá notar que os diagramas de implementação apresentam um elevado nível de eficácia que auxiliam na visualização, especificação e documentação de alguns sistemas. São eles:

- **Sistemas incorporados:** adotam o uso de um hardware. Importante mencionar que esse hardware é controlado por meio de um estímulo externo;
- **Sistemas cliente/servidor:** geralmente apresentam uma distinção entre a interface com o usuário e os dados persistentes do sistema;
- **Sistemas distribuídos:** disponibilizam diversos servidores. Além disso, conseguem armazenar uma variedade de versões de artefatos de software que podem migrar de um nó para outro.

Existe a possibilidade do uso desse tipo de diagrama na análise das implicações da distribuição e das alocações de recursos, pois os diagramas de implementação acabam se concentrando em dois aspectos relevantes: 1) a configuração dos nós de processamento de tempo de execução e 2) nos seus componentes aliados aos artefatos.

Existe uma diferença básica entre os diagramas de implementação e os diagramas de componentes. O primeiro expõe os componentes e artefatos, tendo, como referência, o local onde são usados no sistema implantado; e o segundo estabelece a composição dos componentes e artefatos no sistema.

Nos tópicos a seguir é possível descrever os elementos dos modelos presentes nos diagramas de implementação. Vejamos os principais:

Nós presentes nos modelos UML

Considerando os modelos UML, é possível entender que os nós consistem em componentes de um modelo que simbolizam os recursos computacionais de um sistema. Eles podem ser interligados por meio de caminhos de comunicação, com o objetivo de expor estruturas de rede. Importante frisar que os nós podem conter nós aninhados, além de possuírem artefatos implementados dentro deles.

Geralmente um nó contém um nome que expõe a peça de hardware que ele simboliza. Dentro dos diagramas, as divisões apresentam informações referentes aos requisitos, aos elementos implantados, aos nós aninhados e a estrutura interna do nó. É preciso compreender que, à medida em que o usuário cria um software destinado a um sistema distribuído, se torna viável modelar os componentes distintos do sistema, como os nós dentro de um diagrama de implementação.

Como você deve ter observado, esses sistemas distintos de computador são simbolizados pelo uso de nós. Outro aspecto se refere aos artefatos implementados por cada nó. Eles podem ser listados dentro de um compartimento denominado implementação, ou apresentado de maneira explícita por meio de relacionamentos de implementação.

Instâncias do nó

Consiste em um componente que simboliza, dentre outros aspectos, uma instanciação de um nó. Essas instâncias se referenciam em nós já existentes. Importante lembrar que um nó é a representação de um modelo genérico de dispositivo computacional. Já uma instância de nó simboliza um nó determinado e específico dentro de um ambiente do sistema. Existe a possibilidade de uso de instâncias de nós dentro dos diagramas de implementação, cujo objetivo é simbolizar os atributos existentes no momento da execução. Um bom exemplo disso está relacionado ao uso de instâncias de nós com o objetivo de representar um servidor da web e um servidor de dados. Isso ocorre dentro de um diagrama de implementação direcionado a um aplicativo de e-commerce.

Vale salientar que as divisões apresentam informações referentes aos componentes aplicados dentro de uma instância do nó. Geralmente uma instância de nó contém um nome exclusivo padronizado em normas que apresentam termos sublinhados acrescidos do sinal dois pontos (:) e com o nome do nó, por exemplo: `NodeInstance:Node`.

Ambientes de execução

Podemos definir um ambiente de execução como um modelo de nó que simboliza uma determinada plataforma de execução. Para que você compreenda mais facilmente uma plataforma de execução, pense que ela pode ser um sistema operacional ou até mesmo um sistema voltado para o gerenciamento de banco de dados. Estas plataformas (ambientes) também podem ser usadas para apresentar qual o contexto onde a execução de um modelo vai ser realizada.

Outro aspecto a ser mencionado se refere ao fato de os ambientes de execução geralmente fazerem parte de outro nó responsável pela modelagem do hardware computacional dentro de um sistema. Imagine, por exemplo, uma plataforma de execução dentro do processador de um servidor poder disponibilizar os serviços do nível do sistema operacional solicitados com o objetivo de suportar um aplicativo de banco de dados inserido nele. As divisões rea-

lizadas expõem as informações referentes aos requisitos, aos componentes implantados, aos nós aninhados e à estrutura interna referente à plataforma de execução.

Artefatos

Os artefatos são componentes pertencentes a um modelo que simboliza as entidades físicas dentro de um sistema de software. Essas entidades (unidades) físicas de execução podem ser, por exemplo, os arquivos executáveis ou as bibliotecas. Os artefatos normalmente são usados em diagramas de implementação, entretanto, é possível utilizá-los em diagramas de componentes para demonstrar os elementos pertencentes a um modelo exposto no artefato. É importante lembrar que os elementos de modelo podem ser observados em uma variedade de artefatos.

Você deve compreender que os artefatos são implantados em forma de nós e determinam as unidades físicas de informações produzidas ou utilizadas na realização e operação de um sistema. É possível oferecer suporte aos artefatos por meio da implementação de vários modelos de nós. Você observará que as divisões apresentam informações referentes aos requisitos e às operações do artefato dentro dos diagramas. Vale frisar que um artefato contém um nome específico que apresenta o arquivo ou o elemento de software simbolizado por ele.

Instâncias do artefato

Dentro de uma modelagem UML, a instância de artefato pode ser definida como um elemento de modelo que indica, dentre outros aspectos, a realização de um artefato. Essas instâncias se referenciam nos artefatos já existentes. Devemos observar que instâncias distintas de um artefato podem ser implantadas em diversas instâncias de nós. Além disso, cada instância de artefato contém valores de propriedades de maneira distinta.



Dispositivos

Consiste em um tipo de nó que indica um recurso computacional físico dentro um sistema. Um servidor de aplicativos é um exemplo desse recurso. Importante ressaltar que um dispositivo pode desencadear em outros dispositivos. Existem diferenças sutis entre um dispositivo e um nó, entretanto, essa distinção pode ser mais perceptível dentro de um perfil que determina os modelos exclusivos de dispositivos dentro de um ambiente específico.

Especificações de implementação

De maneira conceitual, uma especificação de implementação pode ser definida como um arquivo de configuração (entenda o arquivo de configuração como um documento XML ou até mesmo um arquivo de texto) que determina como um artefato é implantado dentro de um nó. Essa especificação indica as propriedades que estabelecem os limites de execução de um elemento ou até mesmo um artefato implantado em um nó. Esses limites (parâmetros) podem expressar alguns atributos determinados como: coincidência, execução e alternativas exclusivas da transação.

Relacionamentos em diagramas de implementação

Dentro de uma UML podemos conceituar um relacionamento como uma conexão estabelecida entre os elementos de modelo, que inclui uma semântica dentro de um modelo. Entenda que essa semântica estabelece certo nível de estrutura e o comportamento entre os componentes de modelo. Segundo informações trazidas pelo site da IBM, os relacionamentos UML são agrupados da seguinte maneira:

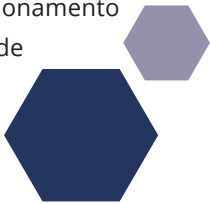


TABELA 1. RELACIONAMENTOS EM DIAGRAMAS DE IMPLEMENTAÇÃO

Categoria	Função
Linhas de atividade	Representam o fluxo entre atividades.
Associações	Indicam que as instâncias de um elemento de modelo estão conectadas a instâncias de outro elemento de modelo.
Dependências	Indicam que uma alteração em um elemento de modelo pode afetar outro elemento de modelo.
Generalizações	Indicam que um elemento de modelo é uma especialização de outro elemento de modelo.
Realizações	Indicam que um elemento de modelo fornece uma especificação que outro elemento de modelo implementa.
Transições	Representam alterações no estado.

Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).



Interaction overview diagram

A interaction overview diagram, ou um diagrama de interação, é formado por um conjunto de objetos e suas relações definidas, que podem ser, por exemplo, a inserção de mensagens enviadas e recebidas entre eles. Diante do conteúdo exposto é possível que você se questione sobre as diferenças existentes entre os diagramas de sequências e comunicação. O primeiro se caracteriza por definir um nível de organização (ordenação) temporal das mensagens remetidas, e o segundo foca na organização estrutural dos objetos, responsáveis pelo envio e recebimento de mensagens.

Um diagrama de interação é responsável por desenvolver uma modelagem dos aspectos dinâmicos inseridos em um sistema. Mas o que de fato essa modelagem informa? Bem, durante certo período, isso vai abranger uma determinada modelagem denominada instâncias concretas. Segundo Booch et. al (2012), além das instâncias concretas, é possível observar as classes, interfaces, componentes e nós em paralelo às mensagens que são estabelecidas entre si, inseridos em um cenário que define um determinado desempenho.

Os diagramas de interações podem ser visualizados isoladamente para o processo de especificação, desenvolvimento e documentação da dinâmica de um conjunto de objetos. Porém, eles também são úteis na modelagem do fluxo de controle inseridos no caso de uso. Vale frisar que esses diagramas também são extremamente importantes no desenvolvimento de sistemas executáveis por meio da engenharia direta e reversa, por exemplo.

Para que fique mais compreensível, vamos utilizar como exemplo um filme projetado. Ao visualizarmos um filme projetado em um cinema ou na televisão, não se visualiza o movimento contínuo característico das ações ao vivo, e sim um conjunto de imagens estáticas inseridas de forma rápida, causando a impressão de movimento contínuo. Os responsáveis pela produção do filme utilizam essa técnica, porém com um índice de fidelidade reduzido. Isso implica dizer que o desenvolvimento do roteiro é considerado a atividade mais importante referente à produção, auxiliando os componentes de uma equipe a observar, determinar, construir e, principalmente, documentar o modelo do filme, desde a sua concepção até a apresentação.

De forma análoga, você irá observar que a modelagem de sistemas complexos de software apresenta uma situação similar onde é possível realizar alguns questionamentos, do tipo, como será realizada a modelagem no que se refere aos aspectos dinâmicos? Idealize um sistema em execução e se, por ventura, você apresentar um depurador interativo ligado ao sistema, é possível visualizar uma seção da memória e verificar como ela altera o seu conteúdo durante um período. Se o usuário mantiver o foco, ele poderá observar diversos objetos de interesse. Durante certo tempo, é possível desenvolver em determinados objetos, as mudanças dos valores dos requisitos e o descarte de alguns deles.

Importante ressaltar que o valor atribuído à observação dos aspectos dinâmicos de um sistema ocorre de maneira bastante restrita ao sistema distribuído, com diversos fluxos de controle aplicados aos concorrentes. Uma maneira mais adequada de realizar a modelagem dos aspectos dinâmicos dentro do sistema ocorre quando desenvolvemos roteiros de cenários criando um nível de interação de determinados objetos de interesse e as mensagens enviadas e recebidas entre eles. É preciso compreender que em uma UML, a modelagem adotada por esses roteiros é realizada por meio do uso de diagramas de interação.

No diagrama a seguir, é possível verificar que esses roteiros podem ser desenvolvidos por meio da ordenação das mensagens (que ocorrem temporariamente) ou através do foco nas relações estruturais estabelecidas entre os objetos que adotam uma interação entre si. De qualquer forma, os diagramas acabam se equivalendo de maneira semântica, e existe a possibilidade de conversão um no outro, sem que as informações sejam extraviadas.

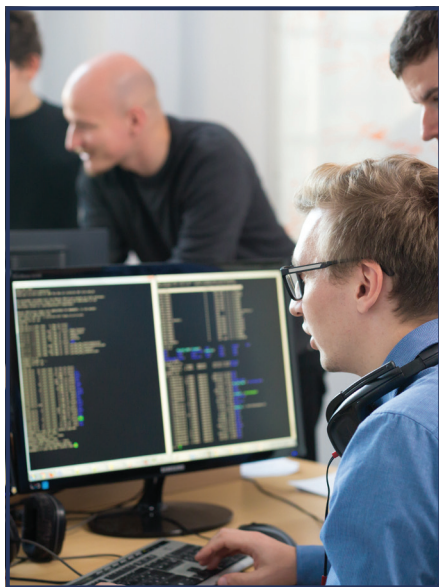
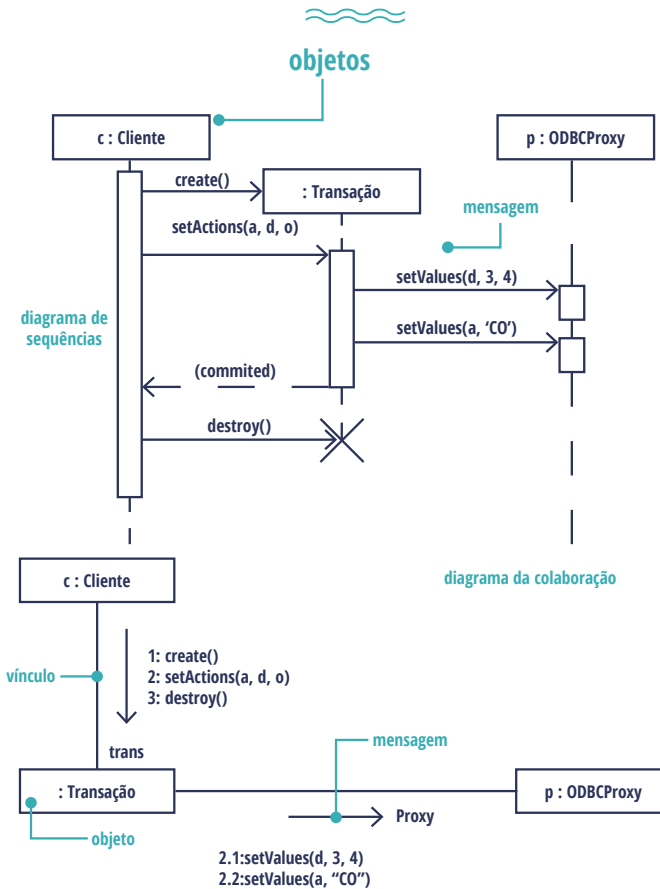


DIAGRAMA 11. UM DIAGRAMA DE INTERAÇÃO



Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).

Você pode notar que um diagrama de interação demonstra a interação formada por meio de um agrupamento de objetos e as suas relações. Isso inclui, por exemplo, as mensagens remetidas e recebidas entre eles. Ele se caracteriza por ser um modelo peculiar de diagrama e divide propriedades comuns pertencentes aos outros diagramas diferenciando-se apenas pelo seu conteúdo privativo. Importante deixar claro que eles podem apresentar notas e limitações.

Os objetos podem ser desenvolvidos ao longo da interação, pois as suas linhas de vida começam a partir do destinatário da mensagem (`create`) e do destinatário da mensagem (`destroy`).

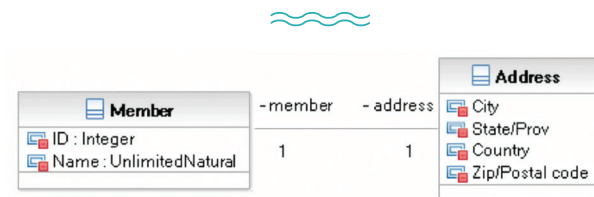


Object diagram

O object diagram, ou diagramas de objetos, se caracterizam por disponibilizar uma forma instantânea de capturar instâncias dentro um sistema e as suas relações entre as instâncias. Ao iniciar os elementos de modelos dentro de um diagrama de classe, é preciso explorar o perfil de um sistema em um determinado período.

É preciso compreender, também, que um diagrama de objetos consiste em um diagrama estrutural de UML que apresenta as instâncias dos classificadores presentes nos modelos. Eles utilizam uma notação similar aos diagramas de classe, porém os diagramas de objetos apresentam instâncias exclusivas desses classificadores e os links entre essas instâncias em certo instante. O usuário pode desenvolver diagramas instanciando os classificadores em alguns diagramas, como os diagramas de classe, implementação, componente e caso de uso. O diagrama a seguir apresenta um único diagrama de classe, formado por duas classes interligadas por meio de uma associação.

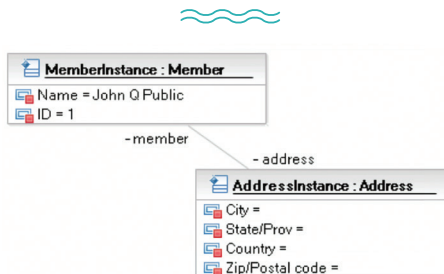
DIAGRAMA 12. ÚNICO DE DIAGRAMA DE CLASSE



Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).

O diagrama seguinte é o de objeto correspondente, que é formado por duas especificações de instâncias ligadas por meio de um relacionamento de link.

DIAGRAMA 13. DIAGRAMA DE OBJETO



Fonte: IBM, [s.d.]. (Adaptado).

É preciso ressaltar que os diagramas de objetos são extremamente úteis em algumas situações específicas, são elas:

- Ao longo da etapa de avaliação de um projeto é possível que você desenvolva um diagrama de classes para apresentar aspectos relacionados à estrutura de um sistema. Simultaneamente, é preciso criar um agrupamento de diagramas de objetos no intuito de verificar a precisão e a integridade do diagrama de classes;
- Antes do desenvolvimento de um diagrama de classes é preciso desenvolver um diagrama de objetos para buscar fatos referentes aos componentes de modelos exclusivos que ilustram exemplos determinados de classificadores solicitados.

Especificações de instâncias na UML

As especificações de instâncias são componentes que simbolizam uma instância presente em um sistema moldado, presente nos modelos UML. À medida que um usuário instancia um classificador dentro de um modelo, a especificação de instância desenvolvida simboliza uma entidade no sistema modelado em certo período, similar a uma busca instantânea da entidade. É importante frisar que você, caro leitor, pode estabelecer uma modelagem para realizar mudanças nas entidades com o tempo, desenvolvendo diversas especificações de instância. As especificações de instâncias incluem algumas informações referentes à entidade, são elas:

- A classificação referente à entidade ocorre através de um ou mais classificadores, onde a entidade citada é uma instância;
- Uma especificação de instância onde o classificador consiste em uma classe, apresenta um objeto referente àquela classe. Quando o classificador é uma associação, esta especificação apresenta um link referente a essa associação;
- Os valores determinados, referentes aos recursos estruturais da entidade, são indicados através de slots. Assim como ocorre com os classificadores, as especificações de instância têm requisitos atribuídos a esses slots.

Importante que você saiba que uma especificação de instância pode apresentar um slot para cada recurso estrutural de seu classificador. Isso significa que você ou qualquer outro usuário pode determinar valores para cada slot dentro de uma especificação de instância, pois o slot pode definir um tipo válido.

Relacionamentos de link na UML

Você pode se questionar o que de fato é um relacionamento de link. Pois bem, trata-se de uma instância referente a um caminho de associação ou co-

municação. Uma associação é definida como um relacionamento que envolve a presença de dois classificadores. Já um link consiste em uma relação que envolve objetos, instâncias de classificadores ou nós. Também podemos definir o relacionamento de link como uma instância de um caminho de comunicação definido entre dois nós.

Relacionamentos de dependência

Um relacionamento no qual um elemento (cliente) utiliza outro elemento (fornecedor). Pode ser utilizado na maioria dos diagramas utilizados, especialmente os diagramas de casos de uso, onde a mudança para o fornecedor implica em uma alteração para o cliente. Este modelo de relacionamento também pode ser usado para que um elemento de modelo preceda o outro. Normalmente os relacionamentos de dependência não apresentam nomenclatura. Veremos, na tabela a seguir, alguns tipos de dependência:

TABELA 2. TIPOS DE DEPENDÊNCIA



Tipo de dependência	Palavra-chave ou estereótipo	Descrição
Abstração	«abstraction», «derive», «refine» ou «trace»	Relaciona dois elementos de modelos ou conjuntos de elementos de modelos que representam o mesmo conceito em níveis diferentes de abstração ou de pontos de vista diferentes
Ligação	«bind»	Conecta argumentos de modelo a parâmetros de modelo para criar elementos de modelos a partir de modelos.
Realização	«realize»	Indica que o elemento de modelo do cliente é uma implementação do elemento de modelo do fornecedor, e que o do fornecedor é a especificação.
Substituição	«substitute»	Indica que o elemento de modelo do cliente substitui o do fornecedor; o elemento de modelo do cliente deve estar em conformidade com o contrato ou interface que o elemento de modelo do fornecedor estabelece.
Uso	«use», «call», «create», «instantia-te» ou «send»	Indica que um elemento de modelo requer outro para sua implementação ou operação completa.

Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).

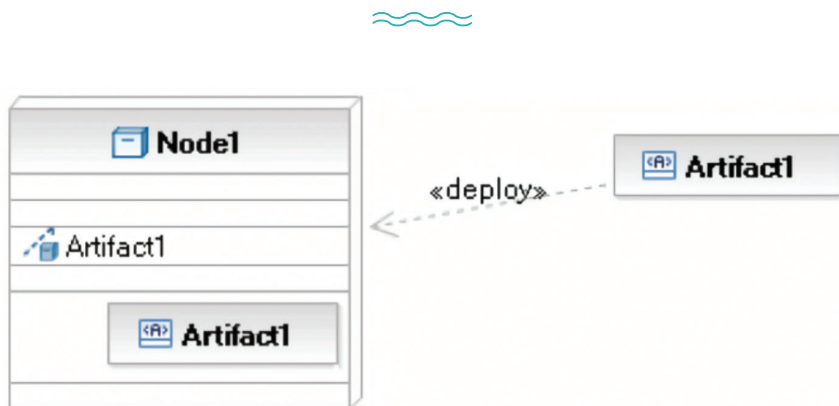
Ao tratarmos dos relacionamentos de dependência em modelos, é preciso compreender que eles podem realizar os seguintes objetivos:

- Elas podem estabelecer uma conexão de dois pacotes para mostrar que pelo menos um componente pertencente ao cliente depende de outros componentes pertencentes ao fornecedor, porém, o relacionamento de dependência não aponta que todos os elementos do cliente serão, de fato, dependentes;
- Uma conexão entre duas classes pode estar em um nível elevado de abstração em comparação a um relacionamento de associação;
- Estabelecer uma conexão de componentes das interfaces em relação a outros componentes pode demonstrar que eles utilizam diversas operações que a interface determina, ou que eles necessitam do outro componente ao longo da compilação.

Relacionamentos de implementação

Os relacionamentos de implementação determinam que um tipo de nó suporta a inserção de um tipo de artefato. Normalmente, os relacionamentos de implementação apresentam nomenclatura. O diagrama a seguir apresenta uma sequência onde um nó denominado Node1 é ligado a um artefato denominado Artifact1 por meio de um relacionamento de implementação. Isso é apresentado como uma linha desenhada em uma seta aberta ao longo da extremidade.

DIAGRAMA 14. RELACIONAMENTOS DE IMPLEMENTAÇÃO



Fonte: IBM. Data de acesso: 2 dez. 2019. (Adaptado).



Verificamos, ao longo da unidade, que os diagramas de atividades estão disponíveis dentro da UML auxiliando na modelagem de aspectos dinâmicos de sistemas formados por meio de um gráfico de fluxo de controle de uma determinada atividade para outra.

O diagrama de classes é visualizado com certa frequência na modelagem de sistemas direcionados a objetos. Eles apresentam uma série de classes, colaborações, relacionamentos e interfaces.

Quando se trata dos diagramas de comunicação, observamos que eles pertencem aos diagramas de interação, assim como os diagramas de sequência. Eles são usados para ajudar na modelagem dos aspectos dinâmicos presentes em um sistema.

Já os diagramas de componente auxiliam na modelagem que envolve aspectos físicos de um sistema orientado à objetos. Esse diagrama se refere a alguns aspectos importantes como a organização e as dependências presentes dentro de uma série de componentes.

Os diagramas de estrutura composta se caracterizam por apresentar uma estrutura interna que apresenta os classificadores estruturados responsáveis por determinar a inserção de um classificador e uma classe, um componente ou até mesmo um nó empregado na implementação.

Observamos, também, que os diagramas de implementação se caracterizam por modelar a arquitetura física de um sistema, apresentando as relações existentes entre os componentes de um software e hardware dentro de um sistema com a distribuição física do processamento.

Apresentamos o diagrama de interação que é composto por um conjunto de objetos e suas relações estabelecidas, verificando as diferenças existentes entre os diagramas de sequências e comunicação.

Por fim, os diagramas de objetos se caracterizam por apresentar uma maneira imediata de capturar instâncias em um sistema e o relacionamento entre as instâncias. Ao iniciar os componentes de modelos em um diagrama de classe é preciso explorar o comportamento de um sistema em um determinado período.

Referências bibliográficas



BOOCH, G. et al. **UML**: guia do usuário. Tradução de Fábio Freitas da Silva. Rio de Janeiro: Campus, 2000.

_. **UML**: guia do usuário. Tradução de Fábio Freitas da Silva e Cristina de Amorim Machado. Rio de Janeiro: Elsevier, 2012.

IBM (IBM KNOWLEDGE CENTER) **Diagramas de estrutura composta**. Disponível em: <https://www.ibm.com/support/knowledgecenter/pt-br/SS5JSH_9.5.0/com.ibm.xttools.modeler.doc/topics/ccompstruc.html>. Acesso em: 2 dez. 2019.

_. **Diagramas de implementação**. Disponível em: <https://www.ibm.com/support/knowledgecenter/pt-br/SS4JE2_7.5.5/com.ibm.xttools.modeler.doc/topics/cdepd.html>. Acesso em: 2 nov. 2019.

_. **Instances specifications in UML**. Disponível em: <https://www.ibm.com/support/knowledgecenter/bg/SS8PJ7_8.5.1/com.ibm.xttools.modeler.doc/topics/cinstancespec.html>. Acesso em: 2 dez. 2019.

_. **Link relationships in UML**. Disponível em: <https://www.ibm.com/support/knowledgecenter/bg/SS8PJ7_8.5.1/com.ibm.xttools.modeler.doc/topics/clink.html>. Acesso em: 2 dez. 2019.

SOMMERVILLE, I. **Engenharia de software**. Tradução de Ivan Bosnic e Kalinka Gonçalves. Revisão técnica de Kechi Hiramã. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

UNIDADE

4

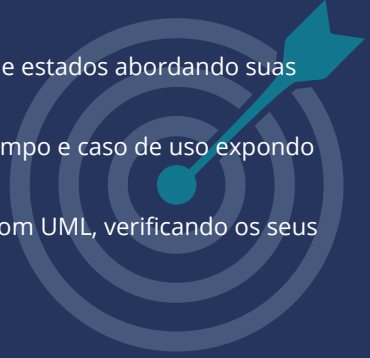
O USO DOS
DIAGRAMAS UML
E A ENGENHARIA
REVERSA



ser
educacional

Objetivos da unidade

- Apresentar os diagramas de pacotes, perfil e de estados abordando suas principais características e aplicações;
- Abordar sobre os diagramas de sequências, tempo e caso de uso expondo suas atribuições e funcionalidades;
- Observar os aspectos da engenharia reversa com UML, verificando os seus principais aspectos.



Tópicos de estudo

- Package diagram
- Profile diagram
- State machine diagram
- Sequence diagram
- Timing diagram
- Use case diagram
- Engenharia reversa com UML





Package diagram



Como é feita a documentação de um sistema? Quais os requisitos básicos para que isto seja realizado? Segundo o livro *UML: guia do usuário*, publicado em 2012 e escrito por Grady Booch, James Rumbaugh e Ivar Jacobson, os atos de observar, especificar, desenvolver e documentar sistemas considerados de grande porte requerem uma série de manipulações elevadas de atributos, aos quais se incluem as classes, diagramas e outros elementos. O uso dos sistemas indica a necessidade de organização dos itens em agrupamentos maiores. Dentro de tal contexto em uma UML, o conceito de pacote é uma ferramenta empregada na organização de itens pertencentes a uma modelagem realizada por grupos.

Além disso, o usuário é capaz de estabelecer um controle de visibilidade desses itens, permitindo que alguns deles sejam percebidos do lado externo do pacote, enquanto outros componentes se encontram ocultos. Os pacotes são aproveitados nas visões distintas da arquitetura dentro do seu sistema. Quando têm estrutura qualificada, reúnem elementos que estão aproximados e que possivelmente irão se alterar em conjunto. Diante disso, a conclusão a que se chega é de que os pacotes bem estruturados são coesos e com baixo acoplamento, além de permitir um acesso com alto nível de controle ao conteúdo do pacote.

Para explicar o conceito de pacotes de maneira mais didática, Booch, Rumbaugh e Jacobson fazem uma analogia à construção de uma casa. Casas de cachorro, por exemplo, demandam um menor nível de complexidade. A partir do momento em que há um nível mais sofisticado de construção, como nas casas e prédios, se pode notar que tais estruturas estão inseridas em componentes maiores, como áreas públicas, em que servem para organizar os planos, mesmo não tendo qualquer relação aparente entre si.

Qualquer sistema segue este padrão de desenvolvimento. Ou seja, a única forma de decodificar um sistema complexo é agrupar as abstrações em conjuntos cada vez maiores, mas boa parte deles não são instâncias reais e só existem com o intuito de interpretar o próprio sistema.

Dentro de uma UML, o conceito de pacotes é uma série de agrupamentos que organizam um modelo para facilitar o seu entendimento, além de proporcionar o controle ao acesso dos conteúdos e às emendas dentro da arquitetura do sistema.

A Figura 1 demonstra a representação gráfica dos pacotes, na qual sua notação permite a observação dos grupos de itens que são manipulados de maneira total ou de uma forma que controle sua visibilidade e acessibilidade dos componentes individuais.

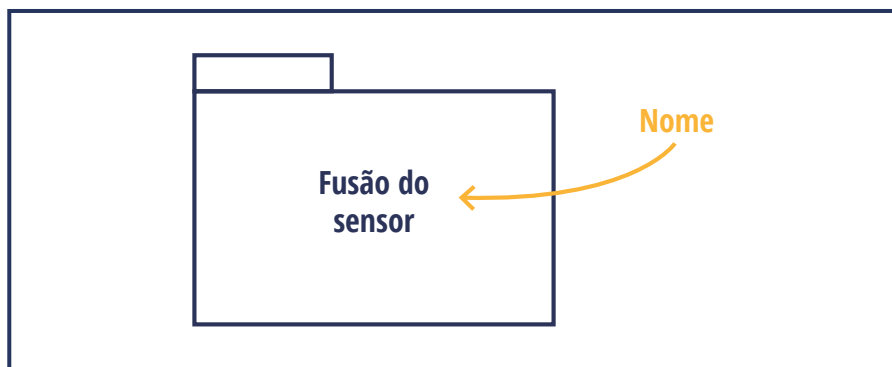


Figura 1. Pacotes. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

Booch, Rumbaugh e Jacobson sintetizam que um pacote nada mais é do que um mecanismo para organizar o próprio modelo dentro de uma hierarquia. Uma pequena advertência a ser feita é relativa ao pacote, simbolizado na condição de uma pasta se o conteúdo da pasta não for exibido ou, senão, como um guia.

Todo e qualquer pacote tem um **nome** que o distingue e que é uma sequência de caracteres de texto, sendo classificado como simples, se estiver sozinho, ou como qualificado, quando tiver como prefixo o nome do pacote que o contém.



DICA

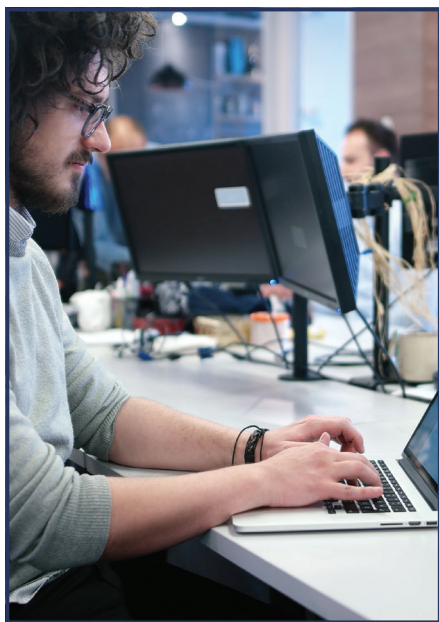
O nome de um pacote é constituído por um texto formado por letras, números e sinais diversificados, chegando a se estender por diversas linhas. De modo prático, os nomes são vistos como expressões, ou pequenos substantivos de grupos, partindo do vocabulário do modelo.

Como já mencionado, um pacote tem diversos componentes, como classes e interfaces. A propriedade de elementos, ou componentes, é considerada como uma relação composta, o que implica afirmar que os elementos estão dispostos no pacote. Se o pacote for destruído automaticamente, seus elementos também serão. Logo, cada elemento pertence apenas a um pacote.

Ainda com base nos autores citados, o pacote fornece um espaço de nome, o que significa dizer que os componentes do mesmo tipo estão de uma única maneira dentro do pacote que os contém e, por isso, os elementos de modelos distintos têm o mesmo nome dentro de um pacote. Na prática, porém, é mais adequado dar nomes únicos para os elementos em relação a todos os tipos de pacote.

Sobre os pacotes que armazenam outros pacotes, há a facilidade de decompor os modelos de maneira hierarquizada, porém é mais adequado evitar pacotes com diversos níveis de aninhamento, usando três camadas, no máximo, a fim de limitar o que será melhor. Além disso, no intuito de organizar os pacotes, se usa a importação.

Há uma espécie de semântica de propriedade que coloca os pacotes na con-



dição de um mecanismo essencial para lidar com escala. Sem ele, isto é, sem a presença dos pacotes, os principais modelos planos seriam extintos, o que obrigaria a todos os elementos à condição de receber nomes únicos, sendo uma situação impossível de ser gerida, em especial quando classes e outros elementos são criados por diversas equipes. Os pacotes têm um papel preponderante, pois controlam elementos que constituem o sistema. Com a evolução em taxas diferentes durante um período, se pode revelar explicitamente o conteúdo de um pacote na forma textual ou gráfica.

Em relação aos pacotes, é possível controlar a visibilidade dos elementos de um pacote de forma similar à visibilidade dos atributos e operações que fazem parte de uma classe. Um elemento de um pacote é público, indicando que o elemento é visível em relação ao conteúdo de qualquer pacote que realizar a importação daquele que contém o elemento. De maneira contrária, os elementos em proteção só são vistos através dos chamados “elementos-filho”, além dos elementos-privados, invisíveis ao exterior do pacote em que são declarados.

Para designar a visibilidade de um elemento que faz parte de um pacote, é recomendável empregar o nome do elemento como prefixo de símbolo de visibilidade. Dentro do contexto, os elementos públicos utilizam (+) como um prefixo que representa seus respectivos nomes. Sendo assim, e de forma coletiva, as partes públicas de um pacote formam a sua interface. Para Booch, Rumbaugh e Jacobson, assim como acontece com as classes, se pode classificar um elemento como protegido ou privado através do prefixo do nome do elemento. Os elementos são visualizados apenas pelos pacotes herdados, ou seja, eles se tornam ocultos fora do pacote, sob qualquer hipótese. A visibilidade do pacote aponta que uma classe se torna aparente em um mesmo pacote, mas é invisível para classes em pacotes distintos.

Para assimilar melhor os processos de importação e exportação, imagine duas classes, posicionadas lado a lado, sendo que a primeira vê aspectos da segunda. Não há uma necessidade específica de qualquer tipo de pacote quando há a presença de duas classes formando um sistema trivial.

Agora, imagine haver algumas centenas de classes, posicionadas lado a lado. Não há limites para desenvolver uma complexa rede de relacionamentos nem há chance alguma de se entender um grupo de classes extenso e sem organização. Como um acesso mais simplificado não cresce em escala, isto acaba se tornando um problema muito sério para os sistemas considerados de grande porte. Logo, visando a organização das abstrações, se segue algum tipo de pacote controlado.

Em uma segunda situação, suponha que foram usados dois grupos, A e B, e que eles sejam declarados como partes públicas dos seus pacotes relacionados, uma situação bastante distinta, pois, apesar de ambas serem públicas, uma não consegue acessar a outra. Entretanto, se o pacote de A realizar o processo de importação do pacote B, será possível vê-lo. A importação garante uma permissão de maneira unilateral, tendo em vista que os elementos de um pacote obtenham acesso aos elementos de pacotes distintos.

Na UML, a modelagem de um relacionamento de importação controla o acesso de um número extenso de abstrações, em que as partes públicas de um pacote são classificadas como exportações. As partes exportadas através de um pacote são visíveis para o seu conteúdo, apesar das dependências de importação e de acesso não serem transitivas.



Profile diagram



Para falar do uso do Profile diagram, ou diagrama de perfil, é preciso interpretar algumas de suas propriedades. Gilleanes Guedes, no livro *UML 2 – Guia Prático*, de 2014, o considera como um diagrama mais abstrato, uma vez que adapta uma UML a uma plataforma, como J2EE, ou até mesmo a um domínio.

É possível explicar o J2EE como a plataforma Java para a criação e realização de aplicações servidoras com capacidade de auxílio ao desenvolvimento de aplicações fortes, com uma série de serviços que disponibilizam uma funcionalidade para a criação de aplicações das multicamadas, que têm a web como referência. O objetivo é tornar simples a criação de soluções no âmbito enterprise por meio de padrões, serviços e elementos modulares. Tais componentes são configuráveis ao longo do desenvolvimento e adotam um modelo de programação em paralelo com seu contêiner.

Originalmente, a linguagem UML não foi desenvolvida para essas plataformas ou domínios, portanto não se pode exigir que a UML modele as suas características. Diante disto, através do desenvolvimento de perfis, há a oportunidade de estender a linguagem ao criar novas metaclasses e estereótipos capazes de modelar novos domínios.

Ao desenvolver um perfil, se cria uma extensão da UML em um nível mais conservador, sem grandes mudanças no metamodelo original, o que torna o diagrama de perfil um mecanismo mais leve de extensão da UML, conforme afirma Guedes. Para maior aprofundamento sobre o assunto, serão estudados os conceitos básicos de um diagrama de perfil, como modelos, metamodelos e metaclasses.

Um modelo captura uma visão do sistema físico, sendo uma abstração, ou amostra, do sistema, cujo objetivo é dar alguns aspectos estruturais ou de desempenho do software. A partir dos modelos, se determina o que é importante ou não para ser incluído.

Já um metamodelo é um modelo que fornece uma linguagem para apresentar modelos. Seu objetivo é estabelecer uma semântica para modelar elementos em um modelo em que está sendo instanciado, ou seja, o modelo é uma instância de um metamodelo, como registrado por Guedes.



```
<<metaclass>>
Actor
```

Figura 2. Metaclasse Actor. Fonte: GUEDES, 2014. (Adaptado).

A Figura 2 retrata a metaclasse Actor, que, se inserida no modelo, simboliza a instância da metaclasse. A seguir, estão outras metaclasses que são inseridas nos metamodelos. São elas:

Metaclasse Classifier

É considerada como uma metaclasse abstrata que designa uma classificação de instâncias. Ela tem uma série de instâncias em comum e, além disso, é um espaço de nomes em que os componentes acrescentam aspectos. Um classificador é um modelo com uma série de generalizações, o que torna possível decidir relacionamentos de generalização para outros classificadores.

Um classificador determina uma hierarquia de generalização através dos classificadores gerais, sendo possível redefinir classificadores aninhados por meio deste tipo de classe. Ainda que um classificador seja um mecanismo que descreva aspectos ligados ao comportamento e à estrutura, as instâncias de um classificador não são lidas como objetos, e sim como elementos UML concretos.

Behaviored Classifier

É considerado como um classificador com especificações de comportamento, como indicado no seu nome. A partir dele é que outras metaclasses, como o Actor e UseCase, são derivadas.

Namespace

É considerada como uma metaclasse abstrata que resume um componente dentro de um modelo com uma série de elementos que são reconhecidos pelo nome.

NamedElement

Representa os componentes nomeados para reconhecer o elemento identificado dentro do espaço de nome onde está demarcado. A partir desta metaclasse foram especializadas duas metaclasses específicas: Extend e Include.

DirectedRelationship

Representa uma relação entre uma coleção de componentes de um modelo-fonte e de um modelo-destino. Ela especializa as metaclasses Extend e Include, proporcionando uma herança diversificada.



State machine diagram



Os diagramas de estados estão entre os disponíveis em uma UML e na modelagem dos aspectos dinâmicos de sistemas, como afirmam Booch, Rumbaugh e Jacobson. Este tipo de diagrama mostra uma máquina de estados, como os diagramas de atividades, que representam um caso específico em que a maioria dos estados se encontra em atividade e as transições são ativadas através da finalização das atividades em estado de origem.

Os diagramas de atividades e os diagramas de estados são valiosos para uma modelagem que estipula o tempo de vida de um objeto. A distinção básica está no diagrama de atividades, cujo fluxo de controle decorre de uma atividade para outra, enquanto o diagrama de estados sujeita o fluxo de controle de um estado para outro. Portanto, os diagramas de estados são aproveitados na modelagem dos aspectos dinâmicos de um sistema.

Na maioria das vezes, isso está ligado aos modelos alusivos ao comportamento dos chamados “objetos reativos” que, para Booch, Rumbaugh e Jacobson, são objetos cujo comportamento responde aos eventos ativados de forma externa. Um objeto reativo tem tempo de vida e desempenho atual influenciado por seu comportamento passado. Os diagramas de estados são anexados a classes, casos de uso ou sistemas inteiros com o propósito de imaginar, especificar, construir e documentar a dinâmica de um objeto individual.

Os diagramas de estados não são relevantes apenas para determinar os modelos dos aspectos dinâmicos de um sistema, pois estão presentes no desenvolvimento dos sistemas do tipo “executáveis” mediante o uso da engenharia direta e reversa.

Para entender os diagramas de maneira mais simples, pense na figura de um investidor responsável pelo financiamento de um novo prédio. Ele não se ocupará em verificar detalhes do processo de construção, que são tarefas pertinentes ao executor do projeto.

Cabe ao investidor proteger o investimento contra possíveis riscos, sendo possível conceber dois perfis de investidor. O primeiro, confiante no executor do construtor, e o outro, mais pragmático e que desejará acompanhar o andamento do projeto antes da liberação do dinheiro, deliberando as fases do projeto. Ao longo do caminho, outras atividades referentes à construção serão seguidas. Apesar disso, é mais relevante para o investidor vislumbrar a fase de mudança do prédio do que as atividades executadas, cuja função pertence ao construtor.

Trazendo tal raciocínio para a modelagem de sistemas complexos de software, se percebe que a maneira mais comum para observar, definir, desenvolver e documentar o desempenho de alguns modelos de objetos consiste em realizar o fluxo de controle, transferindo de um estado para outro, a partir de um fluxograma.

Frente a isso, pense em uma modelagem do desempenho do sistema de segurança embutido doméstico, em que ele é executado de maneira contínua e reage a eventos externos. Outro aspecto trata da ordem dos eventos que altera a forma como o sistema se comporta de estado. Dentro de uma UML, de acordo com Booch, Rumbaugh e Jacobson, a modelagem de desempenhos ordenados através de eventos de um objeto é realizada pelo uso de diagramas de estados.

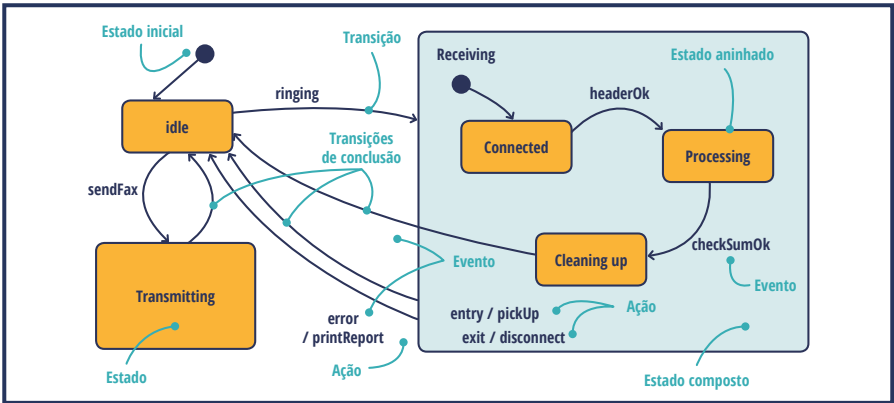


Figura 3. Diagrama de estados. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

Na Figura 3, se pode enxergar que um diagrama de estados consiste na exposição de uma máquina de estados, em que se destaca o fluxo de controle de um estado para outro, como já citado, sempre lembrando que uma máquina de estados é um perfil que sequencia os estados por onde um objeto passa ao longo do seu tempo de vida através das suas respostas direcionadas a esses eventos.

O estado é uma condição ou momento presente na vida de um objeto que atende a alguma condição, executa atividades ou espera algum evento – lembrando que um evento é uma especialização significativa de uma ocorrência que se localiza no tempo e no espaço. Trazendo tal conceito para uma máquina de estados, um evento é um estímulo que consegue ativar uma transição de estado, como salientado por Booch, Rumbaugh e Jacobson.

Uma transição é uma relação entre dois estados, apontando que um objeto no primeiro estado realiza atividades e entra no segundo estado assim que um evento específico for realizado e as condições específicas forem atendidas. Desta forma, é possível conceituar que uma atividade é uma realização não atômica em andamento dentro de uma máquina de estados. Por sua vez, uma ação é uma computação atômica executável que muda o estado do modelo ou o faz regredir a um certo valor.

Este diagrama é um modelo específico que compartilha propriedades similares nos outros diagramas. Um diagrama de estados se caracteriza por conta de seu conteúdo, sendo usado para realizar a modelagem dos aspectos dinâmicos de um sistema envolvendo o desempenho ordenado por eventos relacionados a qualquer modelo de objeto e visão da arquitetura do sistema, incluindo classes e interfaces.

O uso de um diagrama de estados é direcionado para criar modelos de aspectos dinâmicos de um sistema. É possível criar modelos dentro de uma conjuntura virtualizada de qualquer componente da modelagem, porém os diagramas de estados serão utilizados em um sistema, subsistema ou até mesmo dentro de uma classe. O usuário vincula diagramas de estados aos casos de uso e, ao realizar a modelagem dos aspectos dinâmicos, de sistema, classe ou caso de uso, pode valer-se dos diagramas de estados de maneira única.

Um objeto reativo é aquele cujo desempenho responde a eventos ativados de maneira externa, se tornando ocioso de forma típica até um evento em que sua resposta dependa de eventos anteriores. Depois de oferecer uma resposta ao evento, ele retorna ao estado ocioso, à espera da ocorrência de um próximo evento. Para tanto, é necessário focar nos estados estáveis do objeto, verificando aqueles que realizam a transição entre os estados e as ações realizadas em cada mudança de estado.



Nas modelagens de objetos reativos, se observam alguns aspectos: o uso dos diagramas de estados está relacionado à modelagem do perfil de objetos reativos em relação às instâncias de classes, casos de uso e do sistema. Se, por um lado, as interações definem o modelo de comportamento de uma sociedade de objetos atuando em conjunto, o diagrama de estados institui modelos de comportamento durante o seu período de vida. O diagrama de atividades modela o fluxo de controle na transição de uma atividade para outra, enquanto o diagrama de estados modela o fluxo de controle de um evento para outro.

Ao desenvolver modelos de comportamento de um objeto reativo, são ditados três aspectos básicos: primeiro, se especificam os **estados estáveis** em que o objeto atuará; depois, vêm os eventos que acionam a transição de um estado para outro; e, por fim, as atividades que acontecem nas alterações de estado. Esta modelagem envolve a chamada “modelagem do tempo de vida de um objeto”, iniciada junto com o objeto e seguindo até sua extinção, evidenciando os estados estáveis em que o objeto será visto.



DICA

Um estado estável simboliza a condição em que o objeto será visto durante certo período. Na existência de um evento, o objeto executará a transição de um estado para outro. Há a possibilidade de tais eventos acionarem as chamadas “autotransições” e as “transições internas”, em que a origem e a destinação da transição acontecem dentro do mesmo estado. O objeto responde acionando uma atividade, reagindo a um evento ou a uma mudança de estado,

Em concordância com o que está no livro de Booch, Rumbaugh e Jacobson, algumas ações são realizadas para modelar um objeto reativo, como:

- Escolher o contexto para a máquina de estados, seja uma classe, um caso de uso ou o sistema todo;
- Escolher os estados inicial e final do objeto. Para orientar o restante de seu modelo, estabeleça as pré e pós-condições dos estados inicial e final, respectivamente;
- Decidir os estados estáveis do objeto, levando em conta as condições que o objeto se submeterá por algum período identificável de tempo, começando com estados de nível mais alto e depois considere seus subestados;

- Decidir a ordenação parcial significativa dos estados estáveis ao longo do tempo de vida do objeto;
- Decidir os eventos que ativam a transição de um estado para outro. Faça a modelagem como a ativação de transições que passam de uma ordenação legal de estados para outra ordenação;
- Anexar ações às transições, como em uma máquina de Mealy, e/ou a esses estados, como em uma máquina de Moore;
- Considerar formas para simplificar sua máquina, a partir de subestados, ramificações, bifurcações, uniões e estados de histórico;
- Verifique se todos os estados são alcançados sob alguma combinação de eventos;
- Verifique se nenhum estado é um “buraco negro” no qual nenhuma combinação de eventos fará a transição do objeto para outro estado;
- Examine a máquina de estados manualmente ou com ferramentas para verificá-la em relação a sequências esperadas de eventos e respectivas respostas.



Sequence diagram

Quando se trata de Sequence diagram, ou diagrama de sequências, sua principal funcionalidade é a ordenação temporal das mensagens. Na Figura 5, há alguns aspectos relevantes sobre o diagrama de sequências, que é composto da inserção de objetos na interação pertencentes a um nível mais elevado do diagrama, como no chamado “eixo X”. De maneira típica, o objeto responsável pela inicialização da interação é inserido à esquerda, enquanto os objetos mais subordinados vão evoluindo à direita.

Em seguida, as mensagens enviadas e recebidas pelos objetos são dispostas no eixo Y, em uma ordem crescente ao longo do tempo, se deslocando do topo para a base, formando uma indicação visual do fluxo de controle durante um período.

Os diagramas de sequências e os de comunicação têm similaridades e diferenças entre si. Diante de tal contexto, é possível versar sobre dois aspectos que diferenciam os diagramas de sequência dos de comunicação. O primeiro alude à linha de vida do objeto, que é uma linha esboçada verticalmente trazendo a existência de um objeto em um período. Os objetos dentro de um

diagrama de interação, ao qual os diagramas de sequências fazem parte, existirão durante o mesmo período de duração da interação. Portanto, para Booch, Rumbaugh e Jacobson, tais objetos estarão dispostos de maneira alinhada na parte superior do diagrama e suas linhas de vida serão esboçadas da parte superior para a parte inferior do diagrama.

Os objetos são desenvolvidos ao longo de uma interação. Um ponto a se observar é que as linhas de vida começam com o destinatário da mensagem, cujo estereótipo é estipulado como Create. Da mesma forma, eles são extintos ao longo da interação através de um destinatário da mensagem descrito como Destroy.

Outro aspecto aborda a interação, isto é, a história de objetos individuais. Os símbolos de objetos cujos nomes estejam sublinhados serão inseridos no começo da linha da vida. Em boa parte do tempo serão apontadas as interações prototípicas, no entanto são os papéis prototípicos que expressam objetos distintos em cada etapa da interação.

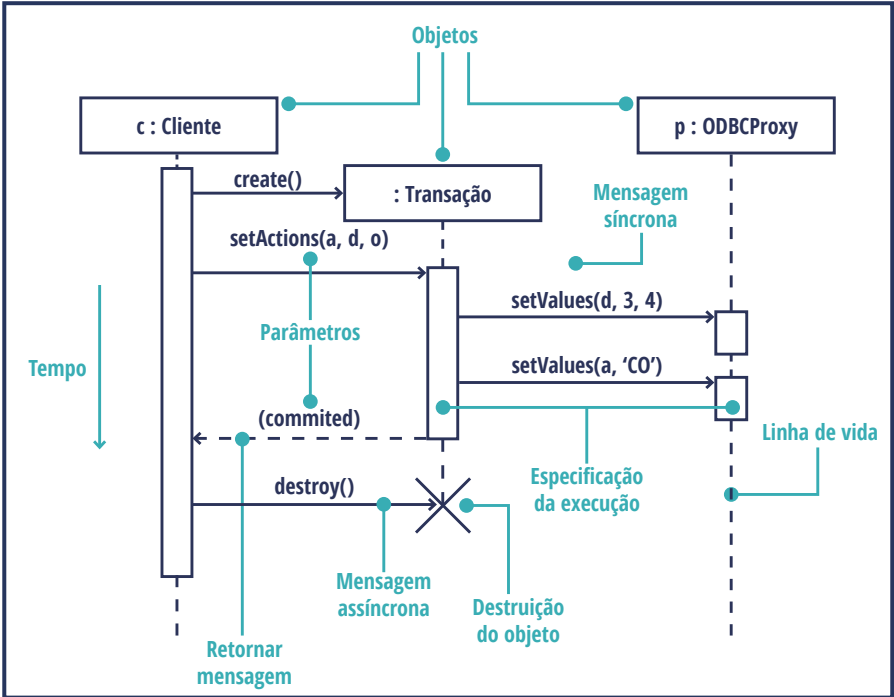


Figura 4. Diagrama de sequências. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

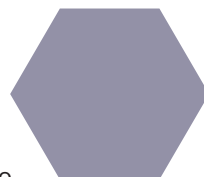
A segunda diferença entre os diagramas de sequências e de comunicação está no foco de controle. Dentro de um diagrama de sequência, o foco de controle consiste em um retângulo, de comprimento elevado e largura estreita, que apresenta o momento em que um objeto executa uma ação, de maneira direta ou através de um processo subordinado, de modo que a área superior do retângulo esteja alinhada com o começo da ação e a parte inferior com a conclusão.

O conteúdo mais relevante dentro de um diagrama de sequências é o conjunto de mensagens. Uma mensagem é exibida através de uma seta de uma linha da vida que pode ser transferida para outra, responsável por mostrar o que as mensagens receberão. Se houver uma mensagem assíncrona, a linha tem uma seta com menor espessura. Do contrário, a configuração da linha será uma seta triangular cheia. Uma mensagem síncrona, ou retorno de chamada, tem uma resposta através de uma linha esboçada por meio de uma seta mais fina. Por seu turno, a mensagem de retorno é excluída, visto que há um retorno subentendido depois de qualquer chamada.

No que diz respeito à ordenação temporal dentro de uma linha de vida única, não há um grande interesse sobre a distância exata, pois as linhas da vida se limitam a sequências relativas, o que significa afirmar que a linha da vida não é um diagrama de escala de tempo. Com base no que foi dito por Booch, Rumbaugh e Jacobson, as posições das mensagens dispostas em pares separados de linhas da vida não resultam em qualquer tipo de sequenciamento de informações. Em suma, as mensagens se sucedem em uma ordem qualquer, fazendo com que a ordenação parcial seja formada por um conjunto inteiro de mensagens dentro das linhas da vida que estão separadas.

Uma sequência de mensagens possui uma sequência linear unificada, mas é preciso demonstrar as condicionais e loops com frequência e, esporadicamente, a execução concorrente de diversas sequências. O modelo de controle de alto nível se dá por operadores de controle estruturados, presentes em diagramas de sequências.

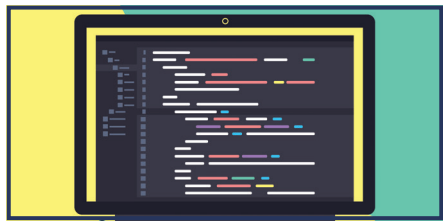
O operador de controle é exposto como uma região retangular dentro de um diagrama de sequências. Reparando com mais detalhamento, ele possui uma tag, que nada mais é do que um rótulo de texto que informa qual o



tipo de operador de controle. Ele é aplicado nas linhas da vida que cruza, sendo que isso será considerado o corpo do operador. Se uma linha da vida não estiver adequada a ele, a mesma é obstruída no topo do operador de controle e volta para a base. Os tipos de controle mais empregados são:

- Execução opcional: a tag que a compõe é denominada de `opt`. Neste tipo de controle, o corpo do operador de controle é realizado se uma condição de guarda for considerada como verdadeira em uma entrada do operador. A condição de guarda é uma expressão booleana que é vista entre colchetes na camada superior de qualquer linha da vida presente no corpo, além de fazer citação aos requisitos do objeto;

- Execução condicional: representada pela tag `alt`, divide o corpo do operador em diversas partes, chamadas de **sub-regiões**, através de linhas esboçadas horizontalmente. Cada sub-região é um ramo de uma condicional, com sua respectiva condição de guarda. Se for verdadeira, haverá uma execução da sub-região, porém vale se atentar que, se houver várias condições de guarda verdadeiras, a seleção de uma sub-região não será o ponto determinante. Não havendo nenhuma condição de guarda verdadeira, o controle se mantém com o operador;



DICA

Uma sub-região tem uma condição de guarda especial ou `else`. Nesta situação, a sub-região será realizada desde que não existam outras condições de proteção verdadeiras.

- Execução paralela: marcada pelo uso da tag `par`, faz com que cada sub-região traduza um modelo de computação denominada de paralela, ou concorrente. Em boa parte dos casos, cada sub-região tem linhas da vida diferentes e, no momento em que o operador de controle entra, as sub-regiões passam a ser realizadas de forma paralela. A execução das mensagens sucedidas dentro de cada sub-região é sequencial, mas a relativa ordem das mensagens dentro das sub-regiões paralelas é impositiva. Entretanto, tal conceito não deve ser usado na hipótese de uma interação entre computações distintas;

• Execução de loop (iterativa): representada pela tag loop, o corpo do loop é executado de maneira repetitiva quando a condição de guarda for considerada como verdadeira, antes de cada iteração. Do contrário, o controle não passa pelo operador.

Na Figura 5, há um exemplo simples de operadores de controle.

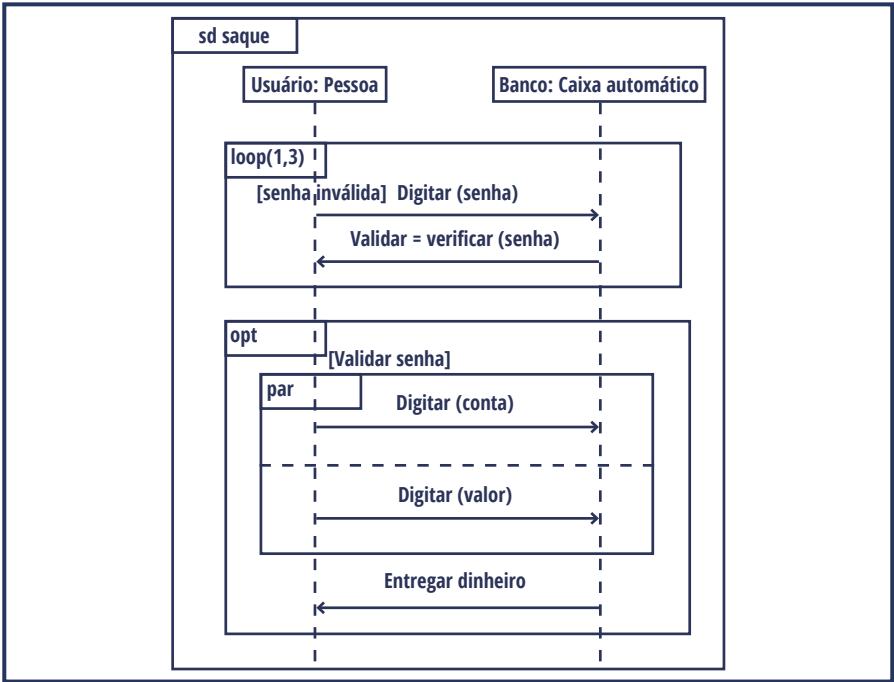


Figura 5. Operadores de controle estruturado. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

O usuário é responsável por iniciar a sequência, na qual o operador inicial é um loop. Dentro do loop, o usuário digita a senha para o sistema verificar. Enquanto a senha estiver incorreta, o loop continuará, mas, depois de três tentativas, o loop termina de qualquer maneira. Diante disso, o próximo operador será um opcional, cujo corpo será executado com a validade da senha. Não havendo validade, o resto do diagrama de sequências passa a ser desconsiderado.

O corpo do operador opcional possui um operador paralelo, com duas sub-regiões. A primeira informa ao usuário sobre a conta, e a outra sobre a informação do montante. Por mais que se apresentem de maneira paralela, elas são executadas em qualquer ordem.

Esse procedimento destaca a existência da concorrência, não assinalando a execução física de forma simultânea, significando que duas ações realizadas sem coordenação se dão em uma ordem qualquer. Havendo independência nas ações, elas se sobrepõem. Nas ações sequenciais, isto acontece de forma aleatória. O operador paralelo será finalizado depois de executar duas ações.



Timing diagram

A respeito do Timing diagram, ou diagrama de tempo, se percebe que ele contém similaridades com o diagrama de máquinas de estado. Entretanto, sua principal diferença está no fato de que o diagrama de tempo muda o estado de um objeto ao longo do tempo, o que reflete em pouco uso para modelar aplicações comerciais, sendo mais empregado na modelagem de sistemas com recursos de multimídia/hipermídia.

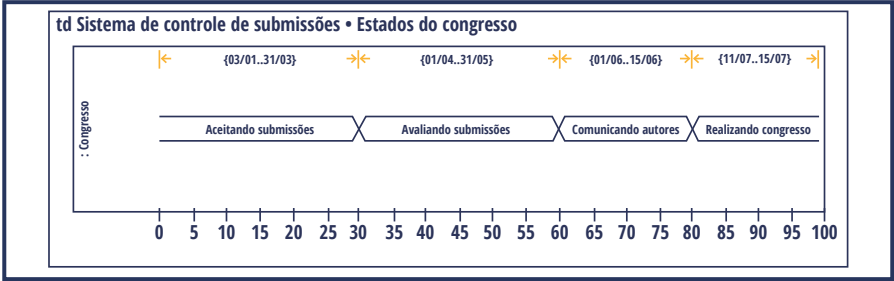


Figura 6. Diagrama de tempo (concisa). Fonte: GUEDES, 2014. (Adaptado).

O diagrama de tempo tem duas maneiras de exibição, mediante a representação concisa, como na Figura 6, ou através de uma notação robusta, em que as etapas estão dispostas como um gráfico.

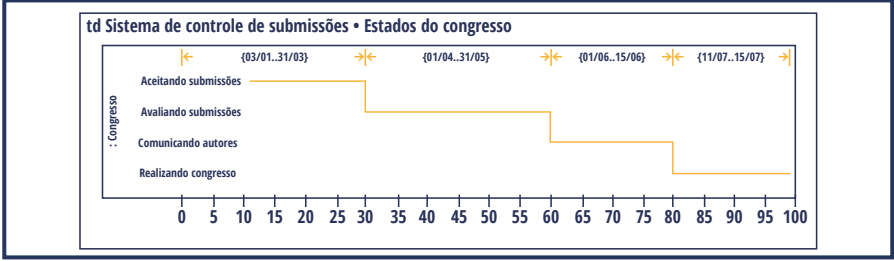


Figura 7. Diagrama de tempo (robusta). Fonte: GUEDES, 2014. (Adaptado).

Um aspecto diz respeito aos formatos aplicados nos diagramas de linha de tempo, visto que o ato de aplicar um formato consegue, de imediato, reorganizar o diagrama de maneira que os padrões estejam dispostos ao longo de um período de tempo.

O primeiro formato é o ordenado, que se caracteriza por ser mais útil quando o usuário tem um diagrama junto com uma sequência de eventos. A sua utilidade é notada:

- Ao adquirir uma quantidade de dados e aplicando em um formato inicial;
- Ao analisar os dados de volume elevado;
- Para avaliar a exibição e impressão.

Com um formato ordenado, se pode aproveitar o layout em um diagrama inteiro. Além disso, é possível restringir o layout para os componentes escolhidos ou até mesmo limitar o layout para os componentes que estão dentro de um intervalo de data e hora. Outra possibilidade é a configuração da distância entre os componentes sucessivos, sendo possível selecionar a configuração da distância entre itens.

Outro tipo de formato é o proporcional, considerado essencial quando há um diagrama associado a uma sequência de eventos e que ajuda a adquirir uma visão da maneira como os eventos são realizados em tempo real, mas pode gerar diagramas mais ampliados. Na importação dos dados, o formato proporcional dá uma observação inicial da maneira que os eventos são difundidos no tempo antes da aplicação em formatos, liberando a inserção no diagrama inteiro ou apenas uma escolha de componentes de diagrama, sendo possível determinar como a largura do diagrama é especificada.

O formato linha de tema é útil quando o usuário contém diversas linhas de tema dentro do seu diagrama, por ser possível categorizá-las através de uma propriedade para alocar a linha de tema com boa parte das interligações surgidas na parte superior do diagrama. Além disso, se pode especificar a separação vertical e alterar a ordem das linhas de tema. Por fim, se alinham os ícones da linha de tema e caixas de evento, sem esquecer de organizar caixas de evento desviadas. O formato de linha de tema não é adequado para diagramas sem itens de controle.



Ao se referir ao layout linha de tema agrupada, se observa que sua utilidade ocorre quando existem diversas linhas de tema dentro de um diagrama e se deseja reduzir o número de vínculos que ultrapassam as linhas de tema, mas lembrando que ele é tratado também com um formato proporcional. Com base nas informações do IBM Knowledge, com este layout é possível:

- Organizar as linhas de tema com o objetivo de reunir o seu diagrama em grupos de linhas de tema com alto nível de conexão;
- Reduzir a quantidade de vínculos que atravessam as linhas de tema;
- Limitar as extensões na linha de tema;
- Alocar os ícones da linha de tema no começo das linhas de tema, antes do vínculo inicial. No momento em que uma linha de tema tem só um vínculo, o ícone da linha de tema será alocado no momento do vínculo.

O formato agrupado temporal organiza componentes da esquerda para a direita dentro do seu diagrama, e os componentes que acontecem em tempos similares são exibidos juntos. O formato agrupado temporal é usado para:

- Coletar dados de alto volume para buscar bursts de atividade que possam estar listados;
- Ser aceito em um diagrama de linha de tempo com diversos eventos que acontecem em intervalos irregulares;
- Limitar as larguras de diagramas que são extensas à medida que são desenhadas de maneira proporcional, dado que o formato mantém uma indicação de períodos de baixa e alta atividade.

Um formato agrupado temporal origina um diagrama no qual a distância entre cada componente é fixada pelo usuário, compondo um formato não proporcional, ou seja, a diferença entre os componentes é diferente em relação ao tempo



entre eles. Com um formato agrupado temporal, se pode implementar o layout no diagrama inteiro, restringi-los para os componentes selecionados ou para os itens em um intervalo de data e hora. Além disso, se pode configurar a diferença de tempo máxima entre itens consecutivos, a distância entre os grupos e entre itens sucessivos em um grupo.



Use case diagram



Os diagramas de casos de uso são apenas um dos diagramas disponíveis dentro da UML direcionados para a modelagem de aspectos dinâmicos de sistemas, de acordo com Booch, Rumbaugh e Jacobson. Além da modelagem do comportamento do sistema, eles são essenciais para um subsistema ou até mesmo uma classe, onde cada um fornece uma série de casos de uso.

É evidente que os diagramas de casos de uso são essenciais na visualização, especificação e documentação do perfil de um elemento. Tais diagramas tornam o conjunto composto por sistemas, subsistemas e classes com elevado nível de acessibilidade e percepção, já que disponibilizam uma visão exterior abordando como os elementos são usados. Os diagramas de casos de uso são relevantes para avaliar sistemas executáveis por meio de engenharia direta e entendê-los através da engenharia reversa.

A partir da UML, é possível inserir os diagramas de casos de uso para examinar o desempenho de um sistema, subsistema ou classe. Isto faz com que os usuários compreendam a forma de uso dos componentes e como os desenvolvedores irão implementá-los. Na Figura 8, o usuário disponibiliza um diagrama de caso de uso ao realizar a modelagem do comportamento da caixa.

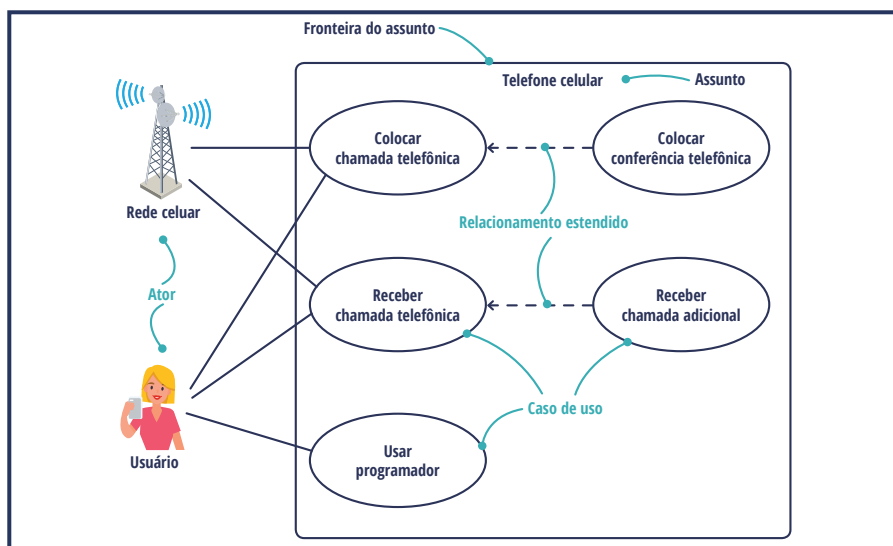


Figura 8. Diagrama de caso de uso. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

Portanto, um diagrama de caso de uso consiste em um modelo especial de diagrama, posto que ele se destaca por compartilhar propriedades similares a todos os diagramas, que se resumem ao nome e conteúdo gráfico da projeção dentro de um modelo.

Os diagramas têm uma série de notas e restrições, assim como em outros diagramas. Eles contêm pacotes usados para reunir componentes do modelo em grupos maiores. De maneira ocasional, se incluem as instâncias de casos de uso dentro dos seus diagramas no momento em que se quer analisar um sistema especial em execução. O usuário assume os diagramas de casos de uso para a realização da modelagem da visão do caso de uso dentro de um panorama, como um sistema, permitindo um suporte para o desempenho.

Na modelagem da visão de caso de uso de um cenário, se usam os diagramas de casos de uso de duas formas distintas: primeiro, se pode realizar a modelagem de um cenário. Ela é responsável pelo desenho de uma linha em torno do sistema, mostrando os atores excluídos do cenário e a maneira como eles interagem; a segunda forma está relacionada à modelagem dos requisitos de um sistema, determinando o que o cenário realizará de **maneira externa**, independentemente de como o cenário será executado.



DICA

Em um sistema, alguns elementos estarão dentro ou fora do sistema, a depender da funcionalidade. Um sistema de validação de cartão de crédito, por exemplo, estará dentro do sistema, enquanto os clientes e as instituições atuam de maneira externa.

As situações ocorridas em um sistema são as executoras do comportamento; já as que se encontram fora aguardam que seja disponibilizado por meio do sistema. As situações externas que realizam a interação com o sistema definem o contexto do sistema que ordena o cenário em que sistema irá atuar, já que é possível, dentro de uma UML, realizar a modelagem de um sistema através de um diagrama de caso de uso, enfatizando os atores no sistema. Definir o que será incluído como ator auxilia na especificação da classe de coisas que são interagidas com o sistema e decidir o que não será incluído é relevante, pois limita o ambiente do sistema para inserir os atores imprescindíveis na vida do sistema. De acordo com Booch, Rumbaugh e Jacobson, para a realização da modelagem do contexto dentro um sistema é preciso:

- Reconhecer as limitações do sistema, fixando quais comportamentos fazem parte dele e quais são executados por entidades externas, demarcando o cenário;
- Reconhecer os atores em torno do sistema, levando em consideração quais grupos carecem deste sistema para a execução de suas atividades e suas funções, vislumbrando o grupo responsável pela interação com algum hardware externo ou outros sistemas de software e aqueles que executam funções auxiliares de gerenciamento e de manutenção;
- Organizar os atores similares dentro de uma hierarquia de especialização;
- Disponibilizar um estereótipo a cada ator, para auxiliar no entendimento.

É necessário completar um diagrama de caso de uso através desses atores e ditar os caminhos de comunicação até os casos de uso do sistema pertencentes a cada ator. A Figura 9 dá o contexto de um sistema de validação de cartões de crédito, em que é possível ver clientes, individual e jurídico, quando interagem com o sistema, ressaltando o desempenho das instituições do sistema, como a instituição de venda e a financeira patrocinadora.

Tal metodologia é aplicada à modelagem do contexto dentro de um subsistema. Se considerarmos um sistema dentro de um nível de abstração, ele é um subsistema que faz parte de um sistema mais extenso em um nível maior de abstração. Portanto, a modelagem do contexto de um subsistema é considerada quando se está desenvolvendo a partir de sistemas interligados.

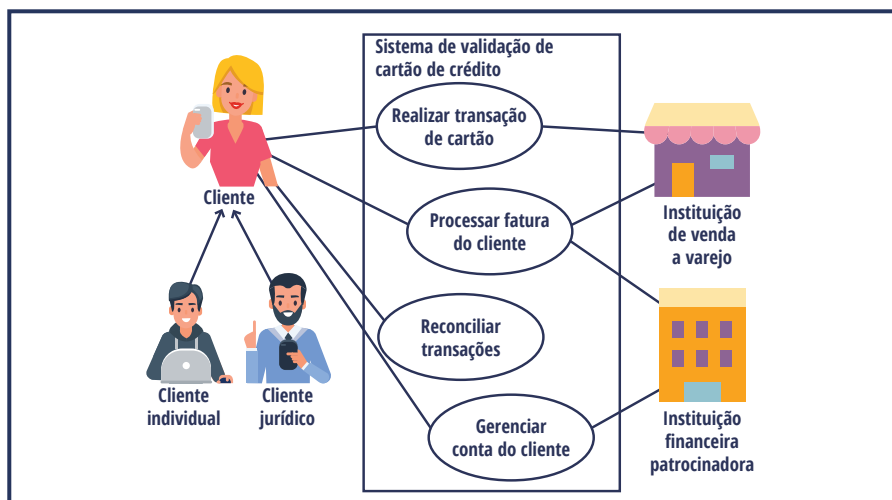


Figura 9. Contexto de um sistema. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

No que diz respeito à modelagem dos requisitos de um sistema, um requisito é um aspecto do projeto, uma propriedade ou um desempenho de um sistema. Ao definir os requisitos, ou atributos, do sistema, é apresentado um contrato, definido entre os elementos externos ao sistema e o sistema em si. Assim, o usuário não se preocupa com as atividades desenvolvidas do sistema, e sim com seu funcionamento.

Um sistema bem delimitado realiza todos os seus atributos de maneira fidedigna, pragmática e confiável. À medida que se desenvolve um sistema, se cria um consenso do que ele realizará. De modo parecido, ao receber um sistema a ser usado, é primordial entender seu comportamento para usá-lo de maneira adequada.

Os requisitos são das mais variadas formas, pois boa parte dos requisitos funcionais de um sistema aparecem como diagramas de casos de uso da UML, que são importantes para a administração dos requisitos. Para executar a modelagem dos requisitos de um sistema, se deve:

- Determinar o contexto do sistema, reconhecendo os atores ao seu redor e levando em consideração, para cada ator, o desempenho que cada um espera que o sistema proporcione;
- Selecionar comportamentos comuns, como casos de uso;
- Realizar a fatoração do desempenho comum em novos casos de uso usados pelos outros;
- Realizar a fatoração do desempenho variante em novos casos de uso que aumentam os fluxos da linha principal;
- Realizar a modelagem que envolve, dentre outros aspectos, os atores e seus relacionamentos dentro de um diagrama de uso;
- Incluir adornos nos casos de uso com notas exibindo requisitos não funcionais.

A Figura 10 amplia o diagrama de caso de uso anterior. Mesmo que se escondam as relações entre os atores e os casos de uso, são aproveitados casos de uso adicionais que não são visíveis ao cliente médio, por não serem comportamentos fundamentais ao sistema. Esse diagrama é considerado como essencial, pois disponibiliza um ponto inicial comum de suas decisões relativas aos requisitos funcionais do sistema para os agentes pertencentes ao mesmo, como usuários finais, especialistas de domínio e desenvolvedores

para visualização, especificação, construção e documentação. A descoberta de uma fraude de cartão, por exemplo, é um procedimento considerável tanto para o setor de venda quanto para a instituição financeira que patrocina. De maneira similar, apresentar o status da conta é outro comportamento requerido pelo sistema através das diversas instituições em seu contexto.

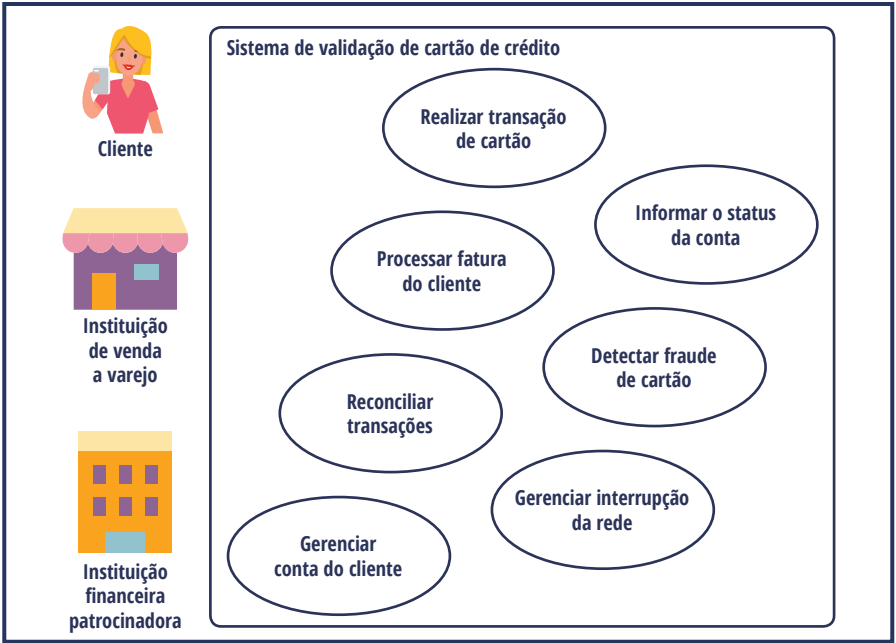


Figura 10. Requisitos de um sistema. Fonte: BOOCH; RUMBAUGH; JACOBSON, 2012. (Adaptado).

O requisito modelado pelo caso de uso denominado “Gerenciar interrupção da rede” é distinto, pois exprime um comportamento acessório do sistema, essencial para sua operação contínua e confiável, como relatado no livro de Booch, Rumbaugh e Jacobson. A partir do momento em que a estrutura do caso de uso é constituída, se descreve o comportamento de cada caso de uso.

É necessário escrever um ou mais diagramas de sequências em cada caso da linha principal. Depois, se escrevem os diagramas de sequências para os chamados “casos variantes”. Por fim, se escreve ao menos um diagrama de sequências para exibir cada modelo de erro ou condição de exceção. O tratamento de erros faz parte do caso de uso e é planejado junto com o desempenho normal.

Ao desenvolver os diagramas de casos de uso dentro da UML, é preciso lembrar que todo diagrama de caso de uso simboliza uma apresentação gráfica da visão estática do caso de uso dentro de um sistema. Booch, Rumbaugh e Jacobson afirmam que um diagrama de caso de uso bem estruturado tem como objetivo comunicar um aspecto da visão estática de caso de uso do sistema, além de conter apenas os casos de uso e atores fundamentais. Além disso, o diagrama visa fornecer detalhes consistentes com seu nível de abstração.



Engenharia reversa com UML

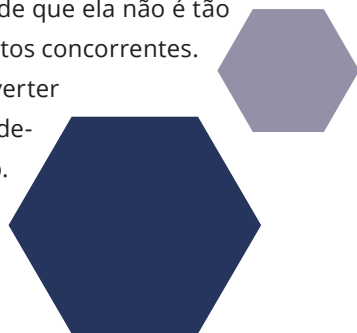


A engenharia reversa, no processo de desenvolvimento de software, tem o objetivo de criar informações com ponto de partida nos códigos, a fim de desenvolver novos itens que colaborem com o projeto. Tal metodologia para códigos-fonte auxilia na construção de método e códigos novos a partir dos modelos.

Ideias sobre os sistemas de software e ferramentas realizam a execução de trechos dos códigos para a criação de documentos através da engenharia reversa. Esta engenharia é um componente de software que cria arquiteturas novas de software de maneira automatizada ou semiautomatizada.

Arquiteturas ou projetos menores de software não desenvolvem documentos sobre o desenvolvimento feito. Sendo assim, cabe à engenharia reversa desenvolver o diagrama de classes partindo de sistemas já adotados, além de servir como método para criar novos modelos de projeto que ajudam na compreensão do sistema. Entretanto, não existem mecanismos que desenvolvam e atualizem os diagramas, o que representa um aspecto negativo das ferramentas de engenharia reversa, além do fato de que ela não é tão usada na abstração de informações de produtos concorrentes.

A engenharia reversa tem a função de reverter um código-fonte de software por meio das determinações com elevado nível de abstração. Sendo assim, se pode reconhecer informações das mudanças de cada código. A Figura 11 apresenta o uso da técnica por meio do código-fonte.



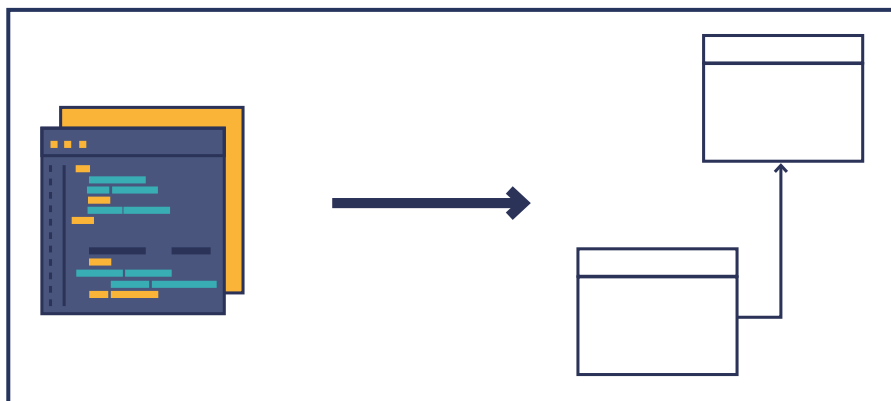


Figura 11. O uso da técnica da engenharia reversa por meio do código-fonte. Fonte: SANTOS, 2018. (Adaptado).

A Figura 11 traz um conceito generalista da engenharia reversa com o código-fonte fazendo a mudança para um diagrama de classes. Em situações eficientes, ela executa mudanças em partes do modelo ao invés do modelo total. A engenharia reversa realiza um processo de mudança, ou seja, o código-fonte é usado para que seja criado um componente novo a partir dele, capturando informações, melhorando as operações e diminuindo o risco e o custo envolvidos no desenvolvimento de um software. No momento atual, a engenharia reversa é útil na busca de definição da linha de produtos de software.



Ao longo dessa unidade, se concluiu que o ato de observar, especificar, desenvolver e documentar sistemas considerados de grande porte exige um conjunto de manipulações de requisitos elevados e que a utilização de sistemas aponta a necessidade de organização dos itens em grupos maiores. Neste contexto, foi possível entender o conceito de pacote, ferramenta usada na organização de itens pertencentes a uma modelagem realizada nos grupos.

Um diagrama de perfil é mais abstrato em comparação com os outros devido à adaptabilidade de uma UML a uma plataforma ou domínio, mas há a possibilidade de aumentar a linguagem ao desenvolver novas metaclasses e estereótipos que permitam a modelagem de novos domínios.

Os diagramas de estados mostram, dentre outras coisas, uma máquina de estados. Os diagramas de atividades, por seu turno, representam um caso específico de diagramas de estados em que a maioria dos estados se encontram em atividade.

Também se notou que o diagrama de tempo contém similaridades com o diagrama de máquinas de estado. A diferença está no diagrama de tempo, que altera o estado de um objeto ao longo do tempo.

Se analisou ainda que os diagramas de casos de uso são estabelecidos como um dos diagramas disponíveis dentro da UML focada para a modelagem de aspectos dinâmicos de sistemas. Além da modelagem do comportamento do sistema, eles são fundamentais em um subsistema ou até mesmo em uma classe na qual cada um se apresenta.

Quanto ao diagrama de sequências, sua principal funcionalidade é desenvolver uma ordenação temporal das mensagens, sendo composto, primeiramente, inserindo os objetos na interação pertencentes a um nível mais elevado do diagrama.

Referências bibliográficas



BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. **UML**: guia do usuário. Trad. Fábio Freitas da Silva e Cristina de Amorim Machado. 12 reimp. Rio de Janeiro: Elsevier, 2012.

GUEDES, G. T. A. **UML 2** – Guia Prático. 2 ed. São Paulo: Novatec Editora, 2014.

IBM KNOWLEDGE CENTER. **Aplicando formatos a diagramas de linha de tempo**. Disponível em: <https://www.ibm.com/support/knowledgecenter/pt-br/SS3J58_9.0.9/com.ibm.i2.anb.doc/apply_layout_timeline_charts.html>. Acesso em: 29 nov. 2019.

SANTOS, M. **TechREF**: uma técnica de engenharia reversa orientada a features. 2018. 106 f. Dissertação (Mestrado) – Programa de Pós-Graduação em Computação Aplicada, Universidade do Vale do Rio dos Sinos, São Leopoldo, RS, 2018. Disponível em: <http://www.repositorio.jesuita.org.br/bitstream/handle/UNISINOS/7024/Maicon%20dos%20Santos_.pdf?sequence=1&isAllowed=y>. Acesso em: 29 nov. 2019.

GERÊNCIA DE CONFIGURAÇÃO (LIVRO 2)



Presidente do Conselho de Administração	Janguiê Diniz
Diretor-presidente	Jânio Diniz
Diretoria Executiva de Ensino	Adriano Azevedo
Diretoria Executiva de Serviços Corporativos	Joaldo Diniz
Diretoria de Ensino a Distância	Enzo Moreira
Autoria	Ronnie Edson de Souza Santos
Projeto Gráfico e Capa	DP Content

DADOS DO FORNECEDOR

Análise de Qualidade, Edição de Texto, Design Instrucional,
Edição de Arte, Diagramação, Design Gráfico e Revisão.

© Ser Educacional 2020

Rua Treze de Maio, nº 254, Santo Amaro

Recife-PE – CEP 50100-160

*Todos os gráficos, tabelas e esquemas são creditados à autoria, salvo quando indicada a referência.

Informamos que é de inteira responsabilidade da autoria a emissão de conceitos.

Nenhuma parte desta publicação poderá ser reproduzida por qualquer meio
ou forma sem autorização.

A violação dos direitos autorais é crime estabelecido pela Lei n.º 9.610/98 e punido pelo
artigo 184 do Código Penal.

Imagens de ícones/capa: © Shutterstock

Boxes



ASSISTA

Indicação de filmes, vídeos ou similares que trazem informações complementares ou aprofundadas sobre o conteúdo estudado.



CITANDO

Dados essenciais e pertinentes sobre a vida de uma determinada pessoa relevante para o estudo do conteúdo abordado.



CONTEXTUALIZANDO

Dados que retratam onde e quando aconteceu determinado fato; demonstra-se a situação histórica do assunto.



CURIOSIDADE

Informação que revela algo desconhecido e interessante sobre o assunto tratado.



DICA

Um detalhe específico da informação, um breve conselho, um alerta, uma informação privilegiada sobre o conteúdo trabalhado.



EXEMPLIFICANDO

Informação que retrata de forma objetiva determinado assunto.



EXPLICANDO

Explicação, elucidação sobre uma palavra ou expressão específica da área de conhecimento trabalhada.

Sumário

Unidade 1 - Introdução à gerência de configuração

Objetivos da unidade	12
Introdução à gerência de configuração	13
Processo de software.....	13
Gerência de configuração	18
Relevância da gerência de configuração para os projetos de software	21
O surgimento da gerência de configuração	21
Importância da gerência de configuração	22
O papel do gerente e da equipe de configuração	28
Sintetizando	34
Referências bibliográficas	35

Sumário

Unidade 2 - Itens de configuração

Objetivos da unidade	37
Identificação de artefatos e itens de configuração	38
A crise do software e o surgimento da engenharia de software.....	39
Artefatos do projeto de software.....	41
Itens de configuração do projeto de software.....	49
Versionamento de itens do projeto de software	51
Controle de configuração e comitê de mudanças	55
Sintetizando.....	57
Referências bibliográficas	58

Sumário

Unidade 3 - Gerenciamento de Ciclo e Vida de Mudanças de Software

Objetivos da unidade	62
Geração de baselines e criação de releases	63
Gerenciamento de mudanças	75
Ciclo de vida das mudanças	79
Procedimentos para mudanças	81
Sintetizando	85
Referências bibliográficas	86

Sumário

Unidade 4 - Gerência de configuração: auditoria, contingência e ferramentas

Objetivos da unidade	88
Auditoria de configuração.....	89
Gerenciamento de riscos e plano de contingência	96
Ferramentas para o gerenciamento de configuração e mudanças	103
Sintetizando.....	107
Referências bibliográficas	108

Apresentação

Olá, aluno(a)!

Nesse material, focado na área de gerência de configuração, você vai encontrar informações importantes sobre esta atividade fundamental para o processo de desenvolvimento de software, especialmente nos dias atuais, nos quais a evolução dos sistemas e das necessidades das pessoas faz com que, cada vez mais, os sistemas sejam desenvolvimentos com rapidez e qualidade.

Aqui você irá conhecer a gerência de configuração e seu papel no ciclo de vida de um software. Nesse âmbito apresentaremos conceitos básicos sobre o processo de desenvolvimento de software, o ciclo de vida e características da gerência de configuração. Logo depois, é apresentada e discutida sua relevância e importância nas atividades de desenvolvimento de software no contexto atual. Por fim, são apresentados os principais profissionais e seu papel dentro do processo de gerência de projetos.

Abordaremos também temas como: itens de configuração e artefatos do projeto, controle de configuração (labels ou rótulos), baselines, releases, solicitações de mudança, auditoria de configuração, plano de contingência e outros assuntos indispensáveis para enriquecer sua formação.

Desejamos que você amplie o seu conhecimento e procure relacionar teoria e exemplos a aspectos de sistemas que você já construiu ou com conceitos de outras disciplinas que você já que você já tenha estudado ao longo de sua formação profissional. Desejamos ainda que você utilize os conceitos aqui apresentados para colocar em prática suas ideias, uma vez que estamos tratando de uma atividade bastante relevante e que requer profissionais bastante preparados no mercado atual.

Bons estudos!

O autor



O professor **Ronnie E. S. Santos** possui doutorado em Ciências da Computação pela Universidade Federal de Pernambuco – UFPE (2019), mestrado em Ciência da Computação pela mesma instituição (2015) e bacharelado em Sistemas de Informação pela Universidade Federal Rural de Pernambuco – UFRPE (2012). Atua há nove anos como pesquisador na área de Engenharia de Software e Gerência de Projetos de Software, tendo publicado cerca de 30 artigos científicos. Como docente, possui experiência, tanto na modalidade presencial quanto EAD, em cursos de Computação no ensino superior. Na indústria de software, atua principalmente como engenheiro de testes e analista de qualidade, tendo ainda experiência como programador Java e analista de requisitos de software.

Currículo Lattes:

<http://lattes.cnpq.br/7740410814678720>

Dedico este material a todos os meus estudantes EAD, que fazem desta modalidade de ensino o caminho para o seu sucesso profissional.

UNIDADE

1

INTRODUÇÃO
À GERÊNCIA DE
CONFIGURAÇÃO



ser
educacional

Objetivos da unidade

- Apresentar a atividade de gerência de configuração;
- Discutir o papel da gerência de configuração no desenvolvimento de software;
- Entender a importância e relevância da gerência de configuração;
- Compreender o papel dos profissionais da gerência de configuração, sobretudo o do gerente de configuração.

VIDEOAULA
Clique aqui



Tópicos de estudo

● Introdução à gerência de configuração

- Processo de software
- Gerência de configuração

● Relevância da gerência de configuração para os projetos de software

- O surgimento da gerência de configuração
- Importância da gerência de configuração

● O papel do gerente e da equipe de configuração





Introdução à gerência de configuração

Você já parou para imaginar qual o passo a passo para desenvolver um software, seja um aplicativo para celular, website ou qualquer outro tipo de sistema?

Incrivelmente, muitas pessoas nunca pararam para refletir como se dá o processo de observar um problema e construir uma solução tecnológica para ele, decorrendo daí todo um sistema. Mas você, que é da área de computação, já deve ter lido ou se deparado com algum tipo de informação sobre o que é comumente chamado de processo de software ou processo de desenvolvimento de software.

Pois bem, chama-se **processo de software** o conjunto de atividades executadas com a finalidade de se obter um sistema funcional, que será aplicado em algum contexto ou atividade específica.

Por exemplo, imagine que o dono do mercadinho de seu bairro realiza manualmente as atividades do negócio e resolveu contratar uma equipe para desenvolver um sistema. Neste sentido, o problema em questão é automatizar as atividades de estoque e de venda do estabelecimento, e a solução é a construção de um software adequado para isso. Para tanto, um conjunto de atividades vai organizar o desenvolvimento desse software. Esse conjunto de atividades e a maneira como elas são organizadas e executadas é conhecido como processo de software.



Processo de software

Um processo de software pode ser resumido como o conjunto estruturado de atividades necessárias para desenvolver um sistema de software. Perceba que a grande questão por trás desse conceito é a sua característica de ser estruturado, com **atividades, funções e papéis** bem definidos, de modo que o resultado final seja um software de valor e qualidade.



EXPLICANDO

Na engenharia de software, um processo de desenvolvimento de software bem definido é aquele que divide o trabalho de desenvolvimento do sistema em fases e atividades distintas, buscando melhorar o design, o gerenciamento de produtos e o gerenciamento de projetos. Esse processo também é conhecido como **ciclo de vida** de desenvolvimento de software.

Estruturando o processo de forma estratégica, é possível obter o máximo benefício do trabalho dos engenheiros de software, visto que cada um deles sabe exatamente o que tem de fazer, o que precisa entregar e o que é esperado do seu trabalho. Note que, comumente, usa-se o termo engenheiro de software para se referir a todos os profissionais que participam do projeto, mas que, de maneira específica, recebem o nome que melhor representa a sua atividade, como programador, designer ou gerente.

Sendo assim, o processo de desenvolvimento de software pode ser dividido em até nove atividades, conforme mostradas no Quadro 1, estruturadas e organizadas de maneira a obter o máximo de produtividade dos profissionais, buscando entregar um software de qualidade e de valor para o cliente.

QUADRO 1. ATIVIDADES DE UM PROCESSO DE SOFTWARE



Modelagem de negócio	Levantamento de requisitos	Análise e projeto
Design	Implementação	Testes
Gerência de projetos	Gerência de configuração	Ambiente

Veja a definição de cada uma dessas atividades a seguir:

- **Modelagem de negócio:** É uma atividade inicial no processo de software, caracterizada pelo processo de entendimento do ambiente onde o software irá funcionar, para que seja definida a viabilidade do desenvolvimento do sistema. Em resumo, como a equipe de desenvolvimento de software pode não compreender previamente o ambiente onde o software irá funcionar, como um sistema para um hospital ou um aplicativo para um escritório de advocacia, é papel da atividade de modelagem de negócio fornecer entendimento e suporte técnico sobre ele e sobre sua viabilidade do ponto de vista de negócio (lucro e custos);
- **Levantamento de requisitos:** Outra atividade inicial do processo de software, esta tem seu foco voltado para a construção de uma lista de ações que o sistema deve realizar e um conjunto de critérios a que deve atender. De maneira geral, é nessa atividade que o problema do cliente será entendido, e decomposto em uma lista de funcionalidades implementadas no software.

Entenda como lista de funcionalidades a lista de funções que o sistema deve fazer e como ele deve fazer. Por exemplo, realizar o cadastro dos usuários (o que fazer), com e-mail e senha composta por oito caracteres numéricos (como fazer);

- **Análise e projeto de software:** Atividade intermediária entre a definição do problema e o início da construção do projeto. Esta é uma atividade que visa à construção de modelos para o sistema, como diagramas e esquemas que, posteriormente, serão usados na programação. Nesta etapa, é como se o sistema fosse todo construído de maneira ilustrada. Se comparada à construção de uma casa, esta etapa seria como o desenvolvimento da planta do imóvel;

- **Design:** Atividade responsável pelas decisões de arte do sistema, como definição de elementos da tela e disposição de botões, imagens e textos. Além disso, esta atividade é responsável por garantir que o sistema atenda visualmente a todas as necessidades dos diferentes tipos de usuários que poderão utilizá-lo. Esta é uma atividade criativa, focada na melhor forma de apresentar o sistema aos usuários, seja através de elementos gráficos ou de outros tipos de comandos, como comandos de voz;

- **Implementação:** Comumente chamada de programação, esta atividade está focada no processo de conversão dos modelos e diagramas em um código executável de linguagem de programação (Java, PHP, dentre outras). Em outras palavras, é o processo de programar e construir o sistema. Isso inclui programar as diversas ações do sistema e a realização de ações no banco de dados, como consultas, armazenamentos e exclusões.

- **Testes de software/qualidade:** Executada ao longo de todo o desenvolvimento, tem como principal finalidade garantir que o sistema está sendo construído da maneira como esperado, e que o resultado seja útil e se comporte de maneira adequada. Além disso, essa atividade tem a função de identificar e reportar falhas existentes no software antes que ele seja entregue. É importante ressaltar que essa atividade deve garantir que o sistema não apresente falhas, ou seja, verificar o sistema e, adicionalmente, garantir que os usuários sintam-se satisfeitos com sua experiência ao utilizá-lo, validando o resultado.

- **Ambiente:** Uma atividade relacionada indiretamente com o desenvolvimento do sistema reúne o conjunto de ações que permitem que o ambiente físico no qual o sistema está sendo desenvolvido possua equipamentos adequados para a construção do sistema, como rede, conexão, software de apoio etc.

• **Gerência de projetos:** Para que todas as atividades do desenvolvimento sejam coordenadas e organizadas, de modo a atender a um objetivo comum, que é a construção do sistema, há a atividade de gerência de projetos, que ocorre durante todo o processo de desenvolvimento, dando apoio e coordenando as demais. É papel do gerente de projetos definir quem é o responsável por cada atividade, quais os riscos e custos envolvidos, e quanto tempo será necessário para a conclusão de cada tarefa. Essa é uma atividade complexa, pois conta com fatores diversos para ser executada, como previsões, recursos financeiros e questões específicas de cada profissional envolvido.

• **Gerência de configuração:** Finalmente, a atividade do processo de desenvolvimento, a gerência de configuração, também chamada de gerência de configuração e mudança, reúne todas as ações necessárias para garantir o correto rastreamento dos **artefatos do desenvolvimento** de software. Isso significa que existe uma série de tarefas que devem ser consideradas para garantir que as diversas partes e versões do sistema não se percam ou sejam confundidas, tendo em vista que nenhum sistema é desenvolvido por completo de uma só vez. O sistema é dividido em partes, construídas ao longo de um período específico, e é preciso garantir o rastreamento da construção e da integração de cada uma delas. Essa atividade afeta diretamente ações de todas as outras atividades, especialmente da gerência de projetos, programação, testes, design, análise e projeto e, finalmente, requisitos.



EXPLICANDO

Entende-se como **artefato de software** os diversos tipos de subprodutos concretos produzidos durante o desenvolvimento de software. Em outras palavras, é tudo aquilo que é produzido pelos profissionais quando estão trabalhando nas suas atividades, como a lista de requisitos de software, o código-fonte criado pelo programador, o desenho de uma tela criado pelo design ou a planilha de atualizações da versão do sistema, criado pelo gerente de configuração.

É importante ressaltar que todas as atividades do desenvolvimento de software têm ligação direta ou indireta entre si. Algumas atividades dividem até certo grau de dependência. Por exemplo, só é possível programar o código do sistema porque existem um ou mais requisitos levantados e descritos.

Além disso, algumas atividades finalizam antes que o sistema seja entregue, como a modelagem de negócio do sistemas, enquanto outras acompanham o processo do início ao fim, como é o caso da gerência de projetos e da gerência de configuração.

Para fixar melhor esse conhecimento, o quadro a seguir resume cada uma das atividades descritas. É importante que você conheça cada uma delas, uma vez que a gerência de configuração trabalha diretamente com todas, sendo necessário rastrear e manter organizadas as diversas versões tanto do código do sistema quanto dos demais artefatos construídos.

QUADRO 2. ATIVIDADES DO PROCESSO DE SOFTWARE



Atividade	Descrição
Modelagem de negócio	Análise da viabilidade de projeto e entendimento do negócio
Levantamento de requisitos	Identificação das necessidades do usuário de software e construção da lista de funcionalidades
Análise e projeto de software	Entendimento do projeto e criação de modelos que irão guiar a construção do sistema
Design	Definição de telas e construção do plano de interação do usuário com o sistema
Implementação	Processo de construção de uma solução para o problema tratado, através de uma linguagem de programação
Testes de software/qualidade	Identificação de falhas e validação do sistema pelo ponto de vista do usuário
Ambiente	Garantia de que toda a equipe de desenvolvimento possui o material necessário para trabalhar
Gerência de projetos	Organização das atividades e da equipe de profissionais, monitoramento de tempo, custos e execuções
Gerência de configuração	Garantia de que as diversas versões do código do sistema e dos demais artefatos são rastreáveis e corretamente armazenadas durante todo o desenvolvimento

Exercite seu aprendizado pesquisando sobre os principais artefatos produzidos por cada fase do desenvolvimento, e que provavelmente serão rastreados e monitorados pelo gerenciamento de configuração. Por exemplo, o principal artefato da implementação é o código do sistema. Em uma linguagem de programação específica, entretanto, os programadores produzem outros. Você pode procurar por artefatos de todas as atividades para saber, de maneira mais específica, o que cada uma produz e qual seu impacto na gerência de configuração.



Gerência de configuração



A gerência de configuração é uma atividade que possui grande influência no sucesso de um software e no processo de desenvolvimento de maneira geral. Por este motivo, a gerência de configuração é sempre considerada como uma boa prática de desenvolvimento ligada à qualidade do sistema. A questão é que, nos dias atuais, a gerência de configuração de projetos de software não é só uma prática, mas uma necessidade, uma ação que reúne um conjunto de atividades projetadas para controlar as mudanças ocorridas durante o desenvolvimento de sistemas.



CONTEXTUALIZANDO

Você sabe como as empresas de telefones celulares fazem para lançar aparelhos com os sistemas atualizados todos os anos? Esse é um processo complexo, bem articulado e que envolve muita gerência de configuração. Pense no conjunto de funcionalidades que seu celular faz para você. A princípio, pode parecer simples, mas dentro desse celular existe não só um sistema, mas vários, que se conectam e interagem entre si, de forma que cada função pode impactar um conjunto de outras ações.

Vamos tentar ilustrar isso de uma maneira bem simples. Perceba que seu telefone é um equipamento complexo, que reúne um conjunto de ações que, há algumas décadas, eram feitas por diversos dispositivos diferentes. Vejamos alguns exemplos dessas ações:

- Realizar e receber ligações, que é uma função básica desde o início do uso do aparelho;
- Receber e enviar mensagens, uma funcionalidade comum, mas que foi adicionada aos celulares depois;
- Realizar cálculos através de um sistema de calculadora embutido no sistema nativo;
- Definir localização, não necessariamente através de um aplicativo de localização, uma vez que os celulares já dispõem de um dispositivo de GPS;
- Dar suporte para chats e conversação com outros indivíduos que dispõem de um mesmo aplicativo de mensagens;
- Permitir acesso à internet através de conexão em rede tanto via wi-fi quanto através de outros tipos de conexão.

Entre outras tantas ações, que podíamos passar algumas horas apenas listando.

A questão é que cada um desses sistemas precisa ser desenvolvido por uma equipe específica e, a cada período de tempo, uma nova versão é liberada para ser testada. É aí que acontece o problema. Quer um exemplo?

Suponhamos que, em uma primeira semana, o sistema da calculadora estava pronto e realizava todas as operações corretamente. Então, na segunda semana, o sistema de música foi finalizado, mas apresentava um defeito: o usuário não conseguia retroceder a música, apenas avançá-las. Ainda assim, os dois sistemas funcionavam muito bem juntos no telefone. Chega a terceira semana e o sistema que acessa a internet foi finalizado. Junto dele, o sistema de música foi corrigido e entregue funcionando perfeitamente. O problema é que o sistema da calculadora parou de funcionar por causa do sistema de música, e agora é necessário voltar para a versão anterior, mesmo com a pequena falha, para observar por que os dois sistemas funcionavam juntos e não funcionam mais. Chega então o grande problema da gerência de configuração: onde está a versão anterior do sistema e qual foi a mudança realizada, fazendo com que os dois não conseguissem mais trabalhar juntos?

Dessa forma, através do entendimento sobre a gerência de configuração e estratégias de mudança, a equipe de desenvolvimento pode realizar diversas alterações no sistema até conseguir entregar uma versão completa, em que todas as funcionalidades estejam trabalhando da maneira como esperado. Assim, a gerência de configuração tem o papel de analisar e responder a questões importantes do desenvolvimento de software, que produzem um impacto direto na qualidade do produto, no sucesso do desenvolvimento e na satisfação do usuário final. Dentre essas questões, estão:

1. Quais mudanças foram realizadas durante o desenvolvimento do software e quando elas aconteceram?
2. Qual a necessidade da realização de uma mudança e como ela irá impactar o software?
3. Quais versões do sistema foram afetadas pela mudança e como elas podem ser localizadas quando necessário?
4. Quem foi o profissional responsável que solicitou a mudança e como ele pode ser contatado, caso necessário?

5. Quem são os profissionais que efetivamente participaram do processo de mudança?

6. É possível retroceder às outras versões, ou seja, é possível voltar para uma etapa anterior, se for necessário?

7. O software em desenvolvimento continua estável depois da mudança ou ela causou algum tipo de comportamento inesperado?

Nesse momento, você pode estar imaginando que essas mudanças podem acontecer o tempo inteiro durante o desenvolvimento do software. E pior, de maneira desordenada, elas podem acabar prejudicando o desenvolvimento do software. Se as mudanças não forem identificadas, controladas e monitoradas, o desenvolvimento do software como um todo poderá ser negativamente impactado, e no fim, o sistema como um todo será prejudicado. Já imaginou se você recebe uma nova atualização para o seu telefone e, ao instalar, percebe que a nova versão desliga o aparelho o tempo todo? Você provavelmente vai querer que a empresa permita você a usar a versão anterior, até que o problema seja corrigido.

Por isso, a motivação por trás do gerenciamento de configuração é a de resolver os problemas que ocorrem em projetos de software devido a alterações realizadas ao longo do desenvolvimento do sistema. Sendo assim, todos os questionamentos mencionados são importantes para o desenvolvimento do software, e todas essas ações estão diretamente ligadas à gerência de configuração. De maneira mais detalhada é possível assumir que os objetivos da gerência de configuração são:

- Apontar, a qualquer momento durante o desenvolvimento, a descrição técnica do sistema e seus componentes, através de uma documentação válida e confiável;
- Controlar, de maneira contínua e efetiva, o processo de evolução de um produto de software, considerando por completo seu código e demais artefatos;
- Promover a rastreabilidade de mudanças e evoluções de todos os documentos, modelos e artefatos produzidos ao longo do desenvolvimento do software;
- Facilitar a consistência entre os diversos componentes do software, como o controle de interfaces, ambientes de rede e demais integrações existentes entre os elementos que compõem o sistema;

- Garantir que a documentação do software seja sempre um reflexo exato do software que está sendo desenvolvido;

- Permitir que qualquer profissional envolvido conheça a capacidade operacional e as limitações de cada item do software e, no caso de não conformidades, saber quais itens são afetados pelas mudanças.



Relevância da gerência de configuração para projetos de software

Até esse ponto, você já deve ter entendido, de maneira geral, o que é gerência de configuração e qual seu papel e objetivos dentro do processo de desenvolvimento de software e da construção de sistemas de qualidade. Neste ponto, é importante centralizar algumas outras informações relacionadas a essa tarefa, especialmente em se tratando da sua relevância.

Você deve estar pensando: “ok, é importante controlar as mudanças, já entendi”. Mas a questão é que a gerência de configuração não para por aqui. Ela é bem maior que apenas observação e controle de mudanças. Já pensou o que acontece se duas pessoas estão trabalhando juntas na mesma parte do sistema? Como essa concorrência pelo recurso pode ser resolvida? E se acontecer um grande problema, como ele vai ser resolvido? E quais padrões vão garantir a qualidade do versionamento?

Percebe que existem muito mais questões do que as que foram discutidas até este ponto? Então, vamos nos estender mais sobre esses novos conhecimentos.



O surgimento da gerência de configuração

A gerência de configuração foi inicialmente criada e desenvolvida na década de 1950 pelas Forças Armadas dos Estados Unidos, visando controlar a documentação produzida pela indústria de mísseis. Esta abordagem de controle de mudanças só foi introduzida na indústria de software a partir de 1980 e, posteriormente, passou a ser reconhecida como um processo de gestão de qualidade em 1995.

Agora, de volta à gerência de configuração, é importante que se tenha em mente que mudanças são inevitáveis durante o desenvolvimento de software. Mais ainda, é importante saber que elas podem acontecer a qualquer momento e por diversas causas, como:

- **Erros de implementação:** por descuido ou simplesmente por acidente, um programador acaba danificando o código de uma parte do sistema que estava funcionando perfeitamente. Agora, é preciso retornar para uma versão segura e sem falhas;

- **Falta de comunicação entre a equipe:** pode resultar em dois profissionais editando o mesmo artefato, ao mesmo tempo, sem se ter ideia de que o outro está trabalhando paralela e concorrentemente;

- **O cliente entende que suas necessidades não são mais atendidas pelo sistema:** agora o software tem uma nova demanda e precisa ser revisto, atualizado e reconstruído;

- **Outras questões como legislação e atualizações:** fazem com que, mesmo sem nenhum tipo de problema, o sistema precise ser atualizado para uma nova versão. Por exemplo, uma nova lei exige que notas fiscais exibam uma informação que antes não era exibida, fazendo necessária uma mudança no software.



Importância da gerência de configuração

Até este ponto, você já entendeu que um projeto de desenvolvimento de software pode produzir diversos itens, uma vez que cada atividade produz seus próprios materiais. Esses itens são chamados de artefatos de engenharia de software e, só para fixar, lembre-se que eles podem ser:

- **Artefatos do programa produzidos pela implementação,** como código-fonte executável, construído em uma linguagem de programação específica (Java, PHP, SQL, dentre outros), ou ainda programas executáveis e módulos do sistema, além de bibliotecas de componentes, dentre outros;

- **Dados do projeto,** como informações sobre os usuários do sistema, a forma como o login deve ser realizado ou dados de teste do sistema, ou seja, material usado para realizar os testes e verificar a qualidade do software;

• **Diagramas do projeto**, itens construídos na etapa intermediária do sistema que servem para guiar seu desenvolvimento. Os diagramas mais comuns produzidos pelo desenvolvimento de software são os chamados **diagramas UML**.

UML é o acrônimo usado para o termo *Unified Modeling Language* que, em português, pode ser traduzido como Linguagem de Modelagem Unificada. Esta é uma linguagem de notação, ou seja, expressa uma maneira de ilustrar e comunicar uma ideia – nesse caso específico, ilustrar ideias sobre o uso do sistema de software. Em linhas gerais, esses diagramas podem ser de dois tipos:

• **Diagramas estruturais**: itens utilizados para especificar detalhes da estrutura do sistema e da construção do código, servem para guiar a programação, mostrando, por exemplo, como as diversas partes do sistema interagem ou como os dados são armazenados no banco de dados;

• **Diagramas comportamentais**: itens utilizados para especificar detalhes do comportamento do sistema e sua relação com o usuário, ou seja, ilustra como cada uma das funcionalidades devem agir e como os usuários utilizam essas funcionalidades dentro do sistema.

No Quadro 3, é possível ver os modelos de diagramas UML disponíveis para cada tipo: estruturais e comportamentais.

QUADRO 3. DIAGRAMAS UML



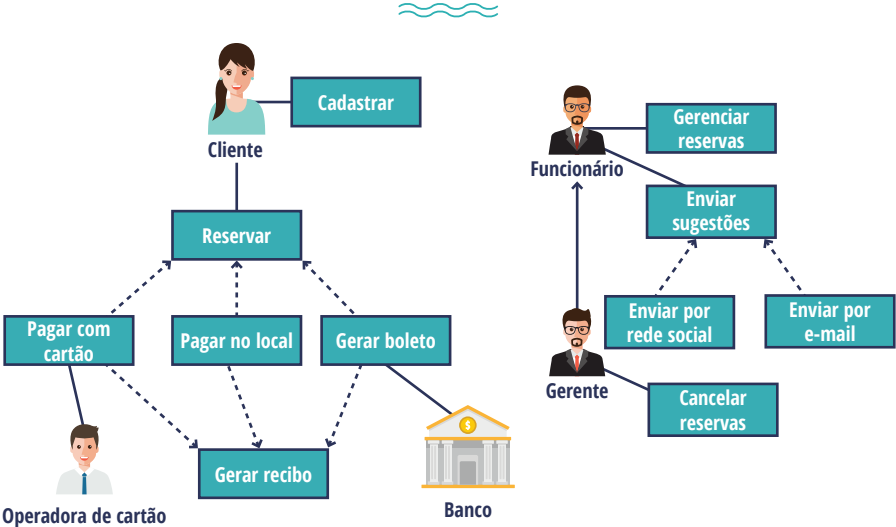
Diagramas estruturais	Diagramas comportamentais
Diagrama de classes	Diagramas de caso de uso
Diagrama de objetos	Diagramas de estado
Diagramas de componentes	Diagramas de atividades
Diagramas de implantação	Diagramas de interação
Diagramas de pacotes	
Diagramas de estrutura	

Para entender os diagramas UML, imagine-se, por exemplo, na seguinte situação: uma equipe de desenvolvimento de software deve construir um sistema de reservas on-line, no qual o cliente pode se cadastrar e realizar reservas. Ao fim de cada reserva, o cliente pode escolher realizar o pagamento com o cartão de crédito, gerar um boleto ou escolher pagar no estabelecimento. Neste esquema, o sistema gera automaticamente os recibos dos pagamentos.

Nesse mesmo sistema, o funcionário do hotel poderá gerenciar as reservas e enviar sugestões sobre pacotes de hospedagens para clientes. Essas informações podem ser enviadas via e-mail ou rede social. O sistema de pagamento tem ligação direta com o sistema da operadora de cartão de crédito do cliente, e o sistema de boletos, com o banco do cliente. O gerente do estabelecimento também tem acesso às funcionalidades do funcionário, porém, ele é o único responsável pelo cancelamento de reservas.

A partir daí, a melhor maneira de representar todas essas interações dos diversos usuários e funcionalidades do sistema é através de um diagrama UML, do tipo comportamental, conhecido como **diagrama de caso de uso**. Nesse esquema, todas as interações descritas no exemplo podem ser representadas como no Diagrama 1.

DIAGRAMA 1. DIAGRAMA DE CASO DE USO DE UM SISTEMA DE HOSPEDAGENS



Agora, comparando o texto do sistema e o diagrama apresentado, perceba que várias versões do mesmo diagrama podem ser criadas, dependendo da interpretação do problema. A função da gerência de configuração é garantir que apenas uma versão válida seja utilizada por todos os profissionais no desenvolvimento do software. Entretanto, é necessário armazenar todas as versões anteriores de maneira consistente e segura.

Nesse processo, todo o conjunto de itens armazenados, rastreados e controlados pela gerência de configuração são chamados, coletivamente, de **configuração do software**. Em outras palavras, a configuração de software é o estado atual de todos os itens que formam o software. A gerência de configuração é, então, o controle da evolução desses itens durante todo o desenvolvimento do projeto. Por exemplo, as várias versões do código de um módulo do sistema ou as várias versões dos diagramas do software, ou ainda as várias versões do documento de teste.

Não esqueça! Artefatos de software são todos os itens produzidos ao longo do desenvolvimento do sistema. Já configuração de software são todos os itens que estão sendo monitorados e controlados pela gerência de configuração.

Dessa maneira, se existir algum tipo de documento que não está sendo monitorado, este documento é um artefato de software, entretanto, não faz parte da configuração do software. É papel do gerente de configuração decidir o que será ou não controlado, de maneira a garantir a eficiência da atividade e do desenvolvimento como um todo.

É importante ter em mente que a gerência de configuração busca, além de controlar as alterações, prever as consequências de uma alteração antes que ela seja feita. Lembre-se: o software está em constante evolução. Durante o desenvolvimento, ele passa de uma lista de itens contendo ações que devem ser promovidas pelo sistema a uma sequência de código estruturado em uma linguagem de programação específica, até finalmente se tornar o sistema utilizado pelo usuário. Além disso, após ser desenvolvido, o sistema ainda pode passar por longos e repetitivos processos de manutenção e de modificações de funcionalidades, seja para atualização ou para correção de erros que não foram percebidos antes.

Estima-se que cerca de 20% de todo o esforço de manutenção de um produto de software é referente ao processo de corrigir erros de implementação e falhas não observadas anteriormente. Por outro lado, 80% do esforço de manutenção é focado na adaptação do software em função de modificações nos requisitos e funcionalidades, no ajuste das regras de negócios e até mesmo na reengenharia do software. Apenas por esse motivo, já é possível justificar o quão importante a gerência de configuração é, antes, durante e depois do desenvolvimento do software.

Acredito que até este ponto já seja possível visualizar a gerência de configuração como uma atividade que controla e notifica a existência de diversas correções, extensões e adaptações realizadas no software. Para tanto, é fundamental prover controle sobre os artefatos produzidos e modificados por diferentes pessoas. Além disso, sua importância está associada aos problemas gerados pela falta de controle das mudanças, especialmente quando dois profissionais trabalham juntos na mesma função ou no mesmo “pedaço” de software.

Para entender o problema das mudanças, imagine a situação em que, em uma empresa de desenvolvimento de software, a gerência de configuração não é utilizada ou então é mal conduzida. Durante o desenvolvimento de um novo aplicativo para um cliente importante, Pedro está trabalhando no sistema de log-in e modifica o código dos módulos M1, M2 e M3. Estes códigos estão em um repositório que é compartilhado com toda a equipe. No mesmo dia, Maria está trabalhando no sistema de cadastros do aplicativo, e está editando o código dos módulos M4, M5 e também do módulo M3 que, coincidentemente, faz parte tanto do sistema de log-in quanto do sistema



de cadastros do aplicativo. Este cenário está ilustrado na Figura 1.

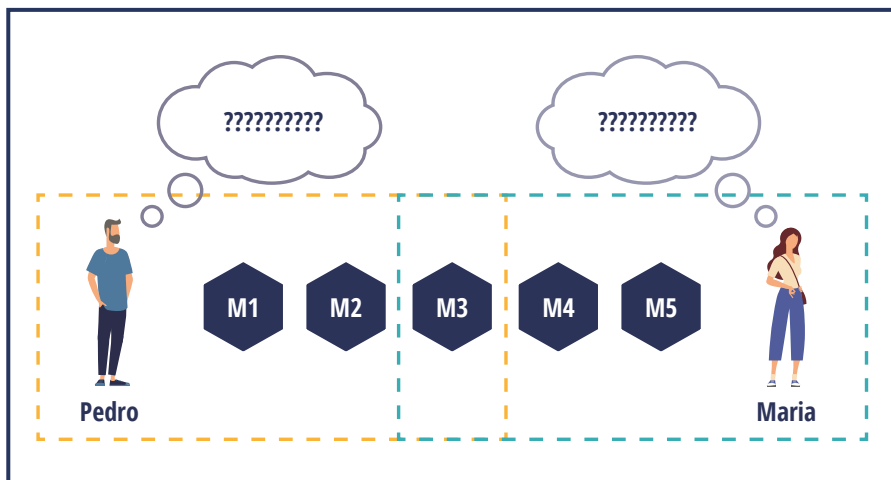


Figura 1. Concorrência por recursos no desenvolvimento de software.

Agora imagine que, antes de sair para almoçar, Pedro fez uma modificação nos módulos em que estava trabalhando e salvou-a no código. Vinte minutos depois, Maria também saiu para almoçar e salvou suas edições, o que incluiu uma alteração no módulo M3, no qual Pedro estava trabalhando. Ao retornar para o trabalho, Pedro nota que o seu código, que antes estava funcionando, não funciona mais, e fica sem entender o que aconteceu.

Em uma situação como esta, em que Maria não comunicou sobre a modificação de M3, tampouco pontuou quais mudanças foram feitas no código compartilhado, Pedro, que está usando o mesmo espaço de trabalho, não conseguirá saber imediatamente por qual razão o código de M3, que ele acabou de fazer, falhou. Em contrapartida, como não há controle de versões, é praticamente impossível retornar para uma versão estável do sistema sem que um dos dois profissionais perca o trabalho feito antes do almoço.

Percebeu o grande problema? Este cenário aconteceu pela falta de controle de versões e planejamento de modificações. Além disso, a ausência de um procedimento específico, que defina como as mudanças devem ser realizadas e concluídas, gerou falta de informações sobre os impactos das mudanças.

Pode-se dizer, então, que esta é a principal razão pela qual a gerência de configuração é tão relevante para o desenvolvimento de software. Além disso, a atividade de gerência de configuração produz outros importantes benefícios para o projeto. Estes benefícios estão listados no Quadro 4.

QUADRO 4. BENEFÍCIOS DA GERÊNCIA DE CONFIGURAÇÃO



Benefício	Justificativa
Aumento de produtividade no desenvolvimento	Falta de comunicação e impactos negativos de mudanças não controladas causam a necessidade de retrabalho
Redução de defeitos	A sincronização de mudanças permitirá que o sistema seja testado várias vezes a cada integração de código
Maior rapidez na identificação de problemas	O versionamento guarda informações sobre alterações realizadas, mantendo um histórico dos problemas ocorridos no sistema
Eficiência na correção de problema	O histórico de modificações permite que problemas sejam identificados de maneira rápida e eficaz
Aumento da disciplina no processo de desenvolvimento	O monitoramento e controle das mudanças deixarão o processo mais organizado e melhor definido
Aumento da memória organizacional	As versões guardam históricos de problemas e de como de suas soluções, aumentando sua eficiência em projetos futuros
Acesso às informações qualitativas e quantitativas referentes ao processo	No futuro, o histórico de mudanças permitirá definir com maior precisão medidas de esforço para se efetuar determinada tarefa
Possibilidade de estabelecer uma trilha de auditoria	Determinado artefato possui histórico de mudanças, falhas e sucesso de implementação
Auxílio à gerência de projeto	Em um ambiente estável, no qual o produto é monitorado constantemente, é possível definir melhor questões relacionadas a custos, tempo e esforço de trabalho
Entrega de produtos com maior qualidade	Cada integração do sistema passa por um processo de verificação e validação completas, reduzindo a chance da entrega de um produto com falhas surpresas e inesperadas

Finalmente, é importante ressaltar que a gerência da configuração está associada com normas e padrões de excelência de processo, justamente por ser uma área fortemente focada no controle. Sendo assim, esta atividade é referenciada em diversas normas, processos, procedimentos, políticas e padrões, como a ISO 12207, CMMI e MPS.Br. Uma empresa de desenvolvimento que deseja ser bem conceituada no mercado precisa cumprir várias dessas normas, em busca de certificações que a coloquem em melhor posição no mercado.



O papel do gerente e da equipe de configuração

Todas as atividades do processo de desenvolvimento de software são executadas por um time de profissionais. Neste esquema, cada membro do time usa sua especialização para agregar valor ao que está sendo construído.

Mesmo dividindo um mesmo objetivo, a natureza dos trabalhos no desenvolvimento de software são diferentes e, por isso, requerem diferentes habilidades e especialidades para serem realizados. Vamos a alguns exemplos:

- Os profissionais que trabalham com levantamento de requisitos têm a característica de serem analíticos e visualizarem o sistema para além da tecnologia, de modo a entender o problema do usuário;
- Profissionais que trabalham com design de interface têm habilidades artísticas para desenho e criação, uma vez que precisam construir sistemas intuitivos e visualmente atraentes;
- Os profissionais que trabalham com implementação são focados na tecnologia, e usam suas habilidades no domínio de ferramentas e linguagens de programação para a construção do código do sistema;
- Os profissionais de teste e qualidade são curiosos, com habilidades exploratórias, de maneira que podem explorar todo o sistema em busca de falhas e problemas que não foram vistos pelos desenvolvedores;
- Os gerentes possuem habilidades de organização e capacidade de tomada de decisão, distribuição de tarefas e monitoramento de processos.

E a gerência de configuração? Você consegue imaginar qual a característica destes profissionais? A equipe de trabalho na gerência de configuração pode apresentar tamanho variável, dependendo do tamanho da empresa e do software que está sendo desenvolvido.

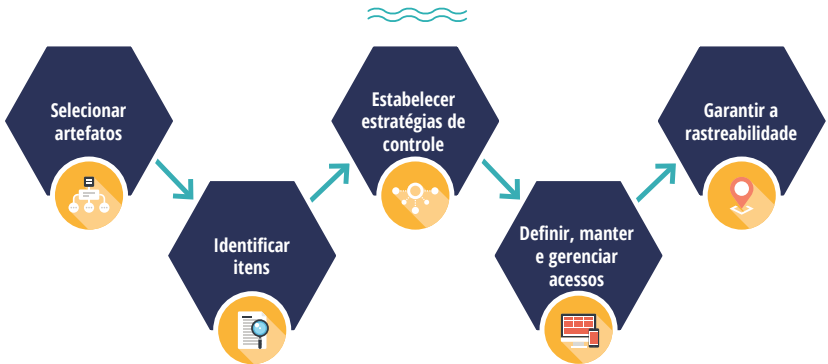
Desse modo, é possível ter gerência de configuração com apenas um profissional e também com vários profissionais trabalhando em conjunto.

Em teoria, a gerência de configuração é uma equipe formada por um gerente de configuração, um gerente de mudanças, um integrador e membros que realizam atividades genéricas.

É importante ressaltar que, em muitos ambientes, o gerente de configuração e o de mudanças são o mesmo profissional, o que vai depender de fatores associados à sua habilidade profissional e ao tamanho do projeto de software em desenvolvimento. É importante que você conheça o papel de ambas as gerências de maneira separada, mas entenda que, em grande parte dos casos, o papel do gerente de configuração engloba ambas as tarefas.

Como você deve ter observado, existe uma série de atividades que têm de ser executadas para se obter um bom resultado da gerência de configuração de software, conforme ilustrado no Diagrama 2, a seguir.

DIAGRAMA 2. ATIVIDADES GERAIS DA GERÊNCIA DE CONFIGURAÇÃO



Por este motivo, existem diversos profissionais envolvidos com o processo de gerência de configuração e mudanças. Vamos conhecer alguns deles.

O gerente de configuração é o profissional responsável por realizar as atividades relacionadas com a tomada de decisão sobre a infraestrutura do ambiente de configuração, ou seja, ele é responsável por definir tudo que é necessário para que esta atividade funcione e seja bem sucedida. Além disso, a restrição de acesso aos itens de configuração e as formas como as diversas mudanças podem afetar o código e os demais artefatos do software são parte de suas atribuições.

Adicionalmente, o gerente de configuração é responsável por garantir que o ambiente de trabalho facilite as atividades de revisão do produto de software e também de rastreamento de mudanças realizadas. Por exemplo, imagine que, após ser lançada a primeira versão de um determinado aplicativo, os usuários começam a experimentar falha na conexão de rede, causada pelo aplicativo em questão. Nesse caso, é necessário garantir que tanto a versão com defeito quanto a última, sem defeitos, possam ser identificadas, para que sejam revisadas em tempo hábil e a falha não prejudique o aplicativo. O gerente de configuração deve então estabelecer processos e diretrizes para que isso seja possível.

De maneira geral, o gerente de configuração tem um perfil profissional voltado para a realização das atividades relacionadas à disponibilização e ao controle da infraestrutura e do ambiente de configuração, o que dará suporte ao desenvolvimento do sistema.

Por isso, ele é o principal responsável por assegurar que este ambiente possibilite a execução das atividades de revisão e de rastreamento de mudanças e defeitos, garantindo maior rapidez no desenvolvimento e maior qualidade do sistema.

Profissionalmente, o gerente de configuração deve conhecer os princípios de gerenciamento de configuração tratados aqui, além de entender como essa atividade se conecta com as demais, de maneira a promover sua correta execução. Além disso, preferencialmente, esse profissional deve possuir experiência e habilidades no uso de ferramentas de gerenciamento de itens de configuração.

No mais, um bom gerente de configuração deve estar atento aos detalhes inerentes ao software, de modo que consiga estabelecer as diretrizes de acesso e controle aos itens. Finalmente, ser assertivo e determinado, de modo a assegurar que os desenvolvedores e demais profissionais das atividades do projeto não ignorem as políticas e os procedimentos de gerenciamento de configuração.

O gerente de controle de mudança é o profissional responsável por supervisionar o processo de mudanças em determinada parte do sistema. Além disso, também faz parte da sua atuação entender quais serão os impactos caso uma mudança seja autorizada, considerando tempo e custo. Dessa forma, pode-se dizer que o papel do gerente de mudança é supervisionar o processo de controle das mudanças que estão acontecendo no sistema.

O gerente de mudança trabalha muito próximo do gerente de configuração, uma vez que suas funções agem de forma muito próxima. Além disso, como o impacto das mudanças afeta não só os itens de configuração (responsabilidade do gerente de configuração), mas também questões associadas a tempo, custo e alocação de pessoas, este gerente ainda trabalha em conjunto com o gerente de projetos de software.

De maneira geral, a pessoa que desempenha este papel precisa compreender também os princípios de gerência de configuração. Adicionalmente, o gerente de mudanças deve ser capaz de medir o impacto das mudanças no cronograma geral do projeto e nos custos do desenvolvimento do software para todas as solicitações de alteração e mudanças requeridas. Neste esquema, este profissional

necessita possuir um perfil comunicativo, visto que trabalha na negociação de prazos e custos para adequar o projeto às mudanças.

É importante ressaltar que, em muitos casos, existe um único profissional, gerente, responsável pelas atividades de configuração e de mudança, afinal, ambas as atividades se complementam. Nesse caso, a pessoa que desempenha esse papel deve entender dos princípios de gerência de configuração e, preferivelmente, ter experiência ou ter sido treinado para usar ferramentas que auxiliem o processo tanto de manutenção da configuração como de controle de mudanças.

Integradores são os profissionais da equipe de configuração responsáveis por realizar a integração dos itens modificados no sistema. Por exemplo, considere um sistema como um quebra-cabeça formado por várias peças diferentes, que se completam para atingir um determinado objetivo. Neste esquema, uma mudança pode ser requisitada em uma única peça do quebra-cabeça ou em um conjunto de peças, enquanto as demais se mantêm intactas e funcionando. Realizar a integração dos itens do sistema, nesse caso, seria o processo de retirar uma determinada função do sistema, realizar a alteração necessária e, então, devolvê-lo, como se estivesse devolvendo a peça removida para o quebra-cabeça ao qual ele faz parte. Esse processo é conhecido como integração de software.

O trabalho dos integradores é, portanto, a execução da entrada e da saída de quaisquer itens relacionados ao produto de software para fins de controle de configuração e mudanças. Esse procedimento é conhecido como check-in e check-out. Neste esquema, após uma solicitação de mudança, por exemplo, os integradores liberam o item que será alterado pelos desenvolvedores. Ao realizar a mudança, os desenvolvedores liberam os componentes modificados, após eles terem sido testados. Estes componentes são adicionados em um espaço de trabalho compartilhado. Neste ponto, os integradores combinam esses componentes para criar um build, que é uma versão estável do programa.

Existem ainda outros membros da equipe de configuração que realizam um conjunto de atividades mais genéricas, relacionadas à verificação e ao controle. Esse grupo envolve os diversos profissionais que trabalham para garantir que as mudanças realizadas no software não prejudiquem o andamento de seu desenvolvimento. Estes profissionais devem estar sempre em sincronia, para garantir que nenhum item seja modificado sem que se tenha ciência e controle sobre isso.

Além disso, o próprio desenvolvedor (programador) também pode ser considerado um membro da equipe de configuração, uma vez que ele é o principal usuário da gerência de configuração de software. O programador produz itens de configuração (códigos do sistema) e é ele também quem realiza as mudanças nesses itens. Entretanto, diversos outros profissionais participam desse processo e podem ser considerados parte da equipe de configuração, como é o caso dos engenheiros de teste e analistas do sistema.

Por fim, lembre-se que toda a equipe de configuração está focada na identificação, controle e manutenção dos itens de configuração do projeto de software. Mas também não se esqueça que a inclusão de todos esses papéis está relacionada com o número de profissionais na equipe, o que está diretamente relacionado com o tamanho do projeto de software. Naturalmente, o gerente de configuração é o elemento-base da equipe de configuração, teoricamente ligado a atividades gerenciais de planejamento e controle, mas que, na prática, pode realizar as atividades de todos os profissionais descritos.



Sintetizando



Nessa unidade, nós vimos alguns elementos básicos ao estudo de gerência de configuração, com o objetivo de fazê-lo entender seus objetivos em projetos de software. No geral, essa é a atividade focada no processo de planejamento e controle de mudanças que podem ocorrer no sistema durante seu desenvolvimento e após a sua entrega.

Para garantir rastreabilidade, segurança, consistência e qualidade, a gerência de configuração deve definir quais artefatos do projeto serão incluídos como itens de configuração e serão monitorados posteriormente. Lembre-se: um artefato do sistema pode ser qualquer elemento resultado de umas das atividades de desenvolvimento, como, por exemplo, o código do programa, um diagrama do projeto, protótipos de tela ou listas de funcionalidades. Uma vez definidos os itens de configuração, é preciso estabelecer medidas de acesso e controle a estes itens, para que não existam inconsistências durante o desenvolvimento, quando, por exemplo, dois profissionais estiverem trabalhando juntos na mesma parte do software.

Qualquer informação, documento ou item importante para desenvolvimento do projeto pode ser incluída na lista de itens de configuração, e é trabalho da equipe de configuração a identificação, controle e manutenção desses itens. Projetos de software que não executam a gerência de configuração podem estar fadados ao fracasso, afinal, o sistema pode ser entregue com diversas falhas vindas de mudanças que não foram observadas e nem contabilizadas. No geral, produtos de baixa qualidade fazem com que usuários procurem alternativas em outros sistemas, e a gerência de configuração é uma das atividades que visa evitar que isso aconteça.

Espero que este material tenha ajudado você a entender a importância da gerência de configuração para o projeto de software. Lembre-se sempre de buscar mais conteúdo para expandir seu conhecimento. Mantenha-se motivado!



Referências bibliográficas



BERSOFF, E. H. Elements of Software Configuration Management. **IEEE Transactions on Software Engineering**, v. 10, n. 1, 1984, p. 79-87. Disponível em: <<https://ieeexplore.ieee.org/document/5010202>>. Acesso em: 12 dez. 2019.

BOURQUE, P.; FAIRLEY, R. E. **Guide to the software engineering body of knowledge: (SWEBOK(R))**: V3.0 Los Alamitos, CA: IEEE Computer Society Press, 2014.

UNIDADE

2

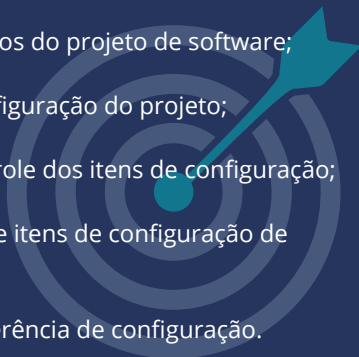
ITENS DE
CONFIGURAÇÃO



ser
educacional

Objetivos da unidade

- Apresentar os conceitos relacionados a artefatos do projeto de software;
- Discutir o processo de seleção de itens de configuração do projeto;
- Entender a importância e a relevância do controle dos itens de configuração;
- Compreender o processo de versionamento de itens de configuração de software;
- Entender o papel do comitê de mudança na gerência de configuração.



Tópicos de estudo

- **Identificação de artefatos e itens de configuração**

- A crise do software e o surgimento da engenharia de software
- Artefatos do projeto de software
- Itens de configuração do projeto de software

- **Versionamento de itens do projeto de software**

- **Controle de configuração e comitê de mudanças**

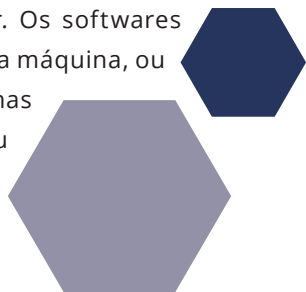




Identificação de artefatos e itens de configuração

Como toda ciência ou como toda nova atividade recém criada, a computação apresentou uma série de problemas até se estabilizar e ser conhecida como a ciência que existe hoje. Logo de início, a partir da criação do primeiro computador, era praticamente impossível separar o hardware do software, ou seja, ambos eram tratados como um mesmo produto e eram construídos, testados e utilizados de maneira inseparável. Com o passar do tempo, ocorreu a separação completa dos conceitos relacionados ao hardware e ao software, bem como sua produção e utilização.

Para entender melhor a separação desses dois itens, imagine seu computador no momento em que você está lendo este material. Sua máquina é formada por estes dois componentes básicos: o hardware e o software. Define-se como hardware todos os componentes físicos que formam a máquina, ou seja, é tudo aquilo que você consegue tocar fisicamente, como seu teclado, seu mouse ou seu monitor. Os softwares são todos os componentes lógicos que formam a máquina, ou seja, tudo aquilo que está funcionando agora, mas que fisicamente não é possível tocar, como seu sistema operacional, seu leitor de texto ou seu navegador da internet. Sendo assim, pode-se dizer que o computador é composto pela união do hardware (parte física) e do software (parte lógica). O hardware precisa de um software que o diga o que fazer, enquanto o software precisa de um hardware que lhe permita funcionar.



Apesar de trabalharem em conjunto, é possível separar facilmente o hardware do software, uma vez que esses elementos são construídos, na maioria das vezes, de maneira independente, passando a funcionar em conjunto apenas depois.

A menos que seja considerado o universo dos **sistemas embarcados**, hardware e software são dois elementos bastante distintos, o que é bem diferente do cenário existente após a Segunda Guerra Mundial, período em que a computação começou a dar seus largos passos.



EXPLICANDO

Sistemas embarcados é o termo utilizado para definir a classe de sistemas computacionais completos e independentes, que muitas vezes podem apresentar uma característica mais simples que um computador comum (desktops ou notebooks), uma vez que são encarregados de executar apenas uma tarefa ou função bastante particular. Esses sistemas são conhecidos como sistemas embutidos, uma vez que o ambiente computacional é completamente encapsulado e totalmente dedicado ao dispositivo que controla. Ou seja, hardware e software não são separados e o software fica dedicado apenas a um hardware específico – é o caso de sistemas que são construídos para aparelhos de televisão, câmeras digitais ou brinquedos.

A separação do hardware e do software, em contextos gerais da computação, permitiu o processo de construção de sistemas mais robustos e dedicados a problemas específicos, fazendo a computação se tornar a área que conhecemos hoje e atingir um alto nível de importância em escala mundial.

Entretanto, mesmo após a separação da produção de hardware e software, o processo de construir programas de computador passou por um longo período de adaptação até se tornar o processo conhecido atualmente. Por muito tempo, cada empresa ou grupo construía o seu software da maneira como acreditava ser melhor, ou seja, com pouca padronização e nenhum tipo de preparo ou de processos de gerenciamento definidos.

Em outras palavras, tudo que conhecemos atualmente sobre desenvolvimento de software, práticas de codificação e programação eficientes, levantamento de requisitos e gerenciamento de mudanças só foi estabelecido após um longo período de desenvolvimento de software sem padronização, o que levou essa indústria para o período conhecido como **crise do software**.



A crise do software e o surgimento da engenharia de software

A crise do software é como ficou conhecido o período que se estendeu ao longo dos anos 70, quando, nas empresas de software e de engenharia de software, eram praticamente inexistentes. Nesse contexto, cada empresa desenvolvia os seus

sistemas da maneira como acreditava que seria melhor, sendo que boas práticas, padronização e processos definidos eram praticamente nulos.

O termo “crise” refletia os problemas e as dificuldades do desenvolvimento de software, bem como sua relação com o rápido crescimento da indústria, que demandava, cada vez mais, sistemas novos e melhores. Além disso, era crescente a complexidade dos problemas que deveriam ser resolvidos por produtos de software, ao passo que não existiam, de maneira geral, técnicas bem estabelecidas para o desenvolvimento de sistemas que funcionassem adequadamente.

Dessa maneira, sem padronização e técnicas de engenharia que auxiliassem o desenvolvimento de sistemas, a crise do software estava ligada à imaturidade das empresas no que diz respeito ao desenvolvimento de seus produtos. Naquele momento, elas apresentavam problemas como:

- **Projetos que ultrapassavam o orçamento** e, muitas vezes, causavam enormes prejuízos às empresas;

- **Projetos que ultrapassavam o prazo**, uma vez que a falta de planejamento e gerenciamento não permitiam uma correta predição de tempo de entrega;

- **Software de baixa qualidade**, sendo a falta de controle a principal responsável pela entrega de sistemas aquém do satisfatório;

- **Software que não satisfazia os requisitos**, resultando em entregas de produtos que não resolviam o problema para o qual foi idealizado;

- **Projetos com código impossível de manter**, devido à grande quantidade de mudanças ao longo do desenvolvimento, falta de controle e, principalmente, falta de padronização.

Assim, no fim desse processo de crise, profissionais e cientistas de todo o mundo começaram o processo de melhoria do desenvolvimento de software, de criação e padronização de atividades e documentos que servem para a construção de sistemas cada vez melhores e de maior qualidade. Dentre as soluções encontradas, é possível citar:

- **Análise econômica de sistemas de informação** em uma atividade que permite identificar, antes que o sistema seja construído, qual a sua viabilidade frente ao problema que resolverá e, conseqüentemente, quais são os custos envolvidos nesse desenvolvimento;

- **O uso de melhores técnicas, métodos e ferramentas**, permitindo que atividades como a arquitetura de software e a gerência de configuração pudes-

sem planejar a construção do sistema e acompanhar as mudanças existentes ao longo do desenvolvimento;

- **Mudança no paradigma de desenvolvimento de software**, resultando em um processo de definição de atividades fundamentais, com objetivos específicos e construídos de maneira bem definida, a ponto de melhorar o produto de software e sua entrega.

É válido ressaltar que, na engenharia de software atual, o desenvolvimento de software segue um processo bem definido, dividindo o trabalho de desenvolvimento do sistema em um conjunto de atividades distintas e interligadas, que buscam obter o melhor produto possível.

Dessa maneira, ao dividir o sistema em fases e atividades distintas, a engenharia de software visa obter os melhores resultados, de modo que cada profissional envolvido no desenvolvimento saiba exatamente o que tem de fazer (tarefas), o que precisa entregar (artefatos) e o que é esperado do seu trabalho (qualidade).



Artefatos do projeto de software

Depois da crise, a evolução da engenharia de software trouxe consigo, dentre algumas mudanças e padronizações, a **divisão do processo de desenvolvimento de software** em atividades distintas e interligadas.

Dessa maneira, cada atividade possui um objetivo específico, com um profissional ou grupo de profissionais singulares, responsáveis pela execução de tal atividade e pela produção de materiais que compõem o sistema, que serve de apoio e suporte ao desenvolvimento. Esses materiais são conhecidos como **artefatos de software**.

Em uma perspectiva geral, pode-se dividir o desenvolvimento de software em até nove atividades específicas, estando oito delas diretamente relacionadas com a produção do sistema e uma delas ligada ao ambiente físico de desenvolvimento. Cada uma das oito atividades diretamente relacionadas ao desenvolvimento do software possui uma característica particular e também uma lista de artefatos de software que pode produzir.



Assim, os artefatos de softwares são responsáveis pela confecção de subprodutos concretos, produzidos enquanto um software está sendo desenvolvido. O Quadro 1 resume cada uma das **atividades relacionadas ao desenvolvimento de software**, seu objetivo geral e os artefatos de software que podem produzir.



DICA
Exercite seu aprendizado, pesquisando sobre o processo de desenvolvimento de software Rational Unified Process, comumente conhecido como RUP. Trata-se de um método de desenvolvimento tradicional surgido após a crise do software e pioneiro na divisão de atividades e definição de artefatos do projeto de software. Por meio dessa pesquisa, é possível conhecer todos os diferentes tipos de profissionais e especialidades que podem estar presentes no desenvolvimento de sistemas, além de entender o que cada um desses profissionais pode produzir durante o seu trabalho em termos de documentação e demais artefatos.

QUADRO 1. ATIVIDADES DO PROCESSO DE SOFTWARE E SEUS ARTEFATOS

Atividade	Descrição/objetivos	Artefatos
Modelagem de negócio	Análise da viabilidade de projeto e entendimento do negócio.	• Glossário de negócio; • Regras de negócio; • Visão de negócio.
Levantamento de requisitos	Identificação das necessidades do usuário do software e construção da lista de funcionalidades.	• Lista de funcionalidades (requisitos); • Plano de gerenciamento de priorização; • Atributos de requisitos.
Análise e projeto de software	Entendimento do projeto e criação de modelos que irão guiar a construção do sistema.	• Diagramas UML; • Documento de arquitetura do sistema; • Modelo de banco de dados.
Design	Definição de telas e construção do plano de interação do usuário com o sistema.	• Interface; • Plano de acessibilidade; • Pacote de design.
Implementação	Processo de construção de uma solução para o problema, por meio de uma linguagem de programação.	• Código-fonte do sistema; • Código e chamadas do banco de dados; • Documento de padrões de desenvolvimento.
Testes de software/qualidade	Identificação de falhas e validação do sistema pelo ponto de vista do usuário final.	• Casos de teste do sistema; • Dados de teste do sistema; • Código de automação de testes.

Gerência de projetos	Organização das atividades, da equipe de profissionais e monitoramento do tempo, custos e execuções.	• Cronograma do projeto; • Dados sobre custos do projeto; • Planilha de alocação de profissionais.
Gerência de configuração	Garantia de que as diversas versões do código do sistema e dos demais artefatos são rastreáveis e corretamente armazenadas durante todo o desenvolvimento.	• Plano de controle de mudança; • Documento de solicitação de mudança; • Registro de acesso e mudança.

Desse modo, é possível dizer que **artefatos de software** são importantes elementos do processo de desenvolvimento de sistemas, sendo definidos como resultados de uma atividade específica que, posteriormente, pode ser consumida por outra atividade do projeto. Isso significa que um artefato produzido no início do projeto, como o documento de requisitos, será utilizado em outro momento do desenvolvimento.

O desenvolvimento de um software pode produzir dezenas de artefatos, a depender do tamanho do projeto, da complexidade envolvida e também do método de desenvolvimento utilizado pela empresa de software. Como exemplos de artefatos mais comuns produzidos no desenvolvimento de software, é possível citar o documento de requisitos, os diagramas de casos de uso, os diagramas de classes e outros modelos UML, o código-fonte produzido pelos programadores e os casos de teste produzidos pelos engenheiros de testes. Além disso, existem outros artefatos que estão mais relacionados com o processo em si, ou com algum tipo de necessidade burocrática do desenvolvimento, podendo ser citados os planos de projetos, documentos de processos de negócios, documentos de análise de risco, dentre outros.

Finalmente, é importante ressaltar que os artefatos de software podem estar disponíveis de forma física (documentos impressos) ou de forma virtual (código-fonte do sistema construído em alguma linguagem de programação específica, como Java ou PHP). A seguir, falaremos sobre alguns artefatos mais comuns em empresas de software.

Documento de requisitos

O documento de requisitos de software é o artefato que enumera e descreve cuidadosamente todas as funcionalidades que o sistema deve apresentar para que possa cumprir o papel de solucionar o problema do cliente, ou seja, satisfazer as necessidades dos usuários. Dentre as características desse artefato, pode-se estabelecer que ele deve ser construído das seguintes maneiras:

- **Precisa**, devendo incluir, de forma não redundante, todas as necessidades dos usuários;
- **Completa**, sendo que nenhuma funcionalidade ou necessidade deve ser ignorada ou esquecida;
- **Consistente**, de forma que uma definição não possa ser entendida de maneira diferente ao longo do documento;
- **Compreensível**, em uma linguagem acessível a todos os envolvidos no processo de desenvolvimento, desde usuários e clientes a gerentes e profissionais técnicos.

O documento de requisitos é um importante artefato do projeto, uma vez que ele reúne o conjunto completo de funcionalidades do sistema e, conseqüentemente, é usado como um **importante mecanismo de comunicação** entre toda a equipe de desenvolvimento. Assim, programadores sabem o que deve ser construído, ao passo que testadores podem saber quando o sistema está se comportando da maneira como deveria.

Dentre as diversas informações contidas nesse documento, estão:

- **Introdução e visão geral do documento**: utilizada para descrever o escopo do projeto e a orientação sobre o uso do documento, o que inclui também informações e termos específicos sobre o contexto de uso do sistema;
- **Descrição de requisitos funcionais**: lista de funcionalidades do que o sistema deve fazer, como realizar cadastro, realizar pagamento, pesquisar produtos, dentre outras ações e funções;
- **Descrição de requisitos não funcionais**: determina como o sistema deve se comportar, além de adicionar restrições às funcionalidades existentes, como realizar cadastro “com CPF válido” (restrição);
- **Documentação de apoio**: deve incluir, caso necessário, todos os documentos que servem para esclarecer dúvidas e processos.

O documento de requisitos é um importante artefato do projeto de software e, dependendo do método de desenvolvimento, ele pode ser apresentado de outras maneiras.

Diagramas UML

Por definição, um diagrama UML é um tipo de representação de um sistema de software através de uma linguagem unificada de modelagem, conhecida no mundo inteiro e que serve para representar graficamente e documentar a estrutura de sistemas. A linguagem UML fornece meios para ilustrar e permitir o entendimento dos requisitos do sistema e que estão descritos de maneira textual no documento de requisitos.



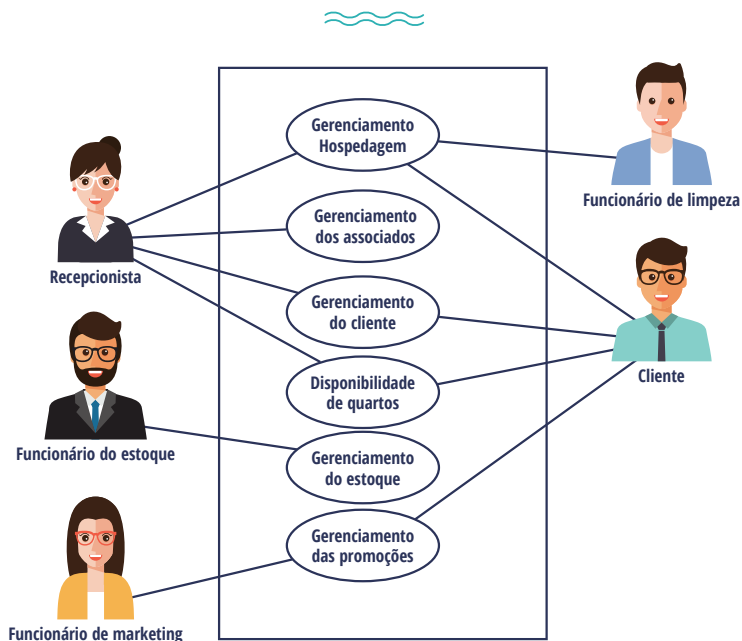
A modelagem UML é comumente desenvolvida em uma atividade seguinte ao levantamento de requisitos, chamada de análise de software, projeto e design ou arquitetura de software. Independentemente de qual termo seja usado para descrever essa atividade, a modelagem UML é responsável por mostrar o comportamento do sistema e sua estruturação.

Nesse sentido, existe uma série de diagramas disponíveis para serem utilizados pela equipe de desenvolvimento, objetivando ilustrar como os usuários utilizam o sistema e como os diversos componentes do sistema interagem entre si. Sendo assim, as diversas notações UML podem ser aplicadas para:

- Esboçar conexão entre as estruturas de um sistema, sendo utilizadas em discussões a respeito do software;
- Servir como documentação de base para atividades de programação, testes e implantação do sistema;
- Servir como base para a atualização de componentes de sistema já desenvolvidos, ou seja, como uma ferramenta de engenharia reversa.

Um dos principais artefatos de projeto de software construídos nesse contexto é o **diagrama de caso de uso**. Esse diagrama UML demonstra como os diversos usuários do sistema interagem com cada uma das funcionalidades. Trata-se de um diagrama simples e que ilustra, com notação simples, todos os requisitos funcionais do sistema, conforme demonstrado no Diagrama 1.

DIAGRAMA 1. DIAGRAMA DE CASO DE USO



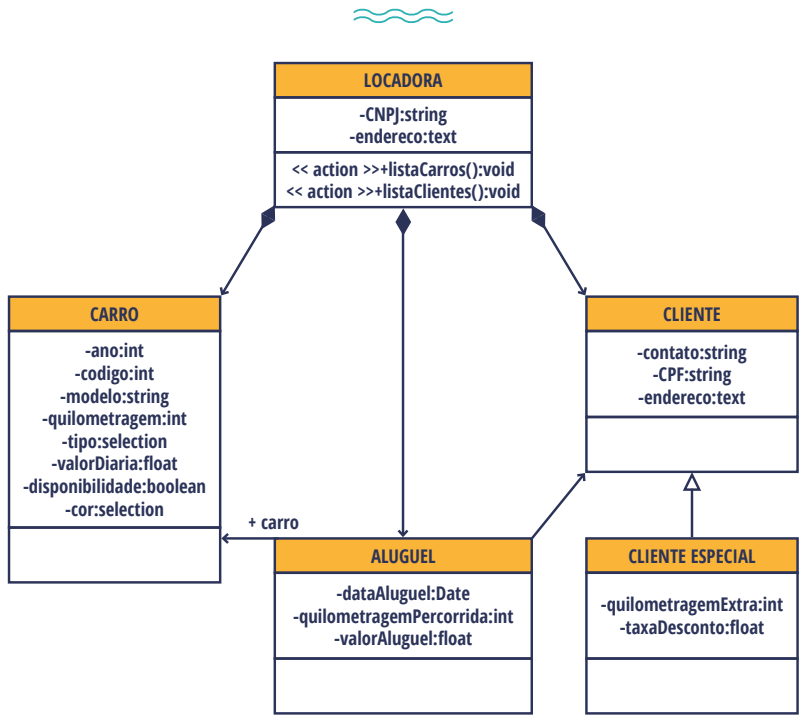
No Diagrama 1, é possível entender que o sistema em desenvolvimento possui, pelo menos, cinco atores distintos e que esses atores interagem com pelo menos seis funcionalidades diferentes. Entretanto, existem restrições quanto às funcionalidades; por exemplo, o usuário funcionário do estoque consegue interagir apenas com a funcionalidade de gerenciamento de estoque, não tendo acesso às demais funcionalidades do sistema.

O diagrama de caso de uso apresenta uma linguagem simples justamente pelo fato de que esse artefato pode ser utilizado para comunicação não somente entre os membros que estão desenvolvendo o projeto como também entre o cliente (que está pagando pelo sistema) ou os usuários (que utilizarão o sistema). Entretanto, existem diversos outros tipos de diagramas UML, como é o caso do diagrama de classes, que apresenta uma linguagem mais técnica para a comunicação entre os programadores do software.

O **diagrama de classes** amplia a ilustração do sistema por meio de uma representação da estrutura do software, mostrando, do ponto de vista da programação, como o código do sistema pode ou deve estar organizado.

Em outras palavras, trata-se de uma representação da programação do sistema sem uma linguagem de programação específica. Assim, uma vez que a estrutura está validada, o programador pode utilizar o diagrama para programar o sistema e construir seu código na linguagem que for determinada pelo projeto. No Diagrama 2, é apresentado um exemplo simples do diagrama de classes.

DIAGRAMA 2. DIAGRAMA DE CLASSES



No Diagrama 2, é possível ver os componentes de um sistema para uma locadora de veículos, os elementos que compõem esse sistema e como eles estão estruturados e interligados. Por exemplo, é possível ver quais informações correspondem ao cadastro do cliente e verificar, também, quais informações possuem os carros e como o aluguel está organizado. Ambos os diagramas, de caso de uso e de classe, além de outros diagramas UML disponíveis, são importantes artefatos de software, produzidos para guiar a programação e, inclusive, os testes de sistema.

Código do sistema

A implementação é a atividade do desenvolvimento de software focada na construção do sistema em si, ou seja, é nessa atividade que o sistema será programado ou codificado. Desse modo, o mais importante artefato dessa atividade, e possivelmente de todo o projeto de software, é chamado de código-fonte. Define-se como código-fonte o conjunto de instruções construído de forma ordenada e lógica, utilizando uma linguagem de programação específica.

Na Figura 1 está apresentado um exemplo de código-fonte de uma função simples, construída para indicar se um determinado número recebido pelo sistema é primo ou não.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 int main(int argc, char *argv[])
6 {
7     int num, sr, flag, i;
8
9     if (argc != 2) return 1;
10    num = atoi(argv[1]);
11    sr = (int)sqrt(num);
12    if (num < 2)
13        flag = 0;
14    else
15    {
16        flag = 1;
17        for (i=2; i<=sr; i++)
18            if (num%i == 0)
19            {
20                flag = 0;
21                break;
22            }
23    }
24    if (flag) printf("%d e' primo\n", num);
25    else printf("%d nao e' primo\n", num);
26    return 0;
27 }
28
```

Figura 1. Captura de tela de código-fonte.

Casos de teste

Os casos de teste são importantes artefatos do projeto de software que são construídos durante a atividade de teste. Esses artefatos servem para verificar se o programa que foi implementado respeita aquilo que foi definido na lista de requisitos do sistema, ou seja, guiam a execução dos testes que determinarão se o sistema apresenta falhas ou não.

De maneira geral, esses casos são os principais artefatos do processo de teste e buscam garantir que o software satisfaça os requisitos do sistema e a necessidade dos usuários.

Dentre outras informações, os casos de teste informam:

- **As informações que serão necessárias** durante os testes, como tipo de dado a ser testado, tipo de caminho a ser seguido no sistema e tipos de parâmetros a serem observados;
- **Os resultados esperados após o teste** e como determinar se um comportamento observado no sistema é comum;
- **A quantidade de testes necessária** e quantas vezes cada teste deverá ser repetido para garantir que o resultado está correto.

Dessa forma, é possível dizer que o caso de teste é um documento de requisitos comparado com o código do sistema, uma vez que para cada funcionalidade do sistema é necessário que exista pelo menos um teste.



Itens de configuração do projeto de software

Até aqui, você conheceu e explorou o conceito de alguns artefatos do projeto de software. É importante, no entanto, considerar que a lista de artefatos pode ser bem maior e incluir muitos outros.

Para isso, é importante conhecer a relação entre artefatos de software e itens de configuração. De maneira geral, artefato de software é tudo aquilo desenvolvido pelos profissionais da engenharia de software. Entretanto, em um determinado momento do projeto, quando a atividade de gerência de configuração iniciar suas ações, alguns desses artefatos serão selecionados, armazenados e monitorados, garantindo, assim, que as mudanças realizadas não interfiram no software final.

Quando um artefato de software passa a ser monitorado pela gerência de configuração, ele passa a ser um **item de configuração** do projeto de software. O procedimento seguido para que um artefato do projeto se torne item de configuração segue as seguintes etapas:

- **Identificação da configuração:** primeira etapa da gerência de configuração que tem como objetivo selecionar quais artefatos poderão se tornar itens de configuração. Nesse processo, é preciso definir uma nomenclatura que possa permitir a correta identificação desses itens, bem como realizar sua descrição formal.

- **Seleção de itens de configuração:** é o processo que segue a identificação dos itens a partir da lista de artefatos de software. Nessa ação, ocorre a definição da criticidade dos itens para o projeto, bem como a determinação de dependências entre esses itens. Por ser uma atividade de planejamento, a seleção de itens também inclui análises do impacto de possíveis modificações de itens, com permissões de modificações e complexidade de mudanças.

- **Controle da configuração:** ação encarregada de controlar e acompanhar a evolução dos itens de configuração. Dessa forma, caso uma mudança seja executada, é possível saber exatamente quais itens serão impactados e de que maneira foram impactados.

- **Acompanhamento da situação da configuração:** atividade focada em armazenar as informações geradas pelas demais ações de identificação, seleção e controle. Dessa forma, por meio do monitoramento, é possível definir estratégias de melhoria no processo de gerência de configuração.

- **Auditoria da configuração:** processo de revisão dos planos de configuração, dos dados dos itens de configuração e dos métodos aplicados nas etapas de identificação, seleção, controle e acompanhamento.

Dessa maneira, por meio da execução dessas ações, os profissionais que trabalham na gerência de configuração do projeto de software podem transformar artefatos gerais do projeto em itens de configuração e acompanhar sua evolução ao longo do desenvolvimento.

Nesse ponto, é importante destacar, mais uma vez, o papel da gerência de configuração no desenvolvimento de software, uma vez que se trata da atividade responsável por planejar e executar todas as ações necessárias para garantir o rastreamento correto dos artefatos de software, evitando e resol-

vendo os problemas que ocorrem em projetos quando alterações são realizadas de maneira inadequada ou sem nenhum tipo de controle.



Versionamento de itens do projeto de software

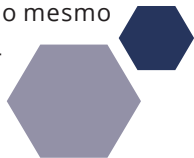


O processo por trás da gerência de configuração de software exige muito cuidado dos profissionais que trabalham na área, principalmente quando o assunto é o planejamento e a execução das ações, para garantir a integridade dos itens de configuração do projeto. Na realidade, desde o processo de seleção de item até o seu controle efetivo é necessário buscar estratégias que evitem ambiguidade e problemas de identificação. Nesse contexto, a principal maneira de executar a correta identificação de itens e melhorar a comunicação e o entendimento de todos sobre esses itens é o chamado **processo de versionamento**.

Define-se como **versionamento de itens** o processo de **criação de nomes específicos** e de uma **terminologia efetiva** para identificar as versões do mesmo item. Dessa forma, dado que um item de configuração é um determinado artefato do projeto de software e que esse artefato pode sofrer diversas alterações até ser finalizado, o versionamento é o processo pelo qual a equipe de gerência de configuração acompanha e rastreia as diversas versões desse item, podendo, assim, garantir sua integridade.

Sendo a gerência de configuração uma atividade de suporte às demais atividades do desenvolvimento, o controle de versões é visto como uma extensão natural do processo, pois permite que os demais profissionais possam realizar modificações paralelas no mesmo item por meio de um processo coerente e padronizado, evitando prejuízos futuros.

Sendo assim, a cada nova alteração de um item, uma nova versão é gerada, integrada e mantida pela gerência de configuração de maneira segura e efetiva. O versionamento é uma estratégia importante para o desenvolvimento de software para equipes que trabalham no mesmo ambiente e, especialmente, quando se trata de um processo de **desenvolvimento de software global**, através do qual os diversos profissionais trabalham geograficamente separados.





CONTEXTUALIZANDO

No contexto de desenvolvimento de software atual, muitas empresas, especialmente multinacionais, trabalham com o conceito de desenvolvimento de software distribuído. Nesses ambientes, os diversos profissionais podem trabalhar no mesmo produto mesmo estando geograficamente separados. É comum, por exemplo, que empresas de telefonia desenvolvam seus sistemas em um país e realizem os testes em outro local, ou que módulos do mesmo sistema sejam desenvolvidos separadamente em locais diferentes. Para tanto, é necessário um eficiente processo de gerência de configuração.

Por meio do esquema de **versionamento de itens**, cada versão indica o estado de um item em um dado momento, como se fosse uma “fotografia” do item de configuração no momento em que foi armazenado. Por exemplo, a versão X do sistema não possui o método de log-in com e-mail e senha implementados; no entanto, a versão Y possui o método com log-in e senha prontos. Dessa forma, a atividade de controle de configuração cuida do processo de versionamento, buscando manter seguras as diversas versões de um mesmo item e determinando, de maneira eficiente, a correta nomenclatura para cada um.

Um exemplo comum de versionamento de itens nesse contexto é o padrão XYZ, em que, geralmente:

- X é a posição do versionamento que recebe o número que representa grandes mudanças em uma determinada versão do item. Inicia-se com 1 e é incrementado conforme novas versões forem desenvolvidas e entregues. Geralmente, acontece quando grandes mudanças são realizadas, como a inclusão de uma nova funcionalidade no software ou o lançamento de uma atualização completa;
- Y é a posição do versionamento que recebe o número modificado a cada nova versão implantada e que esteja em produção. Por exemplo: é utilizado para a entrega de uma versão de testes de alguns usuários específicos, ou em alguma apresentação geral do andamento do projeto para o cliente. O Y significa que não houve mudanças globais no sistema, estando o software, e consequentemente seus artefatos, com as mesmas funcionalidades que foram planejadas;
- Z é a posição do versionamento com mudanças ou correções de emergência, realizadas em versões (releases) e liberadas para uso paralelamente ao desenvolvimento da próxima versão.

Para entender, na prática, como o versionamento XYZ funciona, suponha que um jogo para celular bastante esperado por fãs de todo o mundo esteja sendo desenvolvido e será liberado em breve. Nesse esquema, ele passará pelas seguintes situações:

- Na entrega da primeira versão do produto liberado para download, o jogo receberá o label de versionamento 1.0.0, ou simplesmente 1.0;
- A segunda versão do jogo será liberada sem nenhuma nova funcionalidade implementada, contendo apenas novas telas. Desse modo, ela receberá o label de versionamento 1.1;
- Em um determinado momento, após a liberação da versão 1.1, os usuários começarão a relatar uma falha na versão, informando que o jogo está comprometendo seu sinal de Wi-Fi. Sendo assim, a versão com a falha de Wi-Fi corrigida será lançada como 1.1.1, representando a correção emergencial necessária;
- Finalmente, seis meses após a primeira versão do jogo ter sido liberada, uma nova atualização contendo a nova funcionalidade de chat entre os jogadores foi implementada e finalizada. Nesse caso, a nova versão será a 2.0.0, ou simplesmente 2.0, uma vez que uma alteração global foi efetuada no sistema. Dessa forma, a partir desse ponto, as novas versões serão editadas como 2.1, 2.2 e assim por diante, até que uma versão com alterações globais seja produzida.

Nesse ponto, é importante ressaltar que, mesmo com um exemplo focado no lançamento de um sistema, a prática de versionamento XYZ funciona e pode ser utilizada para todos os demais itens de configuração do projeto, como versões do documento de requisitos ou versões dos diagramas UML.

Finalmente, como parte do conteúdo sobre gerência de configuração e o processo de versionamento de itens, existem alguns conceitos e terminologias usados pelos membros da equipe responsáveis por gerenciar as mudanças do projeto, tais como:

- **Workspace:** termo usado para se referir ao ambiente de trabalho;
- **Baseline:** é o conjunto principal de arquivos e artefatos do projeto (diagramas, documentos) que estão ligados diretamente a um módulo específico do software e que podem ser alterados ao longo do tempo. Sendo

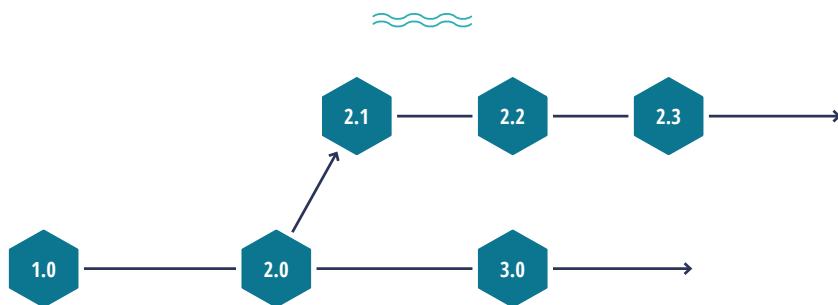
assim, contém todos os itens de configuração de uma determinada versão e indica, de forma lógica e ordenada, a evolução das mudanças que ocorreram ao longo do seu desenvolvimento;

- **Tag:** rótulo que identifica uma baseline e suas várias revisões – de maneira geral, o nome dado a cada versão de um item modificado. Costuma-se usar tags para definir todos os arquivos que compõem um item de configuração, rotulando, assim, todos os arquivos associados a esse item;

- **Build:** é uma versão ainda incompleta de um sistema que está sendo desenvolvido. Entretanto, para ser uma build, essa versão precisa possuir certa estabilidade, podendo ser instalada e executada mesmo sem ter sido finalizada. Uma build pode apresentar algumas limitações, que são conhecidas pela equipe de desenvolvimento e que serão resolvidas conforme o software vai sendo desenvolvido;

- **Branch:** versão de uma baseline que inicia na versão central e evolui, de maneira independente, para diversas versões ao longo do tempo. É como se a versão 2.0 continuasse passando por evoluções mesmo com a nova versão 3.0 disponível. Isso acontece com muitos sistemas operacionais ou aparelhos celulares, uma vez que a empresa precisa dar suporte para usuários que usam o sistema antigo. O Diagrama 3 ilustra a definição de Branch;

DIAGRAMA 3. BRANCH DE SOFTWARE

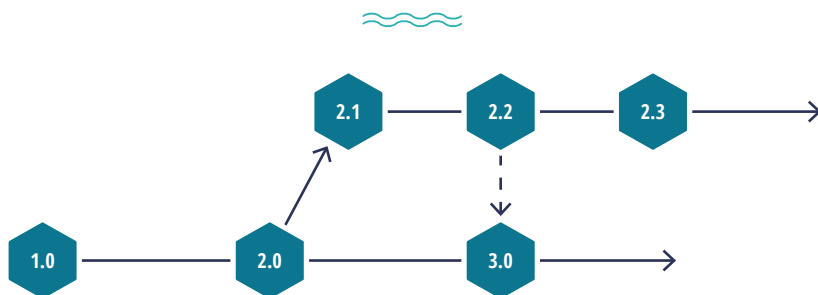


- **Release:** é como é chamada uma determinada versão do sistema entregue para que o usuário possa utilizá-la. Em outras palavras, trata-se de uma versão do sistema liberada para uso;

- **Merge:** é o processo de unificação de diferentes versões de um mesmo item de configuração. Por exemplo, suponha que a versão emergência

2.2 esteja unificada e adicionada na versão 3.0, para que a junção corrija a falha existente, observada anteriormente. Esse processo de junção de versões é chamado de merge e pode ser visto no Diagrama 4.

DIAGRAMA 4. MERGE DE SOFTWARE



Controle de configuração e comitê de mudanças

A grande característica da gerência de configuração é o controle das mudanças que acontecem ao longo do desenvolvimento do software. Nesse sentido, a sincronização requerida nesse processo é, sem dúvida, um grande desafio tanto para mudanças e alterações individuais em itens quanto para mudanças em baselines e releases de versões para os usuários. Esse processo de sincronização e cuidado com versões é denominado **controle de configuração**.

Imagine o que aconteceria se cada membro envolvido no desenvolvimento do software fizesse a alteração que quisesse no momento que quisesse? O resultado seria catastrófico! Seria praticamente impossível saber quando uma mudança foi feita e o que ela impactou. Seria difícil liberar novas versões de sistema para os usuários, uma vez que não seria possível saber adequadamente qual versão apresenta as correções necessárias – o desenvolvimento de software se tornaria uma grande confusão. Dessa forma, o controle de configuração trabalha para manter um registro atualizado de todas as solicitações e realizações de mudanças em qualquer termo.

O controle da configuração estabelece os processos para que os profis-

sionais possam propor, requerer, avaliar e aprovar mudanças nos itens de configuração. Concomitantemente, ele estabelece os procedimentos para a publicação, o acompanhamento e a implementação das mudanças. Esse processo também identifica os profissionais responsáveis por mudanças em diversos níveis. Sob a responsabilidade do controle da configuração está, também, a identificação dos membros do comitê de mudanças.

O comitê de mudanças é um grupo de profissionais especialmente encarregado de avaliar as propostas e os requerimentos de mudanças de itens da configuração, bem como acompanhar o processo de mudança, buscando garantir a qualidade do processo.

Adicionalmente, cabe ao comitê estabelecer regras e critérios para mudanças e para o controle de configuração, de maneira a fornecer, para os diversos profissionais de engenharia de software, um serviço complementar ao serviço oferecido por sistemas de controle de versão utilizados nas empresas, tais como o Redmine, o Mantis, o Bugzilla ou o JIRA. Todas essas ferramentas têm uma excelente aceitação, sendo consideradas robustas e eficientes.

O controle de configuração e o comitê de mudanças estão comumente associados a uma atividade comum, conhecida como **integração contínua**. Essa atividade é responsável por garantir que as mudanças no projeto sejam construídas, testadas e relatadas o mais rápido possível para evitar que versões sejam perdidas.



EXPLICANDO

A integração contínua é um processo realizado por meio da combinação de duas ferramentas: uma utilizada para a construção do software e outra utilizada para monitorar as alterações nas versões existentes. Em outras palavras, sempre que uma mudança acontecer em uma versão, a ferramenta de monitoramento envia os dados para a ferramenta de construção de software para que essa mudança seja incorporada pelo processo de desenvolvimento como um todo.

Finalmente, é importante ressaltar que uma solicitação de mudanças não será sempre aprovada. Desse modo, uma solicitação pode ser negada e, diante disso, a gerência de configuração deve explicar por qual motivo

determinada mudança não é passível de ser realizada. Nesse sentido, a negação pode ocorrer devido a questões relacionadas ao custo, tempo ou de impacto em outros itens. Caso não seja negada, uma mudança deverá ter seus reais impactos confirmados após sua conclusão. No fim do processo, é necessário realizar a atualização dos documentos de registro de versões e encerrar o pedido de mudança, registrando seu resultado.

De maneira geral, o controle de configuração e o comitê de mudanças têm o mesmo objetivo da gerência de configuração, que pode ser resumido na resolução de algumas perguntas básicas, conforme apresentado no Quadro 2.

QUADRO 2. OBJETIVO DO CONTROLE DE CONFIGURAÇÃO E DO COMITÊ DE MUDANÇAS



Aspecto	Fator analisado
Ocorrência	Quais mudanças ocorreram durante o desenvolvimento do software e quando tais mudanças ocorreram?
Necessidade	Qual é a necessidade da realização de uma determinada mudança e qual é o seu impacto no produto?
Extensão	Quais versões do sistema e dos itens de configuração foram afetadas e como serão atualizadas?
Responsáveis	Quem foi o profissional que solicitou a mudança e quem foi o profissional que efetivamente realizou tal mudança?
Contingência	Quais as alternativas para retroceder para uma versão anterior à mudança, caso seja necessário?
Resultado	O software em desenvolvimento continua estável depois que a mudança foi concluída?

O resultado de todo o processo de gerência de configuração é garantir a entrega de um produto de software de qualidade que consiga resolver o problema planejado no início do projeto. A qualidade obtida nessa atividade

é complementar aos processos de teste de software, que visam efetivamente testar o sistema e todas as suas funcionalidades. Nesse caso, a gerência de configuração fornece suporte à qualidade do produto ao garantir que erros não sejam inseridos no sistema por falhas na manutenção das diversas versões do produto.

Além disso, a gerência de configuração ainda presta apoio às atividades gerenciais do processo de desenvolvimento de software, podendo apontar, por exemplo, como uma mudança poderá impactar negativamente no cronograma do projeto ou nos custos de desenvolvimento de software. Entretanto, as ações necessárias para contornar esses possíveis problemas são de responsabilidade do gerente de projetos. Cabe à gerência de configuração, nesse sentido, avaliar se uma mudança pode ser realizada de maneiras diferentes ou em momentos diferentes, além do mais importante, manter a sincronização das ações de mudança e o controle das versões.

Sintetizando



Parabéns! Você acabou de finalizar essa unidade, que trata de várias questões específicas sobre a gerência de configuração de software. No começo da unidade, você conheceu, de maneira aprofundada, os artefatos do projeto de software e os principais artefatos produzidos por alguns profissionais do processo.

Em seguida, aprendeu como esses artefatos se tornam itens de configuração e passam a ser monitorados pela equipe de gerência de configuração. Por fim, entendeu o conceito de controle de configuração e o trabalho do comitê de mudanças no processo de versionamento de itens e de acompanhamento de mudanças do projeto.

De maneira geral, é importante lembrar que o desenvolvimento de software pode produzir dezenas de documentos e outros subprodutos enquanto desenvolve o software que será entregue para os usuários. Nesse processo, nem todos os artefatos necessariamente se tornarão itens de configuração, entretanto aqueles que forem selecionados como item de configuração deverão seguir a nomenclatura e o processo de versionamento definido pela equipe.

Considerando o sistema como um todo, o versionamento é o processo de dar nomes ou códigos a cada versão que é liberada, ou no ambiente de desenvolvimento ou para o cliente. Existem maneiras específicas de nomear os produtos que são liberados para uso dos usuários, para nomear versões que precisam de correção emergencial ou para nomear versões que receberam atualização completa. Essa é uma atividade que requer qualidade e controle para garantir que não existam problemas entre as diversas versões tanto do software quanto dos demais itens do projeto.

Espero que este material tenha ajudado você a entender a seleção de itens de configuração e o seu controle ao longo do desenvolvimento do software. Lembre-se sempre de buscar mais conteúdo para expandir o seu conhecimento. Mantenha-se motivado!

Referências bibliográficas



BERSOFF, E. H. Elements of software configuration management. **IEEE Transactions on Software Engineering**, v. 10, n. 1, 1984.

BOURQUE, P.; FAIRLEY, R. E. **Guide to the software engineering body of knowledge**. 3. ed. Los Alamitos: IEE Computer Society Press, 2014.

DANTAS, C. Gerência de configuração de software. **Engenharia de Software Magazine**, 2009.

SANCHES, R. Gerência de configuração. In: ROCHA, A. R. C. et al. **Qualidade de Software: teoria e prática**. São Paulo: Prentice-Hall, 2001.

UNIDADE

3

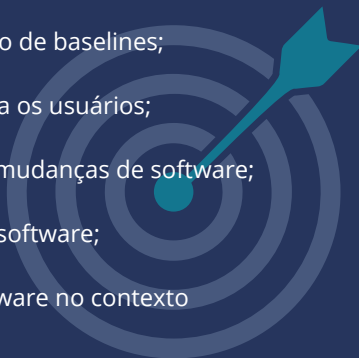
GERENCIAMENTO
DE CICLO E VIDA
DE MUDANÇAS DE
SOFTWARE



ser
educacional

Objetivos da unidade

- Apresentar os conceitos relacionados à geração de baselines;
- Discutir o processo de release de software para os usuários;
- Entender a importância do gerenciamento de mudanças de software;
- Compreender o ciclo de vida de mudanças de software;
- Entender a importância das mudanças de software no contexto organizacional.



Tópicos de estudo

- **Geração de baselines e criação de releases**
- **Gerenciamento de mudanças**
- **Ciclo de vida das mudanças**
- **Procedimentos para mudanças**





Geração de baselines e criação de releases



Construir um sistema e colocá-lo no mercado não é exatamente uma tarefa fácil, especialmente se levarmos em consideração a grande quantidade de empresas concorrentes que existem hoje em dia em um mercado cada vez mais globalizado. Posto isso, torna-se relevante evidenciar, mais uma vez, o quanto o mercado de desenvolvimento de software é extremamente globalizado. Todavia, isso não quer dizer que empresas de software pequenas e regionais não possuam espaço para vendas. Pelo contrário, essas empresas têm como vantagem o oferecimento de produtos mais especializados ou direcionados para seus clientes. Entretanto, independentemente de seu tamanho, as empresas de software podem enfrentar uma grande desvantagem: caso haja um sério problema no sistema que está sendo vendido, isso poderá ocasionar a migração dos usuários para um sistema semelhante.

Hoje em dia, entregar produtos de qualidade e que sejam realmente úteis para seus usuários finais é o grande objetivo das empresas de desenvolvimento de software. Para empresas pequenas, produtos de baixa qualidade ou que apresentem falhas depois de entregues podem acarretar na perda de um contrato importante. Para empresas grandes e já consolidadas no mercado, isso pode ocasionar uma propaganda extremamente negativa, além de causar danos na imagem da empresa. No entanto, é importante se ter em mente que muitas vezes o problema evidenciado não é exatamente uma falha, mas simplesmente uma não adaptação à determinada tecnologia por parte dos usuários.

Como exemplo disso, há o caso de uma grande empresa de tecnologia que tem como seu principal produto um dos sistemas operacionais mais utilizados no mundo. Há alguns anos atrás, esta empresa ofereceu aos seus clientes uma nova versão do sistema operacional que não possuía a opção de menu Iniciar visível na tela, em formato de botão. Para obter acesso a esta opção, os usuários deveriam arrastar o cursor do mouse para o canto inferior esquerdo da tela e o menu apareceria instantaneamente. O problema foi que os usuários não gostaram da ideia e, após inúmeras reclamações, a empresa teve que lançar uma nova versão do mesmo sistema, adicionando o botão no mesmo lugar em que ele estava em versões anteriores.

Esse é um caso muito interessante sobre o que acontece quando dois conceitos relativos ao sistema e ao design gráfico, User Experience e User Interface, não dialogam. Nesse exemplo, o sistema não apresentava nenhuma falha, ou seja: a interface do sistema em momento algum foi comprometida. Entretanto, os usuários começaram a relatar que a ausência do botão do menu estava interferindo diretamente em sua experiência ao utilizar o sistema. Para ilustrar, vamos entender a diferença entre **User Interface** e **User Experience** na Tabela 1.

TABELA 1. DIFERENÇAS ENTRE UI E UX



User Interface	User Experience
Foco no design visual do software	Foco na interação com o software
Criação de layouts de tela	Criação de protótipos
Definição do design gráfico do sistema	Definição de arquitetura da tela
Pesquisa sobre elementos gráficos	Pesquisa sobre os usuários finais
Definição das cores da tela	Definição de cenários de uso



ASSISTA

Problemas relativos às interfaces gráficas do sistema e à experiência do usuário com o software são mais comuns do que se imagina, e podem ocorrer a qualquer momento. É devido a isso que as áreas de UI (User Interface) e UX (User Experience) são extremamente importantes no processo de desenvolvimento de software e é também por isso que seus artefatos são comumente adicionados na lista de itens de configuração. Posto isso, é possível compreender porque é essencial ter conhecimento sobre essas áreas. Então, entenda a diferença entre esses dois conceitos com o *UX/UI Design*? *Saiba a diferença entre UI e UX Design*, do canal Chief of Design.



No caso real da empresa que removeu um elemento gráfico da tela do sistema, e posteriormente enfrentou resistência por parte dos usuários, pode-se afirmar que a equipe de desenvolvimento precisou de confiança extrema em suas estratégias de gerência de configuração. É através dessas estratégias que

se torna possível rever a baseline de diversas versões do sistema e encontrar aquela que pudesse ser utilizada para corrigir o problema relacionado à ausência de botão, denunciado pelos usuários. Somente desta maneira seria viável lançar uma nova release do sistema. Todo esse processo de mudança foi apoiado pelas práticas da gerência de configuração.

O que aconteceu no exemplo exposto, desde o problema até a elaboração da solução, foi o seguinte:

1) A empresa, enquanto desenvolve a nova versão do sistema que irá lançar, faz uso de estratégias de gerência de comunicação;

2) A gerência de configuração garante o correto versionamento do sistema, armazenando diversas versões e preparando-se para rastrear essas versões, caso necessário;

3) A empresa libera a nova versão do sistema para os usuários e, em todo o mundo, pessoas começam a utilizá-la;

4) Os usuários ficam insatisfeitos com a nova tela do sistema, especialmente com a ausência de um elemento gráfico, o botão, com o qual eles estavam acostumados;

5) A empresa, visando garantir a satisfação dos usuários, prepara uma nova versão deste sistema; desta vez com o botão, cuja ausência estava causando estranheza nos usuários;

6) A empresa procura o time de gerência de configuração, que rapidamente consegue identificar uma versão anterior do sistema lançado, estável e com o botão em seu lugar;

7) A equipe de desenvolvimento realiza a mudança na versão identificada pela gerência de configuração, preparando uma nova versão corrigida para ser lançada;

8) A empresa relança o sistema e, ao seguir os pedidos dos usuários, mantém sua posição e reputação no mercado.

Esse exemplo real demonstra como a gerência de configuração está alinhada com o desenvolvimento e o sucesso de um produto de software. Neste contexto, com alterações e mudanças que podem aparecer por diversos motivos durante o desenvolvimento do software, ou mesmo após seu lançamento, existem dois conceitos comuns quando tratamos de versões do sistema, denominados de baselines e releases.

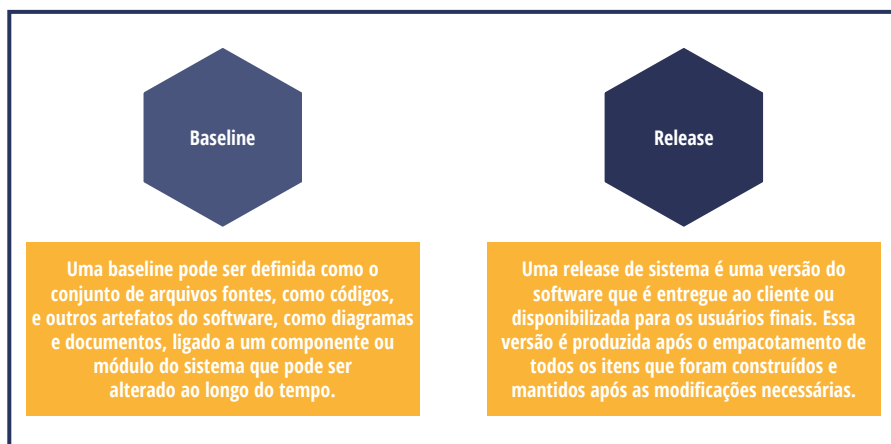


Figura 1. Baseline e release.

O conceito de baseline está diretamente ligado ao gerenciamento de configuração. Além disso, **um artefato do projeto de software** pode passar por diversas modificações ao longo de seu desenvolvimento, ou seja, pode ser submetido a diferentes configurações, em diferentes momentos e através de diferentes ações.



EXPLICANDO

Lembre-se, artefato do projeto de software é tudo aquilo que é produzido pelos profissionais ao longo do desenvolvimento do sistema, como, por exemplo, o código do programa, os diagramas de arquitetura e os protótipos de telas, assim como os demais documentos. Quando esses artefatos são armazenados e controlados pela gerência de configuração, eles passam a ser considerados, também, itens de configuração do projeto.

De maneira geral, uma baseline é vista como um marco no desenvolvimento do software. Este marco indica que um conjunto de itens de configuração do software foi definido e será armazenado de maneira consistente para permitir alterações, caso sejam necessárias. Além disso, a aprovação destes itens foi obtida através de uma revisão técnica formal. Neste sentido, pode-se afirmar que a baseline é o empacotamento adequado de todos os itens que fazem parte de uma dada versão específica do projeto e que, além disso, possui as características necessárias para ser acompanhada e monitorada pela equipe de configuração.

Posto que os elementos de uma especificação de projeto devem ser armazenados na baseline, pode-se afirmar que, quando uma determinada versão do sistema é armazenada, a sua baseline deve conter o documento de requisitos que apresenta a descrição formal de todas as funcionalidades que foram programadas e finalizadas nessa versão, por exemplo. Assim, se em algum momento falhas forem encontradas na versão do sistema, ou mesmo se alguma mudança for realizada na versão presente no pacote (baseline), é necessário garantir que todas as partes da especificação serão revisadas, corrigidas e acordadas. Portanto, a baseline pode ser atualizada somente após todos os itens serem revisados. Nesse sentido, toda vez que uma nova configuração de itens é aprovada ou sempre que houver alterações em um item de configuração uma nova baseline é gerada, para fins de comparação com a anterior e a consequente aprovação, ou não, das mudanças.

O processo de criação de baselines deve incluir informações relevantes que permitam a identificação de suas configurações. Entre estes dados importantes, devem ser destacados:

- O **evento que cria a baseline**, uma vez que ele determina, por exemplo, se o sistema sofreu uma atualização ou se está passando por uma mudança. Assim sendo, o evento determina quando uma configuração poderá ser aprovada;

- Os **itens de configuração que farão parte da baseline**. Ou seja, é preciso definir o que vai estar dentro do pacote da baseline como, por exemplo, o código do sistema em conjunto com os protótipos de tela ou apenas o código do sistema e assim por diante. Considere que uma grande quantidade de artefatos pode ser produzida no processo de desenvolvimento de software, mas nem todos esses artefatos precisam ser transformados em itens que estarão na mesma baseline ou pacote;

- Os **procedimentos utilizados para manter e gerenciar a baseline**. Essa informação refere-se às permissões de acesso e modificação, assim como aos métodos utilizados para que a baseline seja gerenciada;

- Por fim, outra informação importante é o **nível de autoridade necessário** para aprovar mudanças nos itens de configuração sob o controle da baseline, ou seja: as definições de acesso e de aceitação de mudanças. Essa informação refere-se a quem tem o poder de realizar o que neste processo.

Um exemplo simples para entender as baselines é imaginar uma empresa que desenvolve softwares para telefones celulares. Por serem equipamentos utilizados no mundo todo, a empresa deve considerar que precisa produzir sistemas que se adaptem às legislações e realidades de diversos países. Todavia, para simplificar, considere que o aparelho dessa empresa é vendido unicamente nos Estados Unidos e no Brasil. Levando em consideração somente esses dois países, já é possível divisar mudanças expressivas no mesmo sistema para o mesmo aparelho celular, como apresentado na Tabela 2.

TABELA 2. EXEMPLO DE DIFERENÇAS ENTRE BASELINES



Fator (Relativo a um sistema para celular)	Descrição
Rede e dados móveis	As configurações de redes móveis, acesso e utilização de dados são diferentes para os dois países.
Idioma	As especificações de idioma e variações de linguagem são diferentes.
Localização	A forma de utilização do GPS, assim como dados de navegação e geolocalização, são diferentes.
Chamadas e Serviços	As configuração relativas ao formato de ligações, chamadas e envio de mensagens pelo celular são diferentes nos dois países.

A Tabela 2 apresenta alguns exemplos clássicos, mas é possível elaborar uma lista extensa ao imaginar as diferenças que o mesmo sistema para o mesmo celular pode apresentar, apenas levando em consideração a mudança de um país para o outro. O que pode-se concluir com isto é que somente uma baseline para o código e demais itens do sistema não será suficiente para acompanhar o desenvolvimento do software. Seguindo o exemplo apresentado, os itens de configuração da baseline do sistema que será lançado nos Estados Unidos possivelmente serão diferentes dos itens da baseline que será lançada no Brasil, uma vez que leva em consideração, por exemplo, restrições de sinal, banda larga e serviço de operadoras, entre outros itens. E é devido a isso que mais de uma configuração de produto precisa ser gerada, ou seja: mais de uma baseline é criada, fazendo com que o mesmo sistema apresente pacotes diferentes. Posto isso, cabe à equipe de gerenciamento de configuração controlar as mudanças nas duas baselines e em quantas outras precisarem ser criadas ao longo do processo de desenvolvimento.

Este não é o único exemplo prático que podemos utilizar para ilustrar a criação de baselines diferentes. Na realidade, a separação de baselines de software valendo-se do local de lançamento como critério é apenas um dos muitos exemplos que podem ser citados. Imagine, por exemplo, que a mesma empresa que desenvolve softwares para celulares vendidos no Brasil e nos Estados Unidos também possua uma variedade de modelos de outros aparelhos, alguns com mp3 player, outros com TV Digital, uns com uma câmera, outros com duas câmeras. Neste exemplo, cada modelo apresenta como particularidade algum tipo de especificação diferente e exclusiva, e é por isso que o mesmo software que está sendo desenvolvido com baselines diferentes para cada país também precisa ser especializado, a fim de garantir as especificações de cada modelo diferente. Sendo assim, novas baselines podem ser criadas, como ilustrado na Figura 2.

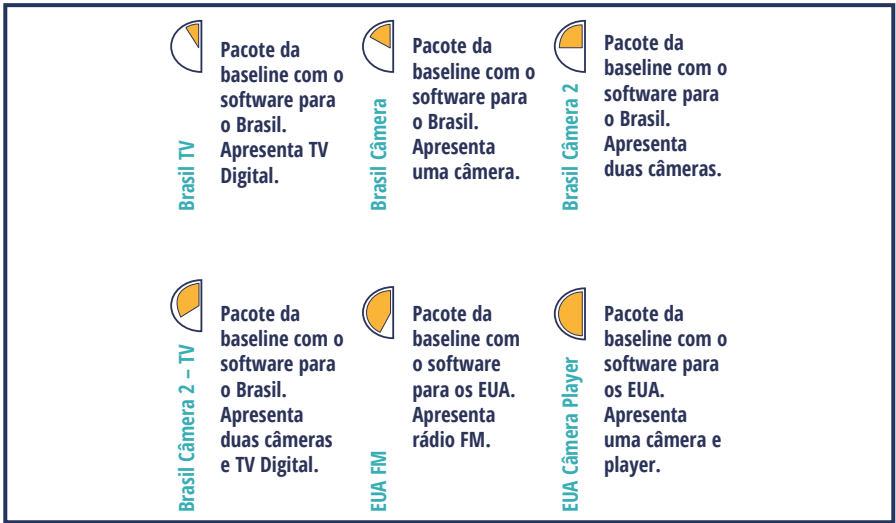


Figura 2. Exemplos de baselines diferentes criadas pelas particularidades do software.

Em um projeto de software é comum a existência de baselines heterogêneas, com configurações diferentes e que passam por atualizações e mudanças ao longo de todo o processo, como ilustrado na Figura 2. Neste sentido, é muito comum a existência de baselines com foco no código do sistema que está sendo programado. Entretanto, essas baselines podem incluir também dados de testes, diagramas UML, protótipos de tela, documentos de requisitos e uma série de itens de configuração diferentes.

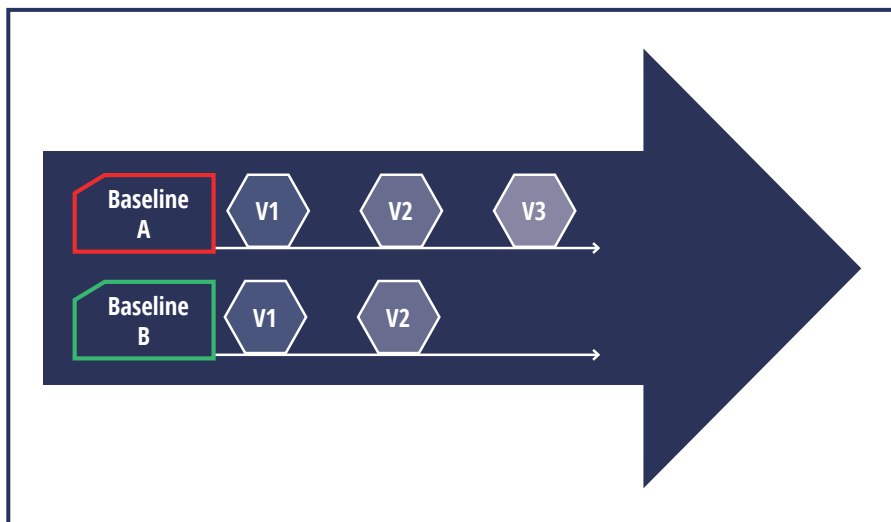


Figura 3. Evolução de baselines ao longo do desenvolvimento.

Na Figura 3 é possível observar o processo de evolução de duas baselines diferentes de um mesmo produto de software, as quais estão evoluindo com o passar do tempo e armazenando versões distintas. No caso da Base A, é possível perceber que o sistema já possui três versões diferentes, ao passo que a Base B possui duas. Não existe um padrão para criação de versões, visto que uma baseline pode produzir quantas forem necessárias. Nesse caso, o importante é garantir o rastreio dessas versões, além de acompanhar e gerenciar seu processo de mudança e evolução.

É importante ressaltar que a grande maioria das versões de uma baseline é produzida no ambiente de desenvolvimento do sistema, o que significa que essas versões são criadas dentro da empresa e não são liberadas para uso. Muitas delas ainda são intermediárias e, apesar de apresentarem bom funcionamento, podem estar incompletas. Considerando as baselines apresentadas na Figura 2, uma versão intermediária da baseline “Brasil TV” poderia conter uma versão do sistema que consegue utilizar o hardware de TV do aparelho celular, mas ainda não é capaz de localizar canais, por exemplo. Sendo assim, somente após várias versões dessa baseline seria possível ter uma versão que poderia ser liberada para uso externo, ou seja, para fora do ambiente de desenvolvimento. Quando uma versão de sistema extrapola o ambiente de desenvolvimento e é disponibilizada para os usuários, ela passa a ser conhecida como release de sistema.

Um release do sistema é uma versão estável do software distribuída para os usuários finais, em que cada release de software deve incorporar um conjunto de funcionalidades bem definidas e completamente construídas para ser executado. Um release é, então, o resultado da promoção de uma das versões produzidas na baseline. Considere, por exemplo a Figura 4.

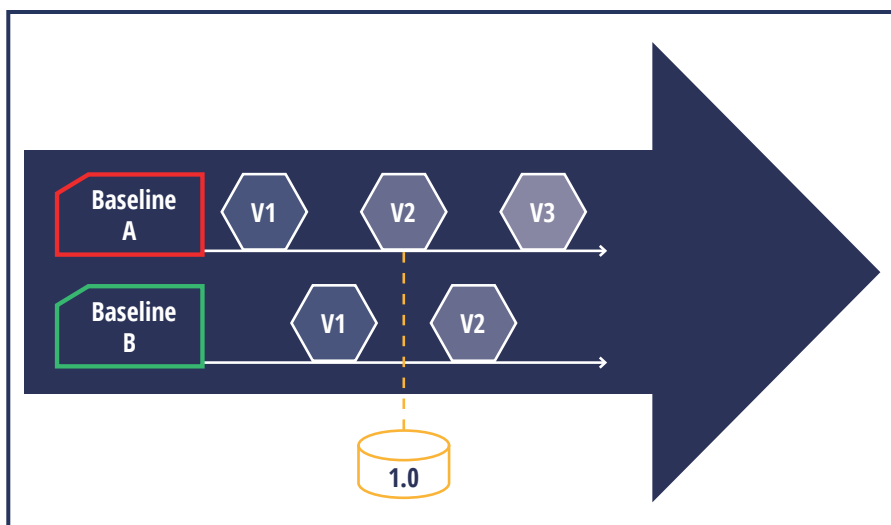


Figura 4. Criação de release.

Voltando ao exemplo da empresa de celulares, considere que na Figura 4:

- 1) Sua totalidade, a seta grande, representa o sistema finalizado do celular, ou seja, o software completo, que executa todas as atividades;
- 2) A Baseline A representa o subsistema responsável pelas atividades da câmera do aparelho;
- 3) A Baseline B representa o subsistema responsável pelas ações do sistema de localização e GPS.

Perceba que, considerando o software por completo, a câmera do aparelho já possui seu software na terceira versão. Enquanto isso, o GPS está com seu software na segunda versão. Agora, imagine que a empresa resolveu lançar a primeira versão do novo celular para seus clientes. No momento do lançamento é preciso escolher uma versão em que todos os subprodutos de software estejam estáveis, completos e sem falhas. Ao realizar uma análise na totalidade do sistema, a empresa percebeu que:

1) As versões 1 e 2 da câmera (baseline A) estão funcionando de forma adequada, como planejado;

2) A versão 3 da câmera (baseline A) apresentou um problema e o foco da câmera não está sendo ajustado automaticamente;

3) A primeira versão do GPS (baseline B) está funcionando como o esperado e planejado no desenvolvimento, ou seja, sem falhas;

4) A segunda versão do GPS (baseline B) não está funcionando bem, uma vez que o aparelho não consegue detectar a localização correta do usuário em ambientes fechados.

Antes que o sistema seja liberado para os usuários, a empresa precisa recorrer à gerência de configuração para **criar o pacote de release** com as versões mais atuais e estáveis do sistema, pois não adianta utilizar somente as versões mais atuais, uma vez que elas podem apresentar falhas. Uma release tem de conter a versão do software que pode ser utilizada pelo usuário sem imperfeições. Sendo assim, a empresa decide lançar a versão 1.0 do sistema, empacotando no release a segunda versão da câmera e a primeira versão do GPS, mesmo que outras estivessem disponíveis neste momento, conforme demonstrado na Figura 4.



EXPLICANDO

A criação de releases é um processo de empacotamento do software com tudo que será necessário para sua execução pelo usuário, juntamente com todos seus documentos de apoio. O código executável de programas e todos os arquivos de dados associados devem ser coletados e identificados.

Antes da geração do release, a equipe de configuração deve verificar se todas as baselines constituintes foram adicionadas à versão que será gerada e entregue para os usuários. Além disso, é preciso apurar se todas as solicitações e execuções de mudanças ao longo do desenvolvimento foram devidamente verificadas, validadas e estão em conformidade com o projeto. Caso algum problema seja encontrado, a equipe deve procurar resolvê-lo antes da geração da release. É importante ressaltar que problemas que ocorrerem após o lançamento do sistema podem custar muito mais caro do que aqueles identificados antes do lançamento da release.

Para realizar o lançamento da versão do usuário, a equipe de configuração deve atribuir uma tag para a nova release, baseada no esquema de versionamento que está sendo utilizado no projeto, e com o objetivo de manter a rastreabilidade dos

itens da baseline com o release. Existe uma série de maneiras de nomear as versões do sistema. Uma versão comum bastante utilizada é o padrão de versionamento X.Y.Z. Neste esquema:

- **X** é a posição do versionamento que recebe o número relativo a grandes mudanças em determinada versão do item. Inicia-se com 1 e é incrementado à medida que forem desenvolvidas e entregues novas versões do item que passaram por alterações globais, como, por exemplo, a inclusão de uma nova funcionalidade no software ou lançamento de uma atualização completa;

- **Y** é a posição do versionamento relativa ao número que é modificado a cada nova versão implantada ainda em produção no ambiente de desenvolvimento, ou seja, antes do sistema ser completamente finalizado. Por exemplo, na entrega de uma versão de testes para alguns usuários específicos ou, então, em uma apresentação geral do andamento do projeto para o cliente. O Y significa que não houve mudanças globais no sistema e, portanto, o software e seus artefatos permanecem com as mesmas funcionalidades que foram planejadas;

- **Z** é a posição do versionamento com número relativo a correções de emergência realizadas e implantadas em versões, ou releases, liberadas para uso, ainda que em paralelo ao desenvolvimento da próxima versão que será lançada. Por exemplo, uma correção de emergência em um botão mal localizado em uma versão já liberada, mas antes do lançamento da próxima versão, que ainda está sendo desenvolvida.

Todavia, existem outras formas de nomear versões de um sistema, como por exemplo os esquemas que utilizam a nomenclatura Alfa, Beta, entre outras, como definidas a seguir:

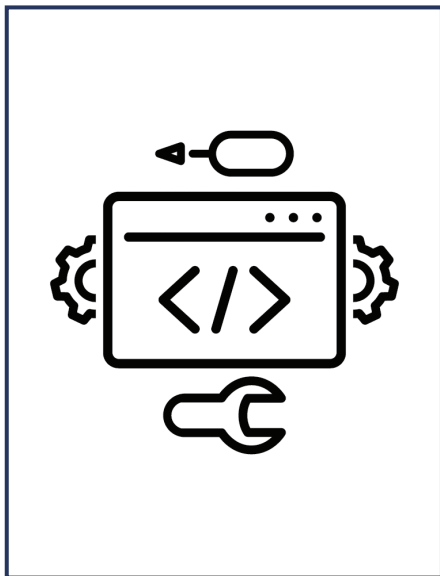
- **Versão Alfa:** o nome alfa deriva de uma letra do alfabeto grego e, nesse esquema, apresenta valor inicial equivalente a um. É graças a isso que esta versão pode ser considerada a primeira release de um software e serve, por exemplo, para que o software desenvolvido possa ser patenteado. Além disso, nessa versão pode-se permitir que o público conheça as funcionalidades do sistema. Muitas vezes, a versão Alfa não é destinada aos clientes ou usuários, mas sim a outros desenvolvedores do projeto ou mesmo utilizada para a formação de baselines. Em alguns casos, esta versão pode ser liberada para um grupo experimental de usuários;

• **Versão Beta:** as versões Beta também derivam do alfabeto grego, e essa letra se encontra posicionada em segundo lugar no esquema. Desta maneira, esta seria a segunda versão do sistema e é considerada aceitável para ser lançada para os usuários finais. Dentre suas características, está o fato de que pode possuir alguns erros ou falhas simples. Entretanto, a equipe de desenvolvimento tem consciência disso e está trabalhando em sua correção. Na versão Beta, os usuários já estão interagindo com o sistema, mesmo que ele ainda esteja em desenvolvimento. Muitos sistemas permanecem em versão Beta durante meses ou anos, passando por diversos ciclos de melhoria.

• **Release Candidate:** o termo Release Candidate significa candidata ao lançamento. Neste esquema, a versão do sistema que recebe a tag de Release Candidate passa a ser considerada como a mais provável a ser liberada para os usuários, uma vez que apresenta todas as funcionalidades desenvolvidas e testadas, além de possuir interface e desempenho sem grandes falhas ou problemas aparentes.

• **Final:** como o nome informa, a versão Final seria o release definitivo de um sistema de software. Neste sentido, o release está pronto para ser utilizado por todos os tipos de usuário, e espera-se que não apresente falhas ou problemas. Em outras palavras, quando uma versão é chamada de Final significa que ela está pronta para ser comercializada.

Um ponto importante a ser tratado com relação ao lançamento de versões do sistema para o usuário é o intervalo entre um lançamento e o próximo. Por exemplo, muitas empresas mantêm seu sistema em versão Beta por um longo período de tempo, garantindo assim que possa evoluir-lo e corrigir falhas enquanto conquista usuários no mercado. Entretanto, deve-se manter um equilíbrio entre o intervalo de um release e outro. Se o sistema passar por lançamentos muito frequentes, os usuários poderão deixar de atualizar os



antigos releases. Por outro lado, se os releases não forem frequentes, poderá ocorrer perda de participação no mercado, à medida que os usuários procurarão utilizar sistemas alternativos e mais atualizados.

Por fim, a disponibilidade de releases deve ser comunicada para que todos os **stakeholders do projeto** tenham acesso à informação relativa a esse lançamento. Além disso, os usuários precisam ser informados de que existe uma nova versão estável e executável do software disponível para uso. É importante ressaltar, ainda, que o processo de criação de releases também depende do tamanho do sistema que está sendo construído. Geralmente, releases frequentes são uma característica de softwares de mercados diversos e com grande número de usuários como, por exemplo, aplicativos populares de redes sociais ou jogos de celular. Outros tipos de sistema, como os softwares para aparelhos celulares, têm lançamentos anuais. Por fim, sistemas muito complexos, como sistemas operacionais para computadores, possuem lançamentos menos frequentes.



EXPLICANDO

Em Engenharia de Software, stakeholder é o termo utilizado para se referir a todos os envolvidos e interessados no sistema que está sendo desenvolvido. Por exemplo: os membros da equipe de desenvolvimento são stakeholders, assim como os investidores e os usuários que irão utilizar o produto. Sendo assim, todos aqueles que têm algum tipo de relação com o sistema que está sendo desenvolvido é um stakeholder.



Gerenciamento de mudanças

Em termos práticos, a gerência de configuração está diretamente associada ao processo de gerenciamento de mudanças. De fato, em um ambiente de desenvolvimento, atividades de configuração, controle e mudança são planejadas e executadas de maneira correlata, uma vez que existe esta ligação quase que indissociável entre esses três fatores. Para explorar a ideia e o conceito por trás deles é importante compreender sua natureza, conforme ilustrado na Figura 5 a seguir.



Figura 5. Configuração, controle e mudança.

Posto isso, pode-se definir cada um dos três conceitos de acordo com seu foco:

- **Configuração:** é o estado atual dos itens de um software. Ou seja, é como os artefatos estão em uma determinada versão;
- **Controle:** é o processo de criar versões e de estabelecer uma maneira eficiente de rastreá-las;
- **Mudança:** é o processo de realizar modificações nos itens do projeto, levando em consideração uma determinada causa ou necessidade.

Apesar de ser mais comum associar o conceito de configuração com um software ainda em desenvolvimento, o conceito de controle com um software prestes a ser lançado, e o conceito de mudança com um software já lançado, mas que precisa passar por alguma modificação, os três conceitos representam ações que afetam o sistema, tanto no ambiente de produção quanto após sua liberação.

Neste ponto, dando um foco maior às questões relacionadas a mudanças, pode-se imaginar que durante todo o desenvolvimento é possível que mudanças ocorram incessantemente. Basta imaginar algumas situações que podem acontecer ao longo do desenvolvimento, como por exemplo:

- O analista de requisitos construiu a lista de funcionalidades do sistema e o software entrou em produção. Semanas depois, em conversa com o cliente, ele percebe que um detalhe importante de determinada funcionalidade não está descrito, e o sistema que está sendo construído precisa ser modificado. Resultado: solicitação de mudança;

- O designer do produto construiu os protótipos de tela, que já estão sendo produzidos e testados. Ao realizar o teste, percebe-se que o protótipo não dispõe de uma opção para usuários com necessidades de acessibilidade. Resultado: solicitação de mudança;

- O programador está construindo o código do sistema e, em dado momento, não sabe como programar uma determinada função, uma vez que o requisito, da maneira como está escrito, permite diferentes interpretações. Resultado: solicitação de mudanças.

Esses são apenas três exemplos dos muitos casos que podem acontecer ao longo do desenvolvimento e que requerem mudanças durante a produção do software. Mas e quando as mudanças ocorrem após o lançamento do produto? Se observarmos a computação como um negócio, de forma que o software em construção é o produto que será vendido e a empresa que desenvolve o sistema precisa se manter no mercado e obter lucros, pode-se afirmar que as mudanças em sistemas já entregues e disponibilizados para os usuários são ações cada mais frequentes no mercado atual, visto que a empresa precisa manter a competitividade em relação a outras organizações que oferecem o mesmo tipo de sistema.

Neste contexto, o **ITIL** define uma mudança como o acréscimo, modificação ou remoção de qualquer processo, arquitetura, ferramenta, métrica, documentação ou outros itens de configuração que possam afetar os serviços oferecidos por um sistema. Pode-se ressaltar a complexidade que as mudanças em sistemas já lançados podem atingir e, por isso, um processo eficiente para gerenciar tais mudanças se faz necessário. Portanto, o gerenciamento de mudanças é o processo que define o fluxo a ser seguido e as etapas a serem desenvolvidas, a fim de que todos os pedidos de mudança sejam analisados, sistematicamente organizados, priorizados, detalhados e, finalmente, realizados de forma segura, garantindo assim que impactos negativos não atinjam o software ou a empresa como um todo.



EXPLICANDO

ITIL é o acrônimo para Information Technology Infrastructure Library, cujo papel é definir um conjunto de práticas detalhadas para o gerenciamento de serviços de tecnologia, especialmente no que diz respeito ao alinhamento dos serviços tecnológicos com as necessidades de empresas e organizações. O ITIL descreve processos, metodologias, tarefas e serviços que não são especificamente o foco da organização e nem da tecnologia, mas que podem ser aplicados por determinada organização como uma estratégia para agregar valor a seu negócio.

Mas então, por que é tão importante implementar um processo de gerenciamento de mudanças em empresas de desenvolvimento de software? A resposta é bem simples. A gestão de mudança deve ser adotada por sua capacidade de fornecer benefícios através da melhoria do software e, assim, continuar satisfazendo as necessidades do usuário, mesmo após o sistema já estar em uso. Além disso, as mudanças no software são inevitáveis, seja durante o desenvolvimento ou depois da entrega. Em geral, considerando sistemas já disponibilizados para os usuários, dois fatores ressaltam a necessidade das mudanças e a importância do seu gerenciamento, a saber:

- O **princípio da mudança contínua**, responsável por indicar os sistemas que estão sendo utilizados e que devem ser modificados ao longo do tempo. Caso isso não ocorra, eles estarão automaticamente caminhando para um cenário em que podem se tornar menos úteis. Isso explica o porquê de, atualmente, tantos aplicativos de celular passarem por atualizações tão frequentes;

- E o **princípio da complexidade crescente**, encarregado de definir que, através de inevitáveis mudanças que acontecem, a estrutura do sistema começa a evoluir e se tornar mais complexa. Desta forma, a complexidade da estrutura do sistema requer cada vez mais recursos para funcionar. Em alguns casos, é necessário simplificar a estrutura para que o sistema continue funcionando adequadamente.

Dessa forma, levando-se em consideração que mudanças são impossíveis de serem evitadas no contexto de desenvolvimento de software, se faz necessário desenvolver estratégias para que elas possam ser executadas. Por isso, as empresas precisam seguir protocolos específicos para gerenciar e acompanhar o denominado ciclo de vida de mudanças em projetos de software. Neste esquema existem algumas atividades principais que, em conjunto, formam o processo de gestão de mudança, sendo elas:

- Identificar possíveis mudanças;
- Avaliar mudanças;
- Planejar mudanças;
- Implementar mudanças;
- Rever e encerrar mudanças.

Participam dessas atividades diferentes perfis de interessados: o cliente, o gerente, a equipe de configuração, o comitê de mudanças e o responsável por realizar a mudança. Assim, visa-se garantir que o ciclo de vida de uma mudança seja executado de maneira eficiente e com resultados positivos para o sistema.



Ciclo de vida das mudanças



Ciclo de vida é um termo bastante amplo. Se pensarmos em seres vivos, por exemplo, o ciclo de vida pode ser definido como o conjunto de transformações que ocorrem com um indivíduo ou animal de dada espécie ao longo de sua vida. Sendo assim, o ciclo de vida de um ser vivo poderia ser sintetizado como as etapas da sua vida, o que inclui seu nascimento, crescimento, reprodução e morte. No entanto, este conceito pode ser utilizado também para definir os estágios da vida de produtos, serviços e processos.

O ciclo de vida de um software pode ser entendido como toda a estrutura formada por processos, atividades e tarefas, cujo objetivo está focado na concepção, desenvolvimento, entrega e manutenção de um produto de software. Ou seja, o ciclo de vida de um software abrange toda as etapas da vida de um sistema, iniciando-se na definição de seus requisitos e se estendendo até que este se torne inútil e seja descartado. Comumente, o ciclo de vida de um software passa por algumas etapas principais, a saber: concepção, elaboração, construção e transição, todas elas ocorrendo enquanto o sistema está em desenvolvimento. Uma vez que o sistema é entregue, ele passa por diversas etapas de manutenção até que, finalmente, não seja mais utilizado. A Figura 6 ilustra essas etapas, que são descritas logo a seguir.

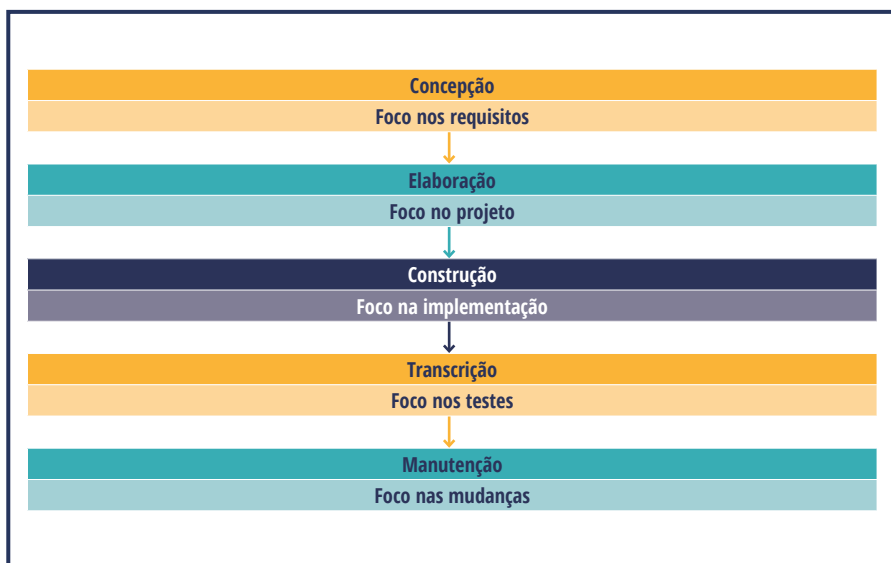


Figura 6. Ciclo de vida do software.

- **Concepção:** é o início de todo o ciclo, quando um problema existente precisa ser resolvido através do sistema de software. Nesta fase, as atividades iniciais do desenvolvimento, como a modelagem de negócio e o levantamento de requisitos, recebem maior foco. Além disso, as atividades gerenciais se preparam para acompanhar o desenvolvimento do sistema, definindo prazos e construindo planejamentos;

- **Elaboração:** é a etapa do ciclo de software que se ocupa da estruturação do desenvolvimento. Este estágio tem foco em atividades de análise e arquitetura, abrangendo a construção de modelos e a definição das estruturas que servirão de base para a implementação do software;

- **Construção:** esta etapa envolve atividades relacionadas ao design, à prototipagem, à codificação e aos primeiros testes. Ou seja, esta etapa está focada em desenvolver o sistema que será entregue. É importante ressaltar que essas atividades devem seguir o que foi estipulado nas etapas anteriores em relação ao planejamento e à estruturação;

- **Transição:** nesta etapa acontecem todos os preparativos para que o sistema deixe o ambiente de produção e possa ser utilizado no ambiente do usuário final, sem que problemas sejam apresentados. Sendo assim, esta etapa tem foco voltado principalmente para testes;

• **Manutenção:** nesta etapa o software já estará sendo utilizado e é necessário prestar o devido suporte aos usuários, o que envolve também corrigir possíveis falhas que possam aparecer. Essa etapa do ciclo de vida também envolve o processo de continuidade do software, através do qual as atualizações permitirão que o sistema possa atender novos requisitos e novas funcionalidades. Assim, essa etapa final acompanha o sistema até que ele pare de ser utilizado.

A compreensão do conceito de ciclo de vida do software, neste ponto, serve dois propósitos. O primeiro está relacionado com a assimilação do conceito de ciclo de vida, que, em linhas gerais, pode ser entendido como o conjunto de atividades a serem executadas, com início, meio e fim bem definidos. Além disso, pode-se perceber que todas as atividades do desenvolvimento de software têm seu próprio ciclo de vida. Por exemplo, a programação do sistema é iniciada, passa por uma etapa intermediária e posteriormente é encerrada.



Procedimentos para mudanças

O mesmo acontece com as demais atividades, como é o caso da gerência de configuração e os procedimentos de mudanças que seguem seu próprio ciclo. O segundo propósito evidencia como mudanças estão presentes no sistema, mesmo após ele ser entregue para os usuários. Nesse sentido, pode-se observar que a etapa final do ciclo de vida do software, a manutenção, é especificamente focada na evolução do sistema e, dessa forma, seja para a adição de novas funcionalidades ou para a correção de falhas, um processo bem definido de realização de mudanças precisa ser estabelecido.

Sendo assim, o ciclo de vida de mudanças é um importante fator dentro do processo de gerência de configuração. Mudanças em sistemas são inevitáveis e constantes, mas para que elas ocorram de forma adequada algumas questões precisam ser respondidas:

- Essa mudança é realmente necessária?
- Caso seja necessária, como ela será realizada?
- Quem é o responsável por realizar a mudança?
- Qual o protocolo a ser seguido para realizar a mudança?
- Ao final da mudança, o que deve acontecer?

Todas essas perguntas podem ser respondidas dentro do ciclo de vida de mudanças em projetos de software. O ciclo de vida de uma mudança geralmente envolve seis atividades principais e algumas decisões a serem tomadas, como evidencia a Figura 7.

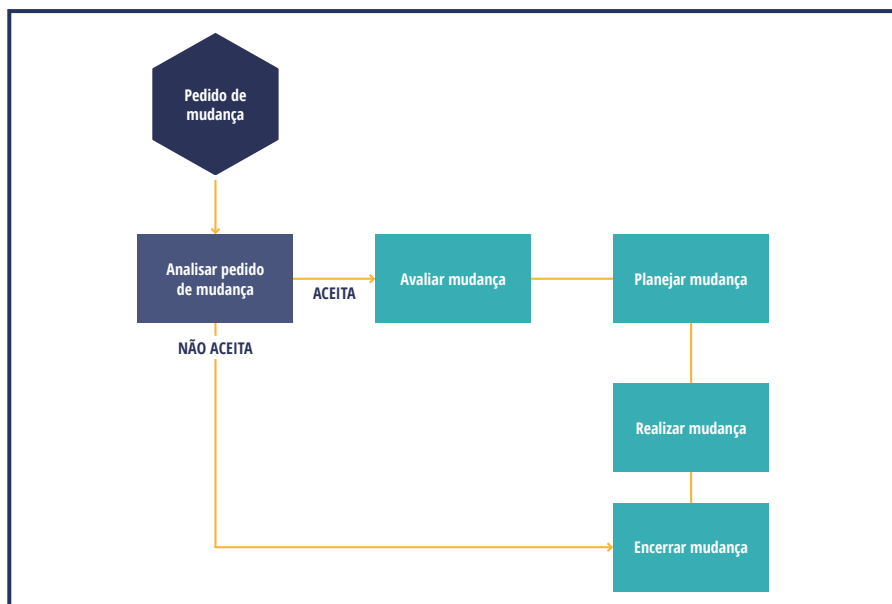


Figura 7. Ciclo de vida da mudança.

Seguindo o esquema apresentado na Figura 7, pode-se estabelecer o passo-a-passo necessário para realizar uma mudança num sistema de software.

1) Uma solicitação de mudança é enviada para a equipe de gerência de configuração, para que a possibilidade de realizá-la seja analisada. Com isso, é preciso analisar o pedido de mudança de modo a identificar seu custo, impacto e, principalmente, a necessidade de se realizar tal mudança naquele dado momento;

2) A equipe de configuração precisar autorizar ou rejeitar a mudança solicitada. Nesse sentido:

- A mudança pode ser negada e o pedido é encerrado. Uma mudança pode ser negada por diversas razões: uma solicitação de mudança no código do software, por exemplo, pode ser negada porque outro desenvolvedor está trabalhando no mesmo código no momento, e duas pessoas não podem alterar o mesmo código simultaneamente. Nesse caso, o pedido de mudança é negado e o ciclo com a solicitação vai para as fases finais de cancelamento;

- A mudança pode ser autorizada e o processo prossegue até o fim. Nesse caso, a realização da mudança deverá cumprir todo o procedimento planejado.

3) Quando aceita, a solicitação de mudança deverá seguir todo o processo de alteração do item de configuração. Ou seja, é neste ponto que a equipe irá planejar como a mudança será efetuada, quem será responsável por realizá-la e quais outros itens deverão ser alterados;

4) No momento em que a mudança estiver sendo efetivamente realizada, a parte do sistema que está sendo alterada deve permanecer bloqueada para outros tipos de solicitações, até que a mudança seja concluída. Por exemplo, se um determinado sistema para celular estiver passando por algum tipo de mudança na câmera, todas as atividades relacionadas com a câmera deverão ser bloqueadas e aguardar a conclusão. Neste processo, outros pedidos de mudança também deverão ser bloqueados, a fim de evitar possíveis conflitos.

5) Uma vez que a mudança foi realizada, a equipe de configuração precisa avaliar o resultado dessa mudança, de modo a garantir que a alteração não causou danos a outras partes do sistema e que o item alterado continua consistente em relação aos demais itens do projeto. Se a equipe identificar problemas ocasionados pela mudança realizada, o responsável pela mudança precisará refazer o trabalho de modo que o resultado da mudança possa tornar-se congruente com o projeto. Se a análise indicar que a mudança foi satisfatória e não gerou nenhum impacto negativo para o projeto, o processamento desta mudança deverá ser continuado até o fechamento da solicitação.

Deve-se ressaltar que a continuidade do processo de mudança se dá com o correto armazenamento do item de configuração atualizado, através do processo de versionamento que irá garantir a rastreabilidade do que foi realizado. Assim, a rastreabilidade indica a capacidade de estabelecer vínculos não apenas entre as mudanças como também entre os diversos itens de configuração e outros artefatos do projeto, que se estendem por todas as atividades, como modelos, documentos, código fonte, sequências de testes ou executáveis. A rastreabilidade é uma das atividades indicadas pelos modelos de qualidade como CMM, CMMI e MPS-BR. Para garantir a rastreabilidade das mudanças, é importante que a equipe construa um relatório de mudança que mostre o status de configuração do item e inclua respostas para as seguintes questões:

- O que aconteceu durante a mudança?
- Quem realizou a mudança?
- Quando a mudança aconteceu?
- O que mais foi afetado pela mudança?

Finalmente, a equipe de configuração poderá encerrar o ciclo de vida da mudança, mantendo armazenadas todas as informações necessárias para um caso de verificação e auditoria no futuro.

Sintetizando



Parabéns! Você acabou de finalizar esta unidade, que trata especificamente do processo de mudanças em sistemas de software. No começo da unidade você conheceu de maneira aprofundada os conceitos de baseline e release e, assim, pôde entender como mudanças podem acontecer durante o desenvolvimento do sistema e mesmo após a sua entrega. Em seguida, a importância do processo de gerenciamento de mudanças como uma das atividades da gerência de configuração foi apresentada e discutida. Por fim, você pôde compreender o ciclo de vida de uma mudança e acompanhar de forma detalhada todo o processo para solicitar, construir e encerrar um pedido de mudança em um sistema.

De maneira geral, é importante lembrar que mudanças são inevitáveis quando um produto de software está em questão, e é preciso determinar um processo que garanta a realização de mudanças de maneira estruturada, consistente e segura, garantido rastreabilidade e manutenção. Sistemas de software que já foram entregues para os seus usuários não estão isentos de mudança, uma vez que os sistemas precisam passar por atualizações para se manter no mercado.

Considerando o ciclo de vida de uma mudança como um todo, é possível afirmar que este processo auxilia empresas de software a produzirem sistemas com maior qualidade, e que de certa forma ele pode trazer vantagens competitivas, posto que no mercado globalizado os usuários anseiam cada vez mais por melhorias e atualizações de seus aplicativos, websites e sistemas. Dessa forma, é preciso ter um processo eficiente que auxilie as empresas de software nessas entregas.

Espero que este material tenha ajudado você a entender o ciclo de vida de mudanças e seu gerenciamento durante o processo. Lembre-se sempre de buscar mais conteúdo para expandir o seu conhecimento. Mantenha-se motivado!

Referências bibliográficas



BERSOFF, E. H. **Elements of software configuration management**. [s.l.]: IEEE Transactions on Software Engineering, v. 10, n. 1, 1984.

BOURQUE, P.; FAIRLEY, R. E. **Guide to the software engineering body of knowledge**. 3. Ed. Los Alamitos: IEEE Computer Society Press, 2014.

DANTAS, C. **Gerência de configuração de software**. [s.l.]: Engenharia de Software Magazine, 2009.

SANCHES, R. Gerencia de configuração. **In: Qualidade de software**. São Paulo: Prentice-Hall, 2001.

UX/UI Design? Saiba a diferença entre UI e UX Design. Postado por Chief of Design. (14m. 46s.) son. color. port. Disponível em: <https://www.youtube.com/watch?v=NDgfm_fjNq8>. Acesso em: 28 ago. 2020.

UNIDADE

4

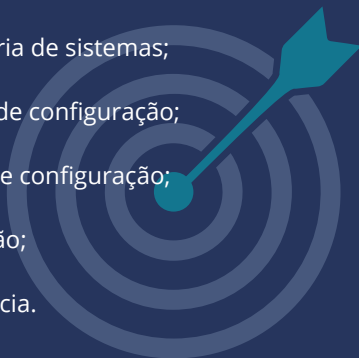
GERÊNCIA DE
CONFIGURAÇÃO:
AUDITORIA,
CONTINGÊNCIA E
FERRAMENTAS



ser
educacional

Objetivos da unidade

- Apresentar os conceitos relacionados à auditoria de sistemas;
- Compreender os riscos associados à gerência de configuração;
- Conhecer ferramentas utilizadas na gerência de configuração;
- Discutir o processo de auditoria de configuração;
- Entender a importância do plano de contingência.



Tópicos de estudo

- Auditoria de configuração
- Gerenciamento de riscos e plano de contingência
- Ferramentas para o gerenciamento de configuração e mudanças





Auditoria de configuração



A gerência de configuração é uma atividade do desenvolvimento de software que garante que todas as mudanças que podem acontecer num sistema sejam planejadas, monitoradas e avaliadas, se associando à construção de produtos de software de alta qualidade, tendo um papel significativo depois de um sistema ter sido lançado, uma vez que ele ainda passa por mudanças, seja para correção de falhas ou atualizações. Logo, se afirma que a gerência de configuração é fundamental na segurança de qualidade de software, tanto que os **modelos de qualidade e de maturidade de software** indicam a sua aplicação a todas as empresas que produzem software.



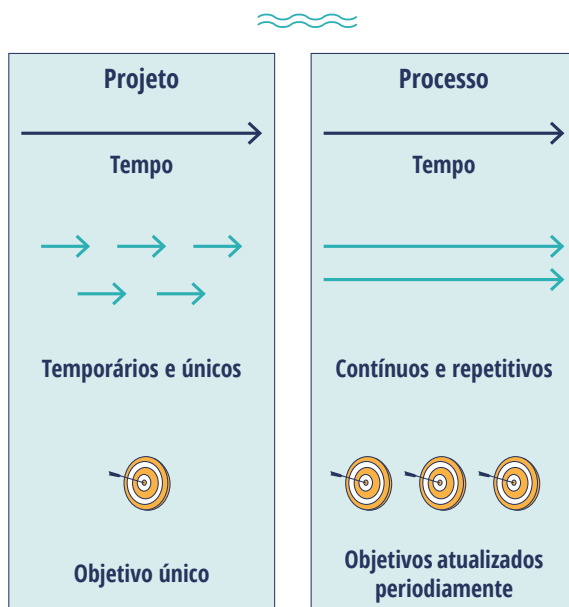
CONTEXTUALIZANDO

Os modelos de qualidade e maturidade são um conjunto de práticas que implantam diretrizes de como as empresas de desenvolvimento de software conduzem suas atividades, descrevendo os principais efeitos esperados da execução de determinado processo da Engenharia de Software, de modo que seja possível comparar o que foi feito em teoria e o que a empresa tem praticado e, assim, descobrir os pontos de melhoria na organização.

Observando a correspondência da gerência de configuração com os processos de qualidade, “qualidade de software” é um termo genérico de toda uma área de conhecimento da engenharia de software, cujo objetivo é certificar que os sistemas sejam construídos e entregues com alto nível de excelência, através da definição e normatização de processos de desenvolvimento e acompanhamento de atividades. Apesar dos modelos aplicados na garantia da qualidade de software atuarem no processo, o principal intuito da área é certificar que o software final, quando entregue, atenda às expectativas do cliente e dos usuários, considerando tudo que foi listado inicialmente.

A garantia da qualidade de software fornece aos níveis de gerência a visibilidade adequada sobre as características dos projetos de software e a execução dos processos de desenvolvimento e dos produtos gerados. Neste ponto, é essencial diferenciar o que é um projeto e o que é um processo.

DIAGRAMA 1. PROJETO X PROCESSO.



Como ilustrado no Diagrama 1, um projeto é um esforço temporário para atingir um alvo num período. Por outro lado, um processo também é executado num tempo específico, mas com a característica de ser contínuo e repetitivo, sendo executado diversas vezes, podendo ter seu objetivo atualizado a cada execução. Um processo tem a característica de ser um guia de instruções a serem seguidas. Enquanto isso, um projeto pode ser:

- A criação de algo novo, inovador e distante do que já existe, como um novo aplicativo lançado no mercado;
- O progresso de algo construído, como a adição de um novo tipo de pagamento nos sistemas de compras em circulação;
- O desenvolvimento de uma infraestrutura, como a construção de uma rede de computadores para dar suporte a um sistema de informação;
- A publicação de uma pesquisa científica, como um artigo num congresso, uma monografia de conclusão de curso ou uma tese de doutorado.

Neste ponto, é importante ressaltar que, embora o termo “projeto” esteja associado, em computação, ao desenvolvimento ou à manutenção de um software, os projetos fazem parte da história da humanidade. As pirâmides

do Egito, monumentos com mais de quatro mil anos, foram concebidas por meio de um projeto arquitetônico bem elaborado, com recursos financeiros e de tempo, administrados, bem como o emprego de tecnologias da época.

Ao longo da história da humanidade, a ideia de projeto se transformou e se especializou nas áreas do conhecimento, mas uma característica que sempre a acompanhou foi a busca por qualidade. Do ponto de vista de sistemas, a qualidade de software atua durante todo seu ciclo de vida, atingindo os quatro grandes alvos de um projeto de desenvolvimento de software:

- Desenvolver softwares de alta qualidade, que agreguem valor ao negócio do cliente e à utilização;
- Ter alta produtividade da equipe, visando atividades individuais e a interligação delas entre si;
- Cumprir o cronograma constituído junto ao cliente, de modo que problemas e atrasos não afetem a qualidade do sistema;
- Dispensar recursos adicionais não previstos, procurando alternativas em falhas do sistema.

Nesse sentido, a gerência de configuração e a qualidade de software são duas áreas da computação que se associam e cujos intentos são afetados quando mudanças no sistema acontecem de maneira aleatória, além da falta de controle com os artefatos do projeto. A qualidade de software e a gerência de configuração, com o intento de atingir seus objetivos, operam em conjunto almejando:

- O planejamento de projeto e o acompanhamento de efeitos, o que inclui o acompanhamento de mudanças;
- Métodos e ferramentas padronizadas no desenvolvimento do sistema e na organização como um todo;
- A adoção de revisões técnicas formais contínuas e eficientes;
- O estabelecimento e a monitoração de estratégias de testes do sistema como um todo, no que diz respeito às validações de mudanças;
- A revisão de artefatos produzidos pelo processo de desenvolvimento e o monitoramento dos itens de configuração;
- Mecanismos adequados de armazenamento e recuperação de dados do projeto de software e da execução de processos;

- A melhoria contínua no processo de desenvolvimento de software através da avaliação de implicações e de auditoria de processos.

No Brasil, empresas de software têm aperfeiçoado seus processos e a qualidade dos sistemas investindo na formação dos profissionais da área. No viés acadêmico e científico, o Simpósio Brasileiro de Qualidade de Software é um evento anual organizado pela Comissão Especial de Engenharia de Software, parte da Sociedade Brasileira de Computação. O evento reúne empresários, profissionais, professores, pesquisadores e estudantes de diversas áreas, interessados em questões sobre **qualidade de software**, num espaço focado na divulgação e troca de experiências entre os envolvidos. Muitas das pesquisas científicas sobre gerência de configuração disponíveis na literatura são discutidas e publicadas nesta esfera.



ASSISTA

A qualidade de software e a gerência de configuração são duas áreas de desenvolvimento de software com conexão direta entre si. Hoje em dia, é impossível separar as duas num ambiente de desenvolvimento, pois a dinâmica das mudanças num sistema de software passa por um processo de verificação e validação, elementos da qualidade de software. O vídeo *O que é Qualidade de Software e porque você precisa ter?* // PAPO INFORMAL, traz mais informações sobre essas questões.



Considerando que o papel fundamental da gerência de configuração é dar coerência e consistência às mudanças no sistema, a própria validação é uma ação ligada à qualidade do sistema. Não obstante, o processo requer uma validação. Ou seja, é preciso garantir o ciclo de vida para promover uma mudança seguindo um procedimento livre de falhas. Para que as solicitações sejam implementadas, é necessária a execução das tarefas de revisão técnica e **auditoria** na configuração do software.



EXPLICANDO

A auditoria é o processo de exame, cuidadoso e sistemático, das atividades em determinada empresa ou num projeto. Sua função é certificar que as atividades estão de acordo com o planejamento e as métricas, implementadas eficaz e adequadamente, conforme os objetivos do projeto e da organização.

A auditoria da gerência de configuração é o processo de estudar se uma mudança solicitada e aprovada foi implementada. Em outras palavras, é a ação de verificar se uma alteração no sistema percorreu todas as etapas do ciclo de vida de mudanças e se o produto final cumpre com o esperado. No Quadro 1, estão descritos o ciclo de vida e suas atividades.

QUADRO 1. CICLO DE VIDA DE MUDANÇAS



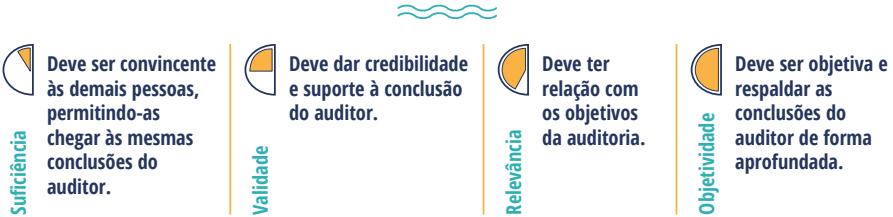
Atividade	Descrição
Pedido de mudança	Uma solicitação é enviada para a equipe de gerência de configuração, a fim de que seja analisada a possibilidade de realizá-la. Com isso, se analisa o pedido para identificar o seu custo, o impacto, e a obrigação de tal mudança no momento
Analisar pedido de mudança	A equipe de configuração autoriza ou rejeita a solicitação. Nesse sentido, ela pode ser negada e o pedido encerrado, ou autorizada e o processo continua até o fim.
Avaliar e planejar mudança	Quando aceita, a solicitação segue o processo de alteração do item de configuração. É nesse ponto que a equipe planeja como a mudança será efetuada, quem será responsável por fazer a alteração e quais itens são alterados.
Executar mudança	No momento em que a mudança estiver sendo efetivada, a parte do sistema alterada permanece bloqueada para outros tipos de solicitações até que ela seja concluída. Se um sistema para celular estiver no meio de uma mudança na câmera, todas as atividades relacionadas à câmera são bloqueadas e aguardam a conclusão. Neste processo, para evitar conflitos, outros pedidos são bloqueados.
Encerrar mudança	A equipe de configuração avalia o resultado, assegurando que a alteração não causou danos a outras partes do sistema e que o item alterado continua consistente com os demais itens do projeto. Se a equipe identificar problemas ocasionados pela mudança, o responsável refaz o trabalho, para que o resultado continue consistente com o projeto. Caso a análise indique que a mudança foi satisfatória e não gerou nenhum impacto negativo, o processamento é continuado até o fechamento da solicitação.

A auditoria da gerência de configuração tem a finalidade de assegurar que cada uma das etapas do ciclo seja executada de forma completa. Mais do que isso, a autoria da gerência de configuração certifica que, mesmo após as mudanças, o produto final está de acordo com os requisitos pré-fixados e que tais requisitos estão em conformidade com a configuração do software, ressaltando que o estado atual dos itens do projeto é demarcado como configuração.

A auditoria é tida como uma fiscalização da atividade de gerência de configuração que garante a qualidade do processo, evitando erros ou desperdícios e afirmando a

confiabilidade do processo de configuração e mudança de software através de evidência comprovada. As evidências são o conjunto de fatos comprovados, suficientes, competentes e pertinentes ao processo conforme caracterizado no Diagrama 2.

DIAGRAMA 2. CARACTERÍSTICAS DA EVIDÊNCIA NA AUDITORIA DE CONFIGURAÇÃO.



Sendo assim, se entende a evidência na auditoria como uma expressão de que houve algum tipo de falha na gerência de configuração ou que o procedimento foi efetivado da maneira correta. Por isso, não se entende a auditoria como uma atividade destrutiva que busca apenas por problemas e falhas, uma vez que a exposição dos processos é derivado da auditoria. É importante advertir, no entanto, que é necessário comprovar o resultado, positivo ou negativo, através de evidências, como exibido no Diagrama 2, sendo que elas são de vários tipos, resumidos no Quadro 2.

QUADRO 2. TIPOS DE EVIDÊNCIA NA AUDITORIA

Tipo	Descrição e exemplos
Evidência física	Inspeção física ou observação direta de profissionais, atividades e projetos. É representada por fotografias, gráficos, memorandos, mapas ou amostras físicas, como uma tela do sistema após a alteração, demonstrando se o resultado foi válido ou não.
Evidência documental	Análise de documentos, contratos e outros materiais documentais, como um documento não atualizado comprovando que a validação de uma mudança não foi feita.
Evidência testemunhal	Aplicação de entrevistas e questionários com pessoas envolvidas no processo. Antes que a mudança seja efetivada, ao consultar a equipe de gerência de configuração sobre o ciclo de vida, se coletam relatos referentes aos documentos que não estão sendo atualizados.
Evidência analítica	Conferência de cálculos, comparações, correlações e análises feitas pelo auditor. Na comparação do número de atualizações do sistema com a quantidade de mudanças registradas, se percebe que nem todas estavam sendo efetuadas dentro do procedimento oficial.

Fundamentadas nas acepções de evidência e na obtenção de dados reais e confiáveis, as auditorias são conduzidas de acordo com processos delimitados e documentados. Neles, os auditores têm responsabilidades, planejadas e executadas. Existem dois tipos de auditoria, expostos no Diagrama 3.

DIAGRAMA 3. AUDITORIA INTERNA E AUDITORIA EXTERNA.



- **Auditoria interna:** feita por profissionais que trabalham na própria empresa, seu propósito é verificar se as normas internas e exigências do ciclo de vida de mudança estão sendo cumpridas, de modo a fornecer elementos para a redução de riscos no processo e contribuir com o aperfeiçoamento das atividades da equipe de gerência de configuração;

- **Auditoria externa:** desempenhada por auditores independentes, sem qualquer vínculo direto com a empresa de software ou com a equipe de desenvolvimento. Neste esquema, para que não haja risco de interferência no conteúdo das evidências, se pega uma visão externa que emita um parecer não enviesado sobre o processo, além de atuar em áreas diferentes do desenvolvimento do software, como custos financeiros efetivos com o ciclo de vida de mudanças.

A auditoria na gerência de configuração tem, pelo menos, dois tipos de avaliação:

- **Avaliação funcional:** garante que o item de software alterado está em conformidade com a rastreabilidade aplicada, no que concerne à identificação da configuração do software, ou estado atual do item. Esta é uma análise formal que confere se um item de configuração possui o desempenho especificado e atende às características funcionais do software implementado e documentado;

- **Avaliação física:** garante que a documentação gerada no processo de desenvolvimento do projeto está consistente com o produto de software produzido. Este é o processo de averiguar se o item de configuração tem as características físicas especificadas, como tamanho ou formato.

É conveniente ressaltar que a gerência de configuração acompanha o desenvolvimento do software e o processo de mudanças após o lançamento. A auditoria de configuração inclui, depois que o sistema já estiver sendo utilizado, uma auditoria de baselines e releases, a fim de apurar o desempenho do software de acordo com as funcionalidades modificadas. Esta auditoria garante que a documentação contínua siga consistente e atualizada, prosseguindo nas atualizações do sistema.



Gerenciamento de riscos e plano de contingência

Os benefícios que a gerência de configurações traz para uma empresa de desenvolvimento de software são expressivos e indiscutíveis, uma vez que possibilita à organização fazer uma análise aprofundada das mudanças durante o processo de desenvolvimento do sistema e após o seu lançamento. Contudo, todo processo e atividade tem riscos associados, da execução aos resultados. Por isso, se destaca a importância de um plano de contingência para a gerência de configuração pois, em situações inesperadas, se pode efetuar a restauração de itens, versões do sistema e demais ações.

Um plano de contingência é um tipo de plano preventivo e preditivo, também chamado de planejamento de riscos, cujo objetivo é descrever as medidas tomadas para fazer com que processos sejam recuperados o mais rápido

possível e voltem a funcionar plenamente, ou num estado aceitável, após um problema incomum ou inevitável acontecer. O plano de contingência está relacionado à análise e gestão de riscos no processo de gerência de configuração.

O risco é todo componente de incerteza ao redor do projeto e que afeta o resultado final. A influência do risco pode ser negativa ou positiva, mas o que o define é o fator de incerteza. Logo, fora as ameaças capazes de prejudicar a execução do projeto, há a chance dos riscos de oportunidades a serem aproveitadas ao longo do ciclo de vida de desenvolvimento de software para proporcionar a conquista de um desempenho superior. O Quadro 3 exibe alguns tipos de riscos envolvidos no gerenciamento de mudança.

QUADRO 3. TIPOS DE RISCOS NA GERÊNCIA DE CONFIGURAÇÃO



Tipo	Descrição e exemplos
Gerencial	Originado do fator gerencial do projeto, como planejamento, acompanhamento e métricas de desenvolvimento: o planejamento inicial do projeto não previu uma mudança indispensável e, por isso, não há recurso financeiro ou tempo disponível.
Externo	Resultantes de elementos externos à empresa de desenvolvimento e que afetam as mudanças no software: uma queda de energia repentina em todo o bairro onde a empresa está localizada impede que uma mudança seja concluída.
Tecnológico	Causado por equipamentos tecnológicos empregues no projeto: a queda de um dos servidores afeta o processo de liberação de uma baseline para testes.
Pessoal	Riscos que rodeiam as pessoas que trabalham na atividade de gerência de projetos: um membro da equipe ficou doente repentinamente e, por isso, as solicitações estão demorando mais tempo do que o de costume para serem analisadas e liberadas.
Legal	Riscos próximos a fatores ligados à lei e a normas judiciais: um novo sistema está sendo desenvolvido e uma nova lei é promulgada deliberando o formato da impressão de recibos dos sistemas, sendo uma mudança não planejada e que precisa ser executada para adequar o sistema às normas legais.

Um risco é incerto, contudo, não é imprevisível. Quantificando e qualificando o risco, se decide a probabilidade de acontecer, o impacto gerado e a preparação para o evento, caso se concretize. Numa situação em que o gerente de configurações de um projeto de software que está para ser lançado, mas

com uma mudança na tela inicial do sistema a ser realizada antes do lançamento, pensando quantitativamente, ele imagina quais os riscos que podem atrapalhar as mudanças e o lançamento do sistema, como:

1. A versão do sistema não ficar pronta na data do lançamento;

2. Um problema não encontrado antes ser identificado durante a mudança e gerar um efeito cascata de correções no sistema;

3. O servidor parar de funcionar no momento em que a baseline está sendo criada;

4. O principal responsável pela mudança ter o seu voo de férias atrasado e não conseguir ir trabalhar;

5. O cliente decidir que a data de lançamento foi antecipada, exigindo que tudo seja feito mais rapidamente;

6. O cliente decidir que a data de lançamento foi adiada, dando mais tempo para a concretização da mudança, assim por diante.

Quando se fala em quantidades, são considerados o número de riscos associados a uma atividade – no episódio retratado, o total de riscos associados à mudança final antes do lançamento do sistema. Do ponto de vista qualitativo, é analisada a força de impacto que o risco vai causar. No número 4, o principal responsável por uma mudança não pôde trabalhar, o risco é menor se o profissional puder ser substituído, mas se torna maior se a equipe tiver de esperar que ele chegue ao trabalho. Outro exemplo é o risco 6, que é positivo e beneficia a equipe de várias formas.

Analisando o risco e seus reflexos, o gerenciamento de riscos em gerência de configuração de software é o processo de conhecer, administrar e se preparar para enfrentar todos elementos incertos que podem atingir os itens de configuração e as mudanças no projeto, seja na baseline ou nas releases. O Diagrama 4 ilustra o processo de gerenciamento de risco, vital para qualquer tipo de projeto ou atividade.



DIAGRAMA 4. GERENCIAMENTO DE RISCOS E PLANO DE CONTINGÊNCIA.



No processo de gerenciamento de riscos, a construção de um plano de contingência é valiosa para reduzir ou até mesmo neutralizar o impacto negativo dos riscos, se ocorrerem. Focando na gerência de configuração, se criam estratégias para que o sistema seja recuperado, na hipótese de um problema durante ou após uma mudança.

No plano de contingência, estão descritas todas as ações caso algum risco atrapalhe as atividades da gerência de configuração, seja envolvendo o processo de identificação de itens de configuração, fazendo o versionamento dos itens, mudanças ou qualquer atividade de responsabilidade da equipe de configuração, sendo de pleno conhecimento de todos os profissionais que trabalham na equipe. Treinamentos para orientar os envolvidos sobre como as ações do plano são executadas são imprescindíveis. O Diagrama 5 ilustra os elementos que compõem o plano de contingência.

DIAGRAMA 5. PLANO DE CONTINGÊNCIA.



O planejamento e gerenciamento de riscos, junto ao estabelecimento de um plano de contingência, são valiosos nas atividades de software, em especial se tratando de atividades ligadas à qualidade do sistema como a gerência de configuração. Tal prestígio se dá graças às experiências práticas de empresas e casos notados ao longo de décadas, uma vez que modelos de qualidade e maturidade do desenvolvimento de software exigem os dois elementos no processo de certificação de empresas de desenvolvimento de software.

O CMMI é uma referência que constitui as práticas para a maturidade em atividades específicas da Engenharia de Software e evolução contínua do processo de desenvolvimento. O CMMI tem um conjunto de níveis associados a como a empresa de software executa e aprimora suas atividades em cada um dos níveis. As principais características dos níveis são:

- **Nível 1 – Inicial:** caracterizado pela imaturidade organizacional, quando os processos são improvisados e não são seguidos, sem contar que compromissos de prazo e custo constantemente não são cumpridos, e que o planejamento não é feito com base em estimativas e métricas confiáveis. Os procedimentos e conhecimentos pertencem às pessoas e não aos projetos, fazendo com que as chances de sucesso dependam das habilidades pessoais dos gerentes e desenvolvedores, e não da estruturação das atividades de caráter confiável;

- **Nível 2 – Gerenciado:** neste nível, há um progresso na fixação de políticas e procedimentos para o desenvolvimento de software. O planejamento é abalizado em estimativas e na experiência anterior de outros projetos, com

processos fixados, disseminados, documentados, medidos e fiscalizados com rotinas de melhoria;

- **Nível 3 – Definido:** a partir deste ponto, os processos são padronizados em toda a organização. Atividades técnicas são administradas em reunião com os processos gerenciais, podendo ser repetidos e replicados para diversos projetos com sucesso;

- **Nível 4 – Quantitativamente gerenciado:** a empresa está numa situação na qual se formulam metas quantitativas para os processos e o software. Medidas de qualidade e produtividade são coletadas em todos os projetos para que haja um controle estatístico. A gestão se dá com alicerces quantitativos confiáveis;

- **Nível 5 – Otimização:** a empresa está engajada no avanço contínuo de seus processos, bem como identificação de pontos fracos e defeitos para a execução de aperfeiçoamentos. Há ações preventivas sobre mudanças, realizadas a partir de análise de custo/benefício embasadas nos dados quantitativos.

Cada um dos níveis do CMMI é uma certificação que uma empresa de software pode obter, sendo o Nível 1 básico e o Nível 5, mais avançado, demonstrando que a empresa em questão tem habilidade, experiência e perícia para desenvolver sistemas e oferecê-los com qualidade. A partir dos pontos sobre os níveis do CMMI, se entende que a gerência de configuração está presente em todos os níveis. No nível 1, ela é implantada e precisa evoluir. No nível 2, ela é um processo gerenciado que requer planejamento, execução, monitoramento e controle. No nível 3, ela é ajustada e alinhada com os processos da organização, incluindo o planejamento de riscos. No nível 4, ela é administrada e controlada de maneira quantitativa. E no nível 5, ela é um processo otimizado.

O COBIT serve como um guia de boas práticas, apresentado como framework para a gestão de tecnologia de informação e focado no nível estratégico. O COBIT não é uma norma, mas ajuda a direcionar ou priorizar os esforços e recursos da tecnologia da informação para atender aos requisitos de um projeto. Sua finalidade é fornecer guias e práticas aos gerentes, permitindo compreender e gerenciar os riscos associados às tecnologias, traçando uma ponte entre os requisitos do projeto, as indigências de controle de problemas e riscos técnicos, reforçando a importância do planejamento de riscos e do plano de contingência. Do ponto de vista da gerência de configuração, ele vincula as ati-

vidades aos processos de entrega e suporte de produtos, além de gerir mudanças e incidentes.

Por fim, não seria possível falar sobre qualidade, gerência de configuração, riscos e plano de contingência sem discutir o modelo de Melhoria do Processo de Software Brasileiro (MPS-BR), metodologia criada para a área de desenvolvimento de software no Brasil e embasada nas principais abordagens internacionais voltadas para definição, avaliação e progresso dos processos de software. O MPS-BR se vale de uma estrutura de níveis de maturidade, semelhantes às do CMMI. Os sete níveis de maturidade previstos pelo MPS-BR são:

- **Nível 1 – Em otimização:** caracterizado pela preocupação com questões como inovação e análise de causas e efeitos no processo;
- **Nível 2 – Gerenciado quantitativamente:** neste nível, é avaliado o desempenho dos processos a partir de dados quantitativos e de como a gerência atua neste contexto;
- **Nível 3 – Definido:** focado no processo de gerenciamento de riscos e na ação da gerência através dos planos de contingência;
- **Nível 4 – Largamente definido:** envolve ações de verificação, validação, liberação, instalação e integração de produtos de software, dentre outras atividades da gerência de configuração;
- **Nível 5 – Parcialmente definido:** abrange processos como treinamento, buscando uma adaptação de processos para a gerência de projetos, se preocupando com a melhoria e o controle do processo organizacional;
- **Nível 6 – Gerenciado:** introduz controles de medição, gerência de configuração, conceitos sobre aquisição de produtos e garantia da qualidade;
- **Nível 7 – Parcialmente gerenciado:** a principal preocupação está em torno do processo de gerenciamento de requisitos e de gerenciamento de projetos.

O MPS-BR é capaz de analisar como uma empresa de desenvolvimento de software faz suas atividades com eficiência, estabelecendo a qualidade do seu software através da qualidade das suas práticas. Neste modelo, a gerência de configuração mantém a integridade dos componentes de software, compondo as atividades de suporte, nas quais estão a garantia de qualidade, os testes de software e as análises de decisão e de riscos.



Ferramentas para o gerenciamento de configuração e mudanças



Construir e desenvolver um software depende da execução de atividades, do trabalho de profissionais capacitados e de instrumentos que auxiliam no desenvolvimento. Há diversas ferramentas para apoiar atividades de gerência de configuração, o processo de mudanças. O Quadro 4 expõe algumas delas, que podem ser usadas individualmente ou juntas, dependendo dos interesses da equipe e do tamanho do projeto de sistema desenvolvido.

QUADRO 4. FERRAMENTAS DA GERÊNCIA DE CONFIGURAÇÃO



Tipo de ferramenta	Open source	Licença para compra
Controle de versão	• Mercurial • Git • Subversion • CVS	• Team Foundation Server – Microsoft • Team Concert - IBM/Rational • ClearCase • StarTeam • Perforce • BitKeeper
Controle de mudança	• Trac • Redmine • Mantis • Bugzilla	• JIRA • FogBUGZ • CaliberRM • Perforce
Integração contínua	• Jenkins • Bitten • SCons • Ant • Maven • CruiseControl • Gump	• Anthill Pro • FinalBuilder • BuildForge

Os materiais do processo de controle de versão monitoram as mudanças feitas ao longo do tempo num arquivo que contém um item de configuração ou num grupo de arquivos, possibilitando o processo de versionamento dos itens e, em seguida, a recuperação de versões específicas do item.

Numa equipe de configuração que recebe um pedido para uma correção emergencial na versão 1.0.1 do sistema, pelo esquema de versionamento X.Y.Z., a versão já passou por uma correção anterior. Agora, com a nova correção, o

seu versionamento é atualizado para a versão 1.0.2. Em consequência, todos os itens afetados, como diagramas e documentos, são atualizados e recebem uma nova tag de versionamento, um procedimento cauteloso e trabalhoso, mas que pode ser apoiado ou automatizado através de umas das ferramentas de controle de versões.

- **Git:** livre e gratuito, permite o trabalho remoto num mesmo item. O sistema suporta a criação de branches e merges, viabilizando esquemas específicos para visualização e navegação de históricos de desenvolvimento e mostrando ligações entre versões do mesmo item, facilitando a aquisição de informações sobre o estado da versão atual do arquivo e dando controle, monitoramento e avaliação das atividades;

- **Mercurial:** disponível para várias plataformas, é usado no gerenciamento de configuração de software nas etapas de desenvolvimento (programação), tendo como características alta performance, escalabilidade, descentralização e desenvolvimento colaborativo distribuído, no qual inúmeras pessoas trabalham ao mesmo tempo com o mesmo item. Além disso, admite operações avançadas com branches e merges;

- **Perforce:** uma ferramenta de controle de versão com uma estrutura firmada na arquitetura cliente/servidor e aproveitada para transferir arquivos entre um repositório e as estações de trabalho, garantindo o backup de versões, fora o gerenciamento de códigos de programação de um projeto, organizando, versionando e documentando todas as alterações.

Existem sistemas preparados para apoiar o cuidadoso processo de controle de mudança e de acompanhamento de alterações do software, registrando e gerenciando modificações durante o desenvolvimento e após o lançamento do software. Os sistemas apoiam a gerência de configuração, fazendo com que o processo tenha maior visibilidade entre as demais atividades e proporcionando estratégias de comunicação hábeis, sendo um registro confiável da evolução do código e dos demais itens de configuração.

- **Jira:** um software aproveitado na gerência de projetos no geral e no gerenciamento de configuração de projetos de software, com suporte para o rastreamento de defeitos e necessidades a serem

implementadas no sistema, dando estimativas referentes ao tempo de desenvolvimento de cada tarefa, já que armazena dados de execuções anteriores. Através dessa ferramenta, se pode planejar mudanças, acompanhar o processo das modificações feitas pelos desenvolvedores, avaliar o resultado e ter registros atualizados sobre essa atividade;

- **Caliber:** uma ferramenta voltada para o gerenciamento de mudanças de requisitos de software, otimizando o processo de coleta e análise de requisitos funcionais e não-funcionais de um software, apresentando recursos para manutenção e rastreamento de alterações de requisitos, o que admite a elaboração de relatórios sobre essas mudanças e o rastreamento dos requisitos para deliberar como tais alterações influenciam a programação e os testes no software;

- **Mantis:** é uma ferramenta popular, de fácil entendimento e gratuita para gerenciamento de falhas no software durante o desenvolvimento, tornando possível o controle total dos erros encontrados durante os testes, ditando as mudanças através da relevância e prioridade dos erros e mensurando o tempo gasto pelos desenvolvedores para efetivar as alterações dos defeitos encontrados no sistema.

Outro conjunto de otimização das atividades da gerência de configuração são as ferramentas de integração contínua, dada pela prática de desenvolvimento de software, que estabelece que versões atualizadas do sistema sejam integradas à versão mais estável. A integração contínua representa práticas de compilação, teste e construção num ambiente separado e independente, integrando as mudanças no código ao sistema mais rápida e ininterruptamente.

Assim, o resultado do trabalho de um engenheiro é adicionado ao pacote geral do sistema durante o desenvolvimento. Como cada indivíduo integra ao menos uma versão do seu trabalho diário, são efetivadas múltiplas integrações por dia. À mão, isso significaria ter que analisar o código do sistema várias vezes durante o dia para adicionar uma nova parte do código ou uma versão modificada sempre que estiver pronta. Portanto, a ferramenta é bastante útil por executar a tarefa com rapidez e segurança.

- **Jenkins:** presente nos processos de gerência de configuração e mudanças, em especial nas atividades de integração contínua de versões. Graças à integração contínua automatizada, tem como benefício a diminuição de riscos, uma vez que o software é integrado contínua e rapidamente. Falhas de integra-

ção também são detectadas e demonstradas, o que gera aumento de produtividade e performance;

- **Maven:** responsável por gerenciar dependências entre o código fonte compilado, permitindo ainda controlar a versão dos itens de configuração, gerar relatórios de produtividade de trabalho, apoiar o processo de execução de testes pós mudanças e auxiliar na manutenção do nível de qualidade do código produzido pelos programadores.

Ao final, a partir do Diagrama 6, que ilustra o contexto das ferramentas da gerência de configuração e sua aplicabilidade, se tem em mente que, na prática, as ferramentas são empregadas em conjunto nas empresas de desenvolvimento, haja vista a atuação de cada uma numa atividade específica da gerência de configuração. Dependendo do tamanho do sistema, todo o processo de controle de versão, controle de mudanças e integração contínua se torna complexo para ser executado manualmente. Deste modo, elas são aproveitadas a fim de otimizar, agilizar e dar segurança ao desenvolvimento de software, visto que as atividades têm relação com a qualidade de software, com a reputação da empresa e de seus produtos de software disponíveis no mercado.

DIAGRAMA 6. FERRAMENTAS DA GERÊNCIA DE CONFIGURAÇÃO.





Nesta unidade, foram tratados conceitos finais ligados à gerência de configuração e mudança no processo de desenvolvimento de software. É importante situar que a preocupação com melhorias do processo vem sendo impulsionada por exigências do mercado por mais qualidade e por fatores internos das empresas de software, como o aumento na produtividade.

Nesse contexto, a gerência de configuração é essencial para manter o desenvolvimento de software controlável, do ponto de vista de mudanças, e eficiente, do ponto de vista de lançamento do sistema. Contudo, ainda é grande o número de empresas que não utilizam nenhum tipo de processo ou apenas o controle de versão nos seus projetos.

As causas para tal situação são o desconhecimento da amplitude e a importância da gerência de configuração, em especial se considerando os riscos associados à não execução da atividade e como os riscos afetam o software de maneira negativa, ao mesmo tempo em que existe um desconhecimento das ferramentas de apoio.

Logo, se conclui que a gerência de configuração deve ser usada em todos os projetos de software, pois faz parte dos processos de qualidade de software, independentemente da estrutura, tamanho e complexidade. Há várias opções de ferramentas, inclusive gratuitas, para a otimização da atividade, tornando a implantação menos complexa. Não obstante, é necessário algum esforço para adequar as ferramentas aos processos já estabelecidos, o que requer treinamento específicos em alguns casos.

Referências bibliográficas



BERSOFF, E. H. Elements of Software Configuration Management. **IEEE Transactions on Software Engineering**, v. 10, n. 1, p. 79-87, 1984.

BOURQUE, P.; FAIRLEY, R. E. **Guide to the Software Engineering Body of Knowledge**. 3. ed. Los Alamitos: IEEE Computer Society Press, 2014.

DANTAS, C. **Gerência de Configuração de Software**. Engenharia de Software Magazine, 2009.

O QUE É Qualidade Software e porque você precisa ter? // PAPO INFORMAL. Postado por TIRio TV (7min. 33s.). son. color. port. Disponível em: <<https://www.youtube.com/watch?v=TmH7Jeb89wY>>. Acesso em: DATA.

SANCHES, R. **Gerência de configuração**. In: Qualidade de Software [s.l.]: s.n.], 2001.